A Programming Paradigm for Spatiotemporal Composability

Yifan Shi1,2, Wei Zhang1, Tianyi Cui2
1Peking University 2DeepSeek-AI

Abstract

Modern software—from plugin systems to self-evolving agent harnesses—increasingly requires dynamic composition, yet its formal foundations remain underdeveloped. We identify two orthogonal dimensions of the problem: temporal composability, the ability to completely revert a component's side efects upon removal, and spatial composability, the ability to declare and reactively manage inter-component dependencies. We address the two dimensions by lifting classical effect and coeffect concepts to runtime mechanisms. In particular, we formalize revertible efects, in which every context transformation carries an inverse that the runtime tracks. We formalize reactive coefects, in which each change of the context notifies a component against its coeffect specification. We unify the effect context and the coeffect context into a single context type, which constitutes a programming paradigm. After that, we combine these mechanisms into the notion of a component and give a calculus of dynamic composition, whose metatheory carries spatiotemporal composability from a single component to a whole system of interleaved components. We implement these ideas in Cordis, a meta-framework of spatiotemporal composability that provides a core library with effect tracking and coeffect resolution, as well as a declarative component loader with configuration reconciliation and hot module replacement.

1. Introduction

Composition—assembling complex systems from simpler parts—is a foundational principle of software engineering [1]. Traditionally, composition is static: function calls, module imports, and class inheritance are resolved at compile time and remain fixed throughout execution. However, modern software increasingly demands dynamic composition, where components are loaded, unloaded, and reconfigured at runtime. Plugin architectures [2] and self-evolving agent harnesses both require systems that can safely add and remove functionality on the fly, yet current practice defers to coarse-grained mechanisms [3] that reconfigure only by restarting, discarding runtime state. Despite the growing practical importance of dynamic composition, its theoretical foundations remain underdeveloped, compared to the rich formal frameworks available for static composition.

1.1. Dimensions of Composability

To characterize the requirements of dynamic composition, we identify two orthogonal dimensions beyond the well-studied algebraic aspects of composition:

• Temporal composability addresses the time dimension: upon removal of a component, the modifications the component made to the shared environment must be completely and safely reversed. This requires tracking every resource allocation, event registration, and state mutation the component performs, and guaranteeing their orderly reclamation upon removal.

• Spatial composability addresses the space dimension: components must be able to declare, discover, and resolve their dependencies on one another in a structured and verifiable manner. This requires managing dependency topology and coordinating component lifecycles in response to dependency changes.

In the static setting, temporal composability reduces to lexical scoping (e.g., RAII [4], bracket patterns [5]),

时空可组合性（Spatiotemporal Composability）的编程范式

石一帆（Yifan Shi）1,2、张伟（Wei Zhang）1、崔天翼（Tianyi Cui）2
1 北京大学（Peking University）2 深度求索（DeepSeek-AI）

摘要（Abstract）

现代软件——从插件系统到自我演化的智能体框架（agent harnesses）——越来越需要动态组合（dynamic composition），然而其形式化基础仍不成熟。我们识别出该问题的两个正交维度：时间可组合性（temporal composability），即组件被移除时其副作用能够被完全逆转的能力；以及空间可组合性（spatial composability），即声明并响应式地管理组件间依赖的能力。我们通过把经典的效应（effect）与余效应（coeffect）概念提升为运行时机制来应对这两个维度。具体而言，我们形式化了可逆效应（revertible effects）：每个上下文变换都携带一个由运行时跟踪的逆。我们形式化了响应式余效应（reactive coeffects）：上下文的每次变化都根据组件的余效应规范通知该组件。我们把效应上下文与余效应上下文统一为单一的上下文类型，这构成了一种编程范式。此后，我们把这些机制组合成组件的概念，并给出一个动态组合演算，其元理论把时空可组合性从单个组件推广到整个交错组件系统。我们在 Cordis 中实现了这些思想——一个时空可组合性的元框架，提供带效应跟踪与余效应解析的核心库，以及带配置对账（configuration reconciliation）与热模块替换（hot module replacement）的声明式组件加载器。

1. 引言（Introduction）

组合（composition）——用较简单的部分组装复杂系统——是软件工程的基本原则 [1]。传统上，组合是静态的：函数调用、模块导入和类继承在编译时解析，并在整个执行期间保持不变。然而，现代软件越来越需要动态组合，即组件在运行时被加载、卸载和重新配置。插件架构 [2] 和自演化智能体框架都要求系统能安全地即时添加和移除功能，但当前实践退而求其次地依赖粗粒度机制 [3]，只能通过重启来重新配置，从而丢弃运行时状态。尽管动态组合的实践重要性日益增长，其理论基础仍然薄弱，与静态组合丰富的形式化框架相比相形见绌。

1.1. 可组合性的维度（Dimensions of Composability）

为刻画动态组合的需求，我们在组合已得到充分研究的代数性质之外，识别出两个正交维度：

• 时间可组合性关注时间维度：在移除一个组件时，该组件对共享环境所做的修改必须被完全且安全地逆转。这要求跟踪组件执行的每一次资源分配、事件注册和状态变更，并保证在移除时有序回收。

• 空间可组合性关注空间维度：组件必须能够以结构化且可验证的方式声明、发现和解析彼此之间的依赖。这要求管理依赖拓扑，并针对依赖变化协调组件生命周期。

在静态设定下，时间可组合性归结为词法作用域（如 RAII [4]、括号模式 [5]），空间可组合性归结为模块导入

and spatial composability reduces to module import resolution [6]. In the dynamic setting, where components arrive and depart at runtime, both dimensions become significantly harder: temporal composability must handle long-lived, stateful efects whose scope is not lexically bounded; and spatial composability must handle dependencies that appear, disappear, or change identity during execution.

1.2. Motivating Examples

1.2.1. Plugin Systems

Plugin systems are a canonical instance of dynamic composition. We use Visual Studio Code (VSCode), one of the most widely-used extensible IDEs, as a representative example.

Temporal limitation. VSCode runs all extensions in a shared process called the extension host. Although extensions can be installed dynamically, this host provides no mechanism to unload an individual extension's code at runtime. Once an extension's activate function has executed, disabling or uninstalling it requires restarting the entire host, afecting all loaded extensions. Purely declarative extensions such as themes, keybindings, and snippets carry no code and can be removed freely. Among the top 100 extensions by install count, however, 87 contain executable code1 and will therefore require such a restart upon removal. Although VSCode provides a deactivate hook, it serves only as a graceful shutdown callback during the host process' termination, and thus does not enable live removal. Moreover, the hook separates effect disposal from effect creation (in activate), violating locality of concern and making complete cleanup dificult to verify.

Spatial limitation. VSCode does provide extensionDependencies for declaring dependencies between extensions, but it sees little use: among the top 100 extensions by install count, only 7 declare extensionDependencies on non-built-in extensions.1 This scarcity reflects the shape of the extension API, which exposes fixed, surface-level extension points such as commands, views, and language features. Extensions contribute to the host through these points rather than depending on one another, so inter-extension dependencies rarely arise. Moreover, VSCode's mechanism for inter-extension interaction provides no structural contract: it exposes an extension's functionality to others through vscode.extensions.getExtension(…).exports, but the returned value is untyped (any by default), so the dependent cannot rely on a checked interface. In short, VSCode steers extensions toward a fixed set of host-provided extension points, and ofers no safe, structured way for them to depend on one another.

These two limitations are not unique to VSCode; they recur across plugin systems generally [2, 7], difering only in degree.

1.2.2. Self-Evolving Agent Harnesses

Modern AI agents rely on runtime agent harnesses [8–10]. These systems may compose diverse tool suites [11] and execution environments, govern permissions and sandboxing, maintain session state and persistence, provide context management and memory systems [12], orchestrate subagents and multi-agent workflows [13], and expose interfaces to users and automation. A future harness may generate and deploy modifications to its own components while continuously serving requests. Model-synthesized reusable tools provide a narrower precursor to component-level self-modification [14]. Each such modification is itself an instance of dynamic composition.

Because these modifications occur continuously and with limited or no human oversight, dynamic composability becomes indispensable. Without temporal composability, each selfmodification forces a full restart that discards all process-local accumulated state; at such frequency the cumulative unavailability becomes substantial, and in-flight tasks are disrupted repeatedly; even worse, a faulty self-modification can disable the very process needed to recover. Without spatial composability, each module must itself detect and adapt to changes in the

解析 [6]。在动态设定下，组件在运行时到达和离开，两个维度都变得显著更困难：时间可组合性必须处理生命周期长、作用域不受词法约束的有状态效应；空间可组合性必须处理在执行期间出现、消失或改变身份的依赖。

1.2. 动机示例（Motivating Examples）

1.2.1. 插件系统（Plugin Systems）

插件系统是动态组合的典型实例。我们用 Visual Studio Code（VSCode）——使用最广泛的可扩展 IDE 之一——作为代表性示例。

时间上的局限。VSCode 在称为扩展宿主（extension host）的共享进程中运行所有扩展。虽然扩展可以动态安装，但这个宿主没有提供在运行时卸载单个扩展代码的机制。一旦扩展的 activate 函数执行完毕，禁用或卸载它就需要重启整个宿主，影响所有已加载的扩展。纯声明式扩展（如主题、按键绑定和代码片段）不携带代码，可以自由移除。然而在按安装量排名的前 100 个扩展中，87 个包含可执行代码 1，因此在移除时需要这样的重启。虽然 VSCode 提供了 deactivate 钩子，但它只是在宿主进程终止期间作为优雅关闭回调，因此不能实现实时移除。此外，该钩子把效应处置与效应创建（在 activate 中）分离开来，违背了关注点局部性，使完全清理难以验证。

空间上的局限。VSCode 确实提供了 extensionDependencies 来声明扩展之间的依赖，但它很少被使用：在按安装量排名的前 100 个扩展中，只有 7 个对非内置扩展声明了 extensionDependencies。1 这种稀缺反映了扩展 API 的形态，它暴露的是固定的、表面级的扩展点（如命令、视图和语言特性）。扩展通过这些点向宿主贡献能力，而不是相互依赖，因此扩展间的依赖很少出现。此外，VSCode 的扩展间交互机制不提供任何结构契约：它通过 vscode.extensions.getExtension(…).exports 向其他扩展暴露功能，但返回的值是无类型的（默认 any），因此依赖方无法依赖被检查的接口。简而言之，VSCode 把扩展引向一组固定的宿主提供的扩展点，并且不为它们之间的相互依赖提供安全、结构化的方式。

这两个局限并非 VSCode 独有；它们在各类插件系统中普遍存在 [2, 7]，只是程度不同。

1.2.2. 自我演化的智能体框架（Self-Evolving Agent Harnesses）

现代 AI 智能体依赖运行时智能体框架 [8–10]。这些系统可能组合多样化的工具套件 [11] 和执行环境，管理权限与沙箱，维护会话状态与持久化，提供上下文管理与记忆系统 [12]，编排子智能体与多智能体工作流 [13]，并向用户和自动化暴露接口。未来的框架可能在持续服务请求的同时，生成并部署对其自身组件的修改。模型合成的可复用工具为组件级自我修改提供了一个较窄的先驱 [14]。每一次这样的修改本身都是动态组合的一个实例。

由于这些修改连续发生，且几乎没有或完全没有人工监督，动态可组合性变得不可或缺。没有时间可组合性，每一次自我修改都迫使完全重启，丢弃所有进程内累积的状态；在这样的频率下，累积的不可用时间变得可观，进行中的任务被反复中断；更糟的是，一个有缺陷的自我修改可能禁用用于恢复的进程本身。没有空间可组合性，每个模块都必须自己检测并适应它所依赖模块的变化——这些模块出现、消失或改变身份——且只能通过临时手段进行；更糟的是，天真的代码替换策略可能静默破坏依赖方，或引入只有在重载时才暴露的循环依赖。

modules it depends on as they appear, disappear, or change identity, and can do so only by ad hoc means; even worse, a naive code-replacement strategy may silently break dependents or introduce circular dependencies that surface only at reload time.

1.2.3. The Coarse-Grained Workaround

One reason dynamic composability has received limited formal attention is that operating systems and container orchestrators already provide a coarse-grained substitute. Operating systems yield temporal composability at the granularity of a process; container orchestrators [3] yield spatial composability at the granularity of a service. In practice, most software tolerates the lack of fine-grained composability by deferring to these coarse-grained mechanisms: a misbehaving module is handled by restarting the process, and a service dependency is managed by the container orchestrator.

However, this workaround imposes substantial costs. Temporally, each restart discards all process-local accumulated state (e.g., caches, connections, partial computations), and rebuilding it takes seconds to minutes [15]; maintaining availability in the interim requires redundant replicas, incurring resource overhead to compensate for the inability to recover a single component. Spatially, container-level orchestration cannot express dependencies between components sharing an address space, and introduces network overhead for interactions that could be local function calls. Both mechanisms operate at the boundary of processes and containers, yet modern systems increasingly compose at a finer level. This granularity mismatch demands a compositional abstraction that manages efects and dependencies at the same level as the components themselves.

1.3. Contributions

The two dimensions of dynamic composability concern, respectively, how computations modify and how they depend on their environment. These two directions are what effect systems [16, 17] and coeffect systems [18, 19] formalize: efects provide the formal vocabulary for reasoning about environmental modifications, and coefects for reasoning about environmental requirements. However, existing formulations restrict reasoning to compile-time analysis over lexically fixed scopes, and do not extend to dynamic scenarios where components arrive and depart at runtime. By lifting efects to a revertible runtime model and coefects to a reactive dependency resolution mechanism, we obtain a unified formal foundation for dynamic composability, one that is language-agnostic and applicable to any software architecture requiring dynamic composition. We make the following contributions:

1. We formalize revertible efects (Section 3.1): every context transformation carries an explicit inverse that the runtime tracks, and both tracking and recovery preserve composition, so the context is recovered upon component removal. This establishes local temporal composability.

2. We formalize reactive coefects (Section 3.2): a component declares the coefects it requires as a specification, and each change of the context notifies the component against that specification as activating, deactivating, or neutral. This establishes local spatial composability.

3. We unify the effect context and the coeffect context into a single context type (Section 3.3), in which an observational equivalence on the coefects supplies the efects with independence, constituting a programming paradigm for spatiotemporal composability.

1.2.3. 粗粒度变通方案（The Coarse-Grained Workaround）

动态组合在形式上受到有限关注的一个原因是，操作系统和容器编排器已经提供了粗粒度的替代品。操作系统以进程为粒度提供时间可组合性；容器编排器 [3] 以服务为粒度提供空间可组合性。在实践中，大多数软件通过退守这些粗粒度机制来容忍细粒度可组合性的缺失：行为异常的模块通过重启进程来处理，服务依赖由容器编排器管理。

然而，这种变通方案代价高昂。在时间上，每次重启都会丢弃所有进程内累积的状态（如缓存、连接、部分计算），重建它需要数秒到数分钟 [15]；期间维持可用性需要冗余副本，为弥补无法恢复单个组件的缺陷而付出资源开销。在空间上，容器级编排无法表达共享同一地址空间的组件之间的依赖，并为本可以成为本地函数调用的交互引入网络开销。这两种机制都在进程和容器的边界处运作，而现代系统越来越在更细的层次上组合。这种粒度不匹配要求一种组合抽象，在与组件本身相同的层次上管理效应和依赖。

1.3. 贡献（Contributions）

动态组合的两个维度分别关注计算如何修改其环境，以及如何依赖其环境。这两个方向正是效应系统 [16, 17] 和余效应系统 [18, 19] 所形式化的：效应提供了推理环境修改的形式化词汇，余效应提供了推理环境需求的词汇。然而，现有表述把推理限制在词法固定作用域上的编译时分析，不扩展到组件在运行时到达和离开的动态场景。通过把效应提升为可逆的运行时模型，把余效应提升为响应式的依赖解析机制，我们为动态组合获得了一个统一的形式化基础——一个与语言无关、适用于任何需要动态组合的软件架构的基础。我们做出以下贡献：

1. 我们形式化了可逆效应（第 3.1 节）：每个上下文变换都携带一个由运行时跟踪的显式逆，跟踪和恢复都保持组合性，因此在组件移除时上下文被恢复。这建立了局部时间可组合性。

2. 我们形式化了响应式余效应（第 3.2 节）：组件把它所需的余效应声明为一个规范，上下文的每次变化都根据该规范把状态分类为激活、停用或中性，并通知该组件。这建立了局部空间可组合性。

3. 我们把效应上下文与余效应上下文统一为单一上下文类型（第 3.3 节），其中余效应上的观测等价为效应提供独立性，构成时空可组合性的编程范式。

4. We give a calculus of dynamic composition (Section 4), which combines the two mechanisms into the notion of a component and equips its lifecycle with an operational semantics. Its metatheory carries spatiotemporal composability from a single component to a whole system of interleaved components.

5. We implement these ideas in Cordis (Section 5), a meta-framework of spatiotemporal composability that provides a core library realizing the formal model with effect tracking and coeffect resolution, as well as a declarative component loader with configuration reconciliation and hot module replacement.

2. Preliminaries

This section provides a concise overview of effect and coeffect systems—the two theoretical pillars underlying our work. We assume familiarity with basic type theory and category theory; the goal here is to fix notation and introduce the key abstractions that Section 3 will operationalize as runtime mechanisms.

2.1. Effects

In the simply typed lambda calculus (STLC) [20, 21], a typing judgment $\Gamma \vdash t : T$ states that term t has type $_T$ under context Γ. An effect system refines the type to describe what side efects a computation may produce, yielding judgments of the form

$$\Gamma \vdash t : T_{\text{effect}} \tag{1}$$

Here, the result type is annotated with an element of an effect algebra that describes which side efects the computation may produce, enabling compositional reasoning about stateful computations. This approach originates with Lucassen and Giford [22], who introduced a kinded type system distinguishing types, efects, and regions to discover scheduling constraints in parallel programs.

Monadic efects. Moggi [16] first modeled computational efects categorically via monads; Wadler [23] popularized the approach in Haskell. A monad (T, η, μ) on a category $\mathcal{C}$ encapsulates an efectful computation as a value of type $T(A)$, with $\eta : AT(A)$ lifting pure values and $\mu : T(T(A)) \to T(A)$ sequencing nested computations. Classic instances include the Maybe monad (for partiality), State monad (for mutable state), and IO monad (for external interaction).

Algebraic efects. Plotkin and Power [17, 24] showed that algebraic operations determine monads, establishing a framework in which effect interfaces are decoupled from their implementations. An effect signature Σ declares a set of operations (e.g., $\text{get} : () \to S, \text{put} : S \to ()$ for state); programs invoke operations freely without committing to a particular interpretation. Plotkin and Pretnar [25] subsequently introduced effect handlers, which interpret operations by providing continuation semantics:

$$\text{handle } e \text{ with } \{\text{op}(v, \kappa) \mapsto \dots\} \tag{2}$$

The handler receives the operation argument v and the delimited continuation κ, which it may invoke zero, one, or multiple times, enabling exceptions, coroutines, and non-determinism within a uniform framework [26]. Languages such as Koka [27, 28], Ef [29], and OCaml 5 [30] have adopted algebraic efects with varying design trade-ofs.

4. 我们给出了动态组合演算（第 4 节），它把这两种机制组合为组件的概念，并为其生命周期配备操作语义。其元理论把时空可组合性从单个组件推广到整个交错组件系统。

5. 我们在 Cordis（第 5 节）中实现了这些思想——一个时空可组合性的元框架，提供实现形式模型的核心库（带效应跟踪与余效应解析），以及带配置对账与热模块替换的声明式组件加载器。

2. 预备知识（Preliminaries）

本节简要概述效应与余效应系统——支撑我们工作的两大理论支柱。我们假定读者熟悉基本的类型论与范畴论；这里的目标是固定记号并引入第 3 节将作为运行时机制操作化的关键抽象。

2.1. 效应（Effects）

在简单类型 λ 演算（STLC）[20, 21] 中，类型判断 $\Gamma \vdash t : T$ 陈述项 t 在上下文 Γ 下具有类型 $_T$。效应系统精化该类型，以描述计算可能产生哪些副作用，产生如下形式的判断

$$\Gamma \vdash t : T_{\text{effect}} \tag{1}$$

这里，结果类型被注解以效应代数的元素，它描述计算可能产生哪些副作用，从而能够对状态性计算进行组合式推理。这种方法起源于 Lucassen 和 Giford [22]，他们引入了一种分类类型系统，区分类型、效应和区域，以发现并行程序中的调度约束。

单子效应。Moggi [16] 首先通过单子对计算效应进行了范畴论建模；Wadler [23] 在 Haskell 中推广了这种方法。范畴 C 上的一个单子 (T, η, μ) 把一次有效应计算封装为 $T(A)$ 类型的值，其中 $\eta : AT(A)$ 提升纯值，$\mu : T(T(A)) \to T(A)$ 对嵌套计算进行排序。经典实例包括 Maybe 单子（表示部分性）、State 单子（表示可变状态）和 IO 单子（表示外部交互）。

代数效应。Plotkin 和 Power [17, 24] 表明代数运算决定单子，建立了效应接口与其实现解耦的框架。效应签名 Σ 声明一组运算 $(\text{e.g.}, \text{get} : () \to S, \text{put} : S \to ()$ 用于状态）；程序自由地调用运算，而不承诺特定的解释。Plotkin 和 Pretnar [25] 随后引入了效应处理器，它通过提供续延语义来解释运算：

$$\text{handle } e \text{ with } \{\text{op}(v, \kappa) \mapsto \dots\} \tag{2}$$

处理器接收运算参数 x 和限定续延（delimited continuation）κ, 它可以调用零次、一次或多次，从而在统一框架内支持异常、协程和非确定性 [26]。Koka [27, 28]、Ef [29] 和 OCaml 5 [30] 等语言以不同的设计取舍采纳了代数效应。

2.2. Coeffects

Dually to efects, a coeffect system [18, 31] enriches the context rather than the type, yielding judgments of the form

$$\Gamma_{\text{coeffect}} \vdash t : T \tag{3}$$

Here, the context is annotated with an element of a coeffect algebra describing what the computation requires from its environment, such as resources to access, permissions to hold, or services to depend on. While efects model a program's impact on the world, coefects model the world's constraints on the program.

Comonadic coefects. The idea of using comonads to structure context-dependent computation was first developed by Uustalu and Vene [32], who proposed symmetric (semi)monoidal comonads as the dual of Moggi′s monadic framework for efects, capturing notions such as dataflow and attribute evaluation. Petricek et al. [18] built on this foundation to propose coefects as a unified static analysis of context-dependence. A comonad (D, ε, δ) captures context-dependent computation: $\varepsilon : D(A) \to A$ extracts the current value from a context, and $\delta : D(A)D(D(A))$ duplicates context for nested access. The Environment comonad $D(X) = E \times X$ models dependence on a fixed environment E; the Stream comonad $D(X) = \mathbb{N} \to X$ models dependence on temporal data.

Graded coefects. For finer-grained tracking, graded coeffect systems use a pre-ordered semiring $\boldsymbol{S} = (S, \mathbf{\Sigma}, +, \times, 0, 1)$ as the coeffect algebra [33], a discipline later unified with graded efects by Gaboardi et al. [19]. Elements of S annotate each variable binding to quantify its usage: 0 for unused, 1 for linear use, n for bounded use, ∞ for unrestricted use. The semiring operations compose coefects sequentially ($\times$) and in parallel ($+$), enabling precise resource tracking, sensitivity analysis [34], and information-flow control [35, 36] within a unified algebraic framework [37].

2.3. Relationship to Dynamic Composability

Effect and coeffect systems organize reasoning about computation along two complementary directions: efects describe how a computation modifies its environment, whereas coefects describe how it depends on its environment. These two directions correspond to the two dimensions of dynamic composability identified in Section 1:

• Temporal composability demands that a component's modifications to the shared environment be revertible upon unloading. The relevant efects are the stateful ones, which durably transform that environment; undoing such a transformation requires it to admit an inverse.

• Spatial composability demands that inter-component dependencies be declared and managed reactively. Such dependencies are the very thing coefects capture, and managing them amounts to resolving each against what the environment supplies.

However, classical effect and coeffect systems are static instruments: efects are tracked within lexically fixed scopes and discharged by compile-time handlers; coeffect annotations are verified against contexts determined before execution. Dynamic composition, by contrast, requires these guarantees to hold for components that arrive and depart at runtime, against contexts that evolve continuously. No fixed lexical scope can delimit a plugin loaded after deployment; no compile-time context can anticipate dependencies that emerge from runtime configuration.

This motivates a shift in perspective: rather than extending static type systems with more annotations, we reify the conceptual structures of efects and coefects so that a runtime can operate on them directly, establishing dynamically the guarantees these systems provide statically.

2.2. 余效应（Coeffects）

与效应相对偶，余效应系统 [18, 31] 丰富的是上下文而非类型，产生如下形式的判断

$$\Gamma_{\text{coeffect}} \vdash t : T \tag{3}$$

这里，上下文被注解以余效应代数的元素，它描述计算从环境需要什么，例如要访问的资源、要持有的权限或要依赖的服务。效应建模程序对世界的影响，而余效应建模世界对程序的约束。

余单子余效应。用余单子结构化上下文相关计算的想法最初由 Uustalu 和 Vene [32] 提出，他们提出对称（半）幺半余单子，作为 Moggi's 单子效应框架的对偶，捕获数据流和属性求值等概念。Petricek 等人 [18] 在此基础之上提出余效应作为上下文依赖的统一静态分析。一个余单子 (D, ε, δ) 捕获上下文相关计算：$\varepsilon : D(A) \to A$ 从上下文提取当前值，$\delta : D(A)D(D(A))$ 为嵌套访问复制上下文。Environment 余单子 $D(X) = E \times X$ 建模对固定环境 E; 的依赖；Stream 余单子 $D(X) = \mathbb{N} \to X$ 建模对时序数据的依赖。

分次余效应。为进行更精细的跟踪，分次余效应系统使用预序半环 $\boldsymbol{S} = (S, \mathbf{\Sigma}, +, \times, 0, 1)$ 作为余效应代数 [33]，这一纪律后来被 Gaboardi 等人 [19] 与分次效应统一。S 的元素注解每个变量绑定以量化其使用：0 表示未使用，1 表示线性使用，ω 表示有界使用，∞ 表示无限制使用。半环运算按顺序 ($\times$) 和并行 ($+$) 组合余效应，从而在统一的代数框架 [37] 内实现精确的资源跟踪、敏感度分析 [34] 和信息流控制 [35, 36]。

2.3. 与动态组合的关系（Relationship to Dynamic Composability）

效应与余效应系统沿两个互补方向组织关于计算的推理：效应描述计算如何修改其环境，而余效应描述计算如何依赖其环境。这两个方向对应第 1 节识别的动态组合的两个维度：

• 时间可组合性要求组件对共享环境的修改在卸载时可逆。相关的效应是有状态的那些，它们持久地变换该环境；撤销这样的变换要求它容许一个逆。

• 空间可组合性要求组件间依赖被声明并响应式地管理。这样的依赖正是余效应所捕获的，而管理它们相当于把每个依赖对环境中提供的东西进行解析。

然而，经典效应与余效应系统是静态工具：效应在词法固定作用域内被跟踪，由编译时处理器清结（discharge）；余效应注解针对执行前确定的上下文进行验证。相比之下，动态组合要求这些保证对在运行时到达和离开的组件成立，而对持续演化的上下文成立。没有任何固定词法作用域可以限定部署后加载的插件；没有任何编译时上下文可以预见运行时配置中涌现的依赖。

这促使视角的转变：与其用更多注解扩展静态类型系统，我们把效应和余效应的概念结构具体化（reify），使运行时能够直接对它们操作，从而在动态中建立这些系统在静态中提供的保证。

3. Revertible Effects and Reactive Coeffects

This section lifts the concepts of efects and coefects introduced in Section 2 to runtime mechanisms, constructing a theory of dynamic composition. The central idea is to turn the typing contexts carrying efects and coefects into context types, i.e., runtime-operable types that reify the context as a first-class entity. For the effect type, we model it as a context transformation paired with an inverse, achieving local temporal composability. For the coeffect context, we model it as a type carrying dependency information, achieving local spatial composability. An observational equivalence on the coefects then supplies the efects with independence. The unified context that carries both efects and coefects constitutes a programming paradigm in its own right.

3.1. Revertible Effects

Temporal composability is the ability to load and unload components at runtime such that, upon unloading, the shared environment is recovered to its pre-composition state. This requires that every modification a component makes to the environment be both trackable and recoverable. We therefore model an effect as a function of type $\Gamma\Gamma \times (\Gamma\Gamma)$: applied to the current context, it yields the modified context together with an explicit inverse. Supplying that inverse is what lets the effect be reverted, and returning it to the runtime is what makes the effect trackable. We call such efects revertible: by tracking and composing these inverses during execution, complete environment recovery becomes a structural guarantee.

3.1.1. Effect Context

Given any impure function $f_{\text{impure}} : XY.$, we transform it into a pure form $f : \Gamma \times X \to \Gamma\times Y$, where Γ is the context and all possible side efects can be represented as transformations on Γ. For any fixed input $x : X$, , the induced map $\gamma \mapsto \text{pr}_1(f(\gamma, x))$) captures the side effect of f independently of the return value. Effects on Γ therefore live in the monoid of transformations $\Gamma \to \Gamma$ under composition $\circ$, where each monoid axiom has a direct reading as a property of efects:

- Closure: the sequential composition of two efects is again an effect;
- Associativity: a composite effect is independent of how it is bracketed;
- Identity: id_Γ, the identity function on Γ, acts as the unit of composition.

To model efects that can be undone, we pair each transformation f with another transformation g that undoes f, and call g a left inverse of f, abbreviated to inverse throughout the paper. Undoing is one-sided: what an inverse is held to is $g \circ f$ and never $f \circ g$. Pairs of transformations carry a multiplication of their own:

Definition 1. Define the twisted composition of pairs of context transformations by

$$(f_1, g_1) \circ (f_2, g_2) := (f_1 \circ f_2, g_2 \circ g_1) \tag{4}$$

As for $\circ$ itself, the left operand acts after the right, and the inverses accumulate in the opposite order. It makes $(\Gamma\Gamma) \times (\Gamma\Gamma)$ a monoid with unit $(\text{id}_\Gamma, \text{id}_\Gamma)$, the product of the monoid of transformations with its opposite, which we call the twisted composition monoid $\mathfrak{T}_\Gamma$ over Γ.

To track efects within the context itself, we introduce the following definition:

Definition 2. Given a context Γ, define its effect context as:

$$\partial\Gamma := \Gamma \times (\Gamma \to \Gamma) \tag{5}$$

It can be understood as a pair (γ, φ) , where:

3. 可逆效应与响应式余效应（Revertible Effects and Reactive Coeffects）

本节把第 2 节引入的效应与余效应概念提升为运行时机制，构建动态组合的理论。核心思想是把携带效应和余效应的类型上下文转变为上下文类型（context types）——把上下文具体化为第一类实体的、可在运行时操作的类型。对效应类型，我们把它建模为带逆的上下文变换，实现局部时间可组合性。对余效应上下文，我们把它建模为携带依赖信息的类型，实现局部空间可组合性。随后余效应上的观测等价为效应提供独立性。同时携带效应和余效应的统一上下文自身构成一种编程范式。

3.1. 可逆效应（Revertible Effects）

时间可组合性是在运行时加载和卸载组件，使得卸载时共享环境恢复到组合前的状态的能力。这要求组件对环境所做的每次修改都可跟踪、可恢复。因此我们把效应建模为类型 $\Gamma\Gamma \times (\Gamma\Gamma)$ 的函数：应用于当前上下文时，它产生修改后的上下文以及一个显式逆。提供该逆使效应可被逆转，把它交给运行时使效应可被跟踪。我们称这样的效应为可逆的：在执行期间跟踪并组合这些逆，完整的环境恢复就成为结构性保证。

3.1.1. 效应上下文（Effect Context）

给定任意非纯函数 $f_{\text{impure}} : XY.$，我们把它变换为纯形式 $f : \Gamma \times X \to \Gamma\times Y$, 其中 Γ 是上下文，所有可能的副作用都可以表示为对 Γ 的变换。对任意固定输入 $x : X$,，诱导映射 $\gamma \mapsto \text{pr}_1(f(\gamma, x))$) 独立于返回值捕获 f 的副作用。因此 Γ 上的效应位于变换 $\Gamma \to \Gamma$ 的幺半群中（在复合 $\circ$ 下），其中每个幺半群公理都可以直接读作效应的一种性质：

- 封闭性（Closure）：两个效应的顺序复合仍是一个效应；
- 结合性（Associativity）：复合效应的结果与其括号方式无关；
- 单位元（Identity）：id_Γ, 即 Γ 上的恒等函数，作为复合的单位。

为建模可被撤销的效应，我们把每个变换 f 与撤销 f 的另一个变换 g 配对，并称 g 为 f 的左逆，在本文中简称为逆。撤销是单侧的：逆被要求满足的是 $g \circ f$ 而绝非 $f \circ g$ 。变换对携带它们自己的乘法：

定义 1. 定义上下文变换对的扭转复合（twisted composition）为

$$(f_1, g_1) \circ (f_2, g_2) := (f_1 \circ f_2, g_2 \circ g_1) \tag{4}$$

与 $\circ$ 本身一样，左操作数在右操作数之后作用，而逆以相反的顺序累积。它使 $(\Gamma\Gamma) \times (\Gamma\Gamma)$ 成为以 $(\text{id}_\Gamma, \text{id}_\Gamma)$ 为单位的幺半群，即变换幺半群与其对偶的积，我们称之为 Γ 上的扭转复合幺半群 $\mathfrak{T}_\Gamma$。

为在上下文内部跟踪效应，我们引入以下定义：

定义 2. 给定上下文 Γ，定义其效应上下文为：

$$\partial\Gamma := \Gamma \times (\Gamma \to \Gamma) \tag{5}$$

它可以理解为二元组 (γ, φ) ，其中：

• $\gamma : \Gamma$ is the current context state;

$\varphi : \Gamma \to \Gamma$ is the accumulator, the composite of the inverses of the efects performed so far, and the function that recovers the context to its initial state.

In particular, the initial effect context can be represented as (γ_0, id)

We also write $\partial^2\Gamma = \partial\Gamma \times (\partial\Gamma \to \partial\Gamma)$, and so on up the tower.

Given the presence of the accumulator φ, all efects performed on $\partial\Gamma$ can be tracked and recovered. We now give the concrete constructions for tracking and recovery.

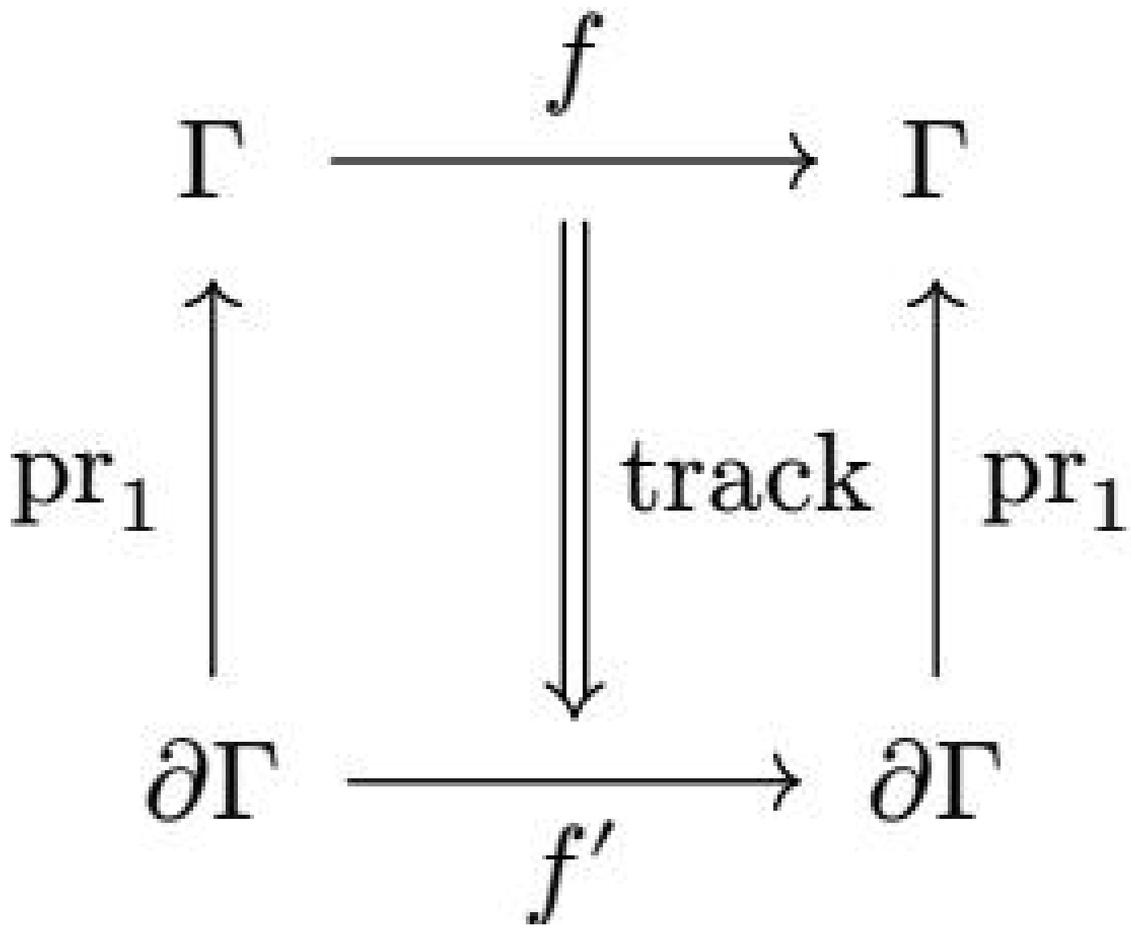
Definition 3. Define the transformation track on pairs of context functions:

$$\begin{array}{lcccccc} \mathrm{track}_\Gamma & : & (\Gamma \to \Gamma) \times (\Gamma \to \Gamma) & \to & \partial\Gamma & \to & \partial\Gamma \\ \mathrm{track}_\Gamma & = & (f, g) & \mapsto & (\gamma, \varphi) & \mapsto & (f(\gamma), \varphi \circ g) \end{array} \tag{6}$$

This transformation converts a forward function f together with a candidate inverse g into a transformation of the effect context $\partial\Gamma$. Applying $\mathrm{track}_\Gamma(f, g)$ to a state (γ, φ) transforms γ by f and composes the inverse g onto φ, thereby tracking the effect of f in the context.

Theorem 4. For every $(f, g) \in (\Gamma \to \Gamma) \times (\Gamma \to \Gamma)$ the following diagram commutes, that is,

$$\mathrm{pr}_1 \circ \mathrm{track}_\Gamma(f, g) = f \circ \mathrm{pr}_1 \tag{7}$$



• $\gamma : \Gamma$ 是当前上下文状态；

$\varphi : \Gamma \to \Gamma$ 是累加器（accumulator），即迄今为止所执行效应之逆的复合，是把上下文恢复到初始状态的函数。

特别地，初始效应上下文可以表示为 $(\gamma_0, \mathrm{id}_\Gamma)$

我们还写 $\partial^2\Gamma = \partial\Gamma \times (\partial\Gamma \to \partial\Gamma)$ ，以此类推向上攀升。

鉴于累加器 φ, 的存在，对 $\partial\Gamma$ 执行的所有效应都可被跟踪和恢复。我们现在给出跟踪与恢复的具体构造。

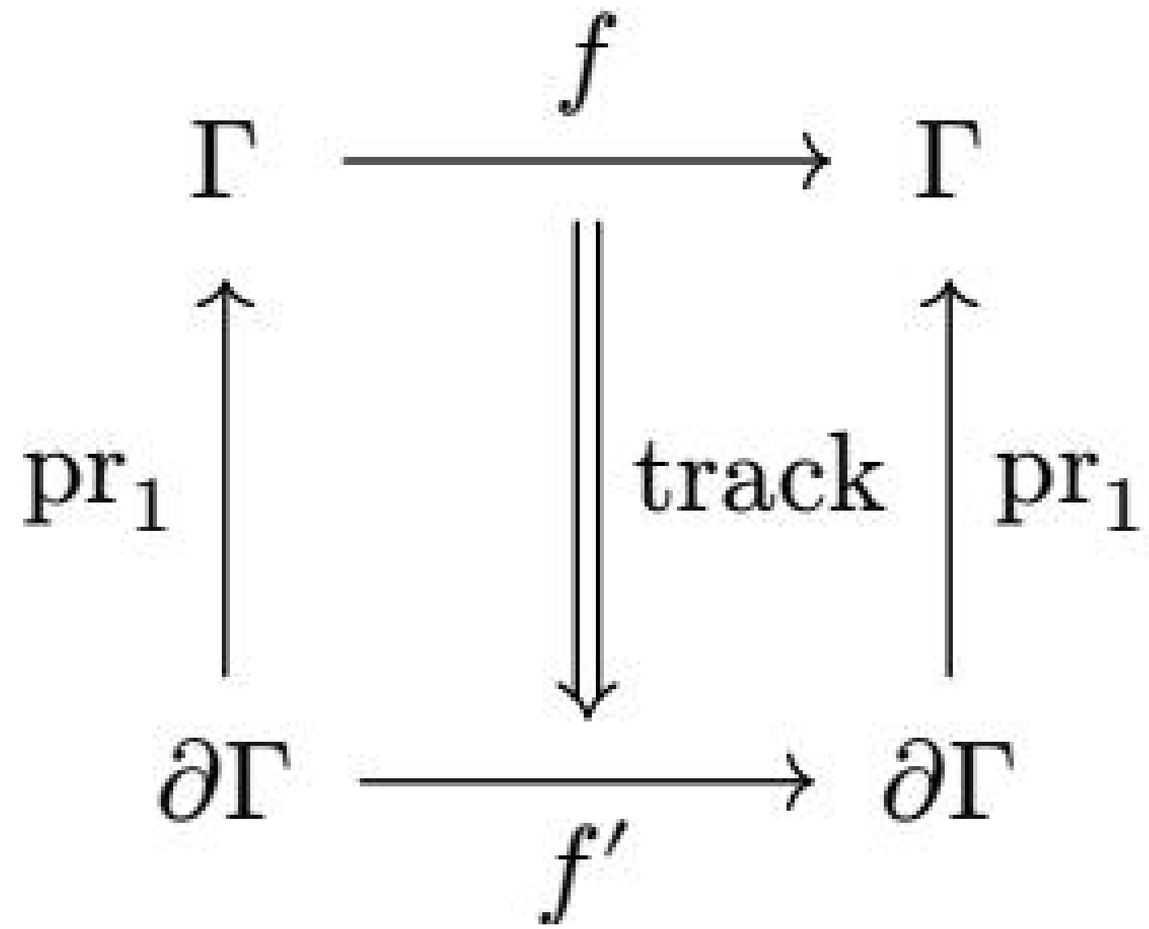
定义 3. 在上下文函数对上定义变换 track：

$$\begin{array}{lcccccc} \mathrm{track}_\Gamma & : & (\Gamma \to \Gamma) \times (\Gamma \to \Gamma) & \to & \partial\Gamma & \to & \partial\Gamma \\ \mathrm{track}_\Gamma & = & (f, g) & \mapsto & (\gamma, \varphi) & \mapsto & (f(\gamma), \varphi \circ g) \end{array} \tag{6}$$

该变换把一个正向函数 f 连同候选逆 g 转换为效应上下文 $\partial\Gamma$ 上的一个变换。把 $\mathrm{track}_\Gamma(f, g)$ 应用于状态 (γ, φ) 用 f 变换 γ 并把逆 g 复合到 φ, 上，从而在上下文中跟踪 f 的效应。

定理 4. 对每个 $(f, g) \in (\Gamma \to \Gamma) \times (\Gamma \to \Gamma)$ 下图可交换，即

$$\mathrm{pr}_1 \circ \mathrm{track}_\Gamma(f, g) = f \circ \mathrm{pr}_1 \tag{7}$$



Proof. For all $(\gamma, \varphi) \in \partial\Gamma$

$$\begin{aligned}(\mathrm{pr}_1 \circ \mathrm{track}_\Gamma(f,g))(\gamma,\varphi) &= \mathrm{pr}_1(f(\gamma), \varphi \circ g) \\ &= f(\gamma) \\ &= (f \circ \mathrm{pr}_1)(\gamma,\varphi)\end{aligned}$$

Theorem 5. track is a monoid homomorphism from $\mathfrak{T}_\Gamma$ into $\partial\Gamma\partial\Gamma$. That is,

1. track $\widehat{\varepsilon}(\mathrm{id}_\Gamma, \mathrm{id}_\Gamma) = \mathrm{id}_{\partial\Gamma}$;

2. for all $(f_1, g_1), (f_2, g_2) \in \mathfrak{T}_\Gamma$,

$$\mathrm{track}_\Gamma\left((f_1,g_1) \circ (f_2,g_2)\right) = \mathrm{track}_\Gamma\left(f_1,g_1\right) \circ \mathrm{track}_\Gamma\left(f_2,g_2\right) \tag{8}$$

Proof.

1. The unit is carried to the unit, since track $\langle \mathrm{id}_\Gamma, \mathrm{id}_\Gamma\rangle(\gamma,\varphi) = (\gamma, \varphi \circ \mathrm{id}_\Gamma) = (\gamma,\varphi)$

2. For the multiplication, take any $(\gamma,\varphi) \in \partial\Gamma$

$$\begin{aligned}(\mathrm{track}_\Gamma(f_1,g_1) \circ \mathrm{track}_\Gamma(f_2,g_2))(\gamma,\varphi) &= \mathrm{track}_\Gamma(f_1,g_1)(f_2(\gamma), \varphi \circ g_2) \\ &= (f_1(f_2(\gamma)), \varphi \circ g_2 \circ g_1) \\ &= \mathrm{track}_\Gamma(f_1 \circ f_2, g_2 \circ g_1)(\gamma,\varphi)\end{aligned}$$

Definition 6. Define the transformation recover on $\partial\Gamma$:

$$\begin{array}{lcccc}\mathrm{recover}_\Gamma & : & \partial\Gamma & \to & \partial\Gamma \\ \mathrm{recover}_\Gamma & = & (\gamma,\varphi) & \mapsto & (\varphi(\gamma), \mathrm{id}_\Gamma)\end{array} \tag{9}$$

This transformation applies the recovery function φ to the current state γ and resets φ to the identity. The following diagram illustrates how recover recovers the context to its initial state after a sequence of efects track $(f_1, g_1), \cdots$, track (f_n, g_n) has been applied to $\partial\Gamma$:

$$\begin{array}{ccc}\Gamma & \xrightarrow{f_1} & \Gamma \to \Gamma \to \Gamma \\ \| & & \| \ \mathrm{track} \\ \partial\Gamma & \xrightarrow{f_1'} & \partial\Gamma \to \partial\Gamma \to \partial\Gamma \\ \| & & \| \ \mathrm{track} \\ \| & & \| \ \mathrm{track} \\ \| & & \| \ \mathrm{recover}\end{array}$$

The diagram shows that the tracked efects followed by recover carry the initial effect context back to itself. What each tracking step preserves is the result of recovery itself, from whatever state it is taken:

证明. 对所有 $(\gamma,\varphi) \in \partial\Gamma$

$$\begin{aligned}(\mathrm{pr}_1 \circ \mathrm{track}_\Gamma(f,g))(\gamma,\varphi) &= \mathrm{pr}_1(f(\gamma), \varphi \circ g) \\ &= f(\gamma) \\ &= (f \circ \mathrm{pr}_1)(\gamma,\varphi)\end{aligned}$$

定理 5. track 是从 $\mathfrak{T}_\Gamma$ 到 $\partial\Gamma\partial\Gamma$ 的幺半群同态。即，

1. track $\hat{\varepsilon}_\Gamma(\mathrm{id}_\Gamma, \mathrm{id}_\Gamma) = \mathrm{id}_{\partial\Gamma}$;

2. 对所有 $(f_1, g_1), (f_2, g_2) \in \mathfrak{T}_\Gamma$,

$$\mathrm{track}_\Gamma\left((f_1,g_1) \circ (f_2,g_2)\right) = \mathrm{track}_\Gamma\left(f_1,g_1\right) \circ \mathrm{track}_\Gamma\left(f_2,g_2\right) \tag{8}$$

证明.

1. 单位元被送到单位元，因为 track $\langle \mathrm{id}_\Gamma, \mathrm{id}_\Gamma\rangle(\gamma,\varphi) = (\gamma, \varphi \circ \mathrm{id}_\Gamma) = (\gamma,\varphi)$

2. 对乘法，取任意 $(\gamma,\varphi) \in \partial\Gamma$

$$\begin{aligned}(\mathrm{track}_\Gamma(f_1,g_1) \circ \mathrm{track}_\Gamma(f_2,g_2))(\gamma,\varphi) &= \mathrm{track}_\Gamma(f_1,g_1)(f_2(\gamma), \varphi \circ g_2) \\ &= (f_1(f_2(\gamma)), \varphi \circ g_2 \circ g_1) \\ &= \mathrm{track}_\Gamma(f_1 \circ f_2, g_2 \circ g_1)(\gamma,\varphi)\end{aligned}$$

定义 6. 在 $\partial\Gamma$ 上定义变换 recover：

$$\begin{array}{lcccc}\mathrm{recover}_\Gamma & : & \partial\Gamma & \to & \partial\Gamma \\ \mathrm{recover}_\Gamma & = & (\gamma,\varphi) & \mapsto & (\varphi(\gamma), \mathrm{id}_\Gamma)\end{array} \tag{9}$$

该变换把恢复函数 φ 应用于当前状态 γ 并把 φ 重置为恒等。下图说明在一系列效应 track $(f_1, g_1), \cdots$，track (f_n, g_n) 应用于 $\partial\Gamma$ 后，recover 如何把上下文恢复到其初始状态：

$$\begin{array}{ccc}\Gamma & \xrightarrow{f_1} & \Gamma \to \Gamma \to \Gamma \\ \| & & \| \ \mathrm{track} \\ \partial\Gamma & \xrightarrow{f_1'} & \partial\Gamma \to \partial\Gamma \to \partial\Gamma \\ \| & & \| \ \mathrm{track} \\ \| & & \| \ \mathrm{track} \\ \| & & \| \ \mathrm{recover}\end{array}$$

该图表明被跟踪的效应随后跟一个 recover 会把初始效应上下文带回到其自身。每个跟踪步骤所保持的是恢复本身的结果，无论从什么状态取之：

Theorem 7. For every $(\gamma, \varphi) \in \partial\Gamma$ and every pair (f, g) with $g(f(\gamma)) = \gamma$,

$$\mathrm{recover}_\Gamma(\mathrm{track}_\Gamma(f,g)(\gamma,\varphi)) = \mathrm{recover}_\Gamma(\gamma,\varphi) \tag{10}$$

Proof.

$$\begin{aligned}\mathrm{recover}_\Gamma(\mathrm{track}_\Gamma(f,g)(\gamma,\varphi)) &= \mathrm{recover}_\Gamma(f(\gamma), \varphi\circ g)\\ &= (\varphi(g(f(\gamma))), \mathrm{id}_\Gamma)\\ &= (\varphi(\gamma), \mathrm{id}_\Gamma) = \mathrm{recover}_\Gamma(\gamma,\varphi)\end{aligned}$$

A sequence of pairs needs no separate argument. Let $(f_1, g_1), \cdots, (f_n, g_n)$ be applied in order from (γ, φ) , and write $\delta_0 = \gamma$ and $\delta_i := \overline{:} f_i(\delta_{i-1})$). By Theorem 5 the composite track $\mathit{F}(f_n, g_n) \circ \cdots \circ \mathrm{track}\cdot(f_1, g_1)$ is track of the twisted composite $(f_n \circ \cdots \circ f_1, g_1 \circ \cdots \circ g_n)$, and if $g_i(\delta_i) = \delta_{i-1}$ for every i then $(g_1 \circ \cdots \circ g_n)(\delta_n) = \delta_0 = \gamma$. That pair therefore meets the hypothesis of Theorem 7 at γ, and one application of the theorem gives

$$\mathrm{recover}_\Gamma\left(\left(\mathrm{track}_\Gamma\left(f_n, g_n\right) \circ \cdots \circ \mathrm{track}_\Gamma\left(f_1, g_1\right)\right)(\gamma,\varphi)\right) = \mathrm{recover}_\Gamma(\gamma,\varphi) \tag{11}$$

Taking $(\gamma, \varphi) = (\gamma_0, \mathrm{id}_\Gamma)$, recovery carries every state reached this way back to (γ_0, id) . A pair with $g \circ f = \mathrm{id}_\Gamma$ meets the hypothesis at every state.

Recovery reads a state through the quantity $\varphi(\gamma)$, and we refer to $\varphi(\gamma) = \gamma_0$ as the soundness invariant of a state in $\partial\Gamma$

3.1.2. Revertible Effect Functions

The track/recover model of the previous section takes inverses as given a priori: $\mathrm{track}_\Gamma(f, g)$ fixes g before any context state is seen, so one g has to serve every state the effect is applied at. In practice, however, the inverse of each effect is not known a priori: it must be supplied by the caller at the point of effect application. Moreover, recover is all-or-nothing: it cannot selectively undo one effect while retaining others. To address both issues, we enhance the model at both the input and output sides:

1. On the input side, we not only transform Γ but also return an inverse function alongside it, so that the inverse is supplied where the effect is applied: $\Gamma \to \Gamma \times (\Gamma \to \Gamma), \mathrm{i.e.}, \Gamma \to \partial\Gamma.$;

2. On the output side, we not only transform $\partial\Gamma$ but also return an inverse function alongside it, so that one effect can be undone while the others are retained: $\$\partial\Gamma\partial\Gamma\times(\ \partial\Gamma\$\ \partial\Gamma), \mathrm{i.e.}, \partial\Gamma\partial^2\Gamma$

This enhancement preserves structural consistency between input and output, so we can still define corresponding theory that maintains the mathematical properties of track. The resulting types are the effect functions $\mathfrak{E}_\Gamma$ and their witnessed refinement $\mathfrak{E}^*_\Gamma$:

Definition 8. Define the effect function $\mathfrak{E}_\Gamma$ and witnessed effect function $\mathfrak{E}^*_\Gamma$ as:

$$\begin{aligned}&\mathfrak{E}_\Gamma := \Gamma \to \Gamma \times (\Gamma \to \Gamma)\\ &\mathfrak{E}^*_\Gamma := (e : \Gamma \to \Gamma \times (\Gamma \to \Gamma))\\ &\qquad \times((\gamma : \Gamma) \to ((\delta : \Gamma) \times (g : \Gamma \to \Gamma) \times ((\delta, g) = e(\gamma) \to g(\delta) = \gamma)))\end{aligned} \tag{12}$$

where $e(\gamma)$ yields a pair (δ, g) representing:

$\delta : \Gamma$ is the new context;

定理 7. 对每个 $(\gamma, \varphi) \in \partial\Gamma$ 和每个满足 $g(f(\gamma)) = \gamma,$ 的对 (f, g)

$$\mathrm{recover}_\Gamma(\mathrm{track}_\Gamma(f,g)(\gamma,\varphi)) = \mathrm{recover}_\Gamma(\gamma,\varphi) \tag{10}$$

证明.

$$\begin{aligned}\mathrm{recover}_\Gamma(\mathrm{track}_\Gamma(f,g)(\gamma,\varphi)) &= \mathrm{recover}_\Gamma(f(\gamma), \varphi\circ g)\\ &= (\varphi(g(f(\gamma))), \mathrm{id}_\Gamma)\\ &= (\varphi(\gamma), \mathrm{id}_\Gamma) = \mathrm{recover}_\Gamma(\gamma,\varphi)\end{aligned}$$

一列对无需单独论证。设 $(f_1, g_1), \cdots, (f_n, g_n)$ 从 (γ, φ) 按顺序应用，并写 $\delta_0 = \gamma$ 且 $\delta_i := \overline{:} f_i(\delta_{i-1})$)。由定理 5 复合 $\mathrm{track}\ \mathit{F}(f_n, g_n) \circ \cdots \circ \mathrm{track}\cdot(f_1, g_1)$ 是扭转复合 $(f_n \circ \cdots \circ f_1, g_1 \circ \cdots \circ g_n)$ 的 track，若对每个 i 有 $g_i(\delta_i) = \delta_{i-1}$ 则 $(g_1 \circ \cdots \circ g_n)(\delta_n) = \delta_0 = \gamma$ 。因此该对在 γ 处满足定理 7 的假设，应用一次该定理即得

$$\mathrm{recover}_\Gamma\left(\left(\mathrm{track}_\Gamma\left(f_n, g_n\right) \circ \cdots \circ \mathrm{track}_\Gamma\left(f_1, g_1\right)\right)(\gamma,\varphi)\right) = \mathrm{recover}_\Gamma(\gamma,\varphi) \tag{11}$$

取 $(\gamma, \varphi) = (\gamma_0, \mathrm{id}_\Gamma)$ ，恢复把每个以此方式达到的状态带回到 $(\gamma_0, \mathrm{id}_\Gamma)$ 。满足 $g \circ f = \mathrm{id}_\Gamma$ 的对在任意状态满足该假设。

恢复通过量 $\varphi(\gamma)$ 读取一个状态，我们把 $\varphi(\gamma) = \gamma_0$ 称为 $\partial\Gamma$ 中状态的可靠性不变量（soundness invariant）。

3.1.2. 可逆效应函数（Revertible Effect Functions）

上一节的 track/recover 模型把逆视为先验给定的：$\mathrm{track}_\Gamma(f, g)$ 在看到任何上下文状态之前就固定了 g，因此一个 g 必须服务于效应所应用的每个状态。然而在实践中，每个效应的逆并非先验已知：它必须在效应应用点由调用者提供。此外，recover 是全有或全无的：它不能选择性地撤销一个效应而保留其他效应。为处理这两个问题，我们在输入侧和输出侧都增强模型：

1. 在输入侧，我们不仅变换 Γ，还同时返回一个逆函数，使逆在效应被应用之处提供：$\Gamma \to \Gamma \times (\Gamma \to \Gamma), \mathrm{i.e.}, \Gamma \to \partial\Gamma.$；

2. 在输出侧，我们不仅变换 $\partial\Gamma$，还同时返回一个逆函数，使一个效应可以被撤销而其他效应被保留：$\partial\Gamma \to \partial\Gamma \times (\partial\Gamma \to \partial\Gamma), \mathrm{i.e.}, \partial\Gamma \to \partial^2\Gamma$

这种增强在输入和输出之间保持结构一致性，因此我们仍然可以定义维护 track 数学性质的相应理论。得到的类型是效应函数 $\mathfrak{E}_\Gamma$ 及其带见证的精化 $\mathfrak{E}^*_\Gamma$：

定义 8. 定义效应函数 $\mathfrak{E}_\Gamma$ 和带见证的效应函数 $\mathfrak{E}^*_\Gamma$ 为：

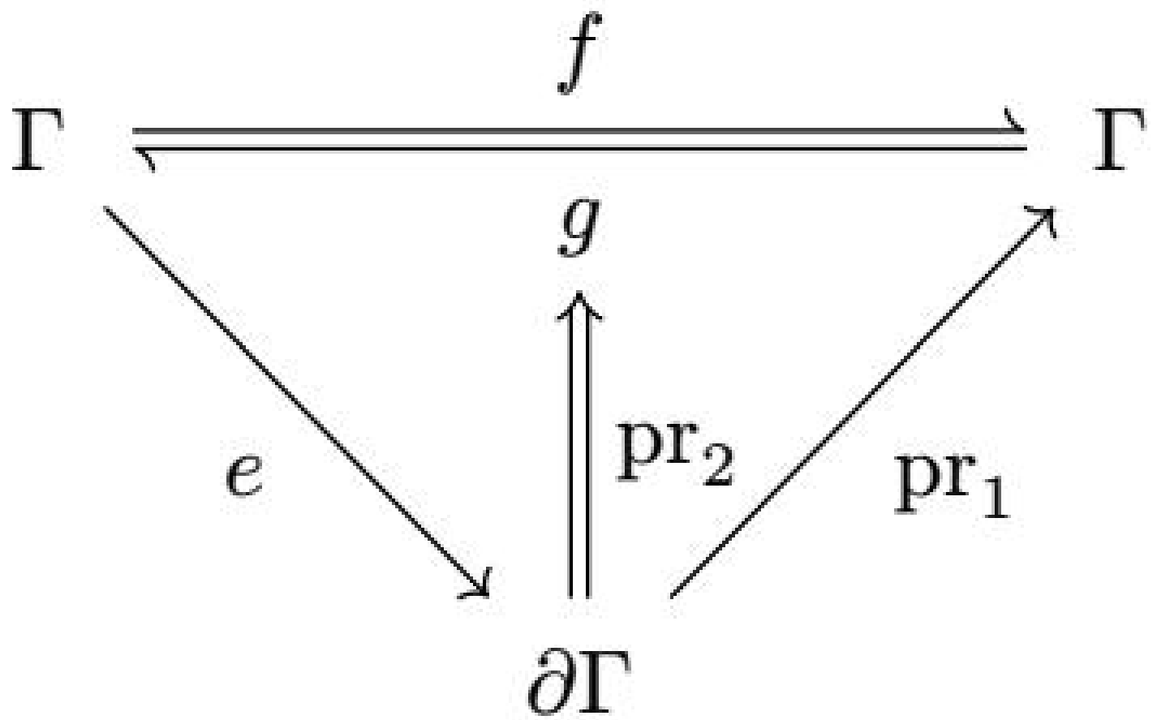
$$\begin{aligned}&\mathfrak{E}_\Gamma := \Gamma \to \Gamma \times (\Gamma \to \Gamma)\\ &\mathfrak{E}^*_\Gamma := (e : \Gamma \to \Gamma \times (\Gamma \to \Gamma))\\ &\qquad \times((\gamma : \Gamma) \to ((\delta : \Gamma) \times (g : \Gamma \to \Gamma) \times ((\delta, g) = e(\gamma) \to g(\delta) = \gamma)))\end{aligned} \tag{12}$$

其中 $e(\gamma)$ 产生二元组 (δ, g) 表示：

$\delta : \Gamma$ 是新的上下文；

$g : \Gamma \to \Gamma$ is the inverse function of the current efect.

An element of $\mathfrak{E}^*_\Gamma$ chooses its inverse per state, and the constraint $g(\delta) = \gamma$ holds that choice to reverting the effect where it was applied, leaving g unconstrained everywhere else. A single g with $g \circ f = \mathrm{id}_\Gamma$ meets the constraint at every state at once, and induces an element of $\mathfrak{E}^*_\Gamma$ by $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$, which Theorem 11 shows to be a homomorphism. The constraint can be visualized as the following commutative diagram, ensuring that the inverse e returns indeed reverses the transformation at the state where e was applied:



Since effect functions $\mathfrak{E}_\Gamma$ are no longer endomorphisms on the context, they cannot be directly composed. We therefore define a new operation for effect composition:

Definition 9. Given functions $f, g \in \mathfrak{E}_\Gamma$, define their effect composition $f \diamond g$ as:

$$\begin{aligned} f \diamond g \quad &: \quad \Gamma \to \partial\Gamma \\ &\quad\ \ \mathbf{let}(\delta, s) = g(\gamma)\mathbf{in} \\ f \diamond g \quad &= \quad \gamma \mapsto \mathbf{let}(\varepsilon, t) = f(\delta)\mathbf{in} \\ &\quad\ \ (\varepsilon, s \circ t) \end{aligned} \tag{13}$$

Theorem 10. Effect composition carries the monoid structure of $\mathfrak{T}_\Gamma$ over to $\mathfrak{E}_\Gamma$. That is,

1. $(\mathfrak{E}_\Gamma, \circ)$ is a monoid with unit $\eta_\Gamma := \gamma \mapsto (\gamma, \mathrm{id}_\Gamma)$;

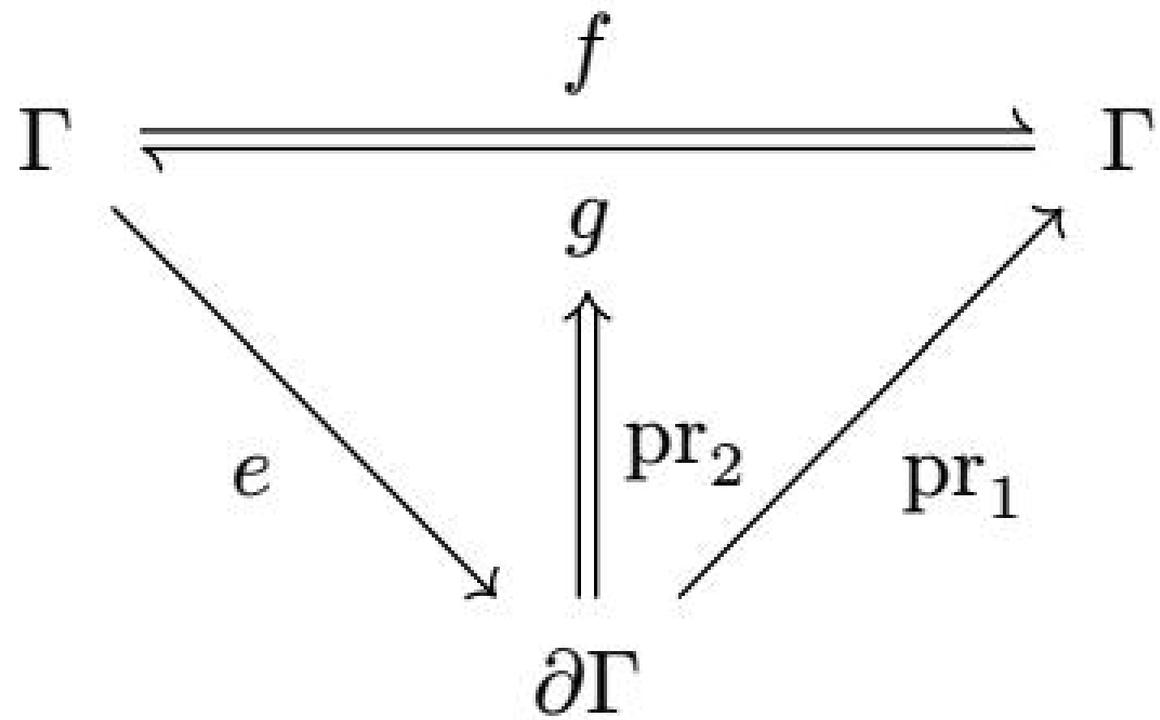
2. the assignment $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$ is a monoid homomorphism from $\mathfrak{T}_\Gamma$ into $\mathfrak{E}_\Gamma$

Proof.

1. Associativity and the unit laws follow componentwise from those of $\circ_\bullet$

$g : \Gamma \to \Gamma$ 是当前效应的逆函数。

$\mathfrak{E}^*_\Gamma$ 的元素按状态选择其逆，约束 $g(\delta) = \gamma$ 把该选择约束为在效应被应用之处逆转效应，而在别处对 g 不加约束。满足 $g \circ f = \mathrm{id}_\Gamma$ 的单一 g 在每个状态同时满足该约束，并通过 $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$ 诱导出 $\mathfrak{E}^*_\Gamma$ 的一个元素，定理 11 将表明这是一个同态。该约束可以可视化为如下交换图，确保逆 g 确实在 f 被应用的状态处逆转该变换：



由于效应函数 $\mathfrak{E}_\Gamma$ 不再是上下文上的自同态，它们不能直接复合。因此我们为效应复合定义一个新运算：

定义 9. 给定函数 $f, g \in \mathfrak{E}_\Gamma$ ，定义其效应复合 $f \diamond g$ 为：

$$\begin{aligned} f \diamond g \quad &: \quad \Gamma \to \partial\Gamma \\ &\quad\ \ \mathbf{let}(\delta, s) = g(\gamma)\mathbf{in} \\ f \diamond g \quad &= \quad \gamma \mapsto \mathbf{let}(\varepsilon, t) = f(\delta)\mathbf{in} \\ &\quad\ \ (\varepsilon, s \circ t) \end{aligned} \tag{13}$$

定理 10. 效应复合把 $\mathfrak{T}_\Gamma$ 的幺半群结构转移到 $\mathfrak{E}_\Gamma$ 。即，

1. $(\mathfrak{E}_\Gamma, \circ)$ 是以 $\eta_\Gamma := \gamma \mapsto (\gamma, \mathrm{id}_\Gamma)$; 为单位的幺半群；

2. 赋值 $(f, g) \mapsto \gamma \mapsto (f(\gamma), g)$ 是从 $\mathfrak{T}_\Gamma$ 到 $\mathfrak{E}_\Gamma$ 的幺半群同态

证明.

1. 结合性和单位律由 $\circ_\bullet$ 的那些逐分量得到。

2. Write $e_i = \gamma \mapsto (f_i(\gamma), g_i).$; then $(e_1 \diamond e_2)(\gamma) = (f_1(f_2(\gamma)), g_2 \circ g_1)$, which is the image of $(f_1, g_1) \circ (f_2, g_2)$, and $(\mathrm{id}_\Gamma, \mathrm{id}_\Gamma)$ maps to $\mathfrak{E}_\Gamma$. □

Theorem 11. Witnessing survives effect composition, and a uniform inverse witnesses at every state. That is,

1. $\mathfrak{E}_\Gamma^*$ is a submonoid of $\mathfrak{E}_\Gamma$: ;

2. the homomorphism of Theorem 10 carries every pair with $g \circ f = \mathrm{id}_\Gamma$ into $\mathfrak{E}_\Gamma^*$.

1. The unit lies in $\mathfrak{E}_\Gamma^*$ since $\mathrm{id}_\Gamma(\gamma) = \gamma.$. For closure, take $f, g \in \mathfrak{E}_\Gamma^*$ and any $\gamma \in \Gamma$, , and let $(\delta, s) = g(\gamma), (\varepsilon, t) = f(\delta)$, so that $(f \diamond g)(\gamma) = (\varepsilon, s \circ t)$. Then $s(\delta) = \gamma$ and $t(\varepsilon) = \delta$, therefore $(s \circ t)(\varepsilon) = s(\delta) = \gamma$

2. $g \circ f = \mathrm{id}_\Gamma$ gives $g(f(\gamma)) = \gamma$ at every γ, so the image of such a pair is witnessed at every state. □

Just as track lifts a pair of transformations on Γ to $\partial\Gamma$, we define effect to lift $\mathfrak{E}_\Gamma$ to $\mathfrak{E}_{\partial\Gamma}$:

Definition 12. Define the effect function transformation effect as:

$$\begin{array}{lcccccc} \mathrm{effect}_\Gamma & : & \mathfrak{E}_\Gamma & \to & \partial\Gamma & \to & \partial^2\Gamma \\ \mathrm{effect}_\Gamma & = & e & \mapsto & (\gamma, \varphi) & \mapsto & \mathbf{let}(\delta, g) = e(\gamma)\mathbf{in} \\ & & & & & & ((\delta, \varphi \circ g), \mathrm{track}_\Gamma(g, \mathrm{pr}_1 \circ e)) \end{array} \tag{14}$$

Since $\mathrm{effect}_\Gamma(e)$ is itself $\mathfrak{E}_{\partial\Gamma}$, , what it returns is an inverse in the sense of Definition 8 read one level up. That inverse is itself a track of the pair obtained by swapping the two directions of the efect. The ordinary tracking rule applies once more: undoing the effect is an effect in its own right, transforming the state by g, and the way to undo that is to perform the effect again, which is what $\mathrm{pr}_1 \circ e$ does. The inverse therefore composes onto the accumulator it is handed, exactly as track prescribes.

We can now prove properties for effect analogous to those of track.

Theorem 13. effect preserves the $\diamond$ operation. That is, $\forall f, g \in \mathfrak{E}_\Gamma$

$$\mathrm{effect}_\Gamma(f) \diamond \mathrm{effect}_\Gamma(g) = \mathrm{effect}_\Gamma(f \diamond g) \tag{15}$$

Proof. Take any $(\gamma, \varphi) \in \partial\Gamma$, and let $(\delta, s) = g(\gamma)$ and $(\varepsilon, t) = f(\delta)$, so that $(f \diamond g)(\gamma) = (\varepsilon, s \circ t)$ and $\mathrm{pr}_1 \circ (f \circ g) = (\mathrm{pr}_1 \circ f) \circ (\mathrm{pr}_1 \circ g)$. Then

$$\begin{aligned} (\mathrm{effect}_\Gamma(f) \diamond \mathrm{effect}_\Gamma(g))(\gamma, \varphi) &= ((\varepsilon, \varphi \circ s \circ t), \mathrm{track}_\Gamma(s, \mathrm{pr}_1 \circ g) \circ \mathrm{track}_\Gamma(t, \mathrm{pr}_1 \circ f)) \\ &= ((\varepsilon, \varphi \circ s \circ t), \mathrm{track}_\Gamma(s \circ t, (\mathrm{pr}_1 \circ f) \circ (\mathrm{pr}_1 \circ g))) \\ &= \mathrm{effect}_\Gamma(f \diamond g)(\gamma, \varphi) \end{aligned}$$

where the first step unfolds Definition 12 at (γ, φ) and at $(\delta, \varphi \circ s)$, the second is Theorem 5, and the third folds Definition 12. □

How the two levels relate is what the following diagram shows. Its upper triangle is the witness condition of e, according to Definition 8, and its lower triangle is the question of whether e' is witnessed the way e is.

2. 写 $e_i = \gamma \mapsto (f_i(\gamma), g_i).$ ；则 $(e_1 \diamond e_2)(\gamma) = (f_1(f_2(\gamma)), g_2 \circ g_1)$ ，它是 $(f_1, g_1) \circ (f_2, g_2)$ 的像，且 $(\mathrm{id}_\Gamma, \mathrm{id}_\Gamma)$ 映到 $\mathfrak{E}_\Gamma$。□

定理 11. 见证在效应复合下保持，且一致逆在每个状态均构成见证。即，

1. $\mathfrak{E}_\Gamma^*$ 是 $\mathfrak{E}_\Gamma$ ：的子幺半群；

2. 定理 10 的同态把每个满足 $g \circ f = \mathrm{id}_\Gamma$ 的对送入 $\mathfrak{E}_\Gamma^*$ 。

1. 单位元位于 $\mathfrak{E}_\Gamma^*$ 中，因为 $\mathrm{id}_\Gamma(\gamma) = \gamma.$ 。对封闭性，取 $f, g \in \mathfrak{E}_\Gamma^*$ 和任意 $\gamma \in \Gamma$, ，并令 $(\delta, s) = g(\gamma), (\varepsilon, t) = f(\delta)$ ，从而 $(f \diamond g)(\gamma) = (\varepsilon, s \circ t)$ 。则 $s(\delta) = \gamma$ 且 $t(\varepsilon) = \delta$, 因此 $(s \circ t)(\varepsilon) = s(\delta) = \gamma$

2. $g \circ f = \mathrm{id}_\Gamma$ gives $g(f(\gamma)) = \gamma$ 在每个 γ 处成立，因此这样的对的像在每个状态都被见证。□

正如 track 把 Γ 上的变换对提升到 $\partial\Gamma$, 我们定义 effect 把 $\mathfrak{E}_\Gamma$ 提升到 $\mathfrak{E}_{\partial\Gamma}$：

定义 12. 定义效应函数变换 effect 为：

$$\begin{array}{lcccccc} \mathrm{effect}_\Gamma & : & \mathfrak{E}_\Gamma & \to & \partial\Gamma & \to & \partial^2\Gamma \\ \mathrm{effect}_\Gamma & = & e & \mapsto & (\gamma, \varphi) & \mapsto & \mathbf{let}(\delta, g) = e(\gamma)\mathbf{in} \\ & & & & & & ((\delta, \varphi \circ g), \mathrm{track}_\Gamma(g, \mathrm{pr}_1 \circ e)) \end{array} \tag{14}$$

由于 $\mathrm{effect}_\Gamma(e)$ 本身是 $\mathfrak{E}_{\partial\Gamma}$, ，它返回的逆是在定义 8 的意义上高一层读出的逆。该逆本身是交换效应的两个方向所得之对的 track。普通跟踪规则再次适用：撤销效应本身就是一个效应，用 g 变换状态，而撤销它的方式是再次执行该效应，这正是 $\mathrm{pr}_1 \circ e$ 所做的。因此逆被复合到交给它的累加器上，正如 track 所规定的那样。

我们现在可以为 effect 证明与 track 类似的性质。

定理 13. effect 保持 $\diamond$ 运算。即，$\forall f, g \in \mathfrak{E}_\Gamma$

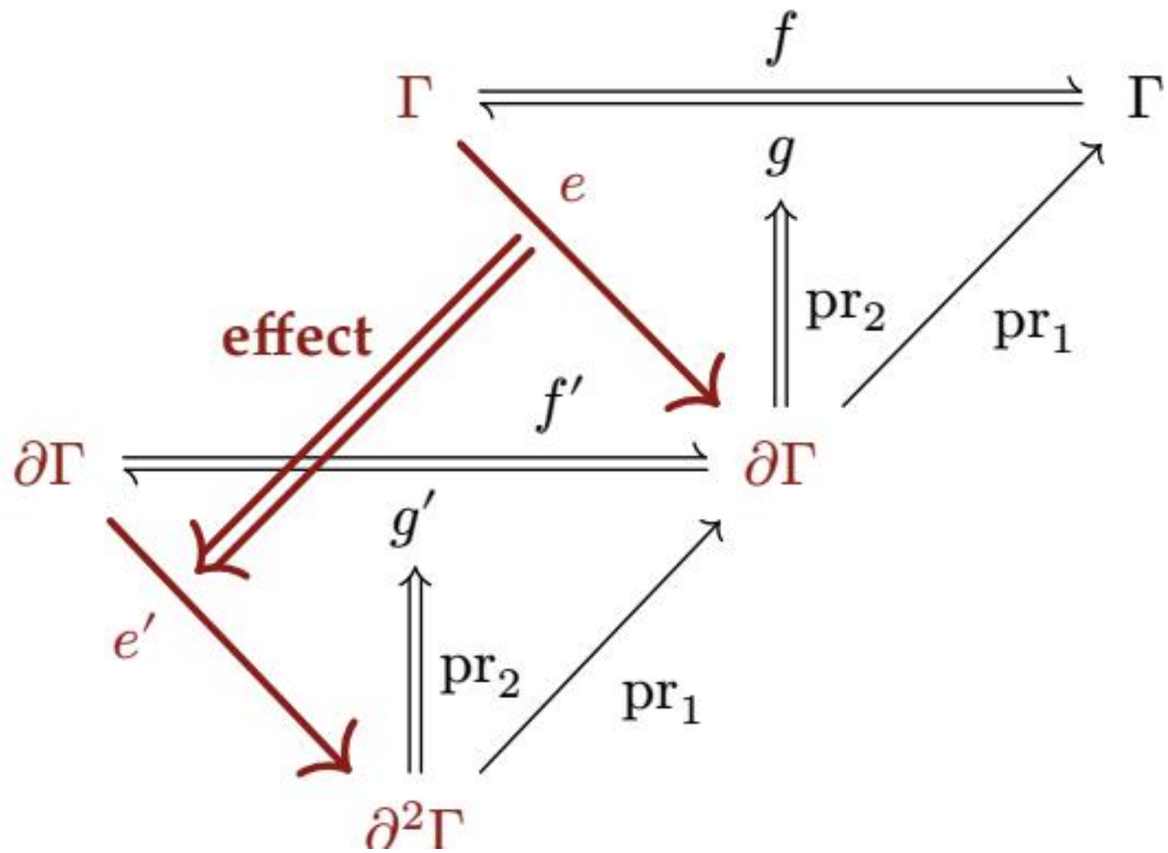
$$\mathrm{effect}_\Gamma(f) \diamond \mathrm{effect}_\Gamma(g) = \mathrm{effect}_\Gamma(f \diamond g) \tag{15}$$

证明. 取任意 $(\gamma, \varphi) \in \partial\Gamma$ ，并令 $(\delta, s) = g(\gamma)$ 和 $(\varepsilon, t) = f(\delta)$ ，从而 $(f \diamond g)(\gamma) = (\varepsilon, s \circ t)$ 且 $\mathrm{pr}_1 \circ (f \circ g) = (\mathrm{pr}_1 \circ f) \circ (\mathrm{pr}_1 \circ g)$ 。则

$$\begin{aligned} (\mathrm{effect}_\Gamma(f) \diamond \mathrm{effect}_\Gamma(g))(\gamma, \varphi) &= ((\varepsilon, \varphi \circ s \circ t), \mathrm{track}_\Gamma(s, \mathrm{pr}_1 \circ g) \circ \mathrm{track}_\Gamma(t, \mathrm{pr}_1 \circ f)) \\ &= ((\varepsilon, \varphi \circ s \circ t), \mathrm{track}_\Gamma(s \circ t, (\mathrm{pr}_1 \circ f) \circ (\mathrm{pr}_1 \circ g))) \\ &= \mathrm{effect}_\Gamma(f \diamond g)(\gamma, \varphi) \end{aligned}$$

其中第一步在 (γ, φ) 和 $(\delta, \varphi \circ s)$ 处展开定义 12，第二步是定理 5, 第三步折叠定义 12。□

两个层次如何关联由下图说明。其上三角形是 e, 的见证条件（按定义 8 ），其下三角形是 e' 是否以 e 的方式被见证的问题。



Between the levels, the projection pr_1 relates each lifted map to the map it lifts, as it does for track in Theorem 4.

Theorem 14. Let $e \in \mathfrak{E}_\Gamma$. , write $f := \mathrm{pr}_1 \circ e,$, and let $e' := \mathrm{effect}_\Gamma(e)$ with forward map $f' := \mathrm{pr}_1 \circ e'$. . Then

1. $\mathrm{pr}_1 \circ f' = f \circ \mathrm{pr}_1$;

2. for each $(\gamma, \varphi) \in \partial\Gamma$, the lifted inverse $g' := \mathrm{pr}_2(e'(\gamma, \varphi))$ and the inverse $g := \mathrm{pr}_2(e(\gamma))$ witnessed there satisfy $\mathrm{pr}_1 \circ g' = g \circ \mathrm{pr}_1$

Proof.

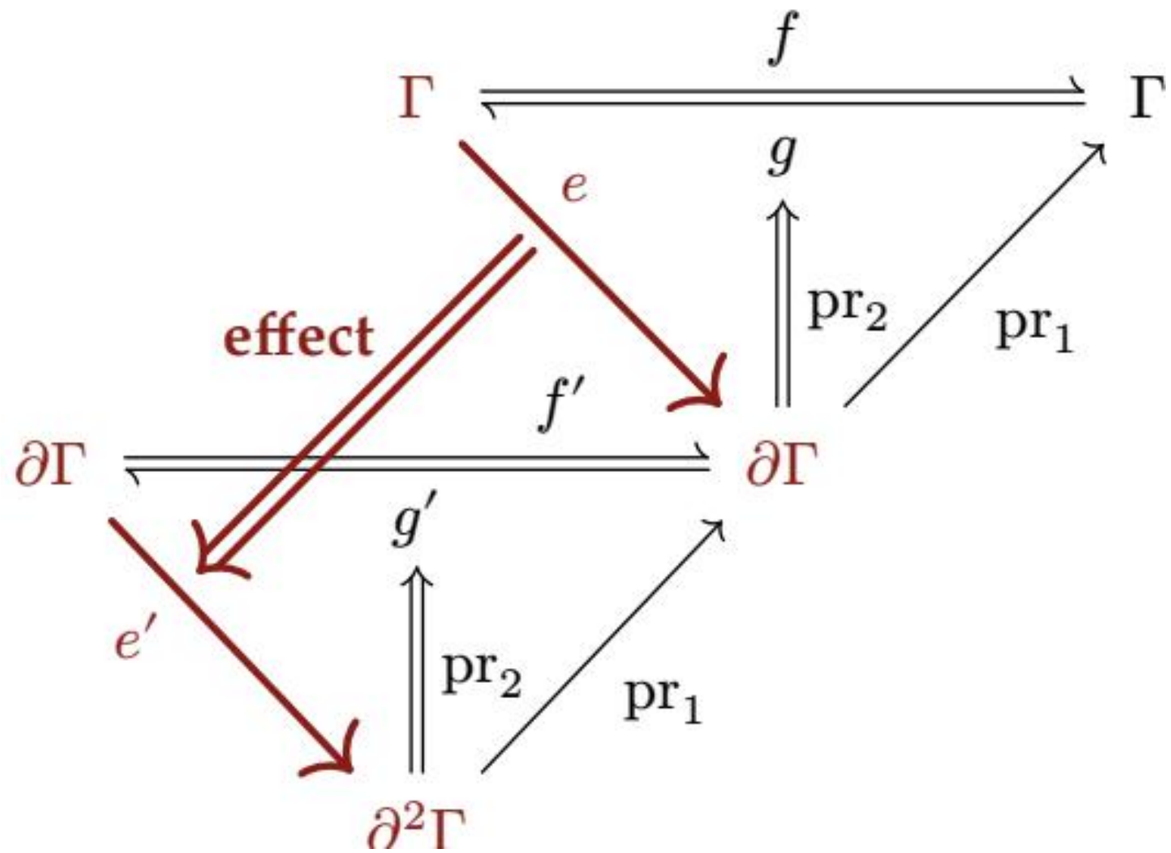
1. By Definition 12, $f'(\gamma, \varphi) = (f(\gamma), \varphi \circ g)$, whose state is $f(\gamma) = (f \circ \mathrm{pr}_1)(\gamma, \varphi)$

2. This is Theorem 4 applied to $g' = \mathrm{track}_\Gamma(g, f)$

Whether the lower triangle closes is settled by computing what the lifted inverse returns:

Theorem 15. Let $e \in \mathfrak{E}_\Gamma^*$ and write $f := \mathrm{pr}_1 \circ e$. Fix $(\gamma, \varphi) \in \partial\Gamma$, let $(\delta, g) = e(\gamma)$, and write (Δ, g') for the value of effe $\mathrm{ct}_\Gamma(e)$at (γ, φ) . Then

$$g'(\Delta) = (\gamma, \varphi \circ g \circ f) \tag{16}$$



在层次之间，投影 pr_1 把每个被提升的映射与它所提升的映射关联起来，正如它对 track 在定理 4 中所做的那样。

定理 14. 令 $e \in \mathfrak{E}_\Gamma$. ， 写 $f := \mathrm{pr}_1 \circ e,$ ， 并令 $e' := \mathrm{effect}_\Gamma(e)$ ， 其正向映射为 $f' := \mathrm{pr}_1 \circ e'$. 。则

1. $\mathrm{pr}_1 \circ f' = f \circ \mathrm{pr}_1$;

2. 对每个 $(\gamma, \varphi) \in \partial\Gamma$ ， 在此见证的提升逆 $g' := \mathrm{pr}_2(e'(\gamma, \varphi))$ 与逆 $g := \mathrm{pr}_2(e(\gamma))$ 满足 $\mathrm{pr}_1 \circ g' = g \circ \mathrm{pr}_1$

证明.

1. 由定义 12， $f'(\gamma, \varphi) = (f(\gamma), \varphi \circ g)$ ， 其状态为 $f(\gamma) = (f \circ \mathrm{pr}_1)(\gamma, \varphi)$

2. 这是把定理 4 应用于 $g' = \mathrm{track}_\Gamma(g, f)$

下三角形是否闭合由计算提升逆返回的内容来确定：

定理 15. 令 $e \in \mathfrak{E}_\Gamma^*$ 并写 $f := \mathrm{pr}_1 \circ e$ 。固定 $(\gamma, \varphi) \in \partial\Gamma$ ，令 $(\delta, g) = e(\gamma)$ ，并把 (Δ, g') 写作 effe $\mathrm{ct}_\Gamma(e)$at (γ, φ) 的值。则

$$g'(\Delta) = (\gamma, \varphi \circ g \circ f) \tag{16}$$

The state is recovered exactly. The accumulator is restored as well, equivalently $\mathrm{effect}_\Gamma(e) \in \mathfrak{E}^*_{\partial\Gamma}$, if and only if $g \circ f = \mathrm{id}_\Gamma$; ; and in every case $(\varphi \circ g \circ f)(\gamma) = \varphi(\gamma)$, so the soundness invariant is preserved.

Proof. By Definition 12, $\Delta = (\delta, \varphi \circ g)$ and $g' = \mathrm{track}_\Gamma(g, f)$, so

$$g'(\Delta) = (g(\delta), \varphi \circ g \circ f) = (\gamma, \varphi \circ g \circ f)$$

using $g(\delta) = \gamma$. Membership in $\mathfrak{E}^*_{\partial\Gamma}$ requires this to equal (γ, φ) at every input; taking $\varphi = \mathrm{id}_\Gamma$ turns the equality of accumulators into $g \circ f = \mathrm{id}_\Gamma$, , and that condition conversely gives the equality of accumulators for every φ. . Finally $(\varphi \circ g \circ f)(\gamma) = \varphi(g(\delta)) = \varphi(\gamma)$ □

The lower triangle therefore closes only when the inverse witnessed at γ reverts f at every state, so effect does not carry $\mathfrak{E}^*_\Gamma$ into $\mathfrak{E}^*_{\partial\Gamma}$. What holds in every case is agreement at γ: recover $\mathrm{i}_\Gamma(g'(\Delta)) = \mathrm{recover}_\Gamma(\gamma, \varphi)$, which is the whole of what Theorem 7 assumes of an accumulator, so reverting leaves the recovery target untouched.

Reverting efects in the reverse of the order in which they were applied requires nothing further, because each inverse then meets the state its own application produced:

Theorem 16. Let $e_1, \cdots, e_n \in \mathfrak{E}^*_\Gamma$ be applied in order from (γ_0, id) and reverted in the reverse order. Then

1. each revert recovers the context state its application ran against;

2. every intermediate state satisfies the soundness invariant.

Proof. Each step is an application or a revert. An application carries (γ, φ) to $(\delta, \varphi \circ g)$ with $g(\delta) = \gamma$, so it preserves $\varphi(\gamma)$ by Theorem [7,] whose hypothesis is exactly the witness of $\mathfrak{E}^*_\Gamma$ Reverting in the reverse order hands each inverse the state its own application produced, so by Theorem 15 that revert recovers the preceding state exactly and preserves $\varphi(\gamma)$ as well; neither conclusion depends on the accumulator the inverse receives. □

3.1.3. Independence of Effects

Reverting an effect at the state its own application produced is what Theorem 16 covers; reverting one at any other state is what this subsection covers. Two situations call for the latter. An inverse may be run while later efects are still in place, which is what withdrawing one component from a running system amounts to; and one sequence may interleave the efects of several components, each keeping the inverses of its own, so that the inverses of one component are separated by the applications of another. In both an inverse meets a state that foreign efects have moved, and whether it still reverts what it was built to revert is a question of commutation: what has to commute is every transformation one effect can perform with every transformation the other can perform, forward map and yielded inverse alike. A single accumulator settles neither situation, φ being a composite that runs every inverse it holds in one order and all at once.

Definition 17. For an effect function $e \in \mathfrak{E}_\Gamma$, the transformation monoid $\mathfrak{M}(e)$ is the submonoid of $\Gamma\Gamma$ generated by the forward map of e together with every inverse e yields, and the generators of $\mathfrak{M}(e)$ are the elements of that generating set:

$$\mathfrak{M}(e) := \langle \{\mathrm{pr}_1 \circ e\} \cup \{\mathrm{pr}_2(e(\gamma)) \mid \gamma \in \Gamma\} \rangle \tag{17}$$

An effect induced by a pair $(f, g) \in \mathfrak{T}_\Gamma$ has f and g for its generators, the inverse it yields being g at every state.

Lemma 18. Commutation is settled on the generators, and $\diamond$ enlarges no transformation monoid. That is,

状态被精确恢复。累加器也被恢复，等价地 $\mathrm{effect}_\Gamma(e) \in \mathfrak{E}^*_{\partial\Gamma}$ ，当且仅当 $g \circ f = \mathrm{id}_\Gamma$; ；且在每种情形下 $(\varphi \circ g \circ f)(\gamma) = \varphi(\gamma)$ ，因此可靠性不变量被保持。

证明. 由定义 12，$\Delta = (\delta, \varphi \circ g)$ 且 $g' = \mathrm{track}_\Gamma(g, f)$ ，因此

$$g'(\Delta) = (g(\delta), \varphi \circ g \circ f) = (\gamma, \varphi \circ g \circ f)$$

其中用到 $g(\delta) = \gamma$ 。属于 $\mathfrak{E}^*_{\partial\Gamma}$ 要求这在每个输入处等于 (γ, φ) ；取 $\varphi = \mathrm{id}_\Gamma$ 把累加器的相等性化为 $g \circ f = \mathrm{id}_\Gamma$, ，反之该条件对每个 φ. 都给出累加器的相等性。最后 $(\varphi \circ g \circ f)(\gamma) = \varphi(g(\delta)) = \varphi(\gamma)$ □

因此下三角形仅在 γ 处见证的逆在每个状态都逆转 f 时才闭合，所以 effect 并不把 $\mathfrak{E}^*_\Gamma$ 送入 $\mathfrak{E}^*_{\partial\Gamma}$ 。每种情形下成立的是在 γ: 处的吻合，即 recover $\mathrm{i}_\Gamma(g'(\Delta)) = \mathrm{recover}_\Gamma(\gamma, \varphi)$ ，这恰是定理 7 对累加器所假设的全部，因此逆转不会触动恢复目标。

以与应用相反的顺序逆转效应无需任何额外条件，因为每个逆此时都面对它自身应用所产生的状态：

定理 16. 令 $e_1, \cdots, e_n \in \mathfrak{E}^*_\Gamma$ 从 $(\gamma_0, \mathrm{id}_\Gamma)$ 按顺序应用并以相反顺序逆转。则

1. 每次逆转都恢复其应用所针对的上下文状态；

2. 每个中间状态满足可靠性不变量。

证明. 每一步是一次应用或一次逆转。一次应用把 (γ, φ) 带到 $(\delta, \varphi \circ g)$ 且 $g(\delta) = \gamma$ ，因此由定理 [7,] 它保持 $\varphi(\gamma)$ ，其假设恰是 $\mathfrak{E}^*_\Gamma$ 的见证。按相反顺序逆转使每个逆面对它自身应用所产生的状态，因此由定理 15 该逆转精确恢复前一个状态并同样保持 $\varphi(\gamma)$ ；这两个结论都不依赖逆所接收的累加器。□

3.1.3. 效应的独立性（Independence of Effects）

在效应自身应用所产生的状态处逆转效应是定理 16 所覆盖的；在任意其他状态处逆转效应是本小节所覆盖的。后一种情况有两种情形。逆可能在后来的效应仍然在位时运行，这相当于把一个组件从运行中的系统撤出；一个序列可能交错多个组件的效应，每个组件保留自己的逆，从而一个组件的逆被另一个组件的应用所分隔。在这两种情形下，逆面对的是外来效应已移动的状态，而它是否仍然逆转它被构造来逆转的东西是一个交换性问题：必须交换的是每个效应能执行的每个变换与另一个效应能执行的每个变换，正向映射与产生的逆都一样。单个累加器都无法解决这两种情形，φ 是以一种顺序、一次性运行它持有的每个逆的复合。

定义 17. 对效应函数 $e \in \mathfrak{E}_\Gamma$ ，变换幺半群 $\mathfrak{M}(e)$ 是由 f 的正向映射连同 e 产生的每个逆所生成的 $\Gamma\Gamma$ 的子幺半群，而 $\mathfrak{M}(e)$ 的生成元是该生成集的元素：

$$\mathfrak{M}(e) := \langle \{\mathrm{pr}_1 \circ e\} \cup \{\mathrm{pr}_2(e(\gamma)) \mid \gamma \in \Gamma\} \rangle \tag{17}$$

由对 $(f, g) \in \mathfrak{T}_\Gamma$ 诱导的效应以 f 和 g 为其生成元，它产生的逆在每个状态都是 g。

引理 18. 交换性在生成元上解决，且 $\diamond$ 不扩大任何变换幺半群。即，

1. if every generator of $\mathfrak{M}(e_1)$ commutes with every generator of $\mathfrak{M}(e_2)$, then every element of $\mathfrak{M}(e_1)$ commutes with every element of $\mathfrak{M}(e_2)$;

2. $\mathfrak{M}(e_1 \circ e_2) \subseteq \langle \mathfrak{M}(e_1) \cup \mathfrak{M}(e_2) \rangle$

Proof.

1. The maps commuting with every generator of $\mathfrak{M}(e_2)$ form a submonoid of $\Gamma\Gamma$, since id_Γ lies in it and $f \circ f'$ does where f and f' do. That submonoid contains the generators of $\mathfrak{M}(e_1)$ by hypothesis and hence contains $\mathfrak{M}(e_1)$. Fixing $f \in \Re(e_1)$, the maps commuting with f likewise form a submonoid containing the generators of $\mathfrak{M}(e_2)$ and hence $\mathfrak{M}(e_2)$

2. By Definition 9 the forward map of $e_1 \diamond e_2 \mathbf{is} \, (\mathrm{pr}_1 \circ e_1) \circ (\mathrm{pr}_1 \circ e_2)$ and the inverse it yields at any state is $s \circ t$ for an s yielded by e_2 and a t yielded by e_1 . Every generator of $\mathfrak{M}(e_1 \circ e_2)$ is therefore a composite of generators of the two. □

Definition 19. Effect functions $e_1, e_2 \in \mathfrak{E}_\Gamma$ are independent when

1. every transformation of one commutes with every transformation of the other,

$$\forall f \in \mathfrak{M}(e_1), g \in \mathfrak{M}(e_2). \quad f \circ g = g \circ f \tag{18}$$

2. neither one's transformations disturb the inverse the other yields,

$$\forall g \in \mathfrak{M}(e_2), \gamma \in \Gamma. \quad \mathrm{pr}_2(e_1(g(\gamma))) = \mathrm{pr}_2(e_1(\gamma)) \tag{19}$$

and the same with e_1 and e_2 exchanged.

A family $(\boldsymbol{e}_l)_{l \in L}$ is pairwise independent when e_l and $e_{l'}$ are independent for every $l \neq l'$. A family may repeat an effect function, and holding one independent of itself is holding M(e) commutative.

For efects induced by pairs (f_1, g_1) and (f_2, g_2) , clause (1) is by Lemma 18(1) the commutation of the four pairs $f_1, f_2; g_1, g_2; f_1, g_2$; and $g_1, f_2,$, and clause (2) holds outright, an induced effect yielding one inverse at every state. Commutation under $\diamond$ is a diferent property. What $e_1 \diamond e_2 = e_2 \diamond e_1$ equates is the composite forward map of the two orders with each other and the composite inverse of the two orders with each other, each inverse entering the composite at the state its own application produced; independence instead relates each transformation of one effect to each transformation of the other, a forward map paired with a foreign inverse included.

Under independence an inverse may be run at a state later efects have moved, and what it withdraws there is its own contribution and nothing else:

Theorem 20. Let $e_1, \cdots, e_n \in \mathfrak{E}_\Gamma^*$ be pairwise independent and applied in order from γ_0 . Write $f_i := \mathbf{pr}_1 \circ e_i,$, let $\delta_i := f_i(\delta_{i-1})$ with $\delta_0 := \gamma_0.$, and let $g_i := \mathrm{pr}_2(e_i(\delta_{i-1}))$ be the inverse e_i yields where it is applied. Fix j and write $\delta'_i := \big(f_i \circ \cdots \circ f_{j+1}\big)\big(\delta_{j-1}\big)$ for the states of the sequence with e_j omitted, so that $\delta'_j = \delta_{j-1}$. Then for every u with $j \leq u \leq n$

1. $\delta_u = f_j(\delta'_u)$ and $g_j(\delta_u) = \delta'_u$,

1. 若 $\mathfrak{M}(e_1)$ 的每个生成元与 $\mathfrak{M}(e_2)$ 的每个生成元交换，则 $\mathfrak{M}(e_1)$ 的每个元素与 $\mathfrak{M}(e_2)$ 的每个元素交换；

2. $\mathfrak{M}(e_1 \circ e_2) \subseteq \langle \mathfrak{M}(e_1) \cup \mathfrak{M}(e_2) \rangle$

证明.

1. 与 $\mathfrak{M}(e_2)$ 的每个生成元交换的映射构成 ΓΓ, 的子幺半群，因为 id_Γ 位于其中，且 f 和 f' 交换处 $f \circ f'$ 也交换。该子幺半群按假设包含 $\mathfrak{M}(e_1)$ 的生成元，因此包含 $\mathfrak{M}(e_1)$ 。固定 $f \in \Re(e_1)$ ，与 f 交换的映射同样构成包含 $\mathfrak{M}(e_2)$ 的生成元、因而包含 $\mathfrak{M}(e_2)$ 的子幺半群

2. 由定义 9，$e_1 \diamond e_2 \mathbf{is} \, (\mathrm{pr}_1 \circ e_1) \circ (\mathrm{pr}_1 \circ e_2)$ 的正向映射是复合，而它在任意状态产生的逆是 $s \circ t$ ，其中 s 由 e_2 产生、t 由 e_1 产生。因此 $\mathfrak{M}(e_1 \circ e_2)$ 的每个生成元都是两者生成元的复合。□

定义 19. 效应函数 $e_1, e_2 \in \mathfrak{E}_\Gamma$ 是独立的，当

1. 一个的每个变换与另一个的每个变换交换，

$$\forall f \in \mathfrak{M}(e_1), g \in \mathfrak{M}(e_2). \quad f \circ g = g \circ f \tag{18}$$

2. 一方的变换不扰动另一方产生的逆，

$$\forall g \in \mathfrak{M}(e_2), \gamma \in \Gamma. \quad \mathrm{pr}_2(e_1(g(\gamma))) = \mathrm{pr}_2(e_1(\gamma)) \tag{19}$$

且交换 e_1 和 e_2 后同样成立。

族 $(\boldsymbol{e}_l)_{l \in L}$ 是两两独立的，当对每个 $l \neq l'$ ，e_l 和 $e_{l'}$ 独立。族可以重复同一个效应函数，而把某效应保持为与自身独立即保持 M(e) 交换。

对由对 (f_1, g_1) 和 (f_2, g_2) 诱导的效应，条款 (1) 由引理 18(1) 即四对的交换性 $f_1, f_2; g_1, g_2; f_1, g_2$; 与 $g_1, f_2,$ ，而条款 (2) 无条件成立，诱导效应在每个状态产生一个逆。$\diamond$ 下的交换是不同性质。$e_1 \diamond e_2 = e_2 \diamond e_1$ 所等同的是两个顺序的复合正向映射彼此、两个顺序的复合逆彼此，每个逆在其自身应用所产生的状态处进入复合；独立性相反地把一个效应的每个变换与另一个效应的每个变换关联起来，包括正向映射与外来逆配对。

在独立性下，逆可以在后来效应已移动的状态处运行，而它在那里撤出的只是它自己的贡献，别无其他：

定理 20. 令 $e_1, \cdots, e_n \in \mathfrak{E}_\Gamma^*$ 两两独立并从 γ_0 按顺序应用。写 $f_i := \mathbf{pr}_1 \circ e_i,$ ，令 $\delta_i := f_i(\delta_{i-1})$ 且 $\delta_0 := \gamma_0.$ ，并令 $g_i := \mathrm{pr}_2(e_i(\delta_{i-1}))$ 为 e_i 在被应用处产生的逆。固定 j 并写 $\delta'_i := \big(f_i \circ \cdots \circ f_{j+1}\big)\big(\delta_{j-1}\big)$ 为省略 e_j 后序列的状态，从而 $\delta'_j = \delta_{j-1}$ 。则对每个满足 $j \leq u \leq n$ 的 u

1. $\delta_u = f_j(\delta'_u)$ 且 $g_j(\delta_u) = \delta'_u$,

2. each e_i with $i > j$ yields at δ'_{i-1} the same inverse g_i it yields at δ_{i-1} .

Proof.

1. The first equation is an induction on u. At $u = j$ it reads $\delta_j = f_j(\delta_{j-1})$, which is the definition of δ_j . For the inductive step, $\delta_{u+1} = f_{u+1}(\delta_u) = f_{u+1}(f_j(\delta'_u)) = f_j(f_{u+1}(\delta'_u)) = f_j\left(\delta'_{u+1}\right)$, the middle equality being clause (1) of Definition 19 for e_{u+1} and e_j, which are distinct efects of the family since $u + 1 > j$. For the second equation, clause (1) carries g_j out through the forward maps applied after e_j, , leaving the witness of e_j to be used at the one state it holds at:

$$g_j\left(\delta_u\right) = \left(g_j \circ f_u \circ \cdots \circ f_{j+1}\right)\left(\delta_j\right) = \left(f_u \circ \cdots \circ f_{j+1}\right)\left(g_j\left(f_j\left(\delta_{j-1}\right)\right)\right) = \delta'_u$$

the last equality resting on $g_j\big(f_j\big(\delta_{j-1}\big)\big) = \delta_{j-1}$, , which is the witness Definition 8 requires of e_j at δ_{j-1}

2. By (1) the state δ_{i-1} is $f_j(\delta'_{i-1})$, and $f_j \in \mathfrak{M}(e_j)$, so clause (2) of Definition 19 for e_i and $e_j$gives$\mathrm{pr}_2\big(e_i\big(f_j(\delta'_{i-1})\big)\big) = \mathrm{pr}_2\big(e_i(\delta'_{i-1})\big)$ □

Clause (1) locates the state an inverse reaches: it is the state the same sequence would have reached had the effect never been applied, whatever efects were applied after it. Clause (2) locates the inverses the others hold there, and together the two let the theorem be applied again to the shorter sequence:

Corollary 21. Let $e_1, \cdots, e_n \in \mathfrak{E}^*_\Gamma$ be pairwise independent and applied in order from γ_0, , and let $g_1, \cdots, g_n$ be as above. Applying the n inverses at δ_n in the order of any permutation of $\{1, \cdots, n\}$ reaches γ_0

Proof. By downward induction on n. Let the permutation begin with j. By Theorem 20(1) applying g_j at δ_n reaches δ'_n. , the state the sequence with e_j omitted reaches, and by Theorem 20(2) the inverses the remaining efects yielded there are the g_i in hand. That sequence is pairwise independent, being a subfamily, so the induction hypothesis applies to it and to the rest of the permutation; the empty sequence reaches γ_0 □

LIFO order is one such permutation, and Theorem 16 reverts in it with no hypothesis at all. What independence buys is every other order, and with it the sequence that interleaves several components, which Section 4.4.2 carries to a trace of a whole system.

Together, these constructions constitute revertible efects: each effect function in $\mathfrak{E}^*_\Gamma$ explicitly provides its own inverse, effect tracks these inverses on the effect context $\partial\Gamma$, , and the $\diamond$ operation composes them while preserving revertibility. What they deliver is local temporal composability, local in that the guarantee is read of one component's efects taken by themselves. We take that to be the following criterion: for every sequence of effect functions a component applies, the accumulator recovers the context it began at (Theorem 7), and reverting the sequence hands each inverse the state its own application ran against (Theorem 16). Loading a component is applying such a sequence and accumulating its inverses in φ; unloading it is applying φ.

Two things the criterion leaves out, and both arrive once several components are in play: reverting out of the order the accumulator imposes, and a sequence that interleaves the efects of others. Independence delivers them (Corollary 21), and it is a condition on the efects rather than a property of the construction, Section 3.3.2 being where the discipline that meets it is identified and Section 4.4.2 where the guarantee is read of a whole system's trace. Where independence fails, the order has to be carried elsewhere: within one component by the accumulator, which reverts in LIFO order whatever the efects (Section 4.3.2), and across components by a declared coeffect, which orders one activation against another (Section 4.3.1).

2. 每个满足 $i > j$ 的 e_i 在 δ'_{i-1} 处产生的逆 g_i 与它在 δ_{i-1} 处产生的一致。

证明.

1. 第一个等式是对 u 的归纳。在 $u = j$ 处它读作 $\delta_j = f_j(\delta_{j-1})$ ，即 δ_j 的定义。对归纳步，$\delta_{u+1} = f_{u+1}(\delta_u) = f_{u+1}(f_j(\delta'_u)) = f_j(f_{u+1}(\delta'_u)) = f_j\left(\delta'_{u+1}\right)$ ，中间等式是定义 19 的条款 (1) 应用于 e_{u+1} 和 e_j, 它们是族中不同效应，因为 $u + 1 > j$ 。对第二个等式，条款 (1) 把 g_j 穿过 e_j 之后应用的正向映射带出，把 e_j 的见证留到它成立的那个状态使用：

$$g_j\left(\delta_u\right) = \left(g_j \circ f_u \circ \cdots \circ f_{j+1}\right)\left(\delta_j\right) = \left(f_u \circ \cdots \circ f_{j+1}\right)\left(g_j\left(f_j\left(\delta_{j-1}\right)\right)\right) = \delta'_u$$

最后一个等式依赖 $g_j\big(f_j\big(\delta_{j-1}\big)\big) = \delta_{j-1}$, ，即定义 8 在 δ_{j-1} 处对 e_j 要求的见证

2. 由 (1)，状态 δ_{i-1} 是 $f_j(\delta'_{i-1})$ ，且 $f_j \in \mathfrak{M}(e_j)$ ，因此定义 19 对 e_i 和 $e_j$gives$\mathrm{pr}_2\big(e_i\big(f_j(\delta'_{i-1})\big)\big) = \mathrm{pr}_2\big(e_i(\delta'_{i-1})\big)$ □

条款 (1) 定位逆所达到的状态：它就是同一序列在效应从未被应用的情况下会达到的状态，无论之后应用了什么效应。条款 (2) 定位其他逆在那里持有的东西，两者合在一起使该定理可以再次应用于更短的序列：

推论 21. 令 $e_1, \cdots, e_n \in \mathfrak{E}^*_\Gamma$ 两两独立并从 γ_0, 按顺序应用，并令 $g_1, \cdots, g_n$ 如上。按 $\{1, \cdots, n\}$ 的任意置换的顺序在 δ_n 处应用这些逆可达到 γ_0

证明. 对 j 向下归纳。令置换以 j 开始。由定理 20(1) 在 δ_n 处应用 g_j 达到 δ'_n. ，即省略 e_j 的序列所达到的状态，并由定理 20(2)，剩余效应在此产生的逆就是手中的 g_i 。该序列两两独立，因为它是子族，所以归纳假设适用于它和置换的其余部分；空序列达到 γ_0 □

LIFO 顺序就是这样一个置换，而定理 16 在无任何假设的情况下按此顺序逆转。独立性带来的是所有其他顺序，以及随之而来的、交错多个组件的序列，第 4.4.2 节将其推广到整个系统的迹。

这些构造合在一起构成可逆效应：$\mathfrak{E}^*_\Gamma$ 中的每个效应函数显式提供自己的逆，effect 在效应上下文 $\partial\Gamma$, 上跟踪这些逆，$\diamond$ 运算在保持可逆性的同时组合它们。它们交付的是局部时间可组合性，说它局部是因为该保证是针对一个组件的效应本身来读的。我们把这个保证表述为如下准则：对组件应用的一列效应函数中的每个序列，累加器恢复它开始的上下文（定理 7），且逆转该序列使每个逆面对它自身应用所针对的状态（定理 16）。加载组件即应用这样的序列并把其逆累积在 φ; 中；卸载组件即应用它。

准则遗漏了两件事，而两者都在多个组件在场时出现：按累加器所强加顺序之外的顺序逆转，以及交错其他效应的一个序列。独立性交付它们（推论 21），且它是关于效应的条件而非构造的性质，第 3.3.2 节识别满足它的纪律，第 4.4.2 节对整个系统迹读出该保证。独立性失败之处，顺序必须由他处承载：在一个组件内由累加器承载，它无论效应如何都按 LIFO 顺序逆转（第 4.3.2 节），在组件间由声明的余效应承载，它把一个激活排到另一个激活之前（第 4.3.1 节）。

3.2. Reactive Coeffects

Spatial composability is the ability for components to declare dependencies on one another and for the system to resolve, provide, and withdraw those dependencies at runtime. This requires that dependency satisfaction be re-evaluated whenever the shared context changes, so that a component activates when its dependencies become available and deactivates when they are withdrawn. We therefore model dependencies of a component as a specification and classify each change to the context, against that specification, as activating, deactivating, or neutral. Classifying against the specification is what detects a change in satisfaction; responding to that classification is what drives activation and deactivation. We call such coefects reactive: by classifying context changes and driving activation and deactivation from them, correct coeffect ordering becomes a structural guarantee.

3.2.1. Coeffect Context

Traditional inversion-of-control (IoC) containers [38] typically model dependencies as simple key-value mappings. This section formalizes IoC as a coeffect context that synergizes with revertible efects to provide a mathematical foundation for dynamic composition.

Definition 22. Given a type family $\nu : K \to \text{Type}$, define the coeffect context as the dependent partial function type:

$$\Sigma := (k : K) \rightharpoonup \mathcal{V}_k \tag{20}$$

where $\sigma : \Sigma$ is a finite partial function assigning to each $k \in \text{dom}(\sigma) \subseteq K$ a value of type ν_k . We write:
$\sigma(k)$ for application (defined when $k \in \text{dom}(\sigma)$)
$\sigma[k \mapsto v]$ for the table binding v at k and agreeing with σ elsewhere;
$\sigma\setminus$ k for restriction (defined when $k \in \text{dom}(\sigma)$).
$k \in \text{dom}(\sigma)$ for membership.

The use of a type family $\mathcal{V}$ ensures that each dependency key k is associated with a specific value type ν_k, , providing static type safety for dependency access. Extension and restriction carry preconditions, imposed by the operations below: a dependency cannot be provided twice (k $\notin$ dom(σ) for extension) nor revoked if absent ($k \in \text{dom}(\sigma)$ for restriction). A violated precondition is signalled as an error and produces no transition, so the effect algebra, which describes the transitions that do occur, applies to these operations unchanged. A reader preferring to internalize the failure may read every $\Sigma\Sigma$ below as $\Sigma\mathsf{Maybe}(\Sigma)$ and compose in the Maybe monad (Section 2.1), at the cost of replacing each identity by the partial identity on the operation's domain. Based on this context structure, we define two core operations:

Definition 23. The get and set operations on Σ are defined as:

$$\begin{array}{llllll} \text{get} & : & (k:K) & \to \Sigma \to & \mathcal{V}_k \\ \text{get} & = & k & \mapsto \sigma \mapsto & \sigma(k) \\ \text{set} & : & (k:K)\times\mathcal{V}_k & \to \Sigma \to & \Sigma \times (\Sigma \rightharpoonup \Sigma) \\ \text{set} & = & (k,v) & \mapsto \sigma \mapsto & (\sigma[k\mapsto v], \lambda\sigma'.\sigma' \setminus k) \end{array} \tag{21}$$

where get(k) requires $k \in \text{dom}(\sigma)$ and set(k, v) requires k $\notin$ dom(σ) as preconditions.

Notably, set (k, v) has type $\mathfrak{E}^*_\Sigma$, precisely an effect function on the coeffect context. We can therefore directly apply the effect machinery from Section 3.1: effect provides automatic tracking and recovery of dependency registrations. This is the synergy between reactive coeffects and revertible efects: coeffect operations are efects,

3.2. 响应式余效应（Reactive Coeffects）

空间可组合性是指组件能够声明彼此之间的依赖，系统能够在运行时解析、提供和撤出这些依赖的能力。这要求每当共享上下文改变时重新评估依赖满足情况，从而使组件在其依赖可用时激活、在其依赖被撤出时停用。因此我们把组件的依赖建模为一个规范，并把上下文的每次变化依据该规范分类为激活、停用或中性。依据规范分类正是检测满足情况变化的方式；响应这种分类驱动激活与停用。我们称这样的余效应是响应式的：通过分类上下文变化并据此驱动激活和停用，正确的余效应排序成为结构性保证。

3.2.1. 余效应上下文（Coeffect Context）

传统的控制反转（inversion-of-control, IoC）容器 [38] 通常把依赖建模为简单的键值映射。本节把 IoC 形式化为与可逆效应协同的余效应上下文，为动态组合提供数学基础。

定义 22. 给定类型族 $\nu : K \to \text{Type}$，定义余效应上下文为依赖的部分函数类型：

$$\Sigma := (k : K) \rightharpoonup \mathcal{V}_k \tag{20}$$

其中 $\sigma : \Sigma$ 是有限部分函数，为每个 $k \in \text{dom}(\sigma) \subseteq K$ 赋予 ν_k 类型的值。我们写：
$\sigma(k)$ 表示应用（当 $k \in \text{dom}(\sigma)$) 时有定义）
$\sigma[k \mapsto v]$ 表示在 v 处绑定 k、在别处与 σ 一致的表格；
$\sigma\setminus$ k 表示限制（当 $k \in \text{dom}(\sigma)$). 时有定义）
$k \in \text{dom}(\sigma)$ 表示成员资格。

类型族 K 的使用确保每个依赖键 k 与特定的值类型 ν_k, 相关联，为依赖访问提供静态类型安全。扩展和限制带有由下面的操作施加的前置条件：依赖不能被提供两次（扩展要求 k $\notin$ dom(σ)），也不能在缺席时被撤销（限制要求 ($k \in \text{dom}(\sigma)$ ）。违反前置条件被报告为错误且不产生转移，因此描述实际发生转移的效应代数对这些操作原封不动地适用。偏好将失败内化的读者可以把下面的每个 $\Sigma\Sigma$ 读作 $\Sigma\mathsf{Maybe}(\Sigma)$ 并在 Maybe 单子（第 2.1 节）中复合，代价是用操作定义域上的部分恒等替换每个恒等。基于此上下文结构，我们定义两个核心操作：

定义 23. 在 Σ 上定义 get 和 set 操作：

$$\begin{array}{llllll} \text{get} & : & (k:K) & \to \Sigma \to & \mathcal{V}_k \\ \text{get} & = & k & \mapsto \sigma \mapsto & \sigma(k) \\ \text{set} & : & (k:K)\times\mathcal{V}_k & \to \Sigma \to & \Sigma \times (\Sigma \rightharpoonup \Sigma) \\ \text{set} & = & (k,v) & \mapsto \sigma \mapsto & (\sigma[k\mapsto v], \lambda\sigma'.\sigma' \setminus k) \end{array} \tag{21}$$

其中 get(k) 要求 $k \in \text{dom}(\sigma)$ 且 set(k, v) 要求 k $\notin$ dom(σ) 作为前置条件。

值得注意的是，set (k, v) 具有类型 $\mathfrak{E}^*_\Sigma$, 恰是余效应上下文上的效应函数。因此我们可以直接应用第 3.1 节的效应机制：effect 提供依赖注册的自动跟踪与恢复。这就是响应式余效应与可逆效应之间的协同：余效应操作是效应，而效应是可逆的。

and efects are revertible.

What get hands a component is a value, and what the component can do with that value is whatever the coeffect at that key provides. A key therefore carries more than a value type:

Definition 24. A coeffect at a key k is a triple $\left(\mathcal{V}_{k,\widetilde{k}}, \mathcal{A}_k\right)$, where ν_k is the value type of Definition 22, $\widetilde{\widetilde{k}}$ is an equivalence relation on ν_k up to which values at k are compared (Section 3.3.2), and $\mathcal{A}_k$ is a set of coeffect operations, the operations the value bound at k provides to a component holding it. An operation $a \in \mathcal{A}_k$ carries an argument type X_a and an outcome type B_a, , and acts on the value alone:

$$a : X_a \to \mathcal{V}_k \rightharpoonup \mathcal{V}_k \times (\mathcal{V}_k \rightharpoonup \mathcal{V}_k) \times B_a \tag{22}$$

its first two constituents forming an effect function on ν_k witnessed as Definition 8 requires, and its third an outcome. Each operation is required to respect $\simeq$: at $\simeq$-related values it is defined at both or at neither, and where defined it yields $\widetilde{\widetilde{k}}$ -related successors, inverses that again carry $\widetilde{\widetilde{k}}$ -related values to $\simeq$-related values, and equal outcomes. An operation acts on the coeffect context through its $lift$

$$a^{\Sigma}(x)(\sigma) := \text{let}(v, g, b) = a(x)(\sigma(k)) \text{ in } (\sigma[k \mapsto v], \lambda\sigma'.\sigma'[k \mapsto g(\sigma'(k))], b) \tag{23}$$

defined when $k \in \text{dom}(\sigma)$, whose first two constituents are an effect function on Σ.

Typing an operation of k on ν_k is what confines it to the binding at k: the lift reads and writes that binding and leaves every other key as it stands, so no side condition is needed to say so. Where isolation is in force the binding it reaches is the one the realm resolves to (Definition 28), two keys sharing a realm sharing one binding. An operation whose behaviour turns on another key reads that key's value into its argument X_a, and the reactive discipline of the next subsection is what holds the value fixed for as long as the component that read it runs (Theorem 63).

3.2.2. Specification and Notification

The preceding definitions describe how individual dependencies are registered and accessed. Accessing an absent dependency, however, is a runtime failure. A component should therefore activate only once all the dependencies it declares are present, rather than accessing them optimistically and failing when one is missing. This raises two questions: whether a component's declared dependencies are jointly satisfied, and how the system should respond when that status changes. The coeffect context Σ carries a natural observational structure that makes both questions tractable: for any coeffect specification $d \subseteq K.$, define the satisfaction predicate:

$$\sigma \vDash d := \forall k \in d.k \in \text{dom}(\sigma) \tag{24}$$

This predicate is decidable (since $\text{dom}(\sigma)$ is finite). Since all mutations to σ pass through effect functions (whose inverses recover the previous domain), changes to satisfaction are detectable at each effect boundary. This is the algebraic basis of reactivity: the effect system guarantees that every coeffect change is observed.

Definition 25. A coeffect specification is:

$$\mathfrak{D}_{\Sigma} := \text{Set}(K) \tag{25}$$

representing the set of dependencies a component declares from the environment.

What makes this specification reactive is how it classifies state transitions. Any effect that transforms σ to σ' can be classified by a specification $d \in \mathfrak{D}_{\Sigma}$ according to whether d's satisfaction status is altered:

get 交给组件的是一个值，而组件能对这个值做什么取决于该键处的余效应提供什么。因此一个键承载的不仅仅是值类型：

定义 24. 键 k 处的余效应是三元组 $\left(\mathcal{V}_{k,\widetilde{k}}, \mathcal{A}_k\right)$ ，其中 ν_k 是定义 22 的值类型，$\widetilde{\widetilde{k}}$ 是 ν_k 上的等价关系，k 处的值在其下进行比较（第 3.3.2 节），而 $\mathcal{A}_k$ 是余效应操作集，即绑定在 k 处的值向持有它的组件提供的操作。操作 $a \in \mathcal{A}_k$ 携带参数类型 X_a 和结果类型 B_a, 并且只作用于该值：

$$a : X_a \to \mathcal{V}_k \rightharpoonup \mathcal{V}_k \times (\mathcal{V}_k \rightharpoonup \mathcal{V}_k) \times B_a \tag{22}$$

其前两个构成部分形成 ν_k 上的效应函数，按定义 8 的要求被见证，第三个是结果。每个操作都被要求尊重 $\simeq$：在 $\simeq$-相关值处，它在两者处都有定义或都没有定义，并且有定义处它产生 $\widetilde{\widetilde{k}}$ -相关的后继、再次把 $\widetilde{\widetilde{k}}$ -相关值带到 $\simeq$-相关值的逆，以及相等的结果。操作通过其 $lift$ 作用于余效应上下文

$$a^{\Sigma}(x)(\sigma) := \text{let}(v, g, b) = a(x)(\sigma(k)) \text{ in } (\sigma[k \mapsto v], \lambda\sigma'.\sigma'[k \mapsto g(\sigma'(k))], b) \tag{23}$$

当 $k \in \text{dom}(\sigma)$ 时有定义，其前两个构成部分是 Σ 上的效应函数。

把 k 的操作类型化在 ν_k 上正是把它限制在 k 处的绑定上：lift 读写该绑定并保持每个其他键原样，因此无需旁条件即可如此说明。隔离生效处，它到达的绑定是领域解析到的那个（定义 28），共享一个领域的两个键共享一个绑定。行为转而依赖另一键的操作把该键的值读入其参数 X_a, 而下一小节的响应式纪律在读取它的组件运行的整个期间保持该值固定（定理 63）。

3.2.2. 规范与通知（Specification and Notification）

前面的定义描述了单个依赖如何被注册和访问。然而，访问缺席的依赖是运行时失败。因此组件应当只在它声明的所有依赖都出现时才激活，而不是乐观地访问并在缺失时失败。这引出两个问题：组件声明的依赖是否被共同满足，以及当该状态改变时系统应如何响应。余效应上下文 Σ 携带一种自然的观测结构，使两个问题都可处理：对任意余效应规范 $d \subseteq K.$ ，定义满足谓词：

$$\sigma \vDash d := \forall k \in d.k \in \text{dom}(\sigma) \tag{24}$$

该谓词是可判定的（因为 $\text{dom}(\sigma)$ 是有限的）。由于对 σ 的所有变更都经过效应函数（其逆恢复先前的定义域），满足情况的变化在每个效应边界都是可检测的。这就是响应性的代数基础：效应系统保证每次余效应变化都被观测到。

定义 25. 余效应规范是：

$$\mathfrak{D}_{\Sigma} := \text{Set}(K) \tag{25}$$

表示组件向环境声明的依赖集。

使这个规范具有响应性的是它如何分类状态转移。任何把 σ 变换为 σ' 的效应都可以由规范 $d \in \mathfrak{D}_{\Sigma}$ 依据 d 的满足状态是否被改变来分类：

Definition 26. Given a coeffect specification $d \subseteq K$ and states $\sigma, \sigma' \in \Sigma$, define:

$$\text{notify}_{\,d}(\sigma, \sigma') := \begin{cases} \text{activating} & \text{if } \sigma \not\models d \wedge \sigma' \models d \\ \text{deactivating} & \text{if } \sigma \models d \wedge \sigma' \not\models d \\ \text{neutral} & \text{otherwise} \end{cases} \tag{26}$$

This is well-defined because $\sigma \models d$ is decidable and all state transitions are mediated by effect functions. The reactive invariant is: an activating transition triggers execution of the component's efects (with full effect tracking), whereas a deactivating transition triggers recovery by applying the accumulator. The precise operational semantics of these transitions depend on their interaction with control flows, and are developed in Section 4.

What set and notify deliver together is local spatial composability, local in the same sense as before, the guarantee being read of one component's coefects taken by themselves. We take that to be the following criterion: a component activates only at a state satisfying its specification, so it never reads a binding that is absent, and every change to the context is classified against that specification, so a loss of satisfaction is detected where it happens and drives a deactivation. Both halves are immediate from the definitions above, satisfaction being a precondition checked where the component would activate and notify$_d$ being defined at every transition.

The criterion covers one direction of the coeffect ordering and not the other. If component A provides a key k and component B declares $k \in d_B$, then B can activate only after A has activated and provided k, since $\boldsymbol{\sigma} \models d_B$ requires $k \in \text{dom}(\sigma)$. The converse fails: unloading A removes A from $\text{dom}(\sigma)$ and so breaks B's satisfaction, but a notification cannot by itself keep B readable for as long as B's own teardown needs it, nor hold B's recovery back until A has finished. Ordering a withdrawal after the deactivations it causes is a condition on other components rather than on the one acting, so it belongs to the global form of the guarantee, and Section 4.3.1 supplies the machinery it takes.

3.2.3. Isolation and Interception

The basic coeffect context Σ models a flat dependency table. In practice, however, the system may need to bind distinct values to the same logical dependency for diferent components. This section extends the coeffect context with two mechanisms: coeffect isolation (the same key resolves diferently in diferent contexts) and coeffect interception (cross-cutting behavior on dependency access).

Realization. The two mechanisms difer from get and set in what they act on. A provision writes the shared table every component reads, so it is an effect on that table and carries an inverse to withdraw it. Isolation and interception instead adjust how a key is resolved for the components under one context, leaving the table itself as it stands. Typing an operation as an effect fixes its denotation, a successor state paired with an inverse, but not its realization, which determines how that inverse is carried out.

Definition 27. An effect function on a context admits two realizations:

• In-place realization mutates the context and returns a nontrivial inverse; the successor aliases the input, and recovery runs the inverse to undo the mutation.

• Derived realization leaves the input intact and returns a fresh context deriving from it, with the identity as its inverse; recovery discards the derived context. A context derived from another is what the recursive structure of Definition 32 carries.

In a purely functional setting the two coincide, and an imperative host may choose either per operation; Section 5.1.2 implements both. Isolation and interception are given derived realization outright: each produces a fresh context whose own table difers from the inherited one, so each is typed below as a map from context to

定义 26. 给定余效应规范 $d \subseteq K$ 和状态 $\sigma, \sigma' \in \Sigma$ ，定义：

$$\text{notify}_{\,d}(\sigma, \sigma') := \begin{cases} \text{activating} & \text{if } \sigma \not\models d \wedge \sigma' \models d \\ \text{deactivating} & \text{if } \sigma \models d \wedge \sigma' \not\models d \\ \text{neutral} & \text{otherwise} \end{cases} \tag{26}$$

这是良定义的，因为 $\sigma \models d$ 可判定且所有状态转移都由效应函数中介。响应式不变量是：激活转移触发组件效应的执行（带完全效应跟踪），而停用转移通过应用累加器触发恢复。这些转移的精确操作语义取决于它们与控制流的交互，将在第 4 节中展开。

set 和 notify 一起交付局部空间可组合性，说它局部与前述同样的意义，该保证是针对一个组件的余效应本身来读的。我们把这个保证表述为如下准则：组件只在满足其规范的状态处激活，因此它从不读取缺席的绑定，且上下文的每次变化都依据该规范被分类，因此满足性的丧失在其发生处被检测并驱动停用。两半都直接由上述定义得出，满足性是组件将要激活处检查的前置条件，而 notify$_d$ 在每个转移处都有定义。

准则覆盖了余效应排序的一个方向而非另一个。若组件 A 提供键 k 且组件 B 声明 $k \in d_B$, 则 B 只能在 A 激活并提供 k, 之后激活，因为 $\boldsymbol{\sigma} \models d_B$ 要求 $k \in \text{dom}(\sigma)$ 。反之则不成立：卸载 A 把 A 从 $\text{dom}(\sigma)$ 中移除从而破坏 B 的满足性，但通知本身既不能使 B 在 B's 自身拆卸所需的整个期间保持可读，也不能推迟 B 的恢复直至 A 完成。把撤出排到它所造成的停用之后是对其他组件而非行动组件的条件，因此属于该保证的全局形式，第 4.3.1 节提供所需的机制。

3.2.3. 隔离与拦截（Isolation and Interception）

基本余效应上下文 Σ 建模扁平的依赖表。然而在实践中，系统可能需要为不同的组件把不同的值绑定到同一个逻辑依赖。本节用两种机制扩展余效应上下文：余效应隔离（同一键在不同上下文中解析不同）和余效应拦截（依赖访问上的横切行为）。

实现。这两种机制与 get 和 set 不同之处在于它们作用的对象。提供（provision）写入每个组件都读取的共享表，因此它是该表上的效应并携带撤出它的逆。相反，隔离与拦截调整一个键在一个上下文下的组件如何被解析，使表本身保持原样。把操作类型化为效应固定其外延（denotation）——后继状态与逆的配对——但不固定其实现，实现决定逆如何被执行。

定义 27. 上下文上的效应函数容许两种实现：

• 原位实现（In-place realization）改变上下文并返回一个非平凡逆；后继与输入共享别名，恢复运行逆以撤销该变更。

• 派生实现（Derived realization）保持输入原样并返回从它派生的全新上下文，以其恒等为逆；恢复丢弃派生上下文。从另一上下文派生的上下文正是定义 32 的递归结构所承载的。

在纯函数设定下两者重合，而命令式宿主可以为每个操作任选其一；第 5.1.2 节实现两者。隔离和拦截被直接赋予派生实现：每个都产生一个全新上下文，其自有表与继承的表不同，因此每个都在下面被类型化为上下文到上下文的映射而非效应函数。共享表中无任何变化，因此没有要跟踪的逆，也没有定义 12 要提升的东西，恢复把

context rather than as an effect function. Nothing in the shared table changes, so there is no inverse to track and nothing for Definition 12 to lift, and recovery discards the derived context along with the adjustment it carried. Assignment on a derived table overrides whatever the inherited table held at the key, which is why neither operation carries a precondition.

Coeffect Isolation. By introducing isolation realms, coeffect isolation allows the same dependency to bind to diferent values in diferent contexts. This has broad applications in multitenant systems, testing environments, and component sandboxes.

Definition 28. Define the coeffect context with isolation as:

$$\Sigma^{\text{iso}} := (K \rightharpoonup R) \times ((r : R) \rightharpoonup \mathcal{V}_r) \tag{27}$$

It can be represented as a pair (ρ, σ) , where:

$\rho : KR$ is the isolation realm table, assigning a realm identifier to each isolated key; a key outside dom (ρ) resolves to its own realm, so we write $\rho(k) = k$ there $(R \supseteq K)$;

$\sigma : (r : R)\mathcal{V}_r$ is the dependency table, a partial dependent function from realm identifiers to typed values.

The two-layer mapping structure decouples the logical layer from the storage layer, making dependency access context-aware. When accessing a key k, the system first resolves $\rho(k)$ to obtain a realm identifier ^{r,} then accesses $\sigma(r)$ for the actual value.

Definition 29. The get, set, and isolate operations on Σ^{iso} are:

$$\begin{array}{rcccccl}
\text{get} & : & (k : K) & \to & \Sigma^{\text{iso}} & \rightharpoonup & \mathcal{V}_{\rho(k)} \\
\text{get} & = & k & \mapsto & (\rho, \sigma) & \mapsto & \sigma(\rho(k)) \\
\text{set} & : & (k : K) \times \mathcal{V}_{\rho(k)} & \to & \Sigma^{\text{iso}} & \rightharpoonup & \Sigma^{\text{iso}} \times (\Sigma^{\text{iso}} \rightharpoonup \Sigma^{\text{iso}}) \\
\text{set} & = & (k, v) & \mapsto & (\rho, \sigma) & \mapsto & ((\rho, \sigma[\rho(k) \mapsto v]), \lambda(\rho', \sigma').(\rho', \sigma' \setminus \rho'(k))) \\
\text{isolate} & : & K \times R & \to & \Sigma^{\text{iso}} & \to & \Sigma^{\text{iso}} \\
\text{isolate} & = & (k, r) & \mapsto & (\rho, \sigma) & \mapsto & (\rho[k \mapsto r], \sigma)
\end{array} \tag{28}$$

where get and set carry the preconditions of Definition 23 transported along ρ, namely $\rho(k) \in \text{dom}(\sigma)$ and $\rho(k) \notin \text{dom}(\sigma)$. The context that isolate (k, r) derives assigns the realm r to k and inherits the dependency table unchanged, so a key already isolated is reassigned rather than refused.

The coeffect isolation mechanism essentially implements a runtime ad-hoc polymorphism system. Through isolation realm identifiers, the same dependency key can resolve to entirely diferent values in diferent contexts, and this polymorphism can be dynamically adjusted at runtime. Compared to traditional dependency injection, coeffect isolation provides finergrained control, enabling customized isolation for specific components; set remains an effect function $\left(\mathfrak{E}^*_{\Sigma^{\text{iso}}}\right)$ and thus inherits revertibility, whereas isolate needs none, deriving a context instead of writing the shared table.

Coeffect Interception. The second mechanism, coeffect interception, attaches cross-cutting metadata to dependency access, adding behavior without modifying the dependency value. This metadata can be either context-carried or component-declared, so we extend both the coeffect context and the coeffect specification:

Definition 30. Define the coeffect context and specification with interception as:

$$\begin{aligned}
\Sigma^{\text{inter}} &:= ((k : K) \to \mathcal{M}_k) \times ((k : K) \rightharpoonup (\mathcal{M}_k \to \mathcal{V}_k)) \\
\mathfrak{D}^{\text{inter}} &:= (k : K) \rightharpoonup \mathcal{M}_k
\end{aligned} \tag{29}$$

The context Σ^{inter} is a pair (ι, σ): ι is the context-carried metadata installed on the context itself, empty

派生上下文连同它承载的调整一起丢弃。在派生表上赋值会覆盖继承表在键处持有的值，这就是两个操作都不携带前置条件的原因。

余效应隔离。通过引入隔离领域，余效应隔离允许同一依赖在不同上下文中绑定不同的值。这在多租户系统、测试环境和组件沙箱中有广泛的应用。

定义 28. 定义带隔离的余效应上下文为：

$$\Sigma^{\text{iso}} := (K \rightharpoonup R) \times ((r : R) \rightharpoonup \mathcal{V}_r) \tag{27}$$

它可以表示为二元组 (ρ, σ) ，其中：

$\rho : KR$ 是隔离领域表，为每个被隔离的键赋予领域标识符；dom (ρ) 之外的键解析到自己的领域，因此那里我们写 $\rho(k) = k$ $(R \supseteq K)$ ；

$\sigma : (r : R)\mathcal{V}_r$ 是依赖表，从领域标识符到类型化值的部分依赖函数。

两层映射结构把逻辑层与存储层解耦，使依赖访问具有上下文感知性。访问键 k, 时，系统首先解析 $\rho(k)$ 获得领域标识符 ^{r,} 然后访问 $\sigma(r)$ 获取实际值。

定义 29. Σ^{iso} 上的 get、set 和 isolate 操作是：

$$\begin{array}{rcccccl}
\text{get} & : & (k : K) & \to & \Sigma^{\text{iso}} & \rightharpoonup & \mathcal{V}_{\rho(k)} \\
\text{get} & = & k & \mapsto & (\rho, \sigma) & \mapsto & \sigma(\rho(k)) \\
\text{set} & : & (k : K) \times \mathcal{V}_{\rho(k)} & \to & \Sigma^{\text{iso}} & \rightharpoonup & \Sigma^{\text{iso}} \times (\Sigma^{\text{iso}} \rightharpoonup \Sigma^{\text{iso}}) \\
\text{set} & = & (k, v) & \mapsto & (\rho, \sigma) & \mapsto & ((\rho, \sigma[\rho(k) \mapsto v]), \lambda(\rho', \sigma').(\rho', \sigma' \setminus \rho'(k))) \\
\text{isolate} & : & K \times R & \to & \Sigma^{\text{iso}} & \to & \Sigma^{\text{iso}} \\
\text{isolate} & = & (k, r) & \mapsto & (\rho, \sigma) & \mapsto & (\rho[k \mapsto r], \sigma)
\end{array} \tag{28}$$

其中 get 和 set 携带沿 ρ, 搬运的定义 23 的前置条件，即 $\rho(k) \in \text{dom}(\sigma)$ 和 $\rho(k) \notin \text{dom}(\sigma)$ 。isolate (k, r) 派生的上下文把领域 r 赋予 k 并原样继承依赖表，因此已隔离的键被重新指派而非拒绝。

余效应隔离机制本质上实现了一个运行时临时多态系统。通过隔离领域标识符，同一依赖键可以在不同上下文中解析到完全不同的值，且这种多态可以在运行时动态调整。与传统的依赖注入相比，余效应隔离提供更细粒度的控制，能为特定组件实现定制隔离；set 仍是效应函数 $\left(\mathfrak{E}^*_{\Sigma^{\text{iso}}}\right)$ 从而继承可逆性，而 isolate 不需要，因为它派生上下文而非写入共享表。

余效应拦截。第二种机制，余效应拦截，把横切元数据附加到依赖访问上，在不修改依赖值的情况下添加行为。该元数据既可由上下文携带也可由组件声明，因此我们同时扩展余效应上下文与余效应规范：

定义 30. 定义带拦截的余效应上下文与规范为：

$$\begin{aligned}
\Sigma^{\text{inter}} &:= ((k : K) \to \mathcal{M}_k) \times ((k : K) \rightharpoonup (\mathcal{M}_k \to \mathcal{V}_k)) \\
\mathfrak{D}^{\text{inter}} &:= (k : K) \rightharpoonup \mathcal{M}_k
\end{aligned} \tag{29}$$

上下文 Σ^{inter} 是二元组 (ι, σ): ι 是安装在上下文本身上的上下文携带元数据，默认为空 (ϵ_k) ； ι 把每个键 k

(ϵ_k) by default; and ι maps each key k to a provider function from metadata $\mathcal{M}_k$ to value ν_k. A specification $d \in \mathfrak{D}^{\text{inter}}$ carries the component-declared metadata, assigning each key its metadata $d(k)$, with dom (d) serving as the dependency set. Each key equips its metadata with a monoid $(\mathcal{M}_k, \oplus_k, \epsilon_k)$: the merge $\oplus_k$ is associative with identity ϵ_k (the empty metadata).

Definition 31. The get, set, and intercept operations on Σ^{inter} are:

$$\begin{array}{rcllll} \text{get} & : & (k:K)\times\mathcal{M}_k & \to & \Sigma^{\text{inter}} & \rightharpoonup \quad \mathcal{V}_k \\ \text{get} & = & (k,\mu) & \mapsto & (\iota,\sigma) & \mapsto \quad \sigma(k)(\mu\oplus_k \iota(k)) \\ \text{set} & : & (k:K)\times(\mathcal{M}_k\to\mathcal{V}_k) & \to & \Sigma^{\text{inter}} & \rightharpoonup \quad \Sigma^{\text{inter}}\times(\Sigma^{\text{inter}}\rightharpoonup\Sigma^{\text{inter}}) \\ \text{set} & = & (k,\psi) & \mapsto & (\iota,\sigma) & \mapsto \quad ((\iota,\sigma[k\mapsto\psi]),\lambda(\iota',\sigma').(\iota',\sigma'\setminus k)) \\ \text{intercept} & : & (k:K)\times\mathcal{M}_k & \to & \Sigma^{\text{inter}} & \to \quad \Sigma^{\text{inter}} \\ \text{intercept} & = & (k,\nu) & \mapsto & (\iota,\sigma) & \mapsto \quad (\iota[k\mapsto\iota(k)\oplus_k\nu],\sigma) \end{array} \tag{30}$$

where get and set carry the preconditions of Definition 23 on the provider table, namely $k \in \text{dom}(\sigma)$ and $k \notin \text{dom}(\sigma)$. The context that intercept (k, ν) derives merges ν onto the metadata inherited at k and inherits the provider table unchanged.

When a component with specification d accesses key k, the system evaluates $\sigma(k)(d(k)\oplus_k \iota(k))$: the component-declared metadata is merged with the context-carried metadata ι, and the provider function is applied to the result. This merge follows each key's own semantics (e.g. scalar fields are overwritten, set-valued fields unioned) and is right-biased, so ι(k) takes priority and can override the component's declaration, letting an enclosing context constrain how a component uses a coeffect without modifying that component (e.g. Section 6.3).

3.3. The Context Paradigm

Section 3.1 and Section 3.2 each act on a context, the first as the carrier of efects and the second as the carrier of coefects, leaving open what a single context carrying both looks like. This section gives that unification a concrete construction, assembles from the coefects an observational equivalence that supplies the effect independence Section 3.1.3 leaves open, and argues that the resulting context type constitutes a programming paradigm in its own right.

3.3.1. Unified Context

For a context Γ, the effect context $\partial\Gamma$ (Section 3.1) provides a higher-level abstraction, carrying the previous-level context and that level's accumulator (Definition 2). Making this structure recursive and combining it with the coeffect context Σ yields the following type:

Definition 32. The context type Γ_∞ is defined as:

$$\Gamma_\infty := \mu\Gamma.\Gamma\times(\Gamma\to\Gamma)\times\Sigma \tag{31}$$

where the three projections are:

- Γ: the current context state (recursive);
- $\Gamma \to \Gamma$: the accumulator, which recovers this level's efects;
- Σ: the coeffect context carrying dependency information.

Under this definition, effect maps $\mathfrak{E}_{\Gamma_\infty}$ to itself, unifying the Γ-tower into a single selfsimilar type. The coeffect context Σ is structurally integrated: dependency operations (set, get) act on Σ, and the accumulator tracks their

映射到从元数据 $\mathcal{M}_k$ 到值 ν_k 的提供函数。规范 $d \in \mathfrak{D}^{\text{inter}}$ 携带组件声明的元数据，为每个键赋予其元数据 $d(k)$ ，以 dom (d) 作为依赖集。每个键为其元数据配备幺半群 $(\mathcal{M}_k, \oplus_k, \epsilon_k)$：合并 $\oplus_k$ 是结合的，以 ϵ_k（空元数据）为单位。

定义 31. Σ^{inter} 上的 get、set 和 intercept 操作是：

$$\begin{array}{rcllll} \text{get} & : & (k:K)\times\mathcal{M}_k & \to & \Sigma^{\text{inter}} & \rightharpoonup \quad \mathcal{V}_k \\ \text{get} & = & (k,\mu) & \mapsto & (\iota,\sigma) & \mapsto \quad \sigma(k)(\mu\oplus_k \iota(k)) \\ \text{set} & : & (k:K)\times(\mathcal{M}_k\to\mathcal{V}_k) & \to & \Sigma^{\text{inter}} & \rightharpoonup \quad \Sigma^{\text{inter}}\times(\Sigma^{\text{inter}}\rightharpoonup\Sigma^{\text{inter}}) \\ \text{set} & = & (k,\psi) & \mapsto & (\iota,\sigma) & \mapsto \quad ((\iota,\sigma[k\mapsto\psi]),\lambda(\iota',\sigma').(\iota',\sigma'\setminus k)) \\ \text{intercept} & : & (k:K)\times\mathcal{M}_k & \to & \Sigma^{\text{inter}} & \to \quad \Sigma^{\text{inter}} \\ \text{intercept} & = & (k,\nu) & \mapsto & (\iota,\sigma) & \mapsto \quad (\iota[k\mapsto\iota(k)\oplus_k\nu],\sigma) \end{array} \tag{30}$$

其中 get 和 set 携带提供函数表上定义 23 的前置条件，即 $k \in \text{dom}(\sigma)$ 和 $k \notin \text{dom}(\sigma)$ 。intercept (k, ν) 派生的上下文把 ν 合并到继承的 k 处元数据上并原样继承提供函数表。

当规范为 d 的组件访问键 k 时，系统求值 $\sigma(k)(d(k)\oplus_k \iota(k))$ ：组件声明的元数据与上下文携带的元数据 ι, 合并，提供函数应用于结果。该合并遵循每个键自身的语义（例如标量字段被覆盖、集合值字段取并集）并且是右偏的，因此 ι(k) 优先并可覆盖组件的声明，让外层上下文能在不修改组件的情况下约束组件如何使用余效应（例如第 6.3 节）。

3.3. 上下文范式（The Context Paradigm）

第 3.1 节与第 3.2 节各自作用于一个上下文，前者作为效应的载体、后者作为余效应的载体，留下一个问题：同时承载两者的单一上下文看起来如何。本节给出这一统一的具体构造，从余效应组装出为第 3.1.3 节留下的效应独立性提供来源的观测等价，并论证所得的上下文类型自身构成一种编程范式。

3.3.1. 统一上下文（Unified Context）

对上下文 Γ，效应上下文 $\partial\Gamma$（第 3.1 节）提供一个更高层的抽象，承载上一层的上下文和该层的累加器（定义 2）。把此结构递归化并与其与余效应上下文 Σ 结合得到如下类型：

定义 32. 上下文类型 Γ_∞ 定义为：

$$\Gamma_\infty := \mu\Gamma.\Gamma\times(\Gamma\to\Gamma)\times\Sigma \tag{31}$$

其中三个投影是：

- Γ：当前上下文状态（递归）；
- $\Gamma \to \Gamma$：累加器，恢复本层的效应；
- Σ：携带依赖信息的余效应上下文。

在此定义下，effect 把 $\mathfrak{E}_{\Gamma_\infty}$ 映到自身，把 Γ-塔统一为单一自相似类型。余效应上下文 Σ 在结构上被整合：依赖操作（set、get）作用于 Σ, 累加器跟踪其逆转。由于支撑 Σ 的类型族 K 不受约束，系统需要在组件间共享的

reversal. Since the type family K underlying Σ is unconstrained, any state the system needs to share across components can be encoded as a dependency with an appropriate value type—Σ subsumes all shared mutable states, not just inter-component dependencies. Every interaction between a component and its environment passes through this single entity.

Hierarchical composition. The recursive structure of Γ_∞ supports hierarchical control: a parent context aggregates multiple child-level efects, forming a tree-shaped control structure that maintains modularity while enabling unified cross-level management. The effect transformation realizes a literal "plug-in" metaphor:

- Loading a component corresponds to executing its efects (plugging in);
- Unloading a component corresponds to recovering its efects (unplugging, without afecting other running components);
- Components at diferent levels of the hierarchy are independently loadable and unloadable; a parent context aggregates and manages the efects of all its children, enabling arbitrarily nested composition.

3.3.2. Observational Equivalence

The recovery guarantee of Section 3.1 asserts an equality of states (Theorem 7), which is an idealization, because the physical state cannot be recovered as it stood. For example, free releases a block to the allocator without restoring the layout the heap had before malloc; and a generative name is not restored by the inverse that discards it, since the next creation draws a fresh one [39]. The equalities of Section 3 are therefore to be read up to an equivalence $\simeq$, and we take $\simeq$ to be an observational equivalence: two states are related when no observer can distinguish them. Comparing behaviour rather than representation is the established route to program equivalence [40], and the relation such a comparison yields depends on what the observer is given to work with [41]. What an observer of a context is given is the coefects it carries, each of which arrives with an equivalence of its own (Definition 24), so the relation on a context is assembled from theirs. Assembling it is the business of this subsection, and quotienting by it is what buys the independence Section 3.1.3 asks for.

Definition 33. Two coeffect contexts are related when they bind the same keys to related values, and two states of a context when their coeffect projections are:

$$\begin{aligned} \sigma \simeq \sigma' &:= \mathrm{dom}(\sigma) = \mathrm{dom}(\sigma') \wedge \forall k \in \mathrm{dom}(\sigma).\sigma(k) \underset{k}{\simeq} \sigma'(k) \\ \gamma \simeq \gamma' &:= \sigma_\gamma \simeq \sigma_{\gamma'} \end{aligned} \tag{32}$$

writing σ_γ for the coeffect projection of γ (Definition 32).

The part of a state that no key binds is thereby forgotten, and forgetting it is what lets Theorem 7 be read up to $\simeq$ at all: the heap layout and the generative name of the examples above lie outside the relation unless some key binds them. What Section 3.2.2 needs of $\simeq$ follows rather than being assumed. Related states have the same domain, so they agree on the satisfaction predicate $\sigma \models d$ and on the classification notify of Definition [26,] and reactivity is a property of $\Sigma/\simeq$

Calling the relation observational is a claim about each $\tilde{k}'$, namely that it separates no more than the operations of k can tell apart. An observer of a value runs those operations and reads their outcomes.

Definition 34. Let k carry a set A of operations in the sense of Definition [24,] and write $\mathfrak{M}(a)$ for the transformation monoid (Definition 17) of the effect functions $a(x)$ over every argument $x : X_a$. A test over A is a finite word over the generators of the monoids $\mathfrak{M}(a), a \in \mathcal{A}$, , each letter applied to the value the letters before it left; its outcomes are those the letters that are forward maps of operations yield along the way, and it is undefined where a precondition fails. Values $v, v' : V$ are indistinguishable, written $v \approx v'$. , when every test over

任何状态都可以编码为具有适当值类型的依赖——Σ 涵盖所有共享可变状态，而不仅仅是组件间依赖。组件与其环境的每次交互都穿过这个单一实体。

层次组合。Γ_∞ 的递归结构支持层次控制：父上下文聚合多个子层效应，形成树形控制结构，在保持模块性的同时支持统一的跨层管理。effect 变换实现了字面的「即插即用」隐喻：

- 加载组件对应于执行其效应（插入）；
- 卸载组件对应于恢复其效应（拔出，不影响其他运行中的组件）；
- 层次中不同层的组件可独立加载和卸载；父上下文聚合并管理其所有子节点的效应，支持任意嵌套的组合。

3.3.2. 观测等价（Observational Equivalence）

第 3.1 节的恢复保证断言状态之间的相等性（定理 7），这是一个理想化，因为物理状态无法被恢复如初。例如，free 把一块内存还给分配器，却不恢复 malloc 之前堆的布局；生成名（generative name）也不会被丢弃它的逆所恢复，因为下一次创建会取得一个新名 [39]。因此第 3 节的相等性要被读作在一个等价 $\simeq$ 之下，且我们取 $\simeq$ 为观测等价：两个状态在无观测者能区分它们时相关。比较行为而非表示是程序等价的既有途径 [40]，而这样的比较所产生的联系取决于观测者被给予什么来工作 [41]。上下文的观测者被给予的是它携带的余效应，每个余效应都自带等价（定义 24），因此上下文上的联系由它们的等价组装而成。组装它是本小节的事，而对其求商正是第 3.1.3 节所要求的独立性之来源。

定义 33. 两个余效应上下文在它们把相同的键绑定到相关的值时相关，两个上下文状态在其余效应投影相关时相关：

$$\begin{aligned} \sigma \simeq \sigma' &:= \mathrm{dom}(\sigma) = \mathrm{dom}(\sigma') \wedge \forall k \in \mathrm{dom}(\sigma).\sigma(k) \underset{k}{\simeq} \sigma'(k) \\ \gamma \simeq \gamma' &:= \sigma_\gamma \simeq \sigma_{\gamma'} \end{aligned} \tag{32}$$

其中 σ_γ 表示 γ 的余效应投影（定义 32）。

状态中没有任何键绑定的部分因此被遗忘，而遗忘它正是使定理 7 能够被读作 $\simeq$ 之下的原因：上例中的堆布局和生成名位于该联系之外，除非某个键绑定它们。第 3.2.2 节对 $\simeq$ 的需要是随之而来的而非被假设的。相关状态具有相同的定义域，因此它们在满足谓词 $\sigma \models d$ 和定义 [26,] 的 notify 分类上一致，而响应性是 $\Sigma/\simeq$ 的性质。

把该联系称为观测的是关于每个 $\tilde{k}'$ 的断言，即它区分的不多于 k 的操作所能区分。值的观测者运行那些操作并读取其结果。

定义 34. 令 k 按定义 [24,] 的意义携带操作集 A ，并写 $\mathfrak{M}(a)$ 为对每个参数 $x : X_a$ 的效应函数 $a(x)$ 的变换幺半群（定义 17）。A 上的测试是幺半群 $\mathfrak{M}(a), a \in \mathcal{A}$, 生成元上的有限词，每个字母应用于此前字母留下的值；其结果是在沿途正向映射为操作的字母所产生的那些，在前置条件失败处无定义。值 $v, v' : V$ 不可区分，记作 $v \approx v'$. ，当 A 上的每个测试在两者处都有定义或都没有定义，且在两者处产生相同的结果。

A is defined at both or at neither and yields the same outcomes at both.

Lemma 35. Indistinguishability is the coarsest relation the operations respect. That is,

1. every operation of A respects $\widetilde{\mathcal{A}}$ in the sense of Definition 24;

2. every equivalence that every operation of A respects is contained in $\widetilde{\widetilde{\lambda}}$. Every admissible choice of $\widetilde{\widetilde{k}}$ is therefore contained in $\widetilde{\mathcal{A}}'_k$ and $\widetilde{\mathcal{A}}_k$ is itself admissible.

Proof.

1. Let v $\tilde{A}{}^{v'}$ and let $a \in \mathcal{A}$ be applied to an argument. Prefixing a test by one letter is again a test, so the values the forward map reaches are indistinguishable, as are the values any one yielded inverse reaches from indistinguishable arguments; the one-letter test gives definedness at both or neither and equality of the outcome.

2. Let R be such an equivalence and vRv'. Each letter of a test is a forward map or a yielded inverse of an operation, and respect carries R along either, keeping the values reached related and the outcomes equal at every letter. Hence every test agrees at v and v'. □

Substituting $\simeq$ for $=$ throughout is not by itself enough, because an effect function returns an inverse as well as a state, and two states that $\simeq$ identifies have to yield inverses $\simeq$ identifies as well.

Definition 36. A map $f : \Gamma \to \Gamma$ respects $\simeq$ when

$$\forall \gamma, \gamma' \in \Gamma. \quad \gamma \simeq \gamma' \Rightarrow f(\gamma) \simeq f(\gamma') \tag{33}$$

Two maps are related when they agree at every state, and two pairs in $\partial\Gamma$ when both components are:

$$\begin{aligned} f \simeq g &:= \forall \gamma \in \Gamma. f(\gamma) \simeq g(\gamma) \\ (\delta, g) \simeq (\delta', g') &:= \delta \simeq \delta' \wedge g \simeq g' \end{aligned} \tag{34}$$

A map respecting $\simeq$ is one that descends to $\Gamma/\simeq$, , and two maps related by $\simeq$ are two that descend to the same map there. An effect function needs both: the first so that the state it computes is determined on the quotient, the second so that the inverse it returns is.

Definition 37. Read Definition 8 up to $\simeq$: an $e \in \mathfrak{E}_\Gamma$ lies in $\mathfrak{E}^*_\Gamma$ when e respects $\simeq$ as a map $\Gamma \to \partial\Gamma$ and, writing $(\delta, g) = e(\gamma)$, for every $\gamma \in \Gamma$

1. $g(\delta) \simeq \gamma$;

2. g respects $\simeq$.

Taking $\simeq$ to be equality on Γ recovers Definition 8.

Lemma 38. With $\mathfrak{E}^*_\Gamma$ read as in Definition [37,] , every equality of states asserted in Section 3.1 holds with $=$ replaced by $\simeq$, and the accumulator of every state reachable from (γ_0, id) respects $\simeq$

Proof. An accumulator is a composition of inverses, each respecting $\simeq$ by Definition 37(2) , and a composition of maps respecting $\simeq$ respects $\simeq$, the base case being id_Γ . The proofs of Section 3.1 then go through unchanged,

引理 35. 不可区分性是操作所尊重的最粗联系。即，

1. A 的每个操作按定义 24 的意义尊重 $\widetilde{\mathcal{A}}$；

2. A 的每个操作所尊重的每个等价都被包含在 $\widetilde{\widetilde{\lambda}}$ 中。$\widetilde{\widetilde{k}}$ 的每个可容许选择因此都被包含在 $\widetilde{\mathcal{A}}'_k$ 中，且 $\widetilde{\mathcal{A}}_k$ 本身可容许。

证明.

1. 令 v $\tilde{A}{}^{v'}$ 并把 $a \in \mathcal{A}$ 应用于一个参数。用一字母作为测试前缀仍是一个测试，因此正向映射所达到的值不可区分，任何产生式逆从不可区分参数达到的值也不可区分；单字母测试给出两者的有定义性及结果的相等性。

2. 令 R 是这样一个等价且 vRv' 。测试的每个字母是操作的正向映射或产生的逆，而尊重关系沿两者携带 R，使所达到的值保持相关、每个字母处的结果保持相等。因此每个测试在 v 和 v' 处一致。□

把 $\simeq$ 代入 $=$ 遍及全文本身并不足够，因为效应函数不仅返回状态还返回逆，而 $\simeq$ 所等同的两个状态必须产生 $\simeq$ 所等同的逆。

定义 36. 映射 $f : \Gamma \to \Gamma$ 尊重 $\simeq$，当

$$\forall \gamma, \gamma' \in \Gamma. \quad \gamma \simeq \gamma' \Rightarrow f(\gamma) \simeq f(\gamma') \tag{33}$$

两个映射在它们在每个状态一致时相关，$\partial\Gamma$ 中的两个对在两个分量都相关时相关：

$$\begin{aligned} f \simeq g &:= \forall \gamma \in \Gamma. f(\gamma) \simeq g(\gamma) \\ (\delta, g) \simeq (\delta', g') &:= \delta \simeq \delta' \wedge g \simeq g' \end{aligned} \tag{34}$$

尊重 $\simeq$ 的映射是下降到 $\Gamma/\simeq$, 的映射，而由 $\simeq$ 相关的两个映射是在那里下降到同一映射的两个。效应函数两者都需要：前者使其计算的状态在商上被确定，后者使其返回的逆如此。

定义 37. 把定义 8 读作 $\simeq$ 之下：当 e 作为映射 $\Gamma \to \partial\Gamma$ 尊重 $\simeq$，且对每个 $\gamma \in \Gamma$ 写作 $(\delta, g) = e(\gamma)$ 时

1. $g(\delta) \simeq \gamma$;

2. g 尊重 $\simeq$。

则 $e \in \mathfrak{E}_\Gamma$ 位于 $\mathfrak{E}^*_\Gamma$ 中。取 $\simeq$ 为 Γ 上的相等性恢复定义 8。

引理 38. 按定义 [37,] 读 $\mathfrak{E}^*_\Gamma$ 时，第 3.1 节断言的每个状态相等性在 $=$ 替换为 $\simeq$, 后成立，且从 $(\gamma_0, \mathrm{id}_\Gamma)$ 可达的每个状态的累加器尊重 $\simeq$

证明. 累加器是逆的复合，每个逆按定义 37(2) 尊重 $\simeq$，而尊重 $\simeq$ 的映射的复合尊重 $\simeq$, 基例是 id_Γ 。第 3.1 节的证明随后原封不动地成立，尊重关系把联系穿过逆：由 $g_2(\delta_2) \simeq \delta_1$ 和 $g_1(\delta_1) \simeq \gamma$ ，尊重关系给出 $(g_1 \circ g_2)(\delta_2) \simeq \gamma$,

respect being what carries a relation through an inverse: from $g_2(\delta_2) \simeq \delta_1$ and $g_1(\delta_1) \simeq \gamma$ respect gives $(g_1 \circ g_2)(\delta_2) \simeq \gamma$, , which is the step each composition of inverses takes, and the soundness invariant of Theorem 7 reads $\varphi(\gamma) \simeq \gamma_0$ by that step. □

The commutation Definition 19 asks for is read up to $\simeq$ by the same lemma, and reading it that way is what makes it attainable at all: two operations may leave values that $\widetilde{\widetilde{k}}$ identifies and still count as commuting. Of two operations it asks one thing more than of the effect functions their lifts induce, an operation yielding an outcome as well.

Definition 39. Operations a and a' are independent when their lifts are independent as effect functions (Definition 19) at every pair of arguments, and neither one's transformations disturb the outcome the other yields:

$$\forall x : X_a, g \in \mathfrak{M}(a'^{\Sigma}), \sigma \in \Sigma. \quad \mathrm{pr}_3(a^{\Sigma}(x)(g(\sigma))) = \mathrm{pr}_3(a^{\Sigma}(x)(\sigma)) \tag{35}$$

and the same with a and a' exchanged, writing $\mathfrak{M}(a^{\Sigma})$ for the transformation monoid of the lifts $a^{\Sigma}(x)$ over every argument as Definition 34 writes $\mathfrak{M}(a)$ for that of the operation itself. A key k is commutative when any two operations of $\mathcal{A}_k$ are independent, an operation being held independent of itself as well.

Across distinct keys the condition holds outright.

Theorem 40. Operations at distinct keys are independent.

Proof. Let a lie in $\mathcal{A}_k$ and a' in $\boldsymbol{A}_{\boldsymbol{k}'}$ with $\mathrm{k} \neq k'$. By Definition 24 every generator of $\mathfrak{M}(a^{\Sigma})$ is of the form $\sigma \mapsto \sigma[k \mapsto u(\sigma(k))]$ for a map u on ν_k, being either the lift of a forward map or the lift of a yielded inverse, and likewise for a' at k' . Two such maps commute, each reading and writing one key alone and the two keys difering, and Lemma 18(1) extends the commutation from the generators to the two monoids. For the second condition, what a^{Σ} yields at σ, inverse and outcome alike, is determined by $\sigma(k)$, which every generator of $\mathfrak{M}(a'^{\Sigma})$ leaves as it stands.□

A key whose value is a table of entries added and removed independently is commutative, registration of a route or of an event listener being the representative case: two registrations in either order leave a table that answers every test alike, and either registration can be withdrawn while the other stands. A key whose value is an ordered chain is not, since a middleware inserted before another sees a diferent request, and neither order can be withdrawn without disturbing the other. The allocator of the opening example divides by what its interface publishes. Where the handles it hands out are compared by no operation of the key, $\widetilde{\widetilde{k}}$ may relate two heaps up to a renaming of handles, which is how CompCert relates the memory states of a program and of its translation [42], and allocation is commutative; where the addresses are outcomes compared by equality, no admissible $\widetilde{\widetilde{k}}$ makes the two orders of allocation agree, and the key is not commutative.

What a component performs is a sequence of operations in which each may depend on what the ones before it yielded, and effect functions of that shape are what the theorem below speaks of.

Definition 41. The coeffect-mediated effect functions form the least set $\mathfrak{E}_{\Sigma}^{\mathcal{A}} \subseteq \mathfrak{E}_{\Sigma}$ that contains the unit η_{Σ} and is closed under the following: for a key k, an operation $a \in \mathcal{A}_k$, an argument $\mathrm{x} : X_a$, and a family $(e_b)_{b \in B_a}$ of members,

$$\sigma \mapsto \mathbf{let}(\delta, s, b) = a^{\Sigma}(x)(\sigma)\mathbf{inlet}(\varepsilon, t) = e_b(\delta)\mathbf{in}(\varepsilon, s \circ t) \tag{36}$$

is again a member. Each stage performs one operation and chooses what follows it by the outcome, so an argument may depend on the outcomes already obtained. The operations occurring in a member are the ones its stages perform, over every choice of outcome.

，即每次逆复合所采取的步骤，而定理 7 的可靠性不变量由该步骤读作 $\varphi(\gamma) \simeq \gamma_0$ □

定义 19 所要求的交换性由同一引理读作 $\simeq$ 之下，而这样读它正是使它可达到的原因：两个操作可能留下 $\widetilde{\widetilde{k}}$ 所等同的值却仍算作交换。对两个操作，它要求比对其提升诱导的效应函数多一点东西，一个操作还产生结果。

定义 39. 操作 a 和 a' 是独立的，当它们的提升作为效应函数（定义 19）在每对参数处独立，且一方的变换不扰动另一方产生的结果：

$$\forall x : X_a, g \in \mathfrak{M}(a'^{\Sigma}), \sigma \in \Sigma. \quad \mathrm{pr}_3(a^{\Sigma}(x)(g(\sigma))) = \mathrm{pr}_3(a^{\Sigma}(x)(\sigma)) \tag{35}$$

交换 a 和 a' 后同样成立，其中 $\mathfrak{M}(a^{\Sigma})$ 表示对每个参数的提升 $a^{\Sigma}(x)$ 的变换幺半群，如定义 34 对操作本身写 $\mathfrak{M}(a)$ 那样。键 k 是交换的，当 $\mathcal{A}_k$ 的任意两个操作独立，操作与自身独立也包括在内。

跨不同键，该条件无条件成立。

定理 40. 不同键处的操作是独立的。

证明. 令 a 位于 $\mathcal{A}_k$ 且 a' 位于 $\boldsymbol{A}_{\boldsymbol{k}'}$ ，其中 $\mathrm{k} \neq k'$ 。由定义 24，$\mathfrak{M}(a^{\Sigma})$ 的每个生成元具有 $\sigma \mapsto \sigma[k \mapsto u(\sigma(k))]$ 的形式，其中 u 是 ν_k, 上的映射，它是正向映射的提升或产生逆的提升，a' 在 k' 处同理。两个这样的映射交换，每个只读写一个键且两个键不同，而引理 18(1) 把交换性从生成元扩展到两个幺半群。对第二个条件，a^{Σ} 在 σ, 处产生的东西（逆和结果都一样）由 $\sigma(k)$ 决定，而 $\mathfrak{M}(a'^{\Sigma})$ 的每个生成元保持其原样。□

值是由独立添加和移除的条目组成的表的键是交换的，路由或事件监听器的注册是代表情形：两次注册无论顺序如何都留下对每个测试应答相同的表，且任一注册都可以在另一注册在位时被撤出。值是有序链的键不是，因为插入到另一中间件之前的中间件会看到不同的请求，且任一顺序都不能在不扰动另一个的情况下被撤出。开篇示例的分配器按其接口发布的内容来划分。它所派发的句柄不被键的任何操作比较时，$\widetilde{\widetilde{k}}$ 可以把两个堆相关到句柄的重命名之下，这正是 CompCert 关联程序及其翻译的内存状态的方式 [42]，而分配是交换的；当地址是被相等性比较的结果时，没有可容许的 $\widetilde{\widetilde{k}}$ 使分配的两种顺序一致，且该键不是交换的。

组件执行的是操作的序列，其中每个操作可能依赖它之前的操作产生的东西，而该形状的效应函数正是下面定理所说的。

定义 41. 余效应中介的效应函数构成最小集 $\mathfrak{E}_{\Sigma}^{\mathcal{A}} \subseteq \mathfrak{E}_{\Sigma}$ ，它包含单位 η_{Σ} 并在以下条件下封闭：对键 k, 操作 $a \in \mathcal{A}_k$, 参数 $\mathrm{x} : X_a$, 和族 $(e_b)_{b \in B_a}$ 的成员，

$$\sigma \mapsto \mathbf{let}(\delta, s, b) = a^{\Sigma}(x)(\sigma)\mathbf{inlet}(\varepsilon, t) = e_b(\delta)\mathbf{in}(\varepsilon, s \circ t) \tag{36}$$

又是一个成员。每个阶段执行一个操作并按结果选择其后续，因此参数可以依赖已获得的结果。成员中出现的操作是它的阶段所执行的，对所有结果选择。

Theorem 42. Let $e_1, e_2 \in \mathfrak{E}^{\mathcal{A}}_{\Sigma}$ and let every key at which operations of both occur be commutative (Definition 39). Then e_1 and e_2 are independent (Definition 19).

Proof. By induction on the construction of Definition 41, $\mathfrak{M}(e_i)$ lies in the submonoid generated by the generators of the operations occurring in $\boldsymbol{e}_i$: the unit generates the trivial monoid, and a stage is a $\diamond$-composite of $a^{\Sigma}(x)$ with a member, to which Lemma 18(2) applies.

For clause (1) of Definition 19 it is therefore enough, by Lemma 18(1), that a generator of an operation occurring in e_1 commute with a generator of one occurring in e_2 . Where the two operations lie at distinct keys this is Theorem [40,] , and where they lie at one key that key carries operations of both and is commutative by hypothesis.

For clause (2), take $g \in \mathfrak{M}(e_2)$, a composite of generators of the operations occurring in e_2, , and induct on the construction of e_1 . The unit yields id_{Σ} at every state. At a stage, let $(\delta, s, b) = a^{\Sigma}(x)(\sigma)$ and $(\varepsilon, t) = e_b(\delta)$, so that the stage yields s $\circ$ t at σ. . Independence of the operations, applied to one generator of g at a time, yields s and t again at $g(\sigma)$, so the same continuation e_b is chosen, and clause (1) puts the state it runs from at $g(\delta)$, where the induction hypothesis yields s again. The stage therefore yields $s \circ t$ *at* $g(\sigma)$ □

Every interaction between a component and its environment passes through the context, and the type family K is unconstrained, so a system may bind every location it shares across components at a key of its own (Section 3.3.1). A component's effect function is then the lift of a coeffect-mediated one along the coeffect projection, and independence transfers to that lift, whose transformations move the projection alone. The assumption Section 3.1.3 leaves open is met that way, and with it the temporal composability of a whole system of components.

What the decomposition divides is a computation's commuting part from its order-sensitive part. The commuting part is carried by the efects: a component performs them in whatever order its task calls for, and Corollary 21 reverts them in whatever order the system finds convenient, no two components constraining each other. The order-sensitive part is carried by the coefects, since a key whose operations do not commute is one whose order has to be imposed from outside the efects, and two places are available for imposing it. Within one component the accumulator imposes it, reverting in LIFO order whatever the efects (Theorem 16). Across components a declared coeffect imposes it, one component providing what another declares and the provision preceding the declaration's satisfaction (Section 3.2.2). Composability is thereby had at the grain of components rather than of single efects, which is the scale Section 4 works at.

Two limits of the theorem are worth naming. Binding every shared location at a key is the paradigm's discipline and not a property of the construction, so a location the system cannot reify as a coeffect lies outside the boundary of Section 6.1 and outside the theorem with it. And commutativity of a key is a property of the interface that key publishes, so meeting it is an obligation on the component providing the key rather than on the components consuming it.

3.3.3. Situating the Context Paradigm

Programming paradigms difer fundamentally in how they handle side efects. Two established poles define the spectrum:

Explicit state threading (functional). To preserve referential transparency, purely functional languages model side efects as explicit transformations on state. The State monad S → (S, S) [23] threads an environment through every computation. This approach yields strong compositional guarantees: efects are visible in types and amenable to equational reasoning. However, it imposes significant ergonomic costs: every function in the call chain must accept and return the state parameter, even when it merely passes the state through unchanged. As the number

定理 42. 令 $e_1, e_2 \in \mathfrak{E}^{\mathcal{A}}_{\Sigma}$ 且让两者的操作都出现的每个键都是交换的（定义 39）。则 e_1 和 e_2 独立（定义 19）。

证明. 对定义 41 的构造归纳，$\mathfrak{M}(e_i)$ 位于 $\boldsymbol{e}_i$: 中出现的操作的生成元所生成的子幺半群内：单位生成平凡幺半群，而阶段是 $a^{\Sigma}(x)$ 与成员的 $\diamond$-复合，引理 18(2) 适用于它。

对定义 19 的条款 (1)，由引理 18(1) 只需 e_1 中出现的操作的生成元与 e_2 中出现的操作的生成元交换。两个操作位于不同键处时这是定理 [40,] ，位于同一键处时该键携带两者的操作且按假设是交换的。

对条款 (2)，取 $g \in \mathfrak{M}(e_2)$ ，即 e_2 中出现的操作生成元的复合，并对 e_1 的构造归纳。单位在每个状态产生 id_{Σ} 。在一个阶段，令 $(\delta, s, b) = a^{\Sigma}(x)(\sigma)$ 且 $(\varepsilon, t) = e_b(\delta)$ ，则阶段在 σ. 处产生 s $\circ$ t 。一次一个生成元地应用操作的独立性，在 $g(\sigma)$ 处再次产生 s 和 t ，因此选择相同的续延 e_b ，而条款 (1) 把其运行的状态放在 $g(\delta)$ ，归纳假设在那里再次产生 s 。因此阶段在 $g(\sigma)$ 处产生 $s \circ t$. □

组件与其环境的每次交互都穿过上下文，而类型族 K 不受约束，因此系统可以在自己的键处绑定它在组件间共享的每个位置（第 3.3.1 节）。组件的效应函数随后是沿余效应投影的余效应中介效应函数的提升，独立性转移到该提升，其变换只移动该投影。第 3.1.3 节留下的假设就这样被满足，随之而来的是整个组件系统的时间可组合性。

分解所划分的是计算的交换部分与其顺序敏感部分。交换部分由效应承载：组件按其任务要求的任何顺序执行它们，推论 21 按系统发现的任何方便的顺序逆转它们，没有两个组件互相约束。顺序敏感部分由余效应承载，因为操作不交换的键是其顺序必须由效应外部强加的键，而强加它有两个地方可用。在一个组件内由累加器强加，无论效应如何都按 LIFO 顺序逆转（定理 16）。在组件间由声明的余效应强加，一个组件提供另一个声明的东西，且提供先于声明的满足（第 3.2.2 节）。组合由此在组件的粒度上而非单个效应的粒度上达成，这正是第 4 节工作的尺度。

定理的两个限制值得指出。把每个共享位置绑定到键是范式的纪律而非构造的性质，因此系统不能具体化为余效应的位置位于第 6.1 节边界之外、随之也在定理之外。键的交换性是该键发布接口的性质，因此满足它是提供该键的组件的义务而非消费它的组件的义务。

3.3.3. 定位上下文范式（Situating the Context Paradigm）

编程范式在处理副作用的方式上根本不同。两个已确立的极点定义了该谱系：

显式状态传递（函数式）。为保持引用透明性，纯函数式语言把副作用建模为状态上的显式变换。State 单子 S → (S, S) [23] 把环境穿过每次计算。这种方法产生强组合保证：效应在类型中可见且适合等式推理。然而它带来显著的人机工程学代价：调用链中的每个函数都必须接受和返回状态参数，即使它只是原封不动地传递状态。随着效应维度数量的增长（日志、配置、I/O），单子栈或效应处理器样板不断增殖。

of effect dimensions grows (logging, configuration, I/O), monadic stacking or efecthandler boilerplate proliferates.

Implicit mutation (imperative/OOP). Mainstream imperative languages permit components to modify shared state and access dependencies without explicit declaration at the call site. On the effect side, a representative example is React's useEffect hook: it registers a persistent side effect on the component's internal fiber, yet neither the effect target nor the registration mechanism appears as an explicit parameter—identification relies on call-order position within hidden runtime state. On the coeffect side, Java's service locator pattern (e.g., Spring's ApplicationContext.getBean(…)) retrieves dependencies from a process-wide registry at runtime, requiring null checks and type casts at each call site; dependency relationships are implicit and scattered across the codebase. More generally, understanding how f() modifies or depends on the system requires reading its implementation transitively. Refactoring becomes fragile because moving or removing a call may silently break distant invariants.

The context paradigm combines the traceability of the functional approach with the ergonomics of the imperative approach. Effects and coefects are both mediated through an explicit context parameter. Each operation is therefore attributable to the specific context on which it was invoked, and hence to the component that context belongs to.

Beyond combining the strengths of both poles, the context paradigm lets the developer handle each effect and dependency individually and composes them into the system's behavior automatically. For revertible efects, the developer supplies the inverse of each atomic operation, and the inverse of any composite follows by composition (Section 3.1), so a component's teardown is derived from its loading rather than written alongside it. For reactive coefects, a component declares only the dependencies it needs, and the runtime resolves and re-wires them automatically (Section 3.2), keeping them consistently wired as providers are added, removed, or replaced. In both directions, correctness that would otherwise rest on developer discipline becomes a structural property of the paradigm.

隐式变更（命令式/OOP）。主流命令式语言允许组件修改共享状态并在调用点无显式声明地访问依赖。在效应侧，代表例是 React 的 useEffect 钩子：它在组件内部纤维上注册一个持久副作用，但效应目标和注册机制都不作为显式参数出现——识别依赖隐藏运行时状态中调用顺序的位置。在余效应侧，Java 的服务定位器模式（例如 Spring 的 ApplicationContext.getBean(…)）在运行时从进程级注册表取回依赖，要求每个调用点做空检查和类型转换；依赖关系是隐式且分散在整个代码库中的。更一般地说，理解 f() 如何修改或依赖系统需要传递式地阅读其实现。重构变得脆弱，因为移动或移除调用可能静默破坏远处的依赖。

上下文范式结合了函数式方法的可追溯性与命令式方法的人机工程学。效应与余效应都通过显式上下文参数被中介。每个操作因此可归因于它被调用的特定上下文，进而归因于该上下文所属的组件。

除了结合两个极点的优势，上下文范式还让开发者单独处理每个效应和依赖，并自动把它们组合成系统的行为。对可逆效应，开发者提供每个原子操作的逆，而任何复合的逆通过组合而得到（第 3.1 节），因此组件的拆卸由加载推导而来而非与之并列书写。对响应式余效应，组件只声明它需要的依赖，运行时自动解析和重连它们（第 3.2 节），在提供者被添加、移除或替换时保持它们一致接线。在两个方向上，原本依赖开发者纪律的正确性成为范式的结构性质。

4. A Calculus of Dynamic Composition

Section 3 establishes spatial and temporal composability in their local form alone. Carrying them to a whole system takes a decomposition of the system into components, each pairing a coeffect specification with a witnessed effect function, so that every interaction with the shared environment is attributable to one of them. The sections below give that decomposition an operational semantics, and establishes spatial and temporal composability in their global form.

Section 4.1 and Section 4.2 present the smallest calculus in which the lifecycle can be given rules, one that takes each transition to be atomic, immediate, and infallible; Section 4.3 drops the three assumptions, atomicity once for each direction a transition may run in, admitting the forms of control flow a runtime interposes between the start of a transition and its end, and arrives at the calculus a real runtime implements; and Section 4.4 establishes the metatheory of that calculus, namely preservation, global temporal and spatial composability, progress, and confluence.

4. 动态组合演算（A Calculus of Dynamic Composition）

第 3 节仅在局部形式确立了空间与时间可组合性。把它们推广到整个系统需要把系统分解为组件，每个组件把一个余效应规范与一个带见证的效应函数配对，从而使与共享环境的每次交互都可归因于其中一个。以下各节给这种分解配以操作语义，并确立其全局形式的空间与时间可组合性。

第 4.1 节和第 4.2 节给出能赋予生命周期规则的最小演算，它把每个转移视为原子的、即时的且不会失败的；第 4.3 节放弃这三个假设，原子性按转移可能运行的方向各抛弃一次，容纳运行时在转移开始与结束之间插入的控制流形式，得到真实运行时实现的演算；第 4.4 节确立该演算的元理论，即保真、全局时间与空间可组合性、进展性和合流性。

4.1. Components and Fibers

This section fixes the objects the rules act on: the component; the fiber, an instantiation of a component carrying a lifecycle state of its own; and the registry, which holds the fibers a state carries and from which the coeffect context is read of.

Components. A component is given as a triple, its coeffect side split into what it reads from the environment

4.1. 组件与纤维（Components and Fibers）

本节固定规则作用的对象：组件；纤维，即携带自身生命周期状态的组件实例化；以及注册表（registry），它承载状态携带的纤维，余效应上下文从中读出。

组件。组件以三元组给出，其余效应侧被拆分为它从环境读取的东西与它向环境提供的东西。

and what it provides to it.

Definition 43. A component over a context Γ carrying both efects and coefects (Definition 32) is defined as:

$$\mathfrak{C}_\Gamma := \mathfrak{D}_\Gamma \times \mathfrak{P}_\Gamma \times \mathfrak{E}^*_\Gamma \tag{37}$$

representing a triple (d, p, e) , where:

$d : \mathfrak{D}_\Gamma$ is the coeffect specification of Definition 25, declaring the dependencies required from the environment;

$p : \mathfrak{P}_\Gamma := \mathsf{Set}(K)$ is the provision, declaring the coeffect keys the component may provide, and no key outside p is one its effect function writes;

$e : \mathfrak{E}^*_\Gamma$ is the witnessed effect function of Definition 8, defining the efects contributed when the component is active together with the inverse that withdraws them.

The two declarations are the two directions of one interface, d what the component reads from the environment and p what the component writes to the environment, and Section 4.2 admits no two fibers of one registry whose provisions meet. Subscripts are taken on Γ throughout, the coeffect context being one of its projections (Definition 32), so the $\mathfrak{D}_\Sigma$ of Definition 25 is written $\mathfrak{D}_\Gamma$ here.

Disjointness of provisions is where this chapter parts company with Section 3.2.3. The isolation of Definition 28 lets one key resolve through a realm table, so that two fibers may provide the same key in diferent realms; a calculus carrying realms would relax disjointness to disjointness within a realm and would resolve a declared key against the realm of the fiber declaring it. We do not introduce realms here, and read every key at one shared realm instead, which is what makes the disjointness above the right condition and each key's provider unique (Definition 45). What it restricts is how often a component may be instantiated: one with a nonempty provision has one fiber at a time, so the many instantiations below are of components providing nothing, which is the common case of a component that only consumes, or that registers others.

A component instantiated in a running system is activated and deactivated over time, so it carries a lifecycle state, and a transition is what moves it from one lifecycle state to another: an activation executes e, accumulating side efects on the context, and a deactivation applies the accumulator to recover the context. In its simplest form the lifecycle is the two-state model of Figure 1, which Section 4.2 gives rules for; Section 4.3 refines it as each control-flow feature is admitted.

Figure 1 | Base component lifecycle

Fibers. One component may be instantiated many times over, each instantiation carrying a lifecycle state of its own. We name such an instantiation a fiber. A fiber records the component that produced it, the fiber it was instantiated under, the coefects it provides, and where in its lifecycle it stands.

Definition 44. Fix a set N of fiber names. A fiber instantiating the component $(d, p, e) \in \mathfrak{C}_\Gamma$ is a tuple $\langle d, p, e, \pi, \sigma, \tau, \theta \rangle$, where

$d : \mathfrak{D}_\Gamma, p : \mathfrak{P}_\Gamma$, and $e : \mathfrak{E}^*_\Gamma$ are the coeffect specification, provision, and effect function of Definition 43;

- π : N $\cup$ {root} is the parent, the fiber this one was instantiated under, or the root marker root;
- σ : Σ is the fiber's own coeffect table (Definition 22), empty until it activates and written by its efects as they run;

$\tau : \{\bot, \top\}$ is the retirement flag, $\bot$ in a fresh fiber and $\top$ once the orchestrator has retired the fiber;

定义 43. 上下文 Γ 上的组件（同时携带效应与余效应，定义 32）定义为：

$$\mathfrak{C}_\Gamma := \mathfrak{D}_\Gamma \times \mathfrak{P}_\Gamma \times \mathfrak{E}^*_\Gamma \tag{37}$$

表示三元组 (d, p, e) ，其中：

$d : \mathfrak{D}_\Gamma$ 是定义 25 的余效应规范，声明从环境要求的依赖；

$p : \mathfrak{P}_\Gamma := \mathsf{Set}(K)$ 是提供（provision），声明组件可能提供的余效应键，且 p 之外的键不是其效应函数写入的键；

$e : \mathfrak{E}^*_\Gamma$ 是定义 8 的带见证效应函数，定义组件激活时贡献的效应以及撤出它们的逆。

两个声明是一个接口的两个方向，d 是组件从环境读取的东西，p 是组件向环境写入的东西，而第 4.2 节不允许一个注册表中两个提供相交的纤维。下标在全文对 Γ 取，余效应上下文是其投影之一（定义 32），因此定义 25 的 $\mathfrak{D}_\Sigma$ 此处写作 $\mathfrak{D}_\Gamma$。

提供的两两不相交是本章与第 3.2.3 节分手之处。定义 28 的隔离让一个键穿过领域表解析，因此两个纤维可以在不同领域中提供同一个键；携带领域的演算会把不相交性放松为领域内的不相交性，并会针对声明纤维的领域解析声明键。我们不在此引入领域，而是在一个共享领域读出每个键，这使上述不相交性是正确条件、每个键的提供者唯一（定义 45）。它限制的是一个组件被实例化的频率：提供非空的组件每次只有一个纤维，因此下面的多次实例化都是不提供任何东西的组件——这是只消费或注册其他组件的组件的常见情形。

运行系统中实例化的组件会随时间被激活和停用，因此它携带生命周期状态，而转移是把它从一个生命周期状态移动到另一个：激活执行 e, 在上下文中累积副作用，停用应用累加器以恢复上下文。在最简形式下，生命周期是图 1 的两状态模型，第 4.2 节为其给出规则；第 4.3 节在容纳每种控制流特性时对其精化。

图 1 | 基础组件生命周期

纤维。一个组件可以被实例化多次，每次实例化携带自己的生命周期状态。我们把这样的实例化命名为纤维。纤维记录产生它的组件、它被实例化所在的纤维、它提供的余效应，以及它处于生命周期的何处。

定义 44. 固定纤维名集合 N。实例化组件 $(d, p, e) \in \mathfrak{C}_\Gamma$ 的纤维是元组 $\langle d, p, e, \pi, \sigma, \tau, \theta \rangle$ ，其中

$d : \mathfrak{D}_\Gamma, p : \mathfrak{P}_\Gamma$ 和 $e : \mathfrak{E}^*_\Gamma$ 是定义 43 的余效应规范、提供和效应函数；

- π : N $\cup$ {root} 是父纤维，即此纤维被实例化所在的纤维，或根标记 root；
- σ : Σ 是纤维自身的余效应表（定义 22），激活前为空，由运行的效应写入；

$\tau : \{\bot, \top\}$ 是退役标志，新纤维中为 $\bot$，编排器已退役纤维后为 $\top$；

$\theta : \Theta_\Gamma$ is the lifecycle state, which in the two-state model of Section 4.2 is

$$\Theta_\Gamma := \text{ Inactive } \mid \text{ Active } (g, \omega) \tag{38}$$

where $g : \Gamma \to \Gamma$ is the accumulator and $\omega :$ d $\to$ N the committed view.

The committed view ω sends each key the fiber declares to the name of the fiber that provided it when the transition committed. Section 4.3 replaces Θ_Γ by the extension that transitions in progress require; the rest of Definition 44 is given once for both, save that ω is read at the richer effect type each layer of Section 4.3 introduces.

Registry. A state holds its fibers under their names, and both the identity of a fiber and the coeffect context of Section 3.2 are read of that arrangement.

Definition 45. Write $\mathfrak{F}_\Gamma$ for the set of fibers over Γ. A state $\gamma \in \Gamma$ carries a registry

$$F_\gamma : \mathfrak{N} \rightharpoonup \mathfrak{F}_\Gamma \tag{39}$$

a finite partial function whose parent pointers form a tree rooted at root, together with whatever else in Γ no fiber's σ names. We write $\gamma(n)$ for $F_\gamma(n)$, and abbreviate a field of $\gamma(n)$ by subscripting it with n where the state is clear, so that $d_n, p_n, e_n, \pi_n, \sigma_n, \tau_n, \theta_n$ are the fields of Definition 44 and g_n, ω_n the accumulator and committed view that θ_n carries; $\gamma[\theta_n \mapsto \theta']$, $\gamma[\mathrm{n} \mapsto \langle \cdots \rangle]$, and $\gamma \setminus n$ are the states difering from γ in one field, one fiber, and the presence of one fiber respectively.

A fiber's name is what gives it an identity that survives its own mutation: every rule below rewrites the lifecycle state of one fiber and leaves the others alone, so the rule has to say which one, and two fields refer to fibers rather than describe them, the parent π and the committed view ω. Names are atoms: no rule computes one, inspects its structure, or relates two of them by anything but equality, and introducing a fiber simply draws one not already in use. This is the discipline of dynamically created local names [39], used here for fiber identity.

Each fiber owning a table means the coeffect context is derived rather than stored: it is what the active fibers jointly provide.

$$\sigma_\gamma := \bigcup \{\sigma_m \mid m \in \mathrm{dom}\,(F_\gamma)\,, \theta_m = \text{ Active } (-,-)\} \tag{40}$$

The union is well defined because a fiber writes only the keys it declares, dom $(\sigma_n) \subseteq p_n$. and the provisions of distinct fibers are disjoint (Definition 43), so each $k \in \mathrm{dom}(\sigma_\gamma)$ lies in the table of exactly one Active fiber, whose name we write provide $_{\mathsf{k}}(\gamma) \in \mathfrak{N}$ and call the provider of k. Each key therefore has one possible provider, fixed by the provisions and not by the state. No rule writes σ_n directly: a fiber's provisions are the set operations its own effect function performs, which land in σ_n and so are already part of the state e_n returns, and they leave again with the accumulator. Only the coeffect part of an effect is recorded this way, because only the coeffect part is what other fibers declare against; efects that mutate state elsewhere in γ are tracked by e like any other, but no fiber can name them in a specification, so they contribute no ordering constraint.

The satisfaction relation of Section 3.2.2 then applies unchanged, with $\gamma \models d$ abbreviating $\sigma_\gamma \vDash$ d. A key lies in dom (σ_γ) exactly when some Active fiber has installed it, its provision being the keys it may install rather than the ones it has, so $\gamma^{\,\mathsf{k}} \vDash$ d already requires that every declared key have an Active provider. Taking the union over Active fibers alone is what lets a fiber cease to provide before it has withdrawn anything, which Section 4.3.1 turns into the ordering discipline.

4.2. The Base Calculus

$\theta : \Theta_\Gamma$ 是生命周期状态，在第 4.2 节的两状态模型下为

$$\Theta_\Gamma := \text{ Inactive } \mid \text{ Active } (g, \omega) \tag{38}$$

其中 $g : \Gamma \to \Gamma$ 是累加器，$\omega :$ d $\to$ N 是已提交视图（committed view）。

已提交视图 ω 把纤维声明的每个键发送到转移提交时提供它的纤维的名字。第 4.3 节用进行中转移所需的扩展替换 Θ_Γ ；定义 44 的其余部分对两者一次性给出，仅 ω 在第 4.3 节各层引入的更丰富效应类型处读出。

注册表。状态在名字下持有其纤维，而纤维的身份与第 3.2 节的余效应上下文都从该安排中读出。

定义 45. 写 $\mathfrak{F}_\Gamma$ 为 Γ 上纤维的集合。状态 $\gamma \in \Gamma$ 携带注册表

$$F_\gamma : \mathfrak{N} \rightharpoonup \mathfrak{F}_\Gamma \tag{39}$$

一个有限部分函数，其父指针形成以 root 为根的树，连同 Γ 中任何纤维的 σ 都不命名的其余部分。我们写 $\gamma(n)$ 表示 $F_\gamma(n)$ ，并在状态清晰时用 n 下标缩写 $\gamma(n)$ 的字段，因此 $d_n, p_n, e_n, \pi_n, \sigma_n, \tau_n, \theta_n$ 是定义 44 的字段，g_n, ω_n 是 θ_n 携带的累加器和已提交视图；$\gamma[\theta_n \mapsto \theta']$ 、$\gamma[\mathrm{n} \mapsto \langle \cdots \rangle]$ 和 $\gamma \setminus n$ 分别是与 γ 在一个字段、一个纤维和有一个纤维在场这三方面不同的状态。

纤维的名字赋予它一个在其自身变更中存活的身份：下面的每个规则都重写一个纤维的生命周期状态而让其他纤维独处，因此规则必须说明是哪一个，而两个字段引用纤维而非描述它们，即父 π 和已提交视图 ω。名字是原子：没有规则计算一个名字、检查其结构或用相等性以外的任何方式关联两个名字，引入纤维只是抽取一个未使用的名字。这是动态创建的局部名字的纪律 [39]，此处用于纤维身份。

每个纤维拥有一个表意味着余效应上下文是派生的而非存储的：它是活动纤维共同提供的东西。

$$\sigma_\gamma := \bigcup \{\sigma_m \mid m \in \mathrm{dom}\,(F_\gamma)\,, \theta_m = \text{ Active } (-,-)\} \tag{40}$$

该并集良定义，因为纤维只写它声明的键，dom $(\sigma_n) \subseteq p_n$. 且不同纤维的提供不相交（定义 43），因此每个 $k \in \mathrm{dom}(\sigma_\gamma)$ 恰好位于一个 Active 纤维的表中，其名字我们写为 provide $\Phi_k(\gamma) \in \mathfrak{N}$ 并称其为 k 的提供者。每个键因此只有一个可能的提供者，由提供决定而非由状态决定。没有规则直接写 σ_n ：纤维的提供是它自己的效应函数执行的集合运算，这些运算落在 σ_n 中因此已经是 e_n 返回状态的一部分，并随累加器一起离开。只有效应的余效应部分以此方式记录，因为只有余效应部分是其他纤维所声明的；在 γ 的其他地方改变状态的效应像任何其他效应一样由 e 跟踪，但没有纤维能在规范中命名它们，因此它们不贡献排序约束。

第 3.2.2 节的满足关系随后原样适用，以 $\gamma \models d$ 缩写 $\sigma_\gamma \vDash$ d。键位于 dom (σ_γ) 当且仅当某个 Active 纤维安装了它，其提供是它可以安装的键而非已安装的键，因此 $\gamma^{\,\mathsf{k}} \vDash$ d 已经要求每个声明键有 Active 提供者。只对 Active 纤维取并集使纤维能在未撤出任何东西前停止提供，第 4.3.1 节把这变成排序纪律。

4.2. 基础演算（The Base Calculus）

This section gives the calculus of the two-state lifecycle of Figure 1 and nothing more: the target each fiber is compared against, and the five rules that move it.

Target views. The rules compare each fiber against a target, namely whether it ought to be running and against which resolution of its dependencies. The target is not a property of the fiber alone, since the keys a fiber declares are resolved against the whole state, so it is a predicate on that state.

Definition 46. The target view of n at γ maps each declared key to its provider, so it is a total map $d_n \to \mathfrak{N}$, and is $\bot$ when n ought not to be running at all:

$$\mathrm{target}_n(\gamma) := \begin{cases} \bot & \text{if} \tau_n \vee \neg(\gamma \vDash d_n) \\ (k \in d_n) \mapsto \mathrm{provider}_k(\gamma) & \text{otherwise} \end{cases} \tag{41}$$

A state is quiescent when every fiber has reached its target view:

$$\mathrm{quiet}(\gamma) := \forall n \in \mathrm{dom}\big(F_\gamma\big). \begin{cases} \mathrm{target}_n(\gamma) = \bot & \text{if} \theta_n = \mathsf{Inactive} \\ \mathrm{target}_n(\gamma) = \omega_n & \text{if} \theta_n = \mathsf{Active}(-, \omega_n) \end{cases} \tag{42}$$

The target answers to two things and to nothing else: retirement, through τ_n, , and coeffect resolution, through $\gamma \vDash d_n$ and provide r_k, each declared key being read of σ_γ at the one shared realm of Definition 43.

The committed view of Definition 44 has the same type as the target view, and the lifecycle is driven by comparing them: ω_n is the resolution n activated against, $\mathrm{target}_n(\gamma)$ the one it should be running against, and every rule below fires on their agreeing or difering. Recording a provider rather than a value is what makes the comparison usable, since a diferent fiber providing an equal value would otherwise compare equal. The value a component reads is reached through the view, since the provider's table holds that value, and the implementation holds the map in fiber.committed and a hash of it in fiber.target (Section 5.1.3).

Rules. The base calculus takes each transition to be atomic, immediate, and infallible: an activation applies its effect function in one step, a deactivation applies the accumulator in one step, and both succeed in doing so. Section 4.3 drops all three.

Five rules generate two relations. An orchestration rule, prefixed O- and written $\gamma \Rightarrow \delta$, is an action the orchestrator may perform; its premises say when the action is legal, not when it occurs. A lifecycle rule, prefixed L- and written $\gamma \longrightarrow \delta$, is a step the system takes unprompted whenever its premises hold. A sequence of steps interleaves the two, and $*$ below means lifecycle steps alone.

$$\frac{n \notin \mathrm{dom}(F_\gamma) \quad \pi \in \mathrm{dom}(F_\gamma) \cup \{\mathrm{root}\} \quad (d,p,e) \in \mathfrak{C}_\Gamma \quad \forall m \in \mathrm{dom}(F_\gamma).\, p \cap p_m = \varnothing}{\gamma \Rightarrow \gamma[n \mapsto \langle d,p,e,\pi,\varnothing,\bot,\mathrm{Inactive}\rangle]}\,\text{O - Insert}$$

$$\frac{n \in \mathrm{dom}(F_\gamma)}{\gamma \Rightarrow \gamma[\tau_n \mapsto \top]}\,\text{O - Retire}$$

$$\frac{\tau_n = \top \quad \theta_n = \mathrm{Inactive} \quad \forall m.\, \pi_m \neq n}{\gamma \Rightarrow \gamma \setminus n}\,\text{O - Remove}$$

Insertion and retirement are the only external inputs: the orchestrator asks for a fiber to exist or to stop existing, and never sets its lifecycle state directly. O-Retire is unconditional on the fiber's state because retiring is a request, and the lifecycle rules are what carry it out. Retirement is separated from removal for the same reason: a retired fiber that is still Active must first be deactivated, and removing it earlier would discard the accumulator and leak. The premise $\forall$m. $\pi_m \neq$ n keeps the tree well-formed by removing children before their parent. The last premise of O-Insert is where the single-source discipline is imposed: a key has one possible provider because the orchestrator may not admit a second component declaring it.

$$\frac{\theta_n = \mathrm{Inactive} \quad \omega = \mathrm{target}_n(\gamma) \neq \bot \quad e_n(\gamma) = (\delta, g)}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathrm{Active}(g,\omega)]}\,\text{L - Reload}$$

$$\frac{\theta_n = \mathrm{Active}(g,\omega) \quad \mathrm{target}_n(\gamma) \neq \omega \quad g(\gamma) = \delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathrm{Inactive}]}\,\text{L -Unload}$$

本节只给出图 1 的两状态生命周期的演算：每个纤维与之比较的目标，以及移动它的五个规则。

目标视图。规则把每个纤维与目标比较，即它是否应当运行以及针对其依赖的哪个解析。目标不是纤维单独的性质，因为纤维声明的键针对整个状态解析，所以它是该状态上的谓词。

定义 46. n 在 γ 处的目标视图把每个声明键映射到其提供者，因此是总映射 $d_n \to \mathfrak{N}$ ，当 n 根本不该运行时为 $\bot$：

$$\mathrm{target}_n(\gamma) := \begin{cases} \bot & \text{if} \tau_n \vee \neg(\gamma \vDash d_n) \\ (k \in d_n) \mapsto \mathrm{provider}_k(\gamma) & \text{otherwise} \end{cases} \tag{41}$$

当每个纤维都达到其目标视图时，状态是静止的（quiescent）：

$$\mathrm{quiet}(\gamma) := \forall n \in \mathrm{dom}\big(F_\gamma\big). \begin{cases} \mathrm{target}_n(\gamma) = \bot & \text{if} \theta_n = \mathsf{Inactive} \\ \mathrm{target}_n(\gamma) = \omega_n & \text{if} \theta_n = \mathsf{Active}(-, \omega_n) \end{cases} \tag{42}$$

目标只应答两件事而不应答别的：通过 τ_n, 退役，以及通过 $\gamma \vDash d_n$ 和 provide r_k, 余效应解析，每个声明键在定义 43 的一个共享领域处从 σ_γ 读出。

定义 44 的已提交视图与目标视图同类型，而生命周期由比较它们驱动：ω_n 是 n 针对其激活的解析，$\mathrm{target}_n(\gamma)$ 是它应针对其运行的解析，下面的每个规则都因它们一致或不一致而触发。记录提供者而非值是使比较可用之处，因为提供相等值的不同纤维否则会比较相等。组件读取的值通过视图达到，因为提供者的表持有该值，而实现把该映射保存在 fiber.committed 中、把其哈希保存在 fiber.target 中（第 5.1.3 节）。

规则。基础演算把每个转移视为原子的、即时的且不会失败的：激活一步应用其效应函数，停用一步应用累加器，两者都成功。第 4.3 节放弃全部三个。

五个规则生成两个关系。编排规则，前缀 O-，写作 $\gamma \Rightarrow \delta$, 是编排器可执行的动作；其前提说明动作何时合法，而非何时发生。生命周期规则，前缀 L-，写作 $\gamma \longrightarrow \delta$, 是系统在其前提成立时主动采取的步骤。步骤序列交错这两者，下面的 $*$ 仅指生命周期步骤。

$$\frac{n \notin \mathrm{dom}(F_\gamma) \quad \pi \in \mathrm{dom}(F_\gamma) \cup \{\mathrm{root}\} \quad (d,p,e) \in \mathfrak{C}_\Gamma \quad \forall m \in \mathrm{dom}(F_\gamma).\, p \cap p_m = \varnothing}{\gamma \Rightarrow \gamma[n \mapsto \langle d,p,e,\pi,\varnothing,\bot,\mathrm{Inactive}\rangle]}\,\text{O - Insert}$$

$$\frac{n \in \mathrm{dom}(F_\gamma)}{\gamma \Rightarrow \gamma[\tau_n \mapsto \top]}\,\text{O - Retire}$$

$$\frac{\tau_n = \top \quad \theta_n = \mathrm{Inactive} \quad \forall m.\, \pi_m \neq n}{\gamma \Rightarrow \gamma \setminus n}\,\text{O - Remove}$$

插入和退役是仅有的外部输入：编排器要求一个纤维存在或停止存在，绝不直接设置其生命周期状态。O-Retire 对纤维状态无条件，因为退役是一个请求，而生命周期规则负责执行它。退役与移除分离出于同样原因：仍 Active 的退役纤维必须首先被停用，过早移除会丢弃累加器并泄漏。前提 $\forall$m. $\pi_m \neq$ n 通过在移除其父之前移除子节点来保持树良构。O-Insert 的最后一个前提是施加单一来源纪律之处：一个键只有一个可能的提供者，因为编排器不能接纳声明它的第二个组件。

$$\frac{\theta_n = \mathrm{Inactive} \quad \omega = \mathrm{target}_n(\gamma) \neq \bot \quad e_n(\gamma) = (\delta, g)}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathrm{Active}(g,\omega)]}\,\text{L - Reload}$$

$$\frac{\theta_n = \mathrm{Active}(g,\omega) \quad \mathrm{target}_n(\gamma) \neq \omega \quad g(\gamma) = \delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathrm{Inactive}]}\,\text{L -Unload}$$

L-Reload installs the committed view alongside the inverse; L-Unload applies the inverse and discards the committed view. Both are driven by the same comparison: L-Reload fires when a fiber holds no committed view and its target view is not $\perp$, L-Unload when the committed view it holds is not its target view. This is the reactive discipline of Section 3.2, read of a target that answers to retirement as well as to the coefects: a transition is initiated whenever the target view changes, regardless of which of the two moved it.

Instantiation. A component may instantiate another while installing its efects, which is what a plugin host does when a plugin loads plugins of its own. The rules so far leave the registry to the orchestration rules alone, so such an instantiation has nowhere to happen. One primitive gives it somewhere.

Definition 47. An application of $\boldsymbol{e}_n$, or one of its iterations where Section 4.3.2 applies, may register a component $(d, p, e) \in \mathfrak{C}_\Gamma$. In place of a state map it takes the O-Insert of that component with $\pi = n$, and it yields as its inverse the O-Retire of the fiber so registered. The rule draws the name, subject to the freshness premise of O-Insert, and hands it to the effect function.

The inverse retires rather than removes, and the reason is that an inverse has to apply wherever it is reached. O-Remove carries premises, so an inverse built from it can fail to: a parent whose child is still Active could not run its accumulator, and no rule would move the child, since Definition 46 does not read the fiber tree. O-Retire has $n \in \mathrm{dom}\,(F_\gamma)$ as its only premise. The entry it leaves behind at the state the registration was taken is retired, Inactive($\perp$), and holds an empty table, which is the vestigial entry of Lemma 57: it difers from the absence of the fiber in control fields alone, and no rule tells the two apart.

Retiring a child sets τ and so takes its target view to $\perp$, after which the ordinary rules carry it back to Inactive. The parent is not made to wait, O-Retire being unconditional, so L-Unload applies to the parent whether or not the child has left. A grandchild is reached one level at a time, the child's own accumulator retiring what the child registered. Theorem 66 covers this cascade and the one Section 4.3.1 imposes along coefects together.

Confinement. With the one exception in hand, the discipline an effect function is held to can be given. It bounds what an application writes, so that the rule applying it accounts for every other change, and what an application reads, so that a fiber sees the coefects it declared and no more of the registry. Bounding the writes is what lets Section 4.4 read Table 1 as a complete inventory of them.

Definition 48. A map $f : \Gamma \to \Gamma$ is confined to n when for every $\gamma \in \Gamma$ with $n \in \mathrm{dom}\,(F_\gamma)$, writing $\delta = f(\gamma)$ 11

1. (Writes.) $\mathrm{dom}\,(F_\delta) = \mathrm{dom}\big(F_\gamma\big), \delta(m) = \gamma(m)$ for every $m \in \mathrm{dom}\big(F_\gamma\big)$ with $\mathrm{m} \neq n$, , and $\delta(n)$ and $\gamma(n)$ difer in θ alone;

2. (Reads.) two states agreeing on σ_n, on the restrictions $\sigma_m|_{d_n}$ for every $m \in \mathrm{dom}\big(F_\gamma\big)$, and on the part of the state that no fiber's table names are carried by f to states agreeing on the same three.

An effect function e is confined to n when every application of it, and of each of its iterations where Section 4.3.2 applies, either registers a component (Definition 47) or has both its state map $\mathrm{pr}_1 \circ e$ and the inverse it yields confined to n. Every fiber's effect function is required to be confined to that fiber.

A registration writes the entry O-Insert writes, at the one name it draws, and nothing else; the O-Retire it yields as its inverse writes the θ of that name and nothing else. An application of either kind therefore writes no control field of a fiber already present, save that one θ, and reads none at all.

Clause (2) is why a component may read the values it declared: those lie in the tables of its providers, so an effect function that reads no table but σ_n would be unable to use its own coefects. What it may not read is a

L-Reload 把已提交视图与逆一起安装；L-Unload 应用逆并丢弃已提交视图。两者由同一比较驱动：L-Reload 在纤维不持有已提交视图且其目标视图非 $\perp$ 时触发，L-Unload 在它持有的已提交视图非其目标视图时触发。这是第 3.2 节的响应式纪律，针对既应答退役也应答余效应的目标读出：每当目标视图改变时启动转移，无论两者中哪一个移动了它。

实例化。组件在安装其效应时可以实例化另一个组件，这正是插件宿主在插件加载自己的插件时所做的。到目前为止的规则把注册表留给编排规则单独处理，因此这样的实例化无处发生。一个原语给了它地方。

定义 47. $\boldsymbol{e}_n$ 的应用，或第 4.3.2 节适用处其迭代之一，可以注册组件 $(d, p, e) \in \mathfrak{C}_\Gamma$ 。它代替状态映射取该组件以 $\pi = n$ 的 O-Insert，并把该纤维的 O-Retire 作为其逆产出。规则抽取名字（受 O-Insert 的新鲜性前提约束）并交给效应函数。

逆退役而非移除，原因是逆必须能在其到达的任何地方应用。O-Remove 携带前提，因此由它构建的逆可能失败：父仍 Active 的子组件无法运行其累加器，且没有规则会移动该子组件，因为定义 46 不读纤维树。O-Retire 唯一的前提是 $n \in \mathrm{dom}\,(F_\gamma)$ 。它在注册被提取的状态留下的条目是已退役的，Inactive($\perp$)，并持有空表，即引理 57: 的残余条目（vestigial entry）：它仅在控制字段上不同于纤维的缺席，且没有规则能区分两者。

退役子组件设置 τ 从而使其目标视图变为 $\perp$，此后普通规则把它带回 Inactive。父组件不被要求等待，O-Retire 无条件，因此 L-Unload 无论子组件是否离开都适用于父组件。孙组件一次一级到达，子组件自己的累加器退役子组件注册的东西。定理 66 连同第 4.3.1 节沿余效应施加的级联一起覆盖该级联。

约束（Confinement）。手握这一个例外，效应函数所受的纪律可以给出。它限制一次应用写什么，使应用它的规则能说明每次其他变化，并限制一次应用读什么，使纤维看到它声明的余效应而不多看到注册表。限制写入是第 4.4 节把表 1 读作它们的完整清单的依据。

定义 48. 映射 $f : \Gamma \to \Gamma$ 对 n 是约束的，当对每个满足 $n \in \mathrm{dom}\,(F_\gamma)$ 的 $\gamma \in \Gamma$ ，写作 $\delta = f(\gamma)$ 11

1. （写入）$\mathrm{dom}\,(F_\delta) = \mathrm{dom}\big(F_\gamma\big), \delta(m) = \gamma(m)$ 对每个满足 $\mathrm{m} \neq n$, 的 $m \in \mathrm{dom}\big(F_\gamma\big)$ ，且 $\delta(n)$ 和 $\gamma(n)$ 仅 θ 不同；

2. （读取）在 σ_n, 上、在对每个 $m \in \mathrm{dom}\big(F_\gamma\big)$ 的限制 $\sigma_m|_{d_n}$ 上、以及在无纤维表命名的状态部分上一致的两个状态被 f 带到在同样三样上一致的状态。

效应函数 e 对 n 是约束的，当它的每次应用、以及第 4.3.2 节适用处它的每次迭代，要么注册一个组件（定义 47），要么其状态映射 $\mathrm{pr}_1 \circ e$ 和它产生的逆都对 n 约束。每个纤维的效应函数被要求对该纤维约束。

注册写入 O-Insert 写入的条目，在它抽取的唯一位名字处，别无其他；它作为逆产出的 O-Retire 只写该名字的 θ，别无其他。两种应用因此都不写已在场纤维的控制字段（除该 θ 外），且完全不读取。

条款 (2) 是组件能读取其声明值的原因：那些值位于其提供者的表中，因此不读任何表（除 σ_n 外）的效应函数将无法使用自己的余效应。它不能读的是 d_n, 之外的表或任何控制字段，这使组件不能分支于它未声明的纤维

table outside d_n, or any control field, which is what keeps a component from branching on the lifecycle state of a fiber it did not declare.

The rules are nondeterministic: several fibers may hold a committed view difering from their target view, and the relation commits to no order among them. They are also reactive only, in that no rule mentions a scheduler; the steps are any sequence of rule applications, so a theorem proved over all such sequences holds for every scheduling policy a runtime might adopt.

4.3. Transitions in Progress

This section extends the base calculus in four settings. The first supplies something Section 3.2 requires and Section 4.2 cannot express, a deactivation spread over an interval its dependents may occupy; the other three drop the idealization that a transition is atomic, immediate, and infallible, none of which a transition in a real runtime is. What is dropped is that a whole transition is one step, not that a step is one application of one rule, and the four share one structural consequence, taken here once: a transition that is not a step needs a state to occupy while it is under way, one for each direction it may run in.

Definition 49. The lifecycle states of this section replace Θ_Γ by

$$\Theta_\Gamma := \text{ Inactive } (\zeta) \mid \text{ Reloading } (i, g, \omega) \mid \text{ Active } (g, \omega) \mid \text{ Unloading } (g, \omega, \zeta) \tag{43}$$

where $i : \mathfrak{E}_\Gamma^{\text{iter}*}$ is the remaining effect iterator (Definition 51 below), $g : \Gamma \to \Gamma$ the accumulator built so far, $\omega : d\mathfrak{N}$ the committed view, and $\zeta : \{\bot\} \cup \Xi$ the outcome, carried by Unloading as the one its deactivation is headed for and by Inactive as the one it reached, either $\bot$ or an error drawn from the set Ξ of errors that Section 4.3.4 supplies.

A fiber is installed when it is in one of the three states carrying an accumulator and a committed view, and failed when it carries an error outcome:

$$\text{installed}_n(\gamma) := \theta_n \neq \text{Inactive}(-), \qquad \text{failed }_n(\gamma) := \exists \xi \in \Xi . \theta_n = \text{Inactive}(\xi) \tag{44}$$

An installed fiber n resolves k to m when $\omega_n(k) = m.$. The quiescence of Definition 46 is read on the wider state space as

$$\text{quiet}(\gamma) := \forall n \in \text{dom}\big(F_\gamma\big). \begin{cases} \zeta \neq \bot \vee \text{target}_n(\gamma) = \bot & \text{if} \theta_n = \text{Inactive}(\zeta) \\ \text{target}_n(\gamma) = \omega_n & \text{if} \theta_n = \text{Active}(-, \omega_n) \\ \bot & \text{otherwise} \end{cases} \tag{45}$$

The definitions of Section 4.1 carry over to this state space, with two readings to fix. First, the Inactive of Section 4.2 is read as Inactive($\bot$) in the conclusion of O-Insert and as Inactive $(-)$ in the premise of O-Remove. Second, σ_γ still unions the tables of Active fibers alone, so a fiber whose transition is under way in either direction reads its coefects through the ω it holds and provides none of its own; a key that its transition has already written is therefore not yet one a dependent may activate against. In the two-state calculus the distinction is empty, every installed fiber being Active there.

Figure 2 draws the lifecycle these states form, and the four subsections below supply the rules on its edges.

的生命周期状态。

规则是非确定性的：多个纤维可能持有与其目标视图不同的已提交视图，而该联系不承诺它们之间的顺序。它们也只具响应性，没有规则提及调度器；步骤是任意规则应用序列，因此在所有此类序列上证明的定理对运行时可能采用的每个调度策略都成立。

4.3. 进行中的转移（Transitions in Progress）

本节在四种设定下扩展基础演算。第一种提供第 3.2 节要求而第 4.2 节无法表达的东西——一个其依赖方可占用的区间上展开的停用；其余三种放弃转移是原子的、即时的且不会失败这一理想化，真实运行时中的转移三者皆非。被放弃的是整个转移是一步，而非一步是一次规则应用，而四种共享一个结构性后果，在此一次性处理：非一步的转移需要在其进行期间占据一个状态，每个可能运行的方向一个。

定义 49. 本节的生命周期状态把 Θ_Γ 替换为

$$\Theta_\Gamma := \text{ Inactive } (\zeta) \mid \text{ Reloading } (i, g, \omega) \mid \text{ Active } (g, \omega) \mid \text{ Unloading } (g, \omega, \zeta) \tag{43}$$

其中 $i : \mathfrak{E}_\Gamma^{\text{iter}*}$ 是剩余效应迭代器（下面的定义 51），$g : \Gamma \to \Gamma$ 是迄今构建的累加器，$\omega : d\mathfrak{N}$ 是已提交视图，$\zeta : \{\bot\} \cup \Xi$ 是结果，由 Unloading 携带作为其停用所指向的结果，由 Inactive 携带作为它所达到的结果，要么 $\bot$，要么从第 4.3.4 节提供的错误集 Ξ 抽取的错误。

纤维在处于三个携带累加器和已提交视图的状态之一时是已安装的（installed），在携带错误结果时是失败的（failed）：

$$\text{installed}_n(\gamma) := \theta_n \neq \text{Inactive}(-), \qquad \text{failed }_n(\gamma) := \exists \xi \in \Xi . \theta_n = \text{Inactive}(\xi) \tag{44}$$

已安装纤维 n 把 k 解析为 m，当 $\omega_n(k) = m.$ 。定义 46 的静止性在更宽的状态空间上读作

$$\text{quiet}(\gamma) := \forall n \in \text{dom}\big(F_\gamma\big). \begin{cases} \zeta \neq \bot \vee \text{target}_n(\gamma) = \bot & \text{if} \theta_n = \text{Inactive}(\zeta) \\ \text{target}_n(\gamma) = \omega_n & \text{if} \theta_n = \text{Active}(-, \omega_n) \\ \bot & \text{otherwise} \end{cases} \tag{45}$$

第 4.1 节的定义延续到这个状态空间，有两个读法需要固定。首先，第 4.2 节的 Inactive 在 O-Insert 的结论中读作 Inactive($\bot$)、在 O-Remove 的前提中读作 Inactive $(-)$ 。其次，σ_γ 仍只并集 Active 纤维的表，因此转移任一个方向进行中的纤维通过它持有的 ω 读取其余效应，且不提供自己的余效应；其转移已写入的键因此还不是依赖方可据以激活的键。在两状态演算中该区分是空洞的，那里每个已安装纤维都是 Active。

图 2 画出这些状态形成的生命周期，下面四个小节在其边上提供规则。

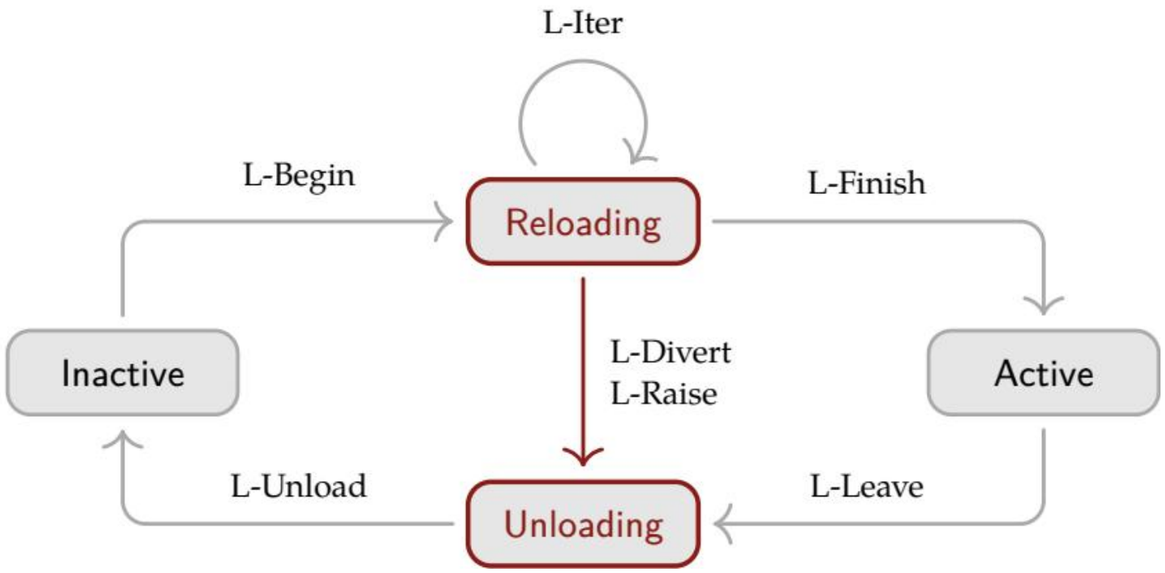


Figure 2 | Lifecycle with transitions in progress; the two transition states are outlined

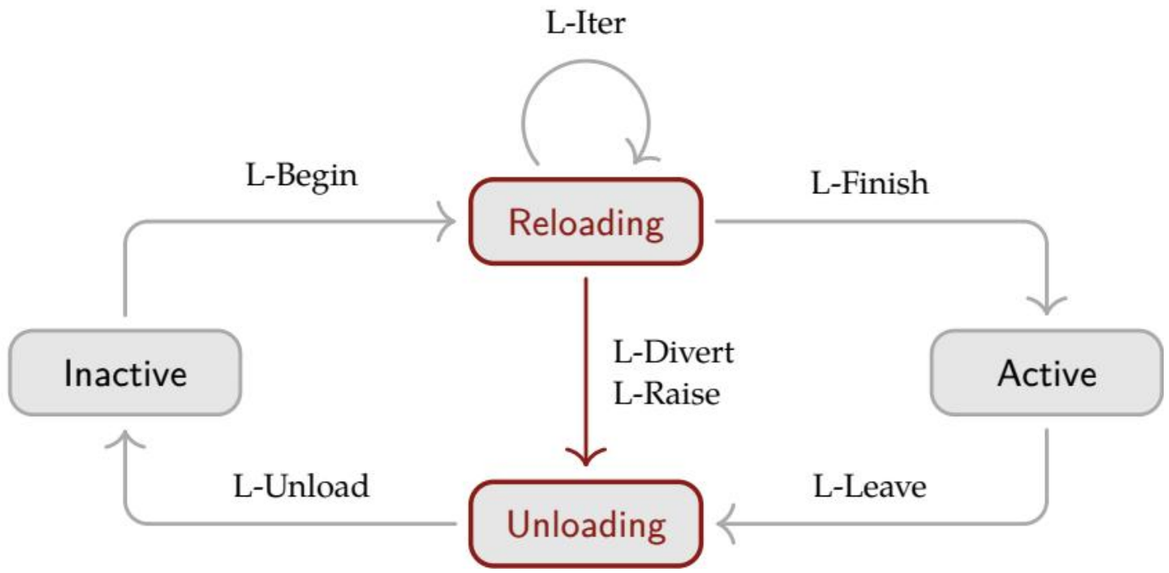


图 2 | 带进行中转移的生命周期；两个转移状态被描出轮廓

4.3.1. Withdrawal

Section 3.2 requires that dependents activate after their dependencies and that dependencies withdraw their provisions only after their dependents have deactivated. The first half holds in the base calculus already: an activation requires $\gamma \models d_n$, so a fiber declaring d cannot activate before some fiber is actively providing k. The second half is the substantive one, and it must deliver more than an ordering of state changes. A component being torn down because its provider is going away is running its own teardown code, which may need the very coeffect that is being withdrawn; closing a connection pool typically means handing the connections back to whatever provided them. What the second half must deliver is that a consumer can still read k throughout its own deactivation, and that the provider's withdrawal of k takes effect only afterwards. The base calculus cannot deliver it at all: its L-Unload removes the provisions and runs the inverse together, leaving no interval between them for a consumer's teardown to occupy.

This layer splits that step in two, and guards the second half by the following condition.

Definition 50. The fiber n is relied upon at γ when some other installed fiber resolves a key to it:

$$\mathrm{relied}_n(\gamma) := \exists m \in \mathrm{dom}\big(F_\gamma\big), k \in d_m.m \neq n \wedge \mathrm{installed}_m(\gamma) \wedge \omega_m(k) = n$$

$$\frac{\theta_n{=}\mathsf{Active}(g,\omega) \quad \mathrm{target}_n(\gamma){\neq}\omega}{\gamma \longrightarrow \gamma[\theta_n \mapsto \mathsf{Unloading}(g,\omega,\bot)]}\text{L - Leave}$$

$$\frac{\theta_n{=}\mathsf{Unloading}(g,\omega,\zeta) \quad \neg\,\mathrm{relied}_n(\gamma) \quad g(\gamma){=}\delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathsf{Inactive}(\zeta)]}\text{L - Unload} \tag{46}$$

L-Leave records the decision to deactivate without acting on it, which stops the fiber providing its coefects while leaving its own committed view and everyone else's intact. L-Unload applies the accumulator, discards the committed view, and leaves the fiber Inactive on the outcome it carries; the outcome is $\bot$ until Section 4.3.4 supplies the other case. It is the only rule in the calculus that applies an accumulator.

The two halves of the ordering are then carried by diferent parts of the form: the visibility half by the committed view, which L-Unload discards as its last act, and the ordering half by the premise ¬ relied (n), which we call the guard and which holds the withdrawal of n back until every consumer that resolves it to n has gone. Theorem 63 establishes both.

4.3.1. 撤出（Withdrawal）

第 3.2 节要求依赖方在其依赖之后激活，且依赖方只在其依赖方已停用后才撤出其提供。前半在基础演算中已经成立：激活要求 $\gamma \models d_n$，因此声明 d 的纤维不能在某个纤维正提供 k 之前激活。后半是实质性的，它必须交付的不只是状态变化的排序。因提供者将消失而被拆除的组件正运行其自身的拆卸代码，而它可能需要正被撤出的余效应本身；关闭连接池通常意味着把连接还给提供它们的人。后半必须交付的是消费者在其自身停用期间仍能读取 k，且提供者对 k 的撤出只在其后生效。基础演算根本无法交付：其 L-Unload 同时移除提供并运行逆，在两者之间不给消费者的拆卸留下区间。

本层把这一步拆成两步，并用以下条件守卫后半。

定义 50. 当某个其他已安装纤维把一个键解析到它时，纤维 n 在 γ 处被依赖（relied）：

$$\mathrm{relied}_n(\gamma) := \exists m \in \mathrm{dom}\big(F_\gamma\big), k \in d_m.m \neq n \wedge \mathrm{installed}_m(\gamma) \wedge \omega_m(k) = n$$

$$\frac{\theta_n{=}\mathsf{Active}(g,\omega) \quad \mathrm{target}_n(\gamma){\neq}\omega}{\gamma \longrightarrow \gamma[\theta_n \mapsto \mathsf{Unloading}(g,\omega,\bot)]}\text{L - Leave}$$

$$\frac{\theta_n{=}\mathsf{Unloading}(g,\omega,\zeta) \quad \neg\,\mathrm{relied}_n(\gamma) \quad g(\gamma){=}\delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathsf{Inactive}(\zeta)]}\text{L - Unload} \tag{46}$$

L-Leave 记录停用决定而不对其行动，这使纤维停止提供其余效应，同时保持它自己的已提交视图和别人的已提交视图原封不动。L-Unload 应用累加器、丢弃已提交视图并使纤维以其携带的结果停在 Inactive；结果在 4.3.4 节提供另一种情形前为 $\bot$。它是演算中唯一应用累加器的规则。

排序的两半随后由形式的不同部分承载：可见性一半由已提交视图承载，L-Unload 作为其最后行动丢弃它；排序一半由前提 ¬ relied (n) 承载，我们称之为守卫，它把 n 的撤出推迟到每个把它解析为 n 的消费者都离开。定理 63 确立两者。

The guard is imposed per binding rather than per fiber: relied $_n(\gamma)$ tests whether some committed view names n, so a fiber that declares none of n's keys is no obstacle, and neither is one that resolved a key of n's in another realm (Section 3.2.3). Under the single-source discipline of Section 4.2 the per-binding reading coincides with the coarser test $\exists \mathrm{m} \neq \mathrm{n},\, k \in d_m$. in $\mathrm{stalled}_m(\gamma) \wedge k \in p_n$. , a key having one possible provider there.

A guard of this kind ordinarily deadlocks. What keeps it from doing so is Unloading together with σ_γ being the union over Active fibers alone: once L-Leave has marked n, its table leaves σ_γ, , so no target view can name n any longer, and every consumer that committed to n is itself on its way out. Theorem 66 turns that into the claim that the guard always releases.

The guard orders deactivations along coefects and not along the fiber tree: a parent may run its inverse while a child of it is still Unloading, since relied speaks only of committed views. Parent and child are accordingly ordered more weakly than Theorem 63 orders a provider and its consumer, and a parent and a child whose efects meet in the ambient state are governed by the independence hypothesis of Definition 60 instead.

守卫按绑定而非按纤维施加：relied $_n(\gamma)$ 测试某个已提交视图是否命名 n, 因此不声明 n 的任何键的纤维不是障碍，在另一个领域（第 3.2.3 节）解析 n's 的键的纤维也不是。在第 4.2 节的单一来源纪律下，按绑定读法恰与更粗的测试 $\exists \mathrm{m} \neq \mathrm{n},\, k \in d_m$ 。在 $\mathrm{stalled}_m(\gamma) \wedge k \in p_n$. 重合，键在那里只有一个可能的提供者。

此类守卫通常会死锁。使它不死锁的是 Unloading 与 σ_γ 只并集 Active 纤维：一旦 L-Leave 标记了 n, 其表离开 σ_γ, 因此没有目标视图能再命名 n，而每个承诺给 n 的消费者本身已在退出之路上。定理 66 把它转化为守卫总是释放的断言。

守卫沿余效应而非沿纤维树对停用排序：父组件可以在其子组件仍 Unloading 时运行其逆，因为 relied 只谈论已提交视图。父与子相应地比定理 63 对提供者与其消费者排序得更弱，而在环境状态中效应相交的父与子由定义 60 的独立性假设管辖。

4.3.2. Iteration

An activation may execute multiple efects in sequence, and the deactivation must recover them. We model such an activation with an effect iterator, each of whose iterations yields the modified context, an inverse, and a continuation:

Definition 51. Define the effect iterator $\mathfrak{E}_\Gamma^{\mathrm{iter}}$ and witnessed effect iterator $\mathfrak{E}_\Gamma^{\mathrm{iter}*}$ as the following recursive types:

$$\begin{aligned}\mathfrak{E}_\Gamma^{\mathrm{iter}} &:= \mu\mathfrak{J}.\Gamma \to \Gamma \times (\Gamma \to \Gamma) \times \mathrm{Maybe}(\mathfrak{J}) \\ \mathfrak{E}_\Gamma^{\mathrm{iter}*} &:= \mu\mathfrak{J}.(e : \Gamma \to \Gamma \times (\Gamma \to \Gamma) \times \mathrm{Maybe}(\mathfrak{J})) \\ &\quad \times((\gamma : \Gamma) \to (\mathbf{l\ e\ t}(\delta, g, o) = e(\gamma)\mathbf{i\ n}g(\delta) \simeq \gamma))\end{aligned} \tag{47}$$

where $e(\gamma)$ yields a triple (δ, g, o) representing:

- δ is the new context;
- g is the inverse function of the current effect;
- o indicates the continuation:

▶ Nothing signals iteration termination;

▶ Just(i) provides the next iteration.

The witness is read at the $\simeq$ of Definition 33, as Definition 37 reads that of $\mathfrak{E}_\Gamma^*$: an $i \in \mathfrak{E}_\Gamma^{\mathrm{iter}}$ lies in $\mathfrak{E}_\Gamma^{\mathrm{iter}*}$ when i respects $\simeq$ and each g it yields respects $\simeq$ and satisfies the clause above. A triple is compared componentwise, Nothing with Nothing alone and Just(i) with Just (i') when $i \simeq i'$ and $\simeq$ on iterators is the greatest relation meeting those clauses. Taking $\simeq$ to be equality on Γ recovers the reading on the nose.

The effect iterator transformation effect $\mathrm{effect}_\Gamma^{\mathrm{iter}}$ extends effect_Γ to the iterator structure through recursive invocation:

Definition 52. Define the effect iterator transformation effect $\mathrm{effect}_\Gamma^{\mathrm{iter}}$ as:

4.3.2. 迭代（Iteration）

激活可以按顺序执行多个效应，而停用必须恢复它们。我们用效应迭代器建模这样的激活，其每次迭代产生修改后的上下文、一个逆和一个续延：

定义 51. 定义效应迭代器 $\mathfrak{E}_\Gamma^{\mathrm{iter}}$ 和带见证的效应迭代器 $\mathfrak{E}_\Gamma^{\mathrm{iter}*}$ 为如下递归类型：

$$\begin{aligned}\mathfrak{E}_\Gamma^{\mathrm{iter}} &:= \mu\mathfrak{J}.\Gamma \to \Gamma \times (\Gamma \to \Gamma) \times \mathrm{Maybe}(\mathfrak{J}) \\ \mathfrak{E}_\Gamma^{\mathrm{iter}*} &:= \mu\mathfrak{J}.(e : \Gamma \to \Gamma \times (\Gamma \to \Gamma) \times \mathrm{Maybe}(\mathfrak{J})) \\ &\quad \times((\gamma : \Gamma) \to (\mathbf{l\ e\ t}(\delta, g, o) = e(\gamma)\mathbf{i\ n}g(\delta) \simeq \gamma))\end{aligned} \tag{47}$$

其中 $e(\gamma)$ 产生三元组 (δ, g, o) 表示：

- δ 是新的上下文；
- g 是当前效应的逆函数；
- o 表示续延：

▶ Nothing 表示迭代终止；

▶ Just(i) 提供下一次迭代。

见证在定义 33 的 $\simeq$ 处读出，正如定义 37 读 $\mathfrak{E}_\Gamma^*$ 的见证：当 i 尊重 $\simeq$ 且它产生的每个 g 尊重 $\simeq$ 并满足上述条款时，$i \in \mathfrak{E}_\Gamma^{\mathrm{iter}}$ 位于 $\mathfrak{E}_\Gamma^{\mathrm{iter}*}$ 中。三元组逐分量比较，Nothing 只与 Nothing 比较，Just(i) 在 $i \simeq i'$ 时与 Just (i') 比较，而迭代器上的 $\simeq$ on 是满足这些条款的最大联系。取 $\simeq$ 为 Γ 上的相等性精确恢复该读法。

效应迭代器变换 $\mathrm{effect}_\Gamma^{\mathrm{iter}}$ 通过递归调用把 effect_Γ 扩展到迭代器结构：

定义 52. 定义效应迭代器变换 $\mathrm{effect}_\Gamma^{\mathrm{iter}}$ 为：

$$
\begin{aligned}
\mathrm{effect}_\Gamma^{\mathrm{iter}} \quad &: \quad \mathfrak{E}_\Gamma^{\mathrm{iter}} \to \partial\Gamma \to \partial^2\Gamma \\
&\quad\ \ \mathrm{let}(\delta, g, o) = i(\gamma)\mathrm{in} \\
&\quad\ \ \mathrm{let}t = \mathrm{track}_\Gamma(g, \mathrm{pr}_1 \circ i)\mathrm{in} \\
&\qquad\qquad\qquad\qquad \mathbf{m\ a\ t\ c\ h}\, o \\
\mathrm{effect}_\Gamma^{\mathrm{iter}} \quad &= \quad i \mapsto (\gamma, \varphi) \mapsto \quad \begin{array}{c} |\mathrm{Nothing} \Rightarrow ((\delta, \varphi \circ g), t) \\ |\mathrm{Just}(i') \Rightarrow \mathrm{let}(s, r) = \mathrm{effect}_\Gamma^{\mathrm{iter}}(i')(\delta, \varphi \circ g)\mathrm{in} \\ (s, t \circ r) \end{array}
\end{aligned}
\tag{48}
$$

At each iteration, the inverse g is composed onto φ in application order, so the accumulator $\varphi \circ g_1 \circ \cdots \circ g_k$ naturally recovers efects in LIFO order when applied. Because $\mathrm{effect}_\Gamma^{\mathrm{iter}}$ lands in the same $\partial\Gamma\partial^2\Gamma$ as effect_Γ does, an iterator is an effect in its own right and can be used wherever an effect can. A component's whole activation is one such use, which is what the rest of this section formalizes, and the implementation admits an iterator at every mutation site (Section 5.1.1). The $\mathsf{Maybe}(\mathfrak{E}^{\mathrm{iter}})$ continuation makes a boundary available between any two consecutive iterations, at which the context is whatever the iterations so far have made it and the accumulator recovers those and nothing more. In this sense the effect iterator is a reified delimited continuation, the structure that mainstream languages expose through the yield operator [43], so the model maps directly onto the generators they already provide.

In the calculus, the e_n of Definition 44 is read at $\mathfrak{E}_\Gamma^{\mathrm{iter}*}$ from here on, and replacing the atomic effect function by an iterator splits the base L-Reload into a begun state that the trace passes through, and gives the fiber a second way out of that state.

$$
\frac{\theta_n{=}\mathrm{Inactive}(\bot) \quad \omega{=}\mathrm{target}_n(\gamma){\neq}\bot}{\gamma \longrightarrow \gamma[\theta_n \mapsto \mathrm{Reloading}(e_n, \mathrm{id}_\Gamma, \omega)]}\text{L - Begin}
$$

$$
\frac{\theta_n{=}\mathrm{Reloading}(i,g,\omega) \quad \mathrm{target}_n(\gamma){\neq}\omega \quad (\delta,h){=}(\gamma,\mathrm{id}_\Gamma){\vee}i(\gamma){=}(\delta,h,{-})}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathrm{Unloading}(g{\circ}h,\omega,\bot)]}\text{L - Divert}
$$

$$
\frac{\theta_n{=}\mathrm{Reloading}(i,g,\omega) \quad \mathrm{target}_n(\gamma){=}\omega \quad i(\gamma){=}(\delta,h,\mathrm{Just}(i'))}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathrm{Reloading}(i',g{\circ}h,\omega)]}\text{L - Iter}
$$

$$
\frac{\theta_n{=}\mathrm{Reloading}(i,g,\omega) \quad \mathrm{target}_n(\gamma){=}\omega \quad i(\gamma){=}(\delta,h,\mathrm{Nothing})}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathrm{Active}(g{\circ}h,\omega)]}\text{L - Finish}
$$

Each iteration composes the newly yielded inverse onto the accumulator as $\boldsymbol{g \circ h}$, , following Definition 52, so that the accumulator applies the inverses in last-in-first-out order. Between any two consecutive iterations the system may divert the transition if its target view has changed, applying the inverse accumulated so far to recover the context. L-Divert routes through Unloading like every other deactivation rather than applying the accumulator where it stands, and the guard it meets there is vacuous, a fiber that has never been Active providing nothing and appearing in no committed view. The first of its two alternatives aborts the iteration the fiber is holding, which only an iteration boundary makes possible, so the granularity at which a divert may fall is that of the iterator; the second lets that iteration land, and Section 4.3.3 is where it is needed.

A plain effect function ($\mathfrak{E}_\Gamma$) is the degenerate case where the first iteration already yields Nothing. Such a transition still passes through Reloading and L-Divert still applies there, but the accumulator is id_Γ and no iteration has run, so nothing is restored and the transition installs either all of its efects or none of them.

4.3.3. Asynchrony

The layers so far let the environment move between one iteration and the next, and assume that each iteration itself completes instantaneously, its launch and its landing being one step. We model non-immediacy abstractly: an iteration yields a value of type Future(A), where Future is an opaque type constructor whose defining property is that between submission and resolution, external state may change.

$$
\begin{aligned}
\mathrm{effect}_\Gamma^{\mathrm{iter}} \quad &: \quad \mathfrak{E}_\Gamma^{\mathrm{iter}} \to \partial\Gamma \to \partial^2\Gamma \\
&\quad\ \ \mathrm{let}(\delta, g, o) = i(\gamma)\mathrm{in} \\
&\quad\ \ \mathrm{let}t = \mathrm{track}_\Gamma(g, \mathrm{pr}_1 \circ i)\mathrm{in} \\
&\qquad\qquad\qquad\qquad \mathbf{m\ a\ t\ c\ h}\, o \\
\mathrm{effect}_\Gamma^{\mathrm{iter}} \quad &= \quad i \mapsto (\gamma, \varphi) \mapsto \quad \begin{array}{c} |\mathrm{Nothing} \Rightarrow ((\delta, \varphi \circ g), t) \\ |\mathrm{Just}(i') \Rightarrow \mathrm{let}(s, r) = \mathrm{effect}_\Gamma^{\mathrm{iter}}(i')(\delta, \varphi \circ g)\mathrm{in} \\ (s, t \circ r) \end{array}
\end{aligned}
\tag{48}
$$

在每次迭代，逆 g 按应用顺序复合到 φ, 上，因此累加器 $\varphi \circ g_1 \circ \cdots \circ g_k$ 应用时自然按 LIFO 顺序恢复效应。由于 $\mathrm{effect}_\Gamma^{\mathrm{iter}}$ 落在与 effect_Γ 相同的 $\partial\Gamma\partial^2\Gamma$ 中，迭代器本身就是一个效应，可用于任何效应可用之处。组件的一次完整激活就是这样一个使用，这正是本节其余部分形式化的东西，而实现在每个变更处接纳迭代器（第 5.1.1 节）。$\mathsf{Maybe}(\mathfrak{E}^{\mathrm{iter}})$ 续延使任意两个连续迭代之间有一个边界可用，在该边界处上下文是迭代至今所制造的样子，累加器恢复它们而不恢复更多。在这个意义上，效应迭代器是具体化的限定续延，即主流语言通过 yield 运算符暴露的结构 [43]，因此该模型直接映射到它们已提供的生成器上。

在演算中，定义 44 的 e_n 从此刻起在 $\mathfrak{E}_\Gamma^{\mathrm{iter}*}$ 处读出，用迭代器替换原子效应函数把基础 L-Reload 拆成一个迹所穿过的开始状态，并给纤维第二个离开该状态的出口。

$$
\frac{\theta_n{=}\mathrm{Inactive}(\bot) \quad \omega{=}\mathrm{target}_n(\gamma){\neq}\bot}{\gamma \longrightarrow \gamma[\theta_n \mapsto \mathrm{Reloading}(e_n, \mathrm{id}_\Gamma, \omega)]}\text{L - Begin}
$$

$$
\frac{\theta_n{=}\mathrm{Reloading}(i,g,\omega) \quad \mathrm{target}_n(\gamma){\neq}\omega \quad (\delta,h){=}(\gamma,\mathrm{id}_\Gamma){\vee}i(\gamma){=}(\delta,h,{-})}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathrm{Unloading}(g{\circ}h,\omega,\bot)]}\text{L - Divert}
$$

$$
\frac{\theta_n{=}\mathrm{Reloading}(i,g,\omega) \quad \mathrm{target}_n(\gamma){=}\omega \quad i(\gamma){=}(\delta,h,\mathrm{Just}(i'))}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathrm{Reloading}(i',g{\circ}h,\omega)]}\text{L - Iter}
$$

$$
\frac{\theta_n{=}\mathrm{Reloading}(i,g,\omega) \quad \mathrm{target}_n(\gamma){=}\omega \quad i(\gamma){=}(\delta,h,\mathrm{Nothing})}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathrm{Active}(g{\circ}h,\omega)]}\text{L - Finish}
$$

每次迭代把新产生的逆作为 $\boldsymbol{g \circ h}$, 复合到累加器上，遵循定义 52，使累加器按后进先出顺序应用逆。任意两个连续迭代之间，系统可以在其目标视图已改变时使转移转向，应用迄今累积的逆以恢复上下文。L-Divert 像每个其他停用一样经 Unloading 路由而非就地应用累加器，且它在那里遇到的守卫是空洞的，从未 Active 的纤维不提供任何东西也不出现在任何已提交视图中。其两个备选中的第一个中止纤维持有的迭代，这只有迭代边界才使之可能，因此转向可落在的粒度是迭代器的粒度；第二个让该迭代落地，第 4.3.3 节是它被需要之处。

普通效应函数 ($\mathfrak{E}_\Gamma$) 是第一次迭代已产生 Nothing 的退化情形。这样的转移仍穿过 Reloading，L-Divert 仍在那里适用，但累加器是 id_Γ 且没有迭代运行，因此没有恢复任何东西，而转移要么安装其全部效应要么完全不安装。

4.3.3. 异步（Asynchrony）

迄今的各层让环境在一次迭代与下一次之间移动，并假设每次迭代本身瞬时完成，其启动和落地是一步。我们抽象地建模非即时性：迭代产生 Future(A) 类型的值，其中 Future 是不透明类型构造器，其定义性质是提交与解析之间外部状态可能改变。

Under this model an iteration is launched at one state and lands at another, and the fiber is Unloading while it is in flight. What the layer adds is inertia: once launched, an iteration lands, and its landing cannot be declined. A target view that turns during the flight therefore cannot be answered by aborting the iteration, and only the alternative of L-Divert that lands one remains available: the iteration lands, and the fiber deactivates afterwards. This layer therefore adds no rule and no type that a rule matches on; at the granularity of Γ inertia is its whole content, and it takes the form of a restriction on which alternative of L-Divert a host may take.

That alternative is what the base calculus could not express. There, a transition whose target view had turned was undone in the same step that discovered it; here the iteration in flight must land first, so the fiber needs somewhere to be while its inverse runs, and the only sound place is Unloading holding the inverse the iteration produced. Routing through Active instead would let the fiber provide its coefects for the length of one step and oblige its dependents to activate against a component that is already leaving. This is the mutual chaining of reload and unload in the implementation.

A deactivation may also chain straight back into an activation, by a composite rather than a rule. L-Unload carries no premise on the target view, so whatever the target view has become while the fiber was deactivating, the accumulator runs and the fiber becomes Inactive, from which L-Begin may immediately start a new transition.

4.3.4. Failure

Every rule so far assumes the effect it runs succeeds, and a runtime cannot. The efects a component installs reach outside the context that tracks them, and what they reach may refuse: a port already bound, a file that is not there, a peer that does not answer. A failing transition must still leave the fiber's efects recovered rather than stranded.

Let Ξ be a set of errors and refine the effect iterator of Definition 51 so that an iteration may raise in place of yielding a triple:

$$\begin{aligned} \mathfrak{E}_{\Gamma}^{\mathrm{fail}} &:= \mu\mathfrak{J}.\Gamma \to \mathrm{Either}(\Xi, \Gamma \times (\Gamma \to \Gamma) \times \mathrm{Maybe}(\mathfrak{J})) \\ \mathfrak{E}_{\Gamma}^{\mathrm{fail}*} &:= \mu\mathfrak{J}.(e : \Gamma \to \mathrm{Either}(\Xi, \Gamma \times (\Gamma \to \Gamma) \times \mathrm{Maybe}(\mathfrak{J}))) \\ &\quad \times((\gamma : \Gamma) \to (\mathrm{let}\ \mathrm{Right}(\delta, g, o) = e(\gamma)\mathrm{in} g(\delta) \simeq \gamma)) \end{aligned} \tag{49}$$

The witness constrains the Right case alone, being vacuous where the pattern does not match, a raise having nothing to undo, and the e that Reloading carries is read at $\mathfrak{E}_{\Gamma}^{\mathrm{fail}*}$ from here on. The lift of Definition 52 carries over with a raise propagated in place of a triple, so a raising iterator is usable wherever an effect is, as an ordinary one is. The layer adds one rule and puts the second outcome of Definition 49 to use, O-Remove needing no widening to admit it. The premises of L-Iter, L-Finish, and L-Divert are read with Right around the triple they match. A raise is something an iteration does, so the rule is an exit from Reloading.

$$\frac{\theta_n = \mathrm{Reloading}(i, g, \omega) \quad i(\gamma) = \mathrm{Left}(\xi)}{\gamma \longrightarrow \gamma[\theta_n \mapsto \mathrm{Unloading}(g, \omega, \xi)]}\ \text{L - Raise}$$

L-Raise recovers before it records. The fiber routes into Unloading carrying the error as its outcome, the accumulator built up to the failing iteration is applied there, and the fiber arrives at Inactive($\perp$) having installed nothing, at a state difering from the one an aborting L Divert would have produced only in the outcome the fiber carries. Routing a failure like every other deactivation is what makes every outcome reachable only through L-Unload, which is the single fact Theorem 59 turns on. L-Begin has Inactive($\perp$) as a premise, so the lifecycle is not re-entered from an error outcome; this is the substance of the outcome, which withholds a fiber whose effect function has shown itself to be unsound in the state it ran against rather than retrying it against an unchanged

在此模型下，迭代在一个状态启动、在另一个状态落地，而纤维在飞行期间是 Unloading。本层增加的是惯性（inertia）：一旦启动，迭代落地，且其落地不能被拒绝。飞行期间转向的目标视图因此不能通过中止迭代来应答，只剩下 L-Divert 中落地一个迭代的备选可用：迭代落地，纤维随后停用。本层因此不增加规则，也不增加规则匹配的类型；在 Γ 的粒度上惯性是它的全部内容，它采取的形式是对宿主可取 L-Divert 哪个备选的限制。

那个备选是基础演算无法表达的。那里，目标视图已转向的转移在发现它的同一步被撤销；这里飞行中的迭代必须先落地，因此纤维需要在逆运行时有处可待，唯一可靠的地方是持有迭代产生的逆的 Unloading。经 Active 路由会让纤维提供其余效应达一步之长，并使其依赖方针对已离开的组件激活。这就是实现中 reload 与 unload 的相互链接。

停用也可以通过复合而非规则直接链回激活。L-Unload 对目标视图不携带前提，因此无论纤维停用时目标视图变成什么，累加器都运行，纤维变为 Inactive，L-Begin 可立即从中启动新转移。

4.3.4. 失败（Failure）

迄今的每个规则都假设它运行的效应成功，而运行时不能。组件安装的效应延伸到跟踪它们的上下文之外，而它们到达的东西可能拒绝：已绑定的端口、不存在的文件、不应答的对端。失败的转移仍必须使纤维的效应被恢复而非搁浅。

令 Ξ 为错误集并把定义 51 的效应迭代器精化，使迭代可以抛错而非产生三元组：

$$\begin{aligned} \mathfrak{E}_{\Gamma}^{\mathrm{fail}} &:= \mu\mathfrak{J}.\Gamma \to \mathrm{Either}(\Xi, \Gamma \times (\Gamma \to \Gamma) \times \mathrm{Maybe}(\mathfrak{J})) \\ \mathfrak{E}_{\Gamma}^{\mathrm{fail}*} &:= \mu\mathfrak{J}.(e : \Gamma \to \mathrm{Either}(\Xi, \Gamma \times (\Gamma \to \Gamma) \times \mathrm{Maybe}(\mathfrak{J}))) \\ &\quad \times((\gamma : \Gamma) \to (\mathrm{let}\ \mathrm{Right}(\delta, g, o) = e(\gamma)\mathrm{in} g(\delta) \simeq \gamma)) \end{aligned} \tag{49}$$

见证只约束 Right 情形，在模式不匹配处是空洞的，抛错没有要撤销的东西，而 Reloading 携带的 e 从此刻起在 $\mathfrak{E}_{\Gamma}^{\mathrm{fail}*}$ 处读出。定义 52 的提升以抛错替代三元组传播过去，因此抛错的迭代器像普通迭代器一样可用于任何效应可用之处。本层增加一条规则并把定义 49 的第二个结果投入使用，O-Remove 无需加宽即可接纳它。L-Iter、L-Finish 和 L-Divert 的前提在其匹配的三元组周围以 Right 读出。抛错是迭代做的事，因此该规则是 Reloading 的一个出口。

$$\frac{\theta_n = \mathrm{Reloading}(i, g, \omega) \quad i(\gamma) = \mathrm{Left}(\xi)}{\gamma \longrightarrow \gamma[\theta_n \mapsto \mathrm{Unloading}(g, \omega, \xi)]}\ \text{L - Raise}$$

L-Raise 在记录之前恢复。纤维携带错误作为其结果路由进 Unloading，累积到失败迭代的累加器在那里被应用，纤维到达 Inactive($\perp$)，未安装任何东西，其状态与中止的 L-Divert 所产生者仅在纤维携带的结果上不同。像每个其他停用一样路由失败，使每个结果只能经 L-Unlock 达到，这是定理 59 所依赖的唯一事实。L-Begin 以 Inactive($\perp$) 为前提，因此生命周期不从错误结果重新进入；这是结果的实质，它不让其效应函数在其运行针对的状态中已显明不健全的纤维对未变的环境重试。失败的纤维也不阻碍任何东西：它 Inactive，因此不携带已提交视图、不能使 relied 成立。

environment. A failed fiber also obstructs nothing: it is Inactive, so it carries no committed view and cannot make relied hold.

A failure is recorded on the fiber rather than propagated to its parent, so a component whose transition fails leaves its siblings running, which is the behavior a plugin host wants and the reason the outcome is per-fiber rather than a property of the whole state.

4.4. Metatheory

Section 4.3 supplies ten rules: the three orchestration rules of Section 4.2; L-Begin, L-Iter, and L-Finish for an activation; L-Divert and L-Raise for the two ways an activation may end early; and L-Leave and L-Unload for a deactivation. This section reads the two dimensions of composability of those rules in their global form, one fiber's guarantee holding whatever the other fibers do in between, and adds what only a whole system can be asked for: that it always reaches the configuration its targets call for, and that the configuration is the one a static assembly would have produced. Every property below is a property of a sequence of steps, so we index the steps and read the fields of a state of that index.

Two conventions carry Section 3.3.2 into this section. Every equality between states below is read up to the observational equivalence $\simeq$ of Definition 33, as Lemma 38 reads those of Section 3.1, and the witness condition an effect function is held to is the one Definition [37] gives, read of an iterator as Definition 51 gives it and of a registering iteration at the $\approx$ below.

Definition 53. Index the steps by t, so that γ^t is the state the first t of them reach, and write

$$\mathrm{step}^t := r(n) \tag{50}$$

for the step taken at γ^t: the rule ρ it applies, one of the ten, and the name $n \in \mathfrak{N}$ it applies that rule at. The sequence starts at a γ^0 with dom $\left(F^0\right) = \varnothing$, so every fiber comes into existence by an O-Insert, whether the orchestrator's or one an iteration takes (Definition 47). A field of γ^t carries the index as a superscript, so that $\theta_n^t, \omega_n^t, \sigma_n^t, g_n^t$, and i_n^t are the lifecycle state, committed view, table, accumulator, and remaining iterator of n at γ^t. , and F^t and σ^t the registry and coeffect context of γ^t itself, the F_γ and σ_γ of Definition 45 read there. Predicates take the state as their argument and everything else as a subscript, so installe $\lessdot_n^t$, target t_n^t , relie l_n^t, , and quiett are the predicates of Definition [46,] , Definition 49, and Definition 50 at γ^t . An episode of n is a maximal interval $[b, u]$ of indices throughout which installe l_n^t holds. It opens at b, where $b > 0$ and $\neg\mathrm{installed}_n^{b-1}$, the empty F^0 leaving no fiber installed at the outset; it closes at t when installe $\mathbb{I}_n^u$ and not installed $\mathrm{l}_n^{\bar{u}+1}$, which a final episode need not do.

Every rule of Section 4.3 concludes in the shape $\gamma \longrightarrow \delta[\cdots]$, , where the premises compute δ from γ and leave it as γ where they compute nothing, and the bracket edits named fields of the registry. The two halves are named separately, and both are maps on all of Γ. The state map of a step taken at γ^t by a rule acting on n is

$$\Psi^t := \begin{cases} \mathrm{pr}_1 \circ i & \text{at L - Iter, L - Finish, and a landing L - Divert} \\ g & \text{at L - Unload} \\ \mathrm{id}_\Gamma & \text{at every other rule} \end{cases} \tag{51}$$

where i and g are the iterator and the accumulator that θ_n^t carries, and the edit edi $\mathrm{t}^t : \Gamma \to \Gamma$ is the bracket read as a function, assigning to the fields it names the values the premises computed at γ^t . Both are therefore fixed by $\mathrm{step}^{\bar{t}}$ together with γ^t and defined at every state, which is what lets Theorem 61 and Lemma 71 evaluate them away from γ^t . Each step factors as

失败记录在纤维上而非传播给其父，因此转移失败的组件让它的兄弟组件继续运行，这是插件宿主想要的行为，也是结果按纤维而非整个状态的性质的原因。

4.4. 元理论（Metatheory）

第 4.3 节提供十条规则：第 4.2 节的三条编排规则；激活的 L-Begin、L-Iter 和 L-Finish；激活提前结束的两种方式的 L-Divert 和 L-Raise；以及停用的 L-Leave 和 L-Unload。本节以全局形式读取这些规则的可组合性两个维度，一个纤维的保证无论其他纤维在其间做什么都成立，并加上只有整个系统才能被要求的东西：它总是达到其目标所要求的配置，且该配置是静态装配会产生的配置。下面的每个性质都是步骤序列的性质，因此我们对步骤编号并读取该编号状态中的字段。

两个约定把第 3.3.2 节带入本节。下面状态间的每个相等性都在定义 33 的观测等价 $\simeq$ 之下读出，正如引理 38 读第 3.1 节的相等性，而效应函数所受的见证条件是定义 [37] 给出的那个，按定义 51 对迭代器读出、在注册迭代处按下面的 $\approx$ 读出。

定义 53. 按 t 对步骤编号，使 γ^t 是前 t 个步骤达到的状态，并写

$$\mathrm{step}^t := r(n) \tag{50}$$

为在 γ^t：处采取的步骤：它应用的规则 ρ（十条之一）以及它在该规则处应用的名字 $n \in \mathfrak{N}$ 。序列从 dom $\left(F^0\right) = \varnothing$, 的 γ^0 开始，因此每个纤维都经 O-Insert 产生，无论是编排器的还是迭代采取的（定义 47）。γ^t 的字段以上标携带索引，因此 $\theta_n^t, \omega_n^t, \sigma_n^t, g_n^t$ 和 i_n^t 是 n 在 γ^t. 的生命周期状态、已提交视图、表、累加器和剩余迭代器，F^t 和 σ^t 是 γ^t 本身的注册表和余效应上下文，即定义 45 在那里读出的 F_γ 和 σ_γ 。谓词以状态为参数、其余为下标，因此 $\mathrm{installed}_n^t$、target_n^t、$\mathrm{reloaded}_n^t$ 和 quiett 是定义 [46,] 、定义 49 和定义 50 在 γ^t 的谓词。n 的片段（episode）是 installe l_n^t 成立的索引的最大区间 $[b, u]$ 。它在 b, 处打开，其中 $b > 0$ 且 $\neg\mathrm{installed}_n^{b-1}$ ，空 F^0 开头不留纤维已安装；它在 installe $\mathbb{I}_n^u$ 且非 installed $\mathrm{l}_n^{\bar{u}+1}$ 的 t 处闭合，最终片段无需如此。

第 4.3 节的每个规则都以 $\gamma \longrightarrow \delta[\cdots]$, 的形状得出结论，其中前提从 γ 计算 δ 并在它们不计算任何东西处把 δ 留为 γ ，而方括号编辑注册表的命名字段。两半分别命名，且都是 Γ 上的映射。由作用于 n 的规则在 γ^t 采取的步骤的状态映射是

$$\Psi^t := \begin{cases} \mathrm{pr}_1 \circ i & \text{at L - Iter, L - Finish, and a landing L - Divert} \\ g & \text{at L - Unload} \\ \mathrm{id}_\Gamma & \text{at every other rule} \end{cases} \tag{51}$$

其中 i 和 g 是 θ_n^t 携带的迭代器和累加器，而编辑 edi $\mathrm{t}^t : \Gamma \to \Gamma$ 是把方括号作为函数读出的，为其命名的字段赋予前提在 γ^t 计算的值。两者因此由 $\mathrm{step}^{\bar{t}}$ 连同 γ^t 固定并在每个状态有定义，这使定理 61 和引理 71 能在远离 γ^t 处求值它们。每步分解为

$$\gamma^{t+1} = \mathrm{edit}^t\big(\Psi^t(\gamma^t)\big) \tag{52}$$

At L-Unload, for instance, editt is $[\theta_n \mapsto \mathrm{Inactive}(\bot)]$, and at O-Remove it is the removal $\backslash n$, which is why the second half is an edit rather than an assignment. The fields divide along the same seam: the tables σ_m, , which no editt writes once the O-Insert creating n has set it empty, and the control fields $\theta_m, \tau_m, \pi_m, d_m, p_m, e_m$ together with dom (F_γ) , which no Ψ^t writes save through the primitive of Definition 47. Write $\gamma \approx \gamma'$ when two states agree on everything but the control fields.

The relation $\approx$ is not the $\simeq$ of Definition 33, and neither refines the other, because each forgets what the other has to keep. Recovery exactness is a claim about efects, so $\approx$ compares the tables and the ambient state exactly and forgets only the registry's record of which fiber installed them. A rule reads the control fields to decide whether it applies, so $\simeq$ has to keep them, and this section reads it as the conjunction of Definition 33 with agreement on the registry's domain and on every control field of every fiber:

$$\gamma \simeq \delta := \sigma_\gamma \simeq \sigma_\delta \wedge \mathrm{dom}\,(F_\gamma) = \mathrm{dom}\,(F_\delta) \wedge \forall n, c \in \{\theta, \tau, \pi, d, p, e\}.c(\gamma(n)) \simeq c(\delta(n)) \tag{53}$$

A field of function type, as e_n and the e inside θ_n are, is compared as Definition 36 compares maps, an iterator as Definition 51 compares two, and a field of any other type by equality. The results below hold up to both relations, one for each half of the state, Lemma 55 establishing the $\simeq$ half once for all ten rules.

Table 1 is the ten rules of Section 4.3 read as such writes. The accumulator, the committed view, and the remaining iterator are constituents of θ_n. , so the third column records the writes to them as well, and $\hbar$ there names the inverse the iteration of the fourth column yields, id_Γ where L-Divert aborts that iteration. Where a Ψ^t built from an iterator registers a fiber (Definition 47), that registration carries the writes of the O-Insert row at the name it draws, and an L-Unload whose accumulator retires one carries those of the O-Retire row. Every case analysis below is a lookup in the table, and five lookups recur often enough to name.

rule	θ_n^t	θ_n^{t+1}	Ψ^t	control fields edited
O-Insert	undefined	Inactive(⊥)	id $_\Gamma$	dom(F $_\gamma$)
O-Retire	unconstrained	unchanged	id $_\Gamma$	τ_n
O-Remove	Inactive(−)	undefined	id $_\Gamma$	dom(F $_\gamma$)
L-Begin	Inactive(⊥)	Reloading(e $_n$, id $_\Gamma$, ω)	id $_\Gamma$	θ_n
L-Iter	Reloading(i, g, ω)	Reloading(i′, g∘h, ω)	pr $_1$ ∘i	θ_n
L-Finish	Reloading(i, g, ω)	Active(g∘h, ω)	pr $_1$ ∘i	θ_n
L-Divert	Reloading(i, g, ω)	Unloading(g∘h, ω, ⊥)	id $_\Gamma$ or pr $_1$ ∘i	θ_n
L-Raise	Reloading(i, g, ω)	Unloading(g, ω, ξ)	id $_\Gamma$	θ_n
L-Leave	Active(g, ω)	Unloading(g, ω, ⊥)	id $_\Gamma$	θ_n
L-Unload	Unloading(g, ω, ζ)	Inactive(ζ)	g	θ_n

Table 1 | The rules as writes on the fiber n they act on, where stept is that rule applied at n.

Lemma 54. Reading Table 1 together with Definition 48, for every step t and all fibers n, m present at γ^t: :

1. $\sigma_m^{t+1} \neq \sigma_m^t$ only where step t acts on m, the write lying inside Ψ^t;

$$\gamma^{t+1} = \mathrm{edit}^t\big(\Psi^t(\gamma^t)\big) \tag{52}$$

例如在 L-Unload 处，editt 是 $[\theta_n \mapsto \mathrm{Inactive}(\bot)]$，在 O-Remove 处它是移除 $\backslash n$, 这就是后半是编辑而非赋值的原因。字段沿同一接缝划分：表 σ_m, 没有 editt 在创建 n 的 O-Insert 将其设为空后写入它们，以及控制字段 $\theta_m, \tau_m, \pi_m, d_m, p_m, e_m$ 连同 dom (F_γ) ，没有 Ψ^t 除通过定义 47 的原语外写入它们。当两个状态在除控制字段外的一切上一致时写 $\gamma \approx \gamma'$。

联系 $\approx$ 不是定义 33 的 $\simeq$，两者互不细化，因为每个都遗忘另一个必须保留的东西。恢复精确性是关于效应的断言，因此 $\approx$ 精确比较表和环境状态，只遗忘注册表对哪个纤维安装它们的记录。规则读取控制字段以决定是否适用，因此 $\simeq$ 必须保留它们，而本节把它读作定义 33 与注册表定义域及每个纤维每个控制字段一致的合取：

$$\gamma \simeq \delta := \sigma_\gamma \simeq \sigma_\delta \wedge \mathrm{dom}\,(F_\gamma) = \mathrm{dom}\,(F_\delta) \wedge \forall n, c \in \{\theta, \tau, \pi, d, p, e\}.c(\gamma(n)) \simeq c(\delta(n)) \tag{53}$$

函数类型的字段，如 e_n 和 θ_n 内部的 e ，按定义 36 比较映射，迭代器按定义 51 比较两个，任何其他类型的字段按相等性比较。下面的结果在这两个联系之下都成立，一个针对状态的一半，引理 55 一次性为全部十条规则确立 $\simeq$ 一半。

表 1 是第 4.3 节的十条规则作为此类写入的解读。累加器、已提交视图和剩余迭代器是 θ_n. 的构成部分，因此第三列也记录对它们的写入，第四列的 $\hbar$ 命名该迭代产生的逆，L-Divert 中止该迭代处为 id_Γ 。由迭代器构建的 Ψ^t 注册纤维处（定义 47），该注册在它抽取的名字携带 O-Insert 行的写入，而累加器退役纤维的 L-Unload 携带 O-Retire 行的写入。下面的每个情形分析都是表中的查找，而五个查找频繁到值得命名。

规则	θ_n^t	θ_n^{t+1}	Ψ^t	控制字段
O-Insert	undefined	Inactive(⊥)	id $_\Gamma$	dom(F $_\gamma$)
O-Retire	unconstrained	unchanged	id $_\Gamma$	τ_n
O-Remove	Inactive(−)	undefined	id $_\Gamma$	dom(F $_\gamma$)
L-Begin	Inactive(⊥)	Reloading(e $_n$, id $_\Gamma$, ω)	id $_\Gamma$	θ_n
L-Iter	Reloading(i, g, ω)	Reloading(i′, g∘h, ω)	pr $_1$ ∘i	θ_n
L-Finish	Reloading(i, g, ω)	Active(g∘h, ω)	pr $_1$ ∘i	θ_n
L-Divert	Reloading(i, g, ω)	Unloading(g∘h, ω, ⊥)	id $_\Gamma$ 或 pr $_1$ ∘i	θ_n
L-Raise	Reloading(i, g, ω)	Unloading(g, ω, ξ)	id $_\Gamma$	θ_n
L-Leave	Active(g, ω)	Unloading(g, ω, ⊥)	id $_\Gamma$	θ_n
L-Unload	Unloading(g, ω, ζ)	Inactive(ζ)	g	θ_n

表 1 | 规则作为对它们作用的纤维 n 的写入，其中 stept 是在 n 处应用该规则。

引理 54. 结合表 1 与定义 48 阅读，对每个步骤 t 和 γ^t: 在场的所有纤维 n、m ：

1. $\sigma_m^{t+1} \neq \sigma_m^t$ 仅在步骤 t 作用于 m, 且写入位于 Ψ^t; 内时成立；

2. ω_n comes into existence only where $\text{step}^t = \text{L}-\text{Begin}(n)$ and ceases only where $\text{step}^t = \text{L-Unload(n)}$, so ω_n^t is constant for t in an episode of n;

3. $\Psi^t = g_n^t$ only where stept = L-Unload(n), and no other step applies g_n to the state;

4. $\neg$ installedt $\wedge$ installe $\mathsf{l}_n^{t+1} \Rightarrow \text{step}^t = \text{L}-\text{Begin}(n)$ and installedt $\wedge \neg$ installed $\frac{1}{n}^{1+1} \Rightarrow$ stept = L-Unload(n);

5. π_n, d_n, p_n, and e_n come into existence with the entry of n and are never written again, and τ_n is monotone, written only at $\top$ and only by an O-Retire.

Proof. Let step t apply ρ at n. By Definition 53 it factors as ed $\boldsymbol{t}^t \circ \boldsymbol{\Psi}^t$, , where editt writes the fields the fifth column of Table 1 names and nothing else, and Ψ^t is id_Γ, , an application of one of n's iterations, or the accumulator g_n^t, , which is a composite of the inverses those iterations yielded. Each of the three is confined to n by Definition ^{48,} so Ψ^t writes no field of a fiber present at γ^t but σ_n, together with the entry a registration adds and the θ its inverse writes. The two halves therefore partition the writes, and each clause is that partition read at one field. One reading of the second and third columns is used twice: Inactive is the one lifecycle state carrying no committed view, L-Begin the one rule leading out of it, and L-Unload the one rule leading into it, while every other row carries the δ of its premise into its conclusion unchanged.

(1) An editt writes no table, the fifth column naming none, and a Ψ^t writes no σ_m for a present m $\neq$ n. So σ_m can move only at $m = n$, and only inside Ψ^t

(2) ω_n is a constituent of θ_n, , which only an editt writes and only at the fiber the step acts on, so by the reading above ω_n comes into existence at an L-Begin of n and ceases at an L-Unload of n. An episode of n is an interval on which installed l_n holds, hence one throughout which ω_n is defined, so neither rule falls in its interior.

(3) The fourth column, where an accumulator appears at L-Unload alone: the other rules take a forward map pr $\circ$ e or id_Γ, and no editt applies a map to the state at all.

(4) installed is $\theta_n \neq \mathsf{Inactive}(-)$, and by the reading above L-Begin and L-Unload are the only rules whose premise and conclusion difer in whether θ_n is Inactive. A step acting on some m $\neq$ n writes no θ_n, and the entry a registration adds is at a name not present at γ^t.

(5) No row of the fifth column names a $\pi, d, p,$ or$e$; those come into existence with the entry O-Insert adds, which its conclusion writes, as does the O-Insert a registration takes. Only O-Retire writes a τ, at $\top$. , whether taken by the orchestrator or as the inverse of a registration (Definition 47); $0\cdot$ -Insert sets $\tau = \perp$ at a name not already present, so no step returns $\mathsf{a}\tau$ to $\pm$.□

Three further lookups say what the rules cannot see. The first is that they read the state only through the observations above, so that the whole calculus descends to $\Gamma/\simeq$

Lemma 55. ($\simeq$-invariance.) Let $\gamma \simeq \gamma'$ as read above. Then a rule of Section 4.3 applies at γ acting on n if

2. ω_n 仅在 $\text{step}^t = \text{L}-\text{Begin}(n)$ 处产生、仅在 $\text{step}^t = \text{L-Unload(n)}$ 处消失，因此 ω_n^t 在 n 的片段内恒定；

3. $\Psi^t = g_n^t$ 仅在 stept = L-Unload(n) 处成立，且没有其他步骤把 g_n 应用于状态；

4. $\neg$ installedt $\wedge$ installe $\mathsf{l}_n^{t+1} \Rightarrow \text{step}^t = \text{L}-\text{Begin}(n)$ 且 installedt $\wedge \neg$ installed $\frac{1}{n}^{1+1} \Rightarrow$ stept = L-Unload(n)；

5. π_n, d_n, p_n, 和 e_n 随 n 的条目产生且永不再写，τ_n 单调，只在 $\top$ 处写且只由 O-Retire 写。

证明. 令步骤 t 在 n 处应用 ρ。由定义 53 它分解为 $\text{edit}^t \circ \boldsymbol{\Psi}^t$, ，其中 editt 只写表 1 第五列命名的字段，Ψ^t 是 id_Γ, 、n's 某次迭代的应用、或累加器 g_n^t, ，后者是那些迭代产生的逆的复合。三者中的每一个都由定义 ^{48,} 对 n 约束，因此 Ψ^t 不写 γ^t 在场纤维的任何字段（除 σ_n, 连同注册添加的条目和其逆写的 θ）。两半因此划分写入，而每个条款是那个划分在一个字段处的解读。第二、三列的一个读法被使用两次：Inactive 是唯一不携带已提交视图的生命周期状态，L-Begin 是唯一离开它的规则，L-Unload 是唯一进入它的规则，而每个其他行把其前提的 δ 原样带入其结论。

(1) editt 不写任何表（第五列不命名任何表），Ψ^t 不写在场 m $\neq$ n 的 σ_m。因此 σ_m 只能在 $m = n$, 处、且只在 Ψ^t 内移动。

(2) ω_n 是 θ_n, 的构成部分，只有 editt 写它且只写步骤作用的纤维，因此由上面的读法，ω_n 在 n 的 L-Begin 处产生、在 n 的 L-Unload 处消失。n 的片段是 installe l_n 成立的区间，因此是 ω_n 有定义的整个区间，两个规则都不落在其内部。

(3) 第四列中，累加器只在 L-Unload 处出现：其他规则取正向映射 pr $\circ$ e 或 id_Γ, 且没有 editt 把映射应用于状态。

(4) installed 是 $\theta_n \neq \mathsf{Inactive}(-)$ ，由上面的读法，L-Begin 和 L-Unload 是唯一其前提与结论在 θ_n 是否 Inactive 上不同的规则。作用于 m $\neq$ n 的步骤不写 θ_n, 而注册添加的条目位于 γ^t. 缺席的名字。

(5) 第五列没有行命名 $\pi, d, p,$ or$e$; 那些随 O-Insert 添加的条目产生，其结论写它们，注册采取的 O-Insert 也一样。只有 O-Retire 写 a τ, 在 $\top$. 处，无论由编排器采取还是作为注册的逆（定义 47); $0\cdot$ -Insert 在缺席名字处设 $\tau = \perp$ ，因此没有步骤把 $\mathsf{a}\tau$ 返回 $\pm$.□

三个进一步的查找说明规则看不到什么。第一个是它们只通过上面的观测读取状态，因此整个演算下降到 $\Gamma/\simeq$

引理 55.（$\simeq$-不变性。）令 $\gamma \simeq \gamma'$ 按上面读法。则第 4.3 节的规则在 γ 作用于 n 处适用当且仅当它在 γ' 作用

and only if it applies at γ' acting on n, and the states the two applications reach are again related by $\simeq$

于 n, 处适用，且两次应用达到的状态又由 $\simeq$ 相关

Proof. Every premise of Section 4.3 is of one of four kinds, and each reads a constituent the relation keeps. A premise matching θ_n or τ_n against a pattern, and the premise $\forall$m. $\pi_m \neq n$ of O-Remove, read control fields. The premises $(d, p, e) \in \mathfrak{C}_\Gamma$ and $\forall$m. $p \cap p_m = \mathcal{O}$ of O-Insert read d, p, and e.A premise mentioning ta get_n or relied_n reads τ_n, , the committed views inside the θ_m, and dom (σ_γ) , which Definition 45 computes from the θ_m and the dom (σ_m) , and Definition 33 relates two coeffect contexts only where their domains agree. The remaining premises read dom (F_γ) . None reads a value $\sigma_\gamma(k)$ otherwise than up to $\overset{\simeq'}{k}$, so no premise separates two $\simeq$-related states.

证明. 第 4.3 节的每个前提都属于四种之一，每个读取联系保留的构成部分。把 θ_n 或 τ_n 与模式匹配的前提，以及 O-Remove 的前提 $\forall$m. $\pi_m \neq n$ ，读取控制字段。O-Insert 的前提 $(d, p, e) \in \mathfrak{C}_\Gamma$ 和 $\forall$m. $p \cap p_m = \mathcal{O}$ 读取 d, p, 和 e.A 提及 ta $;\text{get}_n$ 或 relied_n 的前提读取 τ_n, 、θ_m 内部的已提交视图和 dom (σ_γ) ，后者由定义 45 从 θ_m 和 dom (σ_m) 计算，而定义 33 只在定义域一致时关联两个余效应上下文。剩余前提读取 dom (F_γ) 。没有一个以 $\overset{\simeq'}{k}$ 之外的方式读取值 $\sigma_\gamma(k)$ ，因此没有前提分离两个 $\simeq$-相关状态。

For the conclusion, $\gamma^{t+1} = \text{edit}^t\big(\Psi^t\big(\gamma^t\big)\big)$) by Definition 53. The values an editt assigns are the constituents of the premises it matched, related at the two states by the paragraph above and by Definition 51, which relates the triples an iterator yields at $\simeq$-related states. And Ψ^t respects $\simeq$ it is id , or an iteration of e_n, , which Definition 51 requires to respect $\simeq$, or the accumulator inside θ_n, a composite of inverses each respecting $\simeq$ by the same definition. □

对结论，由定义 53 $\gamma^{t+1} = \text{edit}^t\big(\Psi^t\big(\gamma^t\big)\big)$) 。editt 赋予的值是它匹配的前提的构成部分，由上面一段和定义 51 在两个状态处相关，定义 51 关联迭代器在 $\simeq$-相关状态处产生的三元组。而 Ψ^t 尊重 $\simeq$ 它是 id_Γ 、或 e_n, 的迭代，定义 51 要求其尊重 $\simeq$, 或 θ_n, 内部的累加器，即每个都按同一定义尊重 $\simeq$ 的逆的复合。□

The names a state carries are read by two of those observations, dom (F_γ) and the indexing of the control fields, and the rule that draws a name draws any name not already in use (Definition 47) . Reading the results below up to $\simeq$ therefore also calls for reading them up to a renaming, which is the discipline of Section 4.1 cashed out.

状态携带的名字由其中两个观测读取，dom (F_γ) 和控制字段的索引，而抽取名字的规则抽取任何未使用的名字（定义 47) 。因此把下面的结果读作 $\simeq$ 之下也需要把它们读作重命名之下，这是第 4.1 节纪律的兑现。

Lemma 56. (Equivariance.) Let $\chi : \mathfrak{N} \to \mathfrak{N}$ be a bijection and let $\chi \cdot \gamma$ be the state carrying the registry $F_\gamma \circ \chi^{-1}$, with every name occurring in a π_m or an ω_m replaced by its image. Then $\chi\ \gamma$ is a state, well formed where γ is, and ste $\text{p}^t = r(n)$ carries γ^t to γ^{t+1} if and only if $r(\chi(n))$) carries $\chi \cdot \gamma^t$ to $\chi \cdot \gamma^{t+1}$

引理 56.（等变性。）令 $\chi : \mathfrak{N} \to \mathfrak{N}$ 是双射并令 $\chi \cdot \gamma$ 是携带注册表 $F_\gamma \circ \chi^{-1}$ 的状态，π_m 或 ω_m 中出现的每个名字都被其像替换。则 $\chi\ \gamma$ 是状态，在 γ 良构处良构，且 ste $\text{p}^t = r(n)$ 把 γ^t 带到 γ^{t+1} 当且仅当 $r(\chi(n))$) 把 $\chi \cdot \gamma^t$ 带到 $\chi \cdot \gamma^{t+1}$

Proof. A premise reads a name only by comparing it with another, whether directly, as in the freshness $n \notin$ dom (F_γ) of O-Insert and the $\forall m.\ \pi_m \neq n$ of O-Remove, or through a table of names, as target t_n and relied_n read the π_m and the ω_m.A bijection preserves each such comparison. The only names a rule writes are the π that O-Insert sets and the ω that L-Begin sets, both taken from what its premises read, so the writes commute with χ; an effect function writes no name at all, drawing one only through the primitive of Definition ^{47,} , which Definition 48 confines to the entry that primitive adds. Well-formedness (Definition 58) is four conditions comparing names with names. □

证明. 前提只通过把它与另一名字比较来读取名字，无论是直接的（O-Insert 的新鲜性 $n \notin$ dom (F_γ) 和 O-Remove 的 $\forall m.\ \pi_m \neq n$ ），还是通过名字表（target_n 和 relied_n 读取 π_m 和 ω_m.A 双射保持每个这样的比较。规则只写 O-Insert 设置的 π 和 L-Begin 设置的 ω，两者都取自其前提所读，因此写入与 χ; 交换；效应函数完全不写名字，只通过定义 ^{47,} 的原语抽取一个，定义 48 把其约束到该原语添加的条目。良构性（定义 58）是四个把名字与名字比较的条件。□

A sequence and its renaming therefore take the same rules in the same order and reach states difering by χ alone. Two sequences agreeing save in the names their registrations draw are accordingly identified, and the results below are read up to the renaming that identifies them.

因此序列及其重命名以相同顺序采取相同规则并达到仅差 χ 的状态。除注册抽取的名字外一致的两个序列随之被等同，而下面的结果在该等同它们的重命名之下读出。

The second lookup is that an entry stripped of everything but its name is invisible to the rules, which is what lets Definition 47 retire a fiber where the state it recovers has none, and Lemma 72 remove the registrations a deleted episode made.

第二个查找是剥除名字外一切只剩名字的条目对规则不可见，这使定义 47 能在其恢复的状态没有该纤维处退役一个纤维，使引理 72 移除被删除片段做出的注册。

Lemma 57. (Vestigial entries.) Call n vestigial at γ when $\tau_n = \top, \theta_n = \mathsf{Inactive}(\bot), \sigma_n = \emptyset$, and no m has $\pi_m = n$; a vestigial entry satisfies $\gamma \approx \gamma \setminus n$. If n is vestigial at γ then for every rule and every m $\neq$ n:

引理 57.（残余条目。）当 $\tau_n = \top, \theta_n = \mathsf{Inactive}(\bot), \sigma_n = \emptyset$ 且没有 m 有 $\pi_m = n$; 时称 n 在 γ 处是残余的；残余条目满足 $\gamma \approx \gamma \setminus n$ 。若 n 在 γ 处残余，则对每个规则和每个 m $\neq$ n：

1. a rule applying at γ acting on n applies at $\gamma\setminus$ n acting on m, and the states the two reach difer in the entry at n alone, which stays vestigial;

1. 在 γ 作用于 n 的规则在 $\gamma\setminus$ n 作用于 m, 处适用，且两者达到的状态只在 n 的条目处不同，后者保持残余；

2. conversely a rule applying at $\gamma \setminus n$ acting on n applies at γ, , unless it is an O-Insert drawing the name n or claiming a key of p_n

2. 反之在 $\gamma \setminus n$ 作用于 n 的规则在 γ, 处适用，除非它是抽取名字 n 或主张 p_n 的键的 O-Insert

Proof. A vestigial n contributes to no observation a premise of a rule acting on m $\neq n$ reads. It is not Active,

证明. 残余 n 不贡献作用于 m $\neq n$ 的规则前提读取的任何观测。它不 Active，因此 σ_n 不进任何 σ_γ ，n 不

so σ_n enters no σ_γ and n is the provider of no key, leaving $\gamma \models d_m$ and target_m unmoved; installed l_n fails, so n contributes no disjunct to relied $_m$; no $\pi_{m'}$ names n, so the premise $\forall m'.\pi_{m'} \neq \text{n}$ of an O-Remove of n is unmoved; and θ_n, τ_n, and π_n are read by rules acting on n alone. The two premises clause (2) excepts are the two the removal relaxes, an absent name being fresh and an absent provision meeting every other. By Lemma 54 no rule acting on $m \neq n$ writes a field of n, so the entry survives, and the state map of the step is confined to n by Definition 48, so it leaves σ_n empty. □

Simplifying the lifecycle states, together with the rules that match on them, yields a subcalculus, and not every result survives the simplification. Dropping Section 4.3.1 is the case that matters, which is the division Section 4.3 opens with, read from the metatheory's side: its guard is what establishes clauses (3) and (4) of Definition 58, and Theorem 63 rests on the interval the guard creates, so those three fail without it. What the other three subsections add can be simplified away without disturbing the results below, each of them only adding rules to the one state space Definition 49 fixes.

4.4.1. Preservation

Definition 45 fixes the shape of a registry, and the rules have to be checked against it before the results below can add to it. This subsection identifies the invariant the rules preserve, of which the first clause is that shape and the rest what those results assume.

Definition 58. A registry F_γ is well formed when, for all $m, n \in \text{dom}(F_\gamma)$ and all $k \in K$

1. $\pi_n \in \text{dom}(\text{F}\gamma) \cup \{\text{root}\}$;
2. $\text{m} \neq n \Rightarrow p_m \cap p_n = \emptyset$;
3. ins $\text{talled}_n(\gamma) \Rightarrow \omega_n$ is total on d_n and valued in dom (F_γ) ;
4. $\text{installed}_\text{n}(\gamma) \wedge \text{k} \in \text{dn} \wedge \omega\text{n}(\text{k}) = \text{m} \Rightarrow \text{installed}_\text{m}(\gamma)$.

Clause (1) is the tree of Definition 45 read one edge at a time, keeping a parent pointer landing in the registry. The acyclicity that definition also requires needs no clause, since the fiber a pointer names is registered before the fiber naming it.

Theorem 59. (Preservation.) If F^t is well formed then so is F^{t+1} , whichever rule step t applies. Each clause is established at γ^{t+1} from all four at γ^t .

Proof. Let step t act on n.

(1) By Table 1 only O-Insert and O-Remove write a π or dom (F_γ) . O-Insert has $\pi_n \in \text{dom}(F^t) \mid \cup \{\text{root}\}$ as a premise, which is the clause for the fiber it adds, and it leaves every other π alone while enlarging dom (F_γ) . O-Remove has $\forall \text{m}.\ \pi_m \neq n$, , so no surviving π_m names the fiber it takes away.

(2) The last premise of O-Insert is $\forall \text{m}.\ p_n \cap p_m = \emptyset$, which is the clause for the fiber it adds, and by Table 1 no other rule writes a π or enlarges dom (F_γ) . Two consequences are used below: dom $(\sigma_m) \subseteq p_m$ by Definition ^{43,} so distinct tables are disjoint and σ_γ is a function; and $k \in p_m \cap p_{m'}$ forces $m = m'$, so k has at most one possible provider.

提供任何键，使 $\gamma \models d_m$ 和 target_m 不动；installed l_n 失败，因此 n 不贡献 relied $_m$; 的析取；没有 $\pi_{m'}$ 命名 n, 因此 n 的 O-Remove 的前提 $\forall m'.\pi_{m'} \neq \text{n}$ 不动；而 θ_n, τ_n, 和 π_n 由作用于 n 的规则读取。条款 (2) 例外的两个前提是移除所放松的两个，缺席名字是新名、缺席提供满足每个其他提供。由引理 54，作用于 $m \neq n$ 的规则不写 n 的字段，因此条目存活，且步骤的状态映射由定义 48 对 n 约束，因此它使 σ_n 为空。□

简化生命周期状态以及匹配它们的规则得到一个子演算，且并非每个结果都在简化下存活。放弃第 4.3.1 节是重要情形，即第 4.3 节开头从元理论侧读出的划分：其守卫确立定义 58 的条款 (3) 和 (4)，而定理 63 依赖守卫创造的区间，因此没有它这三者失败。其他三个小节添加的内容可以在不扰动下面结果的情况下简化掉，每个都只是向定义 49 固定的同一状态空间添加规则。

4.4.1. 保真（Preservation）

定义 45 固定注册表的形状，在下面的结果能向它添加内容之前，规则必须对照它检查。本小节识别规则保持的不变量，其第一条款就是那个形状，其余是那些结果所假设的。

定义 58. 注册表 F_γ 是良构的，当对所有 $m, n \in \text{dom}(F_\gamma)$ 和所有 $k \in K$

1. $\pi_n \in \text{dom}(\text{F}\gamma) \cup \{\text{root}\}$；
2. $\text{m} \neq n \Rightarrow p_m \cap p_n = \emptyset$;
3. ins $\text{talled}_n(\gamma) \Rightarrow \omega_n$ 在 d_n 上全定义且取值在 dom (F_γ) ；
4. $\text{installed}_\text{n}(\gamma) \wedge \text{k} \in \text{dn} \wedge \omega\text{n}(\text{k}) = \text{m} \Rightarrow \text{installed}_\text{m}(\gamma)$。

条款 (1) 是定义 45 的树一次一条边读出，保持父指针落在注册表内。该定义还要求的非环性不需要条款，因为指针命名的纤维在命名它的纤维注册之前注册。

定理 59.（保真。）若 F^t 良构则 F^{t+1} 良构，无论步骤 t 应用哪个规则。每个条款在 γ^{t+1} 处由 γ^t 的全部四个确立。

证明. 令步骤 t 作用于 n。

(1) 由表 1 只有 O-Insert 和 O-Remove 写 π 或 dom (F_γ) 。O-Insert 以 $\pi_n \in \text{dom}(F^t) \mid \cup \{\text{root}\}$ 为前提，即它添加的纤维的条款，并在扩大 dom (F_γ) 的同时让每个其他 π 独处。O-Remove 以 $\forall \text{m}.\ \pi_m \neq n$, 为前提，因此没有存活的 π_m 命名它带走的纤维。

(2) O-Insert 的最后一个前提是 $\forall \text{m}.\ p_n \cap p_m = \emptyset$ ，即它添加的纤维的条款，且由表 1 没有其他规则写 π 或扩大 dom (F_γ) 。下面使用两个推论：dom $(\sigma_m) \subseteq p_m$ 由定义 ^{43,} 因此不同表不相交且 σ_γ 是函数；$k \in p_m \cap p_{m'}$ 迫使 $m = m'$ ，因此 k 至多有一个可能的提供者。

(3) By Lemma 54(2) the only rule that writes an ω_n is L-Begin, whose premise $\omega = \text{target}_n^t \neq \bot$ makes it total on d_n and valued in dom $\left(F^t\right)$, target naming providers. By Table 1 the only rule that shrinks dom (F_γ) is O-Remove, whose premise $\theta_n^t = \mathsf{Inactive}(-)$ gives $\neg\text{installed}_n^t$, whence by clause (4) at γ^t no m has $\omega_m^t(k) = n$ for a $k \in d_m$ while installe l_m^t; and n itself carries no ω.

(4) By Lemma 54(2) and (4) the clause can fail at γ^{t+1} only where some installed has fallen, some ω has been written, or a fiber some π names has left dom (F_γ). The last is an O-Remove, whose removed fiber is not installed and hence, by clause (4) at $\boldsymbol{\gamma}^t$, is named by no ω_m^t of an installed $m.$. The first is an L-Unload of n, whose premise $\mod {}_n^t$ reads

$$\forall m \neq n, k \in d_m.\text{installed}_m^t \Rightarrow \omega_m^t(k) \neq n$$

and which writes no ω_m for m $\neq n$ and leaves $\neg\text{installed}_n^{t+1}$, so the clause holds of n as well. The second is an L-Begin of n, writing target_n^t, , whose values are the providers of the keys of d_n and hence Active at γ^t; the step alters no other fiber$'\mathbf{s}\theta$, so they are installed at γ^{t+1} too. □

The guard on L-Unload is what carries clauses (3) and (4). The premise ∀m. $\pi_m \neq$ n of O-Remove speaks only of parent pointers; what keeps a committed view from naming a removed fiber is the guard, imposed several steps earlier and for a diferent reason. Because a failure is routed through Unloading as well, the argument does not have to be repeated for an error outcome. Two things follow that the base calculus does not enjoy. A name freed by O-Remove may be reissued by O-Insert, since no stale committed view can name it; and a fiber may be removed as soon as it is Inactive, without a separate check that nobody depends on it.

4.4.2. Temporal Composability

Local temporal composability recovers one sequence of efects with one accumulator (Section 3.1.3). The registry holds one accumulator per fiber and the fibers interleave: between the moment n composes an inverse onto g_n and the moment g_n runs, other fibers have moved the state. Whether g_n still undoes what it was built to undo there is what the global form of the guarantee asserts, and the condition it turns on is that the intervening steps commute with g_n

Definition 60. For $i \in \mathfrak{C}_\Gamma^{\text{iter}*}$ let reach(i) be the least set of iterators containing i and closed under continuation, and read the transformation monoid M of Definition 17 at an iterator by taking for its generators the forward maps and the yielded inverses of every iterator in reach(i):

$$\begin{aligned} \text{reach}(i) &:= \bigcap\{S \mid i \in S \wedge \forall i' \in S, \gamma \in \Gamma. i'(\gamma) = (-,-,\text{Just}(i'')) \Rightarrow i'' \in S\} \\ \mathfrak{M}(i) &:= \langle\{\text{pr}_1 \circ i' \mid i' \in \text{reach}(i)\} \cup \{\text{pr}_2(i'(\gamma)) \mid i' \in \text{reach}(i), \gamma \in \Gamma\}\rangle \end{aligned} \tag{54}$$

reading Right around the triple where Section 4.3.4 applies, and write len(i) for the supremum of $|C|$ over the chains $C \subseteq \text{reach}(i)$ that continuation orders. Two iterators i, j are independent when they are so in the sense of Definition 19, read with these transformation monoids and with the yield of an iteration being its inverse together with its continuation:

$$\begin{aligned} &\forall f \in \mathfrak{M}(i), g \in \mathfrak{M}(j). \quad f \circ g \simeq g \circ f \\ &\forall i' \in \text{reach}(i), g \in \mathfrak{M}(j), \gamma \in \Gamma. \quad \text{pr}_{2,3}(i'(g(\gamma))) \simeq \text{pr}_{2,3}(i'(\gamma)) \end{aligned} \tag{55}$$

and symmetrically in j, reading $\simeq$ on maps as Definition 36 does, on a continuation as Definition 51 does,

(3) 由引理 54(2) 唯一写 ω_n 的规则是 L-Begin，其前提 $\omega = \text{target}_n^t \neq \bot$ 使它全定义在 d_n 上且取值在 dom $\left(F^t\right)$，target 命名提供者。由表 1 唯一缩小 dom (F_γ) 的规则是 O-Remove，其前提 $\theta_n^t = \mathsf{Inactive}(-)$ 给出 $\neg\text{installed}_n^t$，当由 γ^t 的条款 (4) 没有 n 在 installe l_m^t; 时对 $k \in d_m$ 有 $\omega_m^t(k) = n$；且 n 本身不携带 ω。

(4) 由引理 54(2) 和 (4)，条款只能在 γ^{t+1} 处失败，若某个已安装者倒下、某个 ω 被写、或某 π 命名的纤维离开 dom (F_γ)。最后者是 O-Remove，其被移除纤维不安装，因此由 $\boldsymbol{\gamma}^t$ 的条款 (4) 不被已安装 $m.$ 的任何 ω_m^t 命名。第一个是 $n,$ 的 L-Unload，其前提 $\mod {}_n^t$ 读

$$\forall m \neq n, k \in d_m.\text{installed}_m^t \Rightarrow \omega_m^t(k) \neq n$$

且它不写 m $\neq n$ 的任何 ω_m、留下 $\neg\text{installed}_n^{t+1}$，因此条款也对 n 成立。第二个是 $n,$ 的 L-Begin，写 target_n^t, 其值是 d_n 各键的提供者、因此在 γ^t 处 Active；该步骤不改变其他 fiber$'\mathbf{s}\theta$, 因此它们在 γ^{t+1} 处也已安装。□

L-Unload 上的守卫是承载条款 (3) 和 (4) 的东西。O-Remove 的前提 ∀m. $\pi_m \neq$ n 只谈父指针；使已提交视图不命名被移除纤维的是守卫，它在数步之前因不同理由施加。因为失败也经 Unloading 路由，该论证不必为错误结果重复。基础演算不享有的两件事随之而来。O-Remove 释放的名字可被 O-Insert 重新签发，因为没有过期的已提交视图能命名它；纤维一旦 Inactive 即可移除，无需单独检查是否有人依赖它。

4.4.2. 时间可组合性（Temporal Composability）

局部时间可组合性用一个累加器恢复一个效应序列（第 3.1.3 节）。注册表为每个纤维持有一个累加器，而纤维交错：在 n 把逆复合到 g_n 的时刻与 g_n 运行的时刻之间，其他纤维已移动状态。g_n 是否仍在那里撤销它被构造来撤销的东西是全局形式所断言的，而它所依赖的条件是介入步骤与 g_n 交换。

定义 60. 对 $i \in \mathfrak{C}_\Gamma^{\text{iter}*}$ 令 reach(i) 是包含 i 并在续延下封闭的最小迭代器集，并把定义 17 的变换幺半群 M 在迭代器处读出，以 reach(i) 中每个迭代器的正向映射和产生的逆作为其生成元：

$$\begin{aligned} \text{reach}(i) &:= \bigcap\{S \mid i \in S \wedge \forall i' \in S, \gamma \in \Gamma. i'(\gamma) = (-,-,\text{Just}(i'')) \Rightarrow i'' \in S\} \\ \mathfrak{M}(i) &:= \langle\{\text{pr}_1 \circ i' \mid i' \in \text{reach}(i)\} \cup \{\text{pr}_2(i'(\gamma)) \mid i' \in \text{reach}(i), \gamma \in \Gamma\}\rangle \end{aligned} \tag{54}$$

在第 4.3.4 节适用处把 Right 读作三元组周围，并写 len(i) 为续延排序的链 $C \subseteq \text{reach}(i)$ 上 $|C|$ 的上确界。两个迭代器 i, j 独立，当它们按定义 19 的意义独立，这些变换幺半群读出，迭代的产出是其逆连同其续延：

$$\begin{aligned} &\forall f \in \mathfrak{M}(i), g \in \mathfrak{M}(j). \quad f \circ g \simeq g \circ f \\ &\forall i' \in \text{reach}(i), g \in \mathfrak{M}(j), \gamma \in \Gamma. \quad \text{pr}_{2,3}(i'(g(\gamma))) \simeq \text{pr}_{2,3}(i'(\gamma)) \end{aligned} \tag{55}$$

在 j 中对称，映射上的 $\simeq$ 按定义 36 读、续延上的按定义 51 读、注册迭代（定义 47）上的按它命名的组件

and on a registering iteration (Definition 47) as agreement of the component it names. A family $(i_l)_{l\in L}$ of iterators is pairwise independent when i_l and $i_{l'}$ are independent for every $l \neq l'$. , and a sequence of steps is pairwise independent when $(e_n)_{n\in N}$ is, where N is the set of names the sequence ever holds, one for each fiber the orchestrator inserts and each fiber an iteration registers.

Independence in this sense is what trace theory takes as primitive: commuting actions generate an equivalence on sequences under which reordering two adjacent independent actions preserves the endpoint [44], and Lemma 71 is that reordering for these rules. A family rather than a set is what keeps two names of one component in scope: the condition then requires that component's effect function to be independent of itself, which is to require that M(e) be commutative. The first condition is what Theorem 61 uses and the second what Theorem 73 needs in addition: reordering the steps of two fibers evaluates an iterator at a state the other fiber moved, and commuting the maps does not by itself say that the iterator yields the same inverse and the same continuation there. Checking the first condition calls for no more than the iterations themselves, since Lemma 18(1) carries commutation from the generators to the monoids they generate.

Under these conditions the single-accumulator invariant of Theorem 7 survives the interleaving, in the form that gives temporal composability its content: running an inverse withdraws the fiber's contribution and nothing else.

Theorem 61. (Recovery exactness.) Let the sequence of steps be pairwise independent, let an episode of n open at b, let $u \geq b$ lie in it, and let $t_1 < \cdots < t_l$ be the indices in $[b, u)$ at which the acting fiber is not n. Then

$$g_n^u(\gamma^u) \approx \left(\Psi^{t_l} \circ \cdots \circ \Psi^{t_1}\right)\left(\gamma^b\right) \tag{56}$$

That is, applying n's accumulator at γ^u yields, up to the control fields, the state those same steps would have produced from γ^b . Reading the right side as the state reached had n never begun assumes in addition that no fiber n registers take a step in $[b, u)$, since a fiber n registers is one that would not be there to take it.

Proof. By induction on u, over the indices t with t + 1 in the episode. At t = b the step at $b-1$ an L-Begin, the episode opening by Definition 53, so $g_n^b = \mathrm{id}_\Gamma$ by Table 1, the index set is empty, and the claim is $\bar{\gamma}^b \approx \gamma^b$. Two facts are used at each step. Since editt writes control fields only,

$$\gamma^{t+1} \approx \Psi^t(\gamma^t)$$

and since every map in $\mathfrak{M}(e_n)$ writes no control field but those a registration adds, by Definition 48 together with Definition 47, each such map carries $\approx$-equal states to $\approx$-equal states.

Let step t act on n. Since the episode is open at t and $u + 1$, Lemma 54(4) excludes an L-Begin and an L-Unload of n, and O-Insert and O-Remove read a θ_n that installedt denies, leaving two cases. Where the rule is L-Iter, L-Finish, or a landing L-Divert, Table 1 gives $\Psi^u = \mathrm{pr}_1 \circ i_n^u$ and $g_n^{u+1} = g_n^u \circ \hbar$ for the inverse $\hbar$ that iteration yields. The witness condition of Definition 51 reads $h(\Psi^u(\gamma^u)) = \gamma^u$, up to $\approx$ where the iteration registers a fiber (Lemma 57), and g_n^u carries $\approx$ by the equation above, so

$$g_n^{u+1}(\gamma^{u+1}) \approx (g_n^u \circ h)(\Psi^u(\gamma^u)) = g_n^u(\gamma^u)$$

Where the rule is L-Leave, L-Raise, an aborting L-Divert, or an O-Retire of n, Table 1 gives $\Psi^u = \mathrm{id}_\Gamma$ and $g_n^{u+1} = g_n^u$, so the same equation holds with $h = \mathrm{id}_\Gamma$. Either way the induction hypothesis carries over with the index set unchanged, which is the computation of Theorem 7 one step at a time.

Let step t act on n $\neq$ $_n$. Then $g_n^{u+1} = g_n^u$ by Table 1, and $\Psi^u \in \mathfrak{M}(e_m)$, or $\Psi^u = \mathrm{id}_\mathrm{I}$ where the rule is an

的一致读。迭代器族 $(i_l)_{l\in L}$ 两两独立，当对每个 $l \neq l'$ 的 i_l 和 $i_{l'}$ 独立，而步骤序列两两独立，当 $(e_n)_{n\in N}$ 如此，其中 N 是序列曾持有的名字集，编排器插入的每个纤维和迭代注册的每个纤维各一。

这个意义上的独立性正是迹理论取其原始的东西：交换动作在序列上生成等价，在其下重排两个相邻独立动作保持端点 [44]，而引理 71 对这些规则就是那个重排。族而非集是保持一个组件两个名字在作用域内的东西：条件随后要求该组件的效应函数与自身独立，即要求 M(e) 交换。第一条件定理 61 使用，第二条件定理 73 还需要：重排两个纤维的步骤在另一纤维移动的状态处求值迭代器，而交换映射本身并不说明迭代器在那里产生相同的逆和相同的续延。检查第一条件只需迭代本身，因为引理 18(1) 把交换性从生成元带到它们生成的幺半群。

在这些条件下，定理 7 的单累加器不变量在交错下存活，形式是给时间可组合性以内容的那个：运行逆撤出纤维的贡献而别无其他。

定理 61.（恢复精确性。）令步骤序列两两独立，令 n 的片段在 b 处打开、$u \geq b$ 位于其中，并令 $t_1 < \cdots < t_l$ 是 $[b, u)$ 中作用纤维非 n 的索引。则

$$g_n^u(\gamma^u) \approx \left(\Psi^{t_l} \circ \cdots \circ \Psi^{t_1}\right)\left(\gamma^b\right) \tag{56}$$

即，在 γ^u 处应用 n 的累加器产生，在控制字段之下，那些相同步骤从 γ^b 产生的状态。把右侧读作 n 从未开始会达到的状态还额外假设没有 n 注册的纤维在 $[b, u)$ 采取步骤，因为 n 注册的纤维是本来不会在那里采取它的纤维。

证明. 对 u, 在片段内满足 t + 1 的索引 t 上归纳。在 t = b 处，$b-1$ 的步骤是 L-Begin（片段按定义 53 打开），因此由表 1 $g_n^b = \mathrm{id}_\Gamma$ ，索引集为空，断言是 $\bar{\gamma}^b \approx \gamma^b$ 。每步使用两个事实。由于 editt 只写控制字段，

$$\gamma^{t+1} \approx \Psi^t(\gamma^t)$$

且由于 $\mathfrak{M}(e_n)$ 中每个映射除注册添加的控制字段外不写任何控制字段（由定义 48 连同定义 47），每个这样的映射把 $\approx$-相等状态带到 $\approx$-相等状态。

令步骤 t 作用于 n。由于片段在 t 和 $u + 1$ 打开，引理 54(4) 排除 n, 的 L-Begin 和 L-Unload，而 O-Insert 和 O-Remove 读取 installedt 所否定的 θ_n ，留下两种情形。规则是 L-Iter、L-Finish 或落地 L-Divert 时，表 1 给出 $\Psi^u = \mathrm{pr}_1 \circ i_n^u$ 和 $g_n^{u+1} = g_n^u \circ \hbar$，其中 $\hbar$ 是该迭代产生的逆。定义 51 的见证条件读 $h(\Psi^u(\gamma^u)) = \gamma^u$ ，在迭代注册纤维处读到 $\approx$（引理 57），而 g_n^u 由上式携带 $\approx$，因此

$$g_n^{u+1}(\gamma^{u+1}) \approx (g_n^u \circ h)(\Psi^u(\gamma^u)) = g_n^u(\gamma^u)$$

规则是 L-Leave、L-Raise、中止 L-Divert 或 n, 的 O-Retire 时，表 1 给出 $\Psi^u = \mathrm{id}_\Gamma$ 和 $g_n^{u+1} = g_n^u$, 因此同一等式以 $h = \mathrm{id}_\Gamma$ 成立。两种情形下归纳假设以不变的索引集传递，这就是定理 7 一次一步的计算。

令步骤 t 作用于 n $\neq$ $_n$ 。则表 1 给出 $g_n^{u+1} = g_n^u$ ，且 $\Psi^u \in \mathfrak{M}(e_m)$ ，或规则是编排规则时 $\Psi^u = \mathrm{id}_\mathrm{I}$ ，因此独

orchestration rule, so independence gives

$$g_n^u(\gamma^{u+1}) \approx g_n^u(\Psi^u(\gamma^u)) = \Psi^u(g_n^u(\gamma^u))$$

which is the induction hypothesis with Ψ^u appended.

Corollary 62. (Terminal recovery.) Let the sequence of steps be pairwise independent and let an episode of n open at b and close at u, whatever outcome n arrives at. Then, with $t_1 < \cdots < t_l$ as in Theorem 61,

$$\gamma^{u+1} \approx \left(\Psi^{t_l} \circ \cdots \circ \Psi^{t_1}\right)\left(\gamma^b\right) \tag{57}$$

A fiber removed by O-Remove leaves nothing behind either, its premise admitting only $\theta_n = \text{Inactive}(-)$.

Proof. By Lemma 54(4) step t is an L-Unload of n, whose Ψ^u is g_n^u by Lemma 54(3), $\mathbf{so} \gamma\gamma^{u+1} \approx g_n^u(\gamma^u)$ and Theorem 61 applies. Neither the statement nor $\approx$ mentions ζ, which by Table 1 is the one field in which the states L-Divert and L-Raise lead to difer. □

Pairwise independence is assumed of the components by the results above, and Section 3.3.2 is what discharges it: where every effect a component performs is an operation of a key and every key is commutative, any two effect functions built from those operations are independent (Theorem 42). Carrying that result from effect functions to iterators calls for nothing new, a coeffect-mediated effect function (Definition 41) already choosing what follows each stage by the outcome that stage yields, which is what an iterator carries in its continuation. The coeffect operations of Section 3.2 are the case that needs no hypothesis at all: the maps a component contributes there are composites of set operations and of the corresponding restrictions, two such commute whenever they touch disjoint keys, and clause (2) of Definition 58 makes the provisions of distinct fibers disjoint.

4.4.3. Spatial Composability

Local spatial composability holds a component to its own specification, activating it only where its dependencies are provided and classifying every context change against them (Section 3.2.2). The global form adds what quantifies over other fibers: a provider withdraws a binding only after every dependent that resolved it has deactivated, and the resolution a transition installs its efects against does not shift under it. Two properties of the coeffect side deliver the two, and they are proved together, being two halves of one invariant, namely the fixity of ω_n over an episode that Lemma 54(2) establishes. The ordering theorem is what that fixity buys over the part of the episode in which n is Active and then Unloading, and the coherence theorem what it buys over the part in which n is installing its efects.

Theorem 63. (Ordering.) A fiber begins a transition only where its dependencies are provided:

$$\text{step}^t = \text{L} - \text{Begin}(m) \Rightarrow \gamma^t \vDash d_m \tag{58}$$

Let further $[b', u']$ be an episode of n with $\omega_m^{b'}(k) = n$ for some m $\neq$ n and $k \in d_m$, , let $[b, u]$ be the episode of n containing b', , and let t range over $[b', u']$. Then

1. $\omega_m^t(k) = n$;

2. $b < b'$. , and $u' < u$ if $[b, u]$ closes;

3. $k \in \text{dom}(\sigma_n^t)$ and $\sigma_n^t(k) = \sigma_n^{b'}(k)$

立性给出

$$g_n^u(\gamma^{u+1}) \approx g_n^u(\Psi^u(\gamma^u)) = \Psi^u(g_n^u(\gamma^u))$$

即带追加 Ψ^u 的归纳假设。

推论 62.（终点恢复。）令步骤序列两两独立并令 n 的片段在 b 处打开、在 u, 处闭合，无论 n 到达什么结果。则，$t_1 < \cdots < t_l$ 如定理 61，

$$\gamma^{u+1} \approx \left(\Psi^{t_l} \circ \cdots \circ \Psi^{t_1}\right)\left(\gamma^b\right) \tag{57}$$

O-Remove 移除的纤维也不留下任何东西，其前提只接纳 $\theta_n = \text{Inactive}(-)$。

证明. 由引理 54(4) 步骤 t 是 n, 的 L-Unload，其 Ψ^u 由引理 54(3) 是 $g_n^u, \mathbf{so} \gamma\gamma^{u+1} \approx g_n^u(\gamma^u)$ 而定理 61 适用。声明和 $\approx$ 都不提 ζ, 由表 1 它是 L-Divert 和 L-Raise 导致的状态不同的唯一字段。□

上面的结果假设组件两两独立，而第 3.3.2 节结清它：在组件执行的每个效应是一个键的操作且每个键交换处，任何两个由这些操作构建的效应函数独立（定理 42）。把该结果从效应函数带到迭代器无需新东西，余效应中介的效应函数（定义 41）已经按阶段产生的结果选择其后续，这正是迭代器在其续延中携带的东西。第 3.2 节的余效应操作是完全无需假设的情形：组件在那里贡献的映射是 set 操作和相应限制的复合，两个这样的映射在触碰不相交键时交换，而定义 58 的条款 (2) 使不同纤维的提供不相交。

4.4.3. 空间可组合性（Spatial Composability）

局部空间可组合性把组件保持在其自身规范上，只在依赖被提供处激活它，并依据规范分类每次上下文变化（第 3.2.2 节）。全局形式加入量化其他纤维的东西：提供者只在每个把某键解析到它的依赖方已停用后才撤出该绑定，且转移据以安装其效应的解析不会在其下移动。余效应侧的两个性质交付两者，且它们一起证明，因为它们是一个不变量的两半，即引理 54(2) 确立的 ω_n 在片段上的固定性。排序定理是该固定性在 n 是 Active 然后 Unloading 的片段部分上买来的，相干定理是它在 n 安装其效应的部分上买来的。

定理 63.（排序。）纤维只在依赖被提供处开始转移：

$$\text{step}^t = \text{L} - \text{Begin}(m) \Rightarrow \gamma^t \vDash d_m \tag{58}$$

再令 $[b', u']$ 是 n 的片段，对某个 m $\neq$ n 和 $k \in d_m$ 有 $\omega_m^{b'}(k) = n$, 令 $[b, u]$ 是包含 b', 的 n 的片段，并令 t 遍历 $[b', u']$ 。则

1. $\omega_m^t(k) = n$;

2. $b < b'$. ，且 $[b, u]$ 闭合则 $u' < u$;

3. $k \in \text{dom}(\sigma_n^t)$ 且 $\sigma_n^t(k) = \sigma_n^{b'}(k)$

Proof. The first claim is the premise t $\text{arget}_m^t \neq \perp$ of L-Begin, which by Definition 46 gives $\gamma^t \vDash d_m$

(1) is Lemma 54(2).

For (2), the L-Begin at $b'-1$ writes $\omega_m^{b'} = \text{target}_m^{b'-1}$, whose values are providers, so $\theta_n^{b'} = \mathsf{Active}(-,-)$; the L-Begin at $b-1$ leaves $\theta_n^b = \mathsf{Reloading}(-,-,-)$, so $b \neq b'$ and hence $b < b'$ both episodes opening by Definition 53. Let $[b,u]$ close and suppose $u \leq u'$. Then $u \in [b',u']$. so installed $\mathbb{I}_m^u$ and, by (1), $\omega_m^u(k) = n$; that is reliedn, which the L-Unload at n denies. Hence $u' < u$

For (3), n is the provider of k at $\gamma^{b'}$, so $k \in \text{dom}\left(\sigma_n^{b'}\right)$. No L-Unload of n falls in $[b',u']$: where $[b,u]$ closes it falls at $u > u'$ by (2), and where it does not, Lemma 54(4) leaves n with no L-Unload at all. Since $\theta_n^{b'} = \mathsf{Active}(-,-)$, Table 1 therefore leaves L-Leave as the only rule n can be acted on by within $[b',u']$, and its Ψ^t is id_Γ; by Lemma 54(1) σ_n is constant there. □

A transition spread over steps could otherwise install efects computed against a resolution that has changed under it, and two premises prevent that. L-Iter and L-Finish carry $\text{target}_n(\gamma) = \omega$, , so a transition proceeds only while its committed view is still its target view, and L-Divert carries the negation, so any change to the target view takes the fiber out of the transition. L-Raise is not conditioned on the target view at all, a raise being something the iteration does rather than something the environment asks for, and it exits the transition in any case. The two directions of change are not distinguished: a component whose dependency has gone and one whose dependency has been replaced leave by the same route, because a target view that has become $\perp$ and one that has become some other fiber are equally unequal to ω.

Inertia is what stops this from being a guarantee about every step. An iteration already in flight when the target view turns lands regardless, by L-Divert, and that landing installs an effect computed against a resolution that no longer holds. What the rules deliver is therefore a disjunction, and the second branch is what makes the first safe.

Theorem 64. (Resolution coherence.) Let an episode $[b,u]$ of n open at b with $\omega_n^b = \omega$. Then θ_n is Reloading $(-,-,-)$ on an initial interval $[\bar{b},r]$ of the episode, and every iteration of the transition runs against the one resolution ω:

$$\forall t \in [b,r].\text{step}^t \in \{\text{L}-\text{Iter}(n), \text{L}-\text{Finish}(n)\} \Rightarrow \text{target}_n^t = \omega \tag{59}$$

Where the fiber leaves that interval, so that $r < u$, , exactly one of the following holds:

1. ste $\omega)^r = \text{L}-\text{Finish}(n)$ and $\theta_n^{r+1} = \mathsf{Active}(-,\omega)$; ;

2. st $\mathsf{sp}^r \in \{\text{L}-\text{Divert}(n), \text{L}-\text{Raise}(n)\}$, and the episode closes at some $u > r$ with $\gamma^{u+1} \approx \left(\Psi^{t_l} \circ \cdots \circ \Psi^{t_1}\right)\left(\gamma^b\right)$ as in Corollary 62.

Proof. The L-Begin at $b-1$ writes Reloading, and by Table 1 it is the one rule leading into that lifecycle state; its premise $\theta_n = \mathsf{Inactive}(\perp)$ and Lemma 54(4) put any second application of it outside the episode. So Reloading occupies an initial interval $[b,\bar{r}]$of$[\bar{b},u]$ and is not re-entered.

The first claim is then the premise target $\mathfrak{t}_n(\gamma) = \omega'$ that Table 1 gives L-Iter and L-Finish, together with $\omega' = \omega$ by Lemma 54(2)

For the dichotomy, stept is a rule whose premise has $\theta_n = \mathsf{Reloading}(-,-,-)$ and whose conclusion does not, of which Table 1 ofers L-Finish, L-Divert, and L-Raise; the first lands in $\mathsf{Active}(-,\omega)$ and the other two in

证明. 第一个断言是 L-Begin 的前提 t $\text{arget}_m^t \neq \perp$ ，由定义 46 它给出 $\gamma^t \vDash d_m$

(1) 是引理 54(2)。

对 (2)，$b'-1$ 处的 L-Begin 写 $\omega_m^{b'} = \text{target}_m^{b'-1}$ ，其值是提供者，因此 $\theta_n^{b'} = \mathsf{Active}(-,-)$; $b-1$ 处的 L-Begin 留下 $\theta_n^b = \mathsf{Reloading}(-,-,-)$ ，因此 $b \neq b'$ 从而 $b < b'$ 两者片段都按定义 53 打开。令 $[b,u]$ 闭合并假设 $u \leq u'$ 。则 $u \in [b',u']$. 因此 installed $\mathbb{I}_m^u$ 且由 (1), $\omega_m^u(k) = n$; 那是 reliedn，n 处的 L-Unload 否认它。因此 $u' < u$

对 (3)，n 在 $\gamma^{b'}$ 处是 k 的提供者，因此 $k \in \text{dom}\left(\sigma_n^{b'}\right)$ 。$[b',u']$ 内没有 n 的 L-Unload：$[b,u]$ 闭合处它落在 $u > u'$（由 (2)），不闭合处引理 54(4) 使 n 完全没有 L-Unload。由于 $\theta_n^{b'} = \mathsf{Active}(-,-)$ ，表 1 因此使 L-Leave 成为 n 在 $[b',u']$ 内能被作用上的唯一规则，其 Ψ^t 是 id_Γ; 由引理 54(1) σ_n 在那里恒定。□

否则跨越步骤的转移会安装针对其下已改变的解析计算的效应，而两个前提防止它。L-Iter 和 L-Finish 携带 $\text{target}_n(\gamma) = \omega$, ，因此转移只在其已提交视图仍是其目标视图时进行，L-Divert 携带其否定，因此目标视图的任何改变都把纤维带出转移。L-Raise 完全不受目标视图条件限制，抛错是迭代做的事而非环境要求的，且它无论如何退出转移。变化的两个方向不加区分：依赖消失的组件和依赖被替换的组件走同一条路，因为已变为 $\perp$ 的目标视图和已变为其他纤维的目标视图同样不等于 ω。

惯性是使它不成为关于每步的保证的东西。目标视图转向时已在飞行的迭代无论如何落地（由 L-Divert），而那个落地安装针对不再成立的解析计算的效应。规则因此交付一个析取，第二支使第一支安全。

定理 64.（解析相干性。）令 n 的片段 $[b,u]$ 在 b 处以 $\omega_n^b = \omega$ 打开。则 θ_n 在片段的初始区间 $[\bar{b},r]$ 上是 Reloading $(-,-,-)$ ，且转移的每次迭代都针对唯一解析 ω 运行：

$$\forall t \in [b,r].\text{step}^t \in \{\text{L}-\text{Iter}(n), \text{L}-\text{Finish}(n)\} \Rightarrow \text{target}_n^t = \omega \tag{59}$$

纤维离开该区间（$r < u$, ）处，恰好下列之一成立：

1. ste $\omega)^r = \text{L}-\text{Finish}(n)$ 且 $\theta_n^{r+1} = \mathsf{Active}(-,\omega)$; ;

2. st $\mathsf{sp}^r \in \{\text{L}-\text{Divert}(n), \text{L}-\text{Raise}(n)\}$ ，且片段在某个 $u > r$ 处闭合，$\gamma^{u+1} \approx \left(\Psi^{t_l} \circ \cdots \circ \Psi^{t_1}\right)\left(\gamma^b\right)$ 如推论 62。

证明. $b-1$ 处的 L-Begin 写 Reloading，由表 1 它是进入该生命周期状态的唯一规则；其前提 $\theta_n = \mathsf{Inactive}(\perp)$ 和引理 54(4) 把它的任何第二次应用放在片段外。因此 Reloading 占据 $[b,\bar{r}]$of$[\bar{b},u]$ 的初始区间且不被重新进入。

第一个断言随后是表 1 给 L-Iter 和 L-Finish 的前提 targe $\mathfrak{t}_n(\gamma) = \omega'$ ，连同 $\omega' = \omega$（由引理 54(2) ）

对二分，stept 是其前提有 $\theta_n = \mathsf{Reloading}(-,-,-)$ 而结论没有的规则，表 1 提供 L-Finish、L-Divert 和 L-Raise；前者落在 $\mathsf{Active}(-,\omega)$ 而另两个落在 $\mathsf{Unloading}(-,\omega,-)$ ，从那里引理 54(4) 使 L-Unload 成为唯一出口而

Unloading$(-,\omega,-)$, from which Lemma 54(4) makes an L-Unload the only exit and Corollary 62 supplies the equation. The iteration a landing L-Divert contributes is one of n's own, hence among the maps that accumulator withdraws. Where instead $r = u$, , the sequence ends with the transition still in flight and the first claim is all that is asserted. □

推论 62 提供等式。落地 L-Divert 贡献的迭代是 n's 自己的，因此在累加器撤出的映射之中。而 $r = u$, 处，序列以转移仍在飞行结束，只断言第一个断言。□

4.4.4. Progress

A guard that defers a provider's withdrawal until its dependents are gone delivers Theorem 63 only if it eventually releases. One relation on the fibers of a registry carries the argument.

Definition 65. The precedence relation on the names of a registry is

$$n \prec m := p_n \cap d_m \neq \emptyset \tag{60}$$

so that n may provide a key m declares. It reads π and d alone, which by Lemma 54(5) come into existence with a fiber's entry and are never written again.

Theorem 66 and Theorem 73 are established on the hypothesis that $\prec$ is acyclic, which is an assumption and not something the definition delivers, $n \prec n$ holding of a component that declares a key it provides itself. What $\prec$ orders is the two fibers' activations and not their lifetimes: $n \prec m$ says that n has to become Active before m can, whereas that a provider outlives its consumer is Theorem 63(2), a theorem about the guarded calculus.

A fiber's target view answers to the fiber that created it as well as to its providers. What a creator writes is τ_n, through the primitive of Definition ^{47,} and τ is monotone by Lemma 54(5). A creator can therefore turn its child's target view at most once over that child's whole existence.

Progress is a claim that some rule applies, so it is formulated over the rules a host must ofer: L-Begin, L-Leave, L-Unload, the landing rules L-Iter, L-Finish, and L-Raise, and L-Divert. It appeals to the aborting alternative of L-Divert nowhere, so a host bound by the inertia of Section 4.3.3 is covered as well.

Theorem 66. (Progress.) Assume $\prec$ acyclic, len $(e_n) \leq K$ for every n, and the set N of names of Definition 60 finite; and let every step apply a lifecycle rule. Write $S(n)$ for the number of steps acting on n and

$$V(n) := \left|\left\{t : \text{target}_n^t \neq \text{target}_n^{t+1}\right\}\right| \tag{61}$$

for the number of times its target view turns. Then

1. (No deadlock.) ¬ quiett implies that some lifecycle rule applies at γ^t;

2. (Termination.) $S(n) \leq (K+4)(V(n)+1)$, and both $V(n)$ and $\sum_n S(n)$ are finite. Consequently every maximal sequence of lifecycle steps ends in a quiescent state.

Proof. No deadlock. Let ¬quiet^t, , so some fiber n satisfies neither clause of the quiet of Definition 49. Reading Table 1 against the four kinds it can then be:

$\theta_n^t = \mathsf{Inactive}(\perp)$ with target $\dot{}_n^t \neq \perp$: L · Bt egin applies;

$\theta_n^t = \mathsf{Reloading}(-,-,\omega_n)$ with target $\mathbf{t}_n^t = \boldsymbol{\omega}_n$: whichever of L-Iter, L-Finish, and L-Raise the value of $i_n^t(\gamma^t)$ selects applies;

$\theta_n^t = \mathsf{Reloading}(-,-,\omega_n)$ with target $\mathrm{t}_n^t \neq \omega_n$: L-Raise applies if $i_n^t(\gamma^t)$ raises, and otherwise L-Divert does, landing that iteration rather than aborting it;

$\theta_n^t = \mathsf{Active}(-,\omega_n)$ with targ $\mathrm{et}_n^t \neq \omega_n$: : L-Leave applies.

4.4.4. 进展性（Progress）

把提供者撤出推迟到其依赖方离开的守卫，只有在它最终释放时交付定理 63。注册表纤维上的一个联系承载该论证。

定义 65. 注册表名字上的优先关系是

$$n \prec m := p_n \cap d_m \neq \emptyset \tag{60}$$

因此 n 可能提供 m 声明的键。它只读 π 和 d ，由引理 54(5) 两者随纤维条目产生且永不再写。

定理 66 和定理 73 在 $\prec$ 无环的假设下确立，这是一个假设而非定义交付的东西，$n \prec n$ 对声明自己提供键的组件成立。$\prec$ 排序的是两个纤维的激活而非其寿命：$n \prec m$ 说 n 必须先在 m 之前变为 Active，而提供者比其消费者长寿是定理 63(2)，一个关于受守卫演算的定理。

纤维的目标视图既应答创建它的纤维也应答其提供者。创建者写的是 τ_n, 通过定义 ^{47,} 的原语，而 τ 由引理 54(5) 单调。创建者因此最多在其子组件的整个存在期间把子的目标视图转向一次。

进展性是某个规则适用的断言，因此它针对宿主必须提供的规则表述：L-Begin、L-Leave、L-Unload、落地规则 L-Iter、L-Finish 和 L-Raise，以及 L-Divert。它无处诉诸 L-Divert 的中止备选，因此受第 4.3.3 节惯性约束的宿主也被覆盖。

定理 66.（进展性。）假设 $\prec$ 无环，对每个 n, 有 len $(e_n) \leq K$ ，且定义 60 的名字集 N 有限；并令每个步骤应用生命周期规则。写 $S(n)$ 为作用于 n 的步骤数

$$V(n) := \left|\left\{t : \text{target}_n^t \neq \text{target}_n^{t+1}\right\}\right| \tag{61}$$

为其目标视图转向的次数。则

1. （无死锁。）¬ quiett 蕴含某个生命周期规则在 γ^t; 处适用；

2. （终止。）$S(n) \leq (K+4)(V(n)+1)$ ，且 $V(n)$ 与 $\sum_n S(n)$ 都有限。因此每个最大的生命周期步骤序列都以静止状态结束。

证明. 无死锁。令 ¬quiet^t, 则某个纤维 n 不满足定义 49 的静止性的任一条款。对照表 1 读它可能处于的四种：

$\theta_n^t = \mathsf{Inactive}(\perp)$ 且 targe $\dot{}_n^t \neq \perp$: L · Bt egin 适用；

$\theta_n^t = \mathsf{Reloading}(-,-,\omega_n)$ 且 targe $\mathbf{t}_n^t = \boldsymbol{\omega}_n$ ： $i_n^t(\gamma^t)$ 的值所选择的 L-Iter、L-Finish、L-Raise 中任意一个适用；

$\theta_n^t = \mathsf{Reloading}(-,-,\omega_n)$ 且 targe $\mathrm{t}_n^t \neq \omega_n$: $i_n^t(\gamma^t)$ 抛错则 L-Raise 适用，否则 L-Divert 适用，落地该迭代而非中止它；

$\theta_n^t = \mathsf{Active}(-,\omega_n)$ 且 targ $\mathrm{et}_n^t \neq \omega_n$: ： L-Leave 适用。

Let no fiber be of any of these kinds, leaving some m_0 with $\theta^t_{m_0} = \mathsf{Unloading}(-,-,-)$. Construct $m_0, m_1, \ldots$ as follows: given m_j in Unloading, either $\neg$ relied$^t_{m_j}$, in which case L-Unload applies to m_j and the construction stops, or there are $m_{j+1} \neq m_j$ and k_j with installed$^t_{m_{j+1}}$ and $\omega^t_{m_{j+1}}(k_j) = m_j$. In the latter case

$$k_j \in d_{m_{j+1}} \cap \mathrm{dom}\left(\sigma^t_{m_j}\right) \subseteq d_{m_{j+1}} \cap p_{m_j}$$

the second membership being Theorem 63(3) at the episode of m_{j+1} that t lies in, so that $m_j \prec m_{j+1}$. Moreover tar get$^{\bar{t}}_{m_{j+1}} \neq \omega^t_{m_{j+1}}$: an Unloading fiber is outside the union defining σ_γ, so k_j at γ^t is unprovided or provided by a fiber other than m_j . Were m_{j+1} in Active or Reloading it would then be of one of the four kinds excluded, so it is in Unloading and the construction continues. The m_j are $\prec$-increasing, hence distinct by acyclicity, and dom $\left(F^t\right)$ is finite, so the construction stops.

Termination. Two claims bound $S(n)$

(A) Over a maximal interval on which targett is constant at ω^*, , at most $K+4$ steps act on n. Reading the θ_n columns of Table 1, from $\mathsf{Active}(-,\omega)$ with $\omega \neq \omega^*$ the fiber takes an L-Leave and an L-Unload and then, if $\omega^* \neq \perp$, an L-Begin and at most len $(e_n) \leq K$ landings, plus a second L-Unload where the last landing is an L-Raise; from Reloading against an n $\neq \omega^*$ it takes an L-Divert in place of the L-Leave, and from any other state a sufix of that sequence. No further L-Divert or L-Leave falls in the interval, the ω that the L-Begin writes being target$^t_n = \omega^*$ itself, and at Active $(-,\omega^*)$, at Inactive($\perp$) with $\omega^* = \perp$, and at Inactive($\perp$) no rule applies at all.

(B) If targettn $\neq$ targett+1n and step t acts on m, then either $m \prec n$ or step t writes τ_n . By Definition 46 the value of target is a function of τ_n and of the tables of the providers of the keys of d_n; a provider satisfies $k \in \mathrm{dom}(\sigma_m) \cap d_n$ and hence $m \prec n$, , and a table changes only at a step acting on its own fiber by Lemma 54(1). Acyclicity gives m $\neq$ n in the first case, and the monotonicity of Lemma 54(5) admits the second at one τ per fiber.

By (A) the interval count bounds $S(n)$ as $S(n) \leq (K+4)(V(n)+1)$, and by (B) each turn of target either consumes a step of a fiber strictly $\prec$-below n or is the one turn τ_n afords, so $V(n) \leq 1 + \sum_{m \prec n} S(m)$. Since $\prec$ is acyclic and N is finite, the recursion

$$B(n) := (K+4)\left(2 + \sum_{m \prec n} B(m)\right)$$

is well founded and defines B with $S(n) \leq B(n)$; hence $V(n)$ is finite and $\sum_n S(n) \leq \sum_n B(n)$ By (1) a sequence that cannot be extended is quiescent. □

Finiteness of N is assumed rather than derived, and one condition on the components delivers it. The components a host holds are finitely many programs given before anything runs, so if no component can register, however indirectly, a fiber of a component that registers one of its own, the registrations form a tree of bounded depth, and len $(e_n) \leq K$ bounds its branching. What the assumption rules out is a component that registers instances of itself without bound.

The target records the providing fiber rather than a boolean, and under the single-source discipline of Section 4.2 the two drive the same transitions, a key having one possible provider there. What the view buys is the vocabulary of the results above, Theorem 63 and Theorem 64 both speaking of the resolution a fiber activated

令没有纤维属于任何这类，留下某个 $\theta^t_{m_0} = \mathsf{Unloading}(-,-,-)$ 的 m_0 。如下构造 $m_0, m_1, \ldots$ ：给定 Unloading 中的 m_j ，要么 $\neg$ relied$^t_{m_j}$, 此时 L-Unload 适用于 m_j 且构造停止，要么存在 $m_{j+1} \neq m_j$ 和 k_j 使 installed$^t_{m_{j+1}}$ 且 $\omega^t_{m_{j+1}}(k_j) = m_j$ 。后一种情形

$$k_j \in d_{m_{j+1}} \cap \mathrm{dom}\left(\sigma^t_{m_j}\right) \subseteq d_{m_{j+1}} \cap p_{m_j}$$

第二个成员资格是定理 63(3) 在 m_{j+1} 的 t 所在的片段处，因此 $m_j \prec m_{j+1}$ 。再者 tar get$^{\bar{t}}_{m_{j+1}} \neq \omega^t_{m_{j+1}}$ ：Unloading 纤维在定义 σ_γ 的并集之外，因此 k_j 在 γ^t 处未提供或由非 m_j 的纤维提供。若 m_{j+1} 在 Active 或 Reloading 中，它就会属于被排除的四种之一，因此它在 Unloading 中且构造继续。m_j 是 $\prec$-递增的，因此由非环性两两不同，而 dom $\left(F^t\right)$ 有限，所以构造停止。

终止。两个断言界定 $S(n)$

(A) 在 targett 恒为 ω^*, 的最大区间上，至多 $K+4$ 个步骤作用于 n。读表 1 的 θ_n 列，从 $\mathsf{Active}(-,\omega)$（$\omega \neq \omega^*$）纤维取一个 L-Leave 和一个 L-Unload，然后若 $\omega^* \neq \perp$ 取一个 L-Begin 和至多 len $(e_n) \leq K$ 次落地，若最后一次落地是 L-Raise 再加一个 L-Unload；从针对 n $\neq \omega^*$ 的 Reloading 它取一个 L-Divert 代替 L-Leave，从任何其他状态取该序列的后缀。区间内没有进一步的 L-Divert 或 L-Leave，L-Begin 写的 ω 本身就是 target$^t_n = \omega^*$ ，而在 Active $(-,\omega^*)$ 、$\omega^* = \perp$ 时的 Inactive($\perp$) 和 Inactive($\perp$) 处根本没有规则适用。

(B) 若 targettn $\neq$ targett+1n 且步骤 t 作用于 m, 则 $m \prec n$ 或步骤 t 写 τ_n 。由定义 46，target 的值是 τ_n 和 d_n 各键提供者表的函数；提供者满足 $k \in \mathrm{dom}(\sigma_m) \cap d_n$ 从而 $m \prec n$, ，而表只在作用其自身纤维的步骤处改变（引理 54(1)）。非环性在第一种情形给出 m $\neq$ n ，引理 54(5) 的单调性每纤维只接纳第二种一次。

由 (A) 区间计数把 $S(n)$ 界定为 $S(n) \leq (K+4)(V(n)+1)$ ，由 (B) 每次 target 转向要么消耗严格 $\prec$-低于 n 的纤维的一步，要么是 τ_n 允许的一次转向，因此 $V(n) \leq 1 + \sum_{m \prec n} S(m)$ 。由于 $\prec$ 无环且 N 有限，递归

$$B(n) := (K+4)\left(2 + \sum_{m \prec n} B(m)\right)$$

良基并定义 B 使 $S(n) \leq B(n)$ ；因此 $V(n)$ 有限且 $\sum_n S(n) \leq \sum_n B(n)$ 由 (1)，不可扩展的序列是静止的。□

N 的有限性是假设而非推导，而组件上的一个条件交付它。宿主持有的组件是在任何东西运行前给定的有限多个程序，因此若没有组件能（无论多么间接地）注册一个注册自身之一实例的组件的纤维，注册形成有界深度的树，而 len $(e_n) \leq K$ 界定其分支。假设排除的是无界注册自身实例的组件。

目标记录提供纤维而非布尔，在第 4.2 节的单一来源纪律下两者驱动相同转移，键在那里只有一个可能的提供者。视图买来的是上面结果的词汇，定理 63 和定理 64 都谈纤维据以激活的解析，且它使那些结果在第 3.2.3 节的作用域解析下存活，在那里一个键在不同领域解析到不同提供者而提供不再迫使视图。实现携带该作用域并把

against, and it is what makes those results survive the scoped resolution of Section 3.2.3, under which one key resolves to diferent providers in diferent realms and the provisions no longer force the view. The implementation carries that scoping and holds the view in fiber.committed (Section 5.1.3).

4.4.5. Confluence

The results so far are about individual fibers. The property that characterizes the system as a whole is that its dynamic history leaves no trace: whatever sequence of activations and deactivations a running system has been through, the state it quiesces at is the one the same insertions and retirements would have produced had each component that ends up active been loaded once, in dependency order, and none ever unloaded. The lifecycle relation is confluent, and the normal form it converges on is the statically assembled one. This is the analogue, for dynamic composition, of the consistency with a from-scratch evaluation that change propagation establishes for incremental computation [45].

The claim is about $\longrightarrow$ alone. Orchestration steps are inputs, and two sequences given diferent inputs land in diferent places for no interesting reason; what is at issue is whether the lifecycle rules, which are nondeterministic in which fiber steps next and in which exit a Reloading fiber takes, can be made to disagree.

Three lemmas are needed first. The first fixes the set of fibers that end up Active without reference to any sequence of steps, which is what makes it a function of the input rather than of the schedule.

Definition 67. A fiber is supported at γ when it is not retired, the fiber registering it is supported, and every key it declares is provided by a supported fiber. The support relation on dom (F_γ) is the union of the two relations those clauses read,

$$m \lhd n := m \prec n \vee \pi_n = m \tag{62}$$

and where it is well founded (Lemma 68) we write A for the support set, the fibers supported at γ:

$$n \in A := \neg\tau_n \wedge (\pi_n = \text{root} \vee \pi_n \in A) \wedge \forall k \in d_n.\exists m \in A.k \in p_m \tag{63}$$

where $\pi_n = \texttt{root}$ marks a fiber the orchestrator inserted and π_n otherwise the fiber whose activation registers n. The clauses read no field but τ, π, d, p . Both halves relate a fiber to one immediately below it, a parent rather than an ancestor and a direct provider rather than a transitive one, since that is what the clauses read; where the results below want an order they take the transitive closure, whose minimal elements, maximal elements, and linearizations are those of $\lhd$.

The clauses refer to A itself, so the definition is a recursion along $\lhd$, and it is the following that makes it one with a solution.

Lemma 68. (Support is well founded.) Let $\prec$ be acyclic and let γ be reached by a sequence of steps. Then $\lhd$ is well founded, and A is the one solution of Definition 67, a function of τ, π, d, and p alone.

Proof. Order the names of dom (F_γ) by the index of the step that registered each, which Definition 53 supplies by starting the sequence at an empty registry. The parent half of $\lhd$ descends in that index: an O-Insert has $\pi \in \mathrm{dom}(F_\gamma)$ as a premise, so a parent pointer names a fiber registered earlier, and iterating it reaches the whole ancestry of a name in finitely many steps. A cycle therefore has to use $\prec$, and since $\prec$ is acyclic it has to mix the two, which needs some n to declare a key that a fiber of ms own subtree may provide. Such a fiber is registered by an activation of n or of one of ms descendants, hence at a step after the L-Begin of m; that L-Begin has $\gamma \models d_m$ as a premise, so a fiber providing the key is Active already before it, and clause (2) of Definition 58 leaves the key no second possible provider. The fiber that would close the cycle is therefore never registered, and

视图保存在 fiber.committed 中（第 5.1.3 节）。

4.4.5. 合流性（Confluence）

迄今的结果关于单个纤维。刻画系统整体的性质是它的动态历史不留下痕迹：无论运行系统经历过什么激活与停用序列，它静止处的状态都是相同插入与退役会在每个最终激活的组件按依赖顺序各加载一次、且从不卸载时产生的状态。生命周期联系是合流的，它收敛的范式是静态装配的那个。对动态组合而言，这相当于增量计算 [45] 通过变更传播确立的与从头求值的一致性。

该断言只关于 $\longrightarrow$。编排步骤是输入，给定不同输入的两个序列落在不同地方没有有趣的原因；关键在生命周期规则（对哪个纤维下一步以及 Reloading 纤维取哪个出口是非确定性的）能否被弄得不一致。

首先需要三个引理。第一个在不提及任何步骤序列的情况下固定最终 Active 的纤维集，这使它成为输入的函数而非调度的函数。

定义 67. 纤维在 γ 处是受支撑的（supported），当它未退役、注册它的纤维受支撑、且它声明的每个键都由受支撑的纤维提供。dom (F_γ) 上的支撑联系是这些条款读取的两个联系的并，

$$m \lhd n := m \prec n \vee \pi_n = m \tag{62}$$

在其良基处（引理 68）我们写 A 为支撑集，即 γ 处受支撑的纤维：

$$n \in A := \neg\tau_n \wedge (\pi_n = \text{root} \vee \pi_n \in A) \wedge \forall k \in d_n.\exists m \in A.k \in p_m \tag{63}$$

其中 $\pi_n = \texttt{root}$ 标记编排器插入的纤维，π_n 否则是激活注册 n 的纤维。这些条款不读 τ, π, d, p 以外的字段。两半都把纤维关联到紧邻其下的一者，父而非祖先、直接提供者而非传递提供者，因为那是条款所读；下面的结果想要序时取传递闭包，其最小元、最大元和线性化是 $\lhd$ 的那些。

这些条款引用 A 本身，因此定义是沿 $\lhd$ 的递归，而下面使它成为有解的那个。

引理 68.（支撑良基。）令 $\prec$ 无环且 γ 由步骤序列达到。则 $\lhd$ 良基，且 A 是定义 67, 的唯一解，一个只由 $\tau, \pi, d,$ 和 p 决定且独立于步骤的函数。

证明. 按注册每个名字的步骤的索引对 dom (F_γ) 的名字排序，定义 53 通过让序列始于空注册表提供之。$\lhd$ 的父半部在该索引下降：O-Insert 以 $\pi \in \mathrm{dom}(F_\gamma)$ 为前提，因此父指针命名较早注册的纤维，迭代它在有限步内到达名字的整个祖先。循环因此必须使用 $\prec$, 且因 $\prec$ 无环它必须混合两者，这需要某个 n 声明 ms 自身子树中的纤维可能提供的键。这样的纤维由 n 或 ms 后代之一的激活注册，因此在 m; 的 L-Begin 之后的步骤处注册；该 L-Begin 以 $\gamma \models d_m$ 为前提，因此提供该键的纤维在那之前已 Active，而定义 58 的条款 (2) 不给该键第二个可能的提供者。将闭合循环的纤维因此从不被注册，该边不在 dom (F_γ) 中。良基递归有唯一解，而条款只读那四个字段。□

the edge is absent from dom (F_γ) . A well-founded recursion has one solution, and the clauses read the four fields alone. □

The last clause reads p, the keys a component may provide, whereas the target reads dom (σ_γ) , the keys its fibers have installed, and Definition 43 relates the two by dom $(\sigma_n) \subseteq p_n$ alone. The support set therefore over-approximates the Active fibers in general, and the condition that closes the gap is the following.

Definition 69. A component (d, p, e) is total on its provision when an activation of it that finishes has installed every key of p, so that dom $(\sigma_n) = p_n$ at every Active fiber instantiating it.

Like independence (Definition 60) this is a condition on the components alone, mentioning no lifecycle state and no step, and independence already bounds how far it can fail: were a component to install a key only at context states another component's efects reach, its forward map would not commute with that component's, so the keys a fiber installs are fixed by its component rather than by the schedule. What totality adds is that the fixed set is all of p rather than a proper subset of it.

Lemma 70. (Support at quiescence.) Let $\prec$ be acyclic, let quiet (γ) , let no fiber of γ be failed, and let every component of γ be total on its provision (Definition 69). Then the support set is the set of Active fibers:

$$A = \{n : \theta_n = \text{ Active } (-,-)\} \tag{64}$$

Proof. Write A' for the right-hand side. No fiber being failed, the quiet of Definition 49 leaves Inactive($\perp$) and Active as the only states and reads

$$n \in A' \Longleftrightarrow \text{target}_n(\gamma) \neq \perp$$

By Definition 46 the right side holds exactly when $\neg\tau_n$ and every $k \in d_n$ lies in dom (σ_γ) , and dom $(\sigma_\gamma) = \bigcup_{m \in A'} p_m$ by Definition 69. The middle clause is the one the target no longer carries, and registration supplies it: a fiber with $\pi_n \neq$ root is registered only by an activation of π_n, and if $\pi_n \notin A'$ then π_n is not Active, so its accumulator has run and retired n by Definition ^{47,} , giving τ_n . Hence A' satisfies the clauses of Definition ^{67,} , and Lemma 68 gives them one solution, so $A = A'$ □

Lemma 71. (Transposition.) Let the steps be pairwise independent and F^t well formed, and let steps t and $t+1$ act on distinct fibers n and m.

1. If both apply an activation rule, namely L-Begin, L-Iter, or L-Finish, and step $t+1$ is applicable at γ^t , then step t is applicable at the state step $t+1$ produces from γ^t , and the two orders reach the same $\gamma^{t+\bar{2}}$

2. If step t applies an activation rule at m, step $t+1$ an orchestration rule at n, and step t does not register n, then the same holds of the two.

Proof. For (1), by Table 1 the step of n writes θ_m and, within $\Psi^t \in \mathfrak{M}(e_m)$. , the table σ_m and the effect part. It therefore leaves θ_n and i_n alone, and by the second condition of Definition 60 leaves the inverse and the continuation that i_n yields alone as well, so only the premises of step $t+1$ that mention target_n remain to be checked. Its retirement half cannot fall, no activation rule writing a τ. . Its resolution half cannot move either: step $t+1$ being applicable at γ^t puts every $k \in d_n$ in dom (σ^t) , and clause (2) of Definition 58 makes the fiber providing such a k the only one that can, so k $\notin \boldsymbol{p}_m$ and no write of σ_m reaches a key of d_n . The same argument in the other direction leaves step t applicable. Finally $\Psi^t \in \mathfrak{M}(e_m)$ and $\Psi^{t+1} \in \mathfrak{M}(e_n)$ commute by the first condition of Definition 60, and the two edits write control fields of distinct fibers, so the composite is the same in

最后一条款读 p, 即组件可能提供的键，而目标读 dom (σ_γ) ，即其纤维已安装的键，定义 43 以 dom $(\sigma_n) \subseteq p_n$ 关联两者。支撑集因此一般过度近似 Active 纤维，而闭合差距的条件如下。

定义 69. 组件 (d, p, e) 在其提供上完备（total），当完成激活的它已安装 p, 的每个键，从而在每个实例化它的 Active 纤维处 dom $(\sigma_n) = p_n$ 。

像独立性（定义 60）一样，这是只对组件成立的条件，不提及生命周期状态或步骤，而独立性已经界定它可能失败多远：若组件只在另一组件效应达到的上下文状态安装键，其正向映射就不会与该组件的交换，因此纤维安装的键由其组件而非调度固定。完备性所加的是固定集是全部 p 而非其真子集。

引理 70.（静止处的支撑。）令 $\prec$ 无环、quiet (γ) 、γ 无纤维失败、且 γ 的每个组件在其提供上完备（定义 69）。则支撑集是 Active 纤维集：

$$A = \{n : \theta_n = \text{ Active } (-,-)\} \tag{64}$$

证明. 写 A' 为右侧。由于无纤维失败，定义 49 的静止性只留 Inactive($\perp$) 和 Active 作为状态并读

$$n \in A' \Longleftrightarrow \text{target}_n(\gamma) \neq \perp$$

由定义 46 右侧恰在 $\neg\tau_n$ 且每个 $k \in d_n$ 位于 dom (σ_γ) 时成立，而 dom $(\sigma_\gamma) = \bigcup_{m \in A'} p_m$ 由定义 69。中间条款是目标不再携带的，而注册提供它：$\pi_n \neq$ root 的纤维只由 π_n, 的激活注册，若 $\pi_n \notin A'$ 则 π_n 不 Active，因此其累加器已运行并由定义 ^{47,} 退役 n，给出 τ_n 。因此 A' 满足定义 ^{67,} 的条款，而引理 68 给它们唯一解，所以 $A = A'$ □

引理 71.（转置。）令步骤两两独立且 F^t 良构，并令步骤 t 和 $t+1$ 作用于不同纤维 n 和 m。

1. 若两者都应用激活规则（即 L-Begin、L-Iter 或 L-Finish）且步骤 $t+1$ 在 γ^t 处适用，则步骤 t 在步骤 $t+1$ 从 γ^t 产生的状态处适用，且两个顺序达到相同的 $\gamma^{t+\bar{2}}$

2. 若步骤 t 在 m, 处应用激活规则、步骤 $t+1$ 在 n, 处应用编排规则且步骤 t 不注册 n, 则两者同样成立。

证明. 对 (1)，由表 1，n 的步骤写 θ_m 并在 $\Psi^t \in \mathfrak{M}(e_m)$. 内写表 σ_m 和效应部分。它因此让 θ_n 和 i_n 独处，并由定义 60 的第二条件让 i_n 产生的逆和续延也独处，因此只需检查步骤 $t+1$ 提及 target_n 的前提。其退役半不会失败，没有激活规则写 a τ. 。其解析半也不会移动：步骤 $t+1$ 在 γ^t 处适用使每个 $k \in d_n$ 在 dom (σ^t) 中，而定义 58 的条款 (2) 使提供这样的 k 的纤维成为唯一能提供的，因此 k $\notin \boldsymbol{p}_m$ 且 σ_m 的写入不达 d_n 的键。反方向的同一论证使步骤 t 适用。最后 $\Psi^t \in \mathfrak{M}(e_m)$ 和 $\Psi^{t+1} \in \mathfrak{M}(e_n)$ 由定义 60 的第一条件交换，而两个编辑写不同纤维的控制字段，因此复合在任一顺序相同。

either order.

For (2), the orchestration step has $\Psi^{t+1} = \mathrm{id}_\Gamma$ by Table 1, so the two state maps commute outright, and its editt+1 writes τ_n or dom (F_γ) at n alone, which the activation step neither reads nor writes: the premises of the latter read $\theta_m, i_m, \tau_m,$, and target_m, and an O-Insert of a fresh n moves no target, a fresh fiber providing nothing, whereas an O-Retire or O-Remove of n leaves σ_γ where it was, n being Inactive in the one case and unafected in its table in the other. So step t remains applicable. Conversely each premise of the orchestration step is either read at n, which step t does not write, or is one of the two premises of O-Insert that a smaller registry only relaxes, whence its applicability at γ^{t+1} gives its applicability at γ^t. ; here step t not registering n is what keeps n present at γ^t where O-Retire and O-Remove require it. □

Lemma 72. (Deletion.) Let the sequence of steps be pairwise independent, let every component be total on its provision (Definition 69), let it reach a quiescent γ^T at which no fiber is failed, let $[b, u]$ be an episode of n that closes, let no episode of any m with $n \prec m$ close in the sequence, and let no fiber n registers during $[b, u]$ have an episode. Write R for the names those registrations draw. Then deleting the steps that act on n in $[b, u]$, together with every step acting on a name of R, leaves a sequence of steps reaching a state $\approx$-equal to γ^T and $\simeq$-equal to it outside R.

Proof. The deleted steps leave the state where they found it. Let $t_1 < \cdots < t_l$ be the steps of $[b, u]$ that act on fibers other than n. Corollary 62 reads

$$\gamma^{u+1} \approx \left(\Psi^{t_l} \circ \cdots \circ \Psi^{t_1}\right)\left(\gamma^b\right)$$

whose right side is what the surviving steps of $[b, u]$ produce on their own, $\gamma^{b-1} \approx \gamma^b$ and their edits writing control fields of fibers other than n that the deletion does not touch. By Table 1 the deleted steps of n write no field but θ_n, which Lemma 54(4) restores to Inactive($\perp$) at u, no fiber being failed, and which it held at γ^{b-1}

An invariant carries the sufix. Write γ'^t for the state the surviving steps reach at the point corresponding to n. We claim, for every $t > u$, that $\gamma^t \approx \gamma'^t$, that every name of n is vestigial at γ^t and absent from γ'^t , and that the two states agree on every field of every name outside n. At $t = u+1$ this is the paragraph above together with Definition 47, , which leaves each name of n retired by the accumulator that ran at n, Inactive($\perp$) and holding an empty table, the fibers of n having no episode by hypothesis. The induction step is Lemma 57(1) applied at each name of n in turn: a step acting outside n has the same premises at the two states, reaches states again $\approx$-equal, and leaves the entries of n vestigial. A step acting on a name of n is one of the deleted ones, and Lemma 57(2) is why it has to be deleted rather than kept, an O-Retire or O-Remove of an absent name having no fiber to act on; by (1) again such a step moves no field outside R, so dropping it preserves the invariant. Hence the final states are $\approx$-equal, and equal outside n.

No surviving step loses a premise. A step acting on m $\notin R \cup \{n\}$ reads n only through $\mathrm{target}_m(\gamma)$ or $\mathrm{relied}_m(\gamma)$. The first depends on m when n declares a key m provides, hence $n \prec m$, , and when n registered m, , which puts $m \in R$. . In the first case n's episode does not close, by hypothesis, so it is open at γ^T , where quiet gives $\omega_m = \mathrm{target}_m^T$ and Lemma 70 puts its values among the Active fibers, which n is not; since a key has at most one possible provider, n provided no key of d_m at n's L-Begin either. The second reads n only through the values of ω_n, and deleting the episode can only make relied false, which relaxes the guard on L-Unload rather than blocking it. What such a step reads of a name of n is covered by the invariant. Pairwise independence is a property of the effect functions, so deleting steps preserves it. □

Theorem 73. (Confluence.) Let a sequence of steps reach a quiescent γ^T at which no fiber is failed, let the steps be pairwise independent and every component be total on its provision (Definition 69), and let A be as in

对 (2)，编排步骤由表 1 有 $\Psi^{t+1} = \mathrm{id}_\Gamma$ ，因此两个状态映射无条件交换，而其 editt+1 只写 n, 的 τ_n 或 dom (F_γ) ，激活步骤既不读也不写：后者的前提读 $\theta_m, i_m, \tau_m,$ 和 target_m, 而新 n 的 O-Insert 不移动目标，新纤维不提供任何东西，而 n 的 O-Retire 或 O-Remove 使 σ_γ 留在原处，n 在前一种情形 Inactive 后一种情形表不受影响。因此步骤 t 保持适用。反之，编排步骤的每个前提要么在 n, 处读出（步骤 t 不写），要么是较小注册表只会放松的 O-Insert 的两个前提之一，其在 γ^{t+1} 的适用性给出其在 γ^t. 的适用性；这里步骤 t 不注册 n 是使 n 在 O-Retire 和 O-Remove 所需的 γ^t 处在场的东西。□

引理 72.（删除。）令步骤序列两两独立、每个组件在其提供上完备（定义 69）、它达到无纤维失败的静止 γ^T 、$[b, u]$ 是闭合的 n 的片段、序列中没有满足 $n \prec m$ 的 m 的片段闭合、且 n 在 $[b, u]$ 注册的任何纤维没有片段。写 R 为那些注册抽取的名字。则删除 $[b, u]$ 中作用于 n 的步骤、连同作用于 R 中名字的每个步骤，留下一个达到与 γ^T 状态 $\approx$-相等、在 R 之外 $\simeq$-相等的状态的步骤序列。

证明. 被删除的步骤让状态留在它们发现它的地方。令 $t_1 < \cdots < t_l$ 为 $[b, u]$ 中作用于非 n 纤维的步骤。推论 62 读

$$\gamma^{u+1} \approx \left(\Psi^{t_l} \circ \cdots \circ \Psi^{t_1}\right)\left(\gamma^b\right)$$

其右侧正是 $[b, u]$ 中存活的步骤自己产生的，$\gamma^{b-1} \approx \gamma^b$ 且其编辑写删除未触及的非 n 纤维的控制字段。由表 1，n 的被删除步骤只写 θ_n, 引理 54(4) 在 u, 处把它恢复到 Inactive($\perp$)（无纤维失败），且它在 γ^{b-1} 持有它。

一个不变量携带后缀。写 γ'^t 为存活步骤在对应 n 的点达到的状态。我们断言，对每个 $t > u$, 有 $\gamma^t \approx \gamma'^t$ 、n 的每个名字在 γ^t 处残余且在 γ'^t 处缺席、且两个状态在 n 外每个名字的每个字段上一致。在 $t = u + 1$ 处这是上面一段连同定义 47, ，后者使 n 的每个名字被 n 处运行的累加器退役，Inactive($\perp$) 且持有空表，n 的纤维按假设没有片段。归纳步是对 n 的每个名字依次应用引理 57(1) ：作用于 n 外的步骤在两个状态有相同前提、达到再次 $\approx$-相等的状态、并使 n 的条目保持残余。作用于 n 中名字的步骤是被删除者之一，而引理 57(2) 是它必须被删除而非保留的原因，缺席名字的 O-Retire 或 O-Remove 没有纤维可作用；由 (1) 又知这样的步骤不移动 R, 外的字段，因此丢弃它保持不变量。因此最终状态 $\approx$-相等，且在 n 外相等。

没有存活步骤失去前提。作用于 m $\notin R \cup \{n\}$ 的步骤只通过 $\mathrm{target}_m(\gamma)$ 或 $\mathrm{relied}_m(\gamma)$ 读 n 。前者在 n 声明 m 提供的键时依赖 m ，从而 $n \prec m$, 在 n 注册 m, 时依赖 n ，这使 $m \in R$. 。第一种情形 n 的片段按假设不闭合，因此它在 γ^T 处打开，那里 quiet 给出 $\omega_m = \mathrm{target}_m^T$ 而引理 70 把其值放在 Active 纤维中，n 不是；由于键至多有一个可能的提供者，n 在 n 的 L-Begin 处也没提供 d_m 的任何键。第二种只通过 ω_n 的值读 n ，而删除片段只能使 relied 变假，这放松 L-Unload 的守卫而非阻塞它。这样的步骤读 n 中名字的东西由不变量覆盖。两两独立是效应函数的性质，删除步骤保持它。□

定理 73.（合流性。）令步骤序列达到无纤维失败的静止 γ^T 、步骤两两独立且每个组件在其提供上完备（定义 69）、A 如定义 67 。则

Definition 67 . Then

1. (Canonical form.) γ^T is reached, up to the names whose entries the reduction withdraws, from γ^0 by a sequence that takes the same orchestration steps in their original order, those at a fiber the orchestrator inserted preceding every lifecycle step and each of the rest following the step that registered the fiber it acts on, and that takes, for an enumeration $n_1, ..., n_k$ of A linearizing $\lhd$, one episode of each n_i in that order.

2. (Confluence.) Any two such sequences from γ^0 taking the same orchestration steps reach states related, after a renaming as in Lemma 56, by $\simeq$ and by $\approx$.

Proof. For (1), the episodes of the sequence are of two kinds: those that close and those still open at γ^T , which by quiett and Lemma 70 are one episode of each fiber of A.

Closing episodes go first, by induction on their number. At each stage pick a closing episode of a fiber n that is $\lhd$-maximal among the fibers whose episodes still close; one exists by Lemma 68 and the finiteness of A. The three hypotheses of Lemma 72 are then met. No m with $n \prec m$ has a closing episode, by maximality. And no fiber n registers during [b, u] has an episode: such a fiber is retired by the accumulator that ran at n (Definition 47) and by Lemma 54(5) stays retired, so its target view is $\perp$ and Lemma 70 puts it outside A, whence it has no episode open at γ^T. ; and $\lhd$ relates it to n through its parent pointer, so by maximality it has no closing one either. The lemma removes the episode, together with the steps of the names it registered, leaving γ^T where it was up to those names. The measure drops by one, so no closing episode remains.

A fiber outside A takes no lifecycle step. It has no open episode at γ^T , by Lemma 70 and quiett, and no closing one now remains, so it has no episode at all and is Inactive($\perp$) throughout; L-Begin is the only rule that applies there, and applying it would open an episode.

Orchestration steps go next. An orchestration step at a fiber the orchestrator inserted moves one place earlier past a lifecycle step of a diferent fiber by Lemma 71(2), which applies because a step of a fiber of A registers no such name: registrations draw fresh names, whereas the name here is one an O-Insert of the original sequence introduced. With a lifecycle step of the same fiber there is nothing to exchange, an O-Insert of n already preceding every step of n and an O-Retire or O-Remove of n applying only outside A, which takes no lifecycle step. Moving each to the front in turn preserves their relative order. An orchestration step at a fiber some activation registered cannot go to the front, its premises requiring that fiber to be present, so it stays where the registration put it; it acts outside A by the paragraph above and therefore commutes with everything between it and the registration by the same clause of Lemma 71.

Episodes are sorted and made contiguous, by induction on $|A|$. Let n_1 be $\lhd$-minimal in A. Then $d_{n_1} = \emptyset$ and $\pi_{n_1} = \mathsf{root}$, since Definition 67 puts a provider of a key of d_{n_1} and the fiber registering n_1 in A while $\lhd$ puts both below n_1 . So target t_{n_1} reads no field of another fiber and, no orchestration step remaining to write τ_{n_1} and no fiber below n_1 remaining to retire it, is constant. Every step acting on n_1 is an activation step, no episode closing, and its remaining premises read θ_{n_1} and i_{n_1} , which by Table 1 only n_1 writes; each is therefore applicable at every earlier state, and Lemma 71 moves it one place earlier without moving the endpoint. The number of steps of other fibers preceding a step of n_1 drops by one at each application, so the episode of n_1 becomes an initial contiguous block. The argument repeats on $A \setminus \{n_1\}$ over the sufix that follows the block, where n_1 is Active throughout and takes no further step, so it too contributes a constant target. The enumeration this produces linearizes $\lhd$ by construction.

For (2), both sequences reduce by (1) to a canonical one, and the two reductions run over the same A up to a

1. （规范形。）γ^T 是从 γ^0 经一个序列达到的（在还原撤出的名字条目之上），该序列以原始顺序采取相同的编排步骤，编排器插入的纤维处的那些步骤先于每个生命周期步骤、其余每个随后跟随注册其所作用纤维的步骤，并且对线性化 $\lhd$ 的 A 的枚举 $n_1, ..., n_k$ 按该顺序各取一个 n_i 的片段。

2. （合流性。）任何两个从 γ^0 采取相同编排步骤的此类序列达到的状态在引理 56 式的重命名后由 $\simeq$ 和 $\approx$ 相关。

证明. 对 (1)，序列的片段有两种：闭合的以及仍开在 γ^T 的，后者由 quiett 和引理 70 是 A 每个纤维的一个片段。

闭合片段先走，对其数量归纳。每阶段在片段仍闭合的纤维中选一个其纤维 $\lhd$-最大的闭合片段；一个由引理 68 和 A 的有限性存在。引理 72 的三个假设于是满足。没有满足 $n \prec m$ 的 m 有闭合片段，由最大性。n 在 [b, u] 注册的任何纤维没有片段：这样的纤维被 n 处运行的累加器退役（定义 47）并由引理 54(5) 保持退役，因此其目标视图是 $\perp$ 而引理 70 把它放在 A 外，从而它在 γ^T 处没有打开的片段；且 $\lhd$ 通过其父指针把它关联到 n，因此由最大性它也没有闭合的。引理移除该片段连同它注册名字的步骤，把 γ^T 留在原处（那些名字之上）。度量减一，因此没有闭合片段剩余。

A 外的纤维不取生命周期步骤。它在 γ^T 处没有打开的片段（由引理 70 和 quiett），现在也没有闭合的，因此它完全没有片段、全程 Inactive($\perp$)；L-Begin 是那里唯一适用的规则，而应用它会打开片段。

编排步骤随后。编排器插入的纤维处的编排步骤经引理 71(2) 移到一个更早的位置、越过不同纤维的生命周期步骤，该引理适用因为 A 纤维的步骤不注册这样的名字：注册抽取新名，而这里的名字是原始序列的 O-Insert 引入的。与同一纤维的生命周期步骤则无可交换，n 的 O-Insert 已先于 n 的每个步骤，而 n 的 O-Retire 或 O-Remove 只在 A 外应用，后者不取生命周期步骤。逐一把每个移到前面保持其相对顺序。某激活注册的纤维处的编排步骤不能移到前面，其前提要求该纤维在场，因此它留在注册放它的地方；由上面一段它作用于 A 外，因此由引理 71 的同一条款它与它和注册之间的一切交换。

片段被排序并做成连续的，对 $|A|$ 归纳。令 n_1 在 A 中 $\lhd$-最小。则 $d_{n_1} = \emptyset$ 且 $\pi_{n_1} = \mathsf{root}$, 因为定义 67 把 d_{n_1} 键的提供者和注册 n_1 的纤维放在 A 中而 $\lhd$ 把两者都放在 n_1 下。因此 targe t_{n_1} 不读另一纤维的字段，且没有编排步骤剩下写 τ_{n_1} 、没有 n_1 下的纤维剩下退役它，故恒定。每个作用于 n_1 的步骤都是激活步骤，没有片段闭合，其剩余前提读 θ_{n_1} 和 i_{n_1} ，由表 1 只有 n_1 写；每个因此在前面的每个状态处适用，而引理 71 在不移动端点的情况下把它移前一位。每次应用使 n_1 的步骤之前的其他纤维步骤数减一，因此 n_1 的片段成为初始连续块。该论证在块的后续后缀上对 $A \setminus \{n_1\}$ 重复，n_1 在那里全程 Active 且不取进一步步骤，因此它也贡献恒定目标。此产生的枚举按构造线性化 $\lhd$。

对 (2)，两个序列都经 (1) 化到规范序列，而两个还原在同一 A 上运行（重命名之下）。定义 67 读 $\tau, \pi, d,$ 和

renaming. Definition 67 reads τ, π, d, and p, of which the last three are written once with a fiber's entry (Lemma 54(5))), so what has to be seen is that the same names come into existence carrying the same d, p, and π, and that the same names are retired. Insertions the two sequences share by hypothesis. Registrations they share as well: an activation of a fiber of A registers, at each of its iterations, the component the iterator names there, which the second condition of Definition 60 holds fixed across interleavings, so the tree of registrations below an A-fiber is a function of that fiber's component; the names those registrations draw are not shared, and it is here that Lemma 56 is applied, matching the two trees by a bijection. And a retirement is either an orchestration step, shared, or the O-Retire an accumulator takes, which retires exactly the names the same activation registered. Two enumerations linearizing $\lhd$ difer by transpositions of incomparable episodes, which Lemma 71 again leaves the endpoint unchanged by, so the two canonical sequences agree. With the termination of Theorem 66, the lifecycle relation therefore has unique normal forms. □

Failure is excluded from the statement because it is a genuine source of divergence, and the calculus should not be read as denying it: whether a step raises depends on the state it ran against, so one schedule may fail a fiber where another completes it, and the two quiescent states then difer in that fiber's lifecycle state. They do not difer in anything else, by Corollary 62, which puts a failed fiber's contribution to the state at nothing.

In the base calculus of Section 4.2 the same theorem holds, and the proof needs no substitution beyond dropping one clause. L-Unload carries no guard there, so the last paragraph of Lemma 72 is vacuous; the rest of that lemma appeals to quiett alone, which the base calculus supplies unchanged.

The theorem is what licenses reasoning about a Cordis application as though it were statically assembled. An orchestrator that adds a component, removes it, replaces a provider, and reverts the replacement is guaranteed to arrive at the state it would have obtained by writing the final composition down at the outset, and a component author reasoning about which coefects are in scope may reason about the quiescent state alone. It also delimits the guarantee: it speaks of the state, not of the emissions the system produced along the way, which is the distinction Section 6.1 draws between an acquisition, tracked inside the boundary, and an emission, which crosses it.

5. Implementation and Case Study

This section presents Cordis, which realizes the formal models of Section 3 as a practical programming abstraction. Cordis is a meta-framework of spatiotemporal composability: unlike application frameworks that target a specific domain (e.g., web routing, ORM, UI rendering), it prescribes no concrete scenario; its sole responsibility is to supply universal dynamic composition semantics. The implementation is layered into three tiers: (1) the core library (Section 5.1) implements the effect and coeffect systems directly; (2) the component loader (Section 5.2) extends the core with configuration reconciliation and hot module replacement; and (3) application frameworks such as Koishi (Section 5.3) build domain-specific functionality on top of the former two tiers.

5.1. Core Library

Table 2 summarizes the correspondence between theoretical constructs and their runtime counterparts. In particular, we use the runtime names introduced below throughout this section, reserving the theoretical symbols for the formal correspondence. We also write @@name for a framework-internal symbol key, so the brackets in ctx[@@store] denote symbol-keyed access to an opaque slot on the context, rather than indexing into a string-keyed map.

p, 其中后三个随纤维条目写一次（引理 54(5)）），因此需要看到的是相同的名字带着相同的 d, p, 和 π, 产生、相同的名字被退役。两个序列按假设共享插入。它们也共享注册：A 纤维的激活在其每次迭代注册迭代器在那里命名的组件，定义 60 的第二条件在交错下保持它固定，因此 A-纤维下注册的树是该纤维组件的函数；那些注册抽取的名字不共享，而这里应用引理 56，用双射匹配两棵树。而退役要么是共享的编排步骤，要么是累加器采取的 O-Retire，后者恰好退役相同激活注册的名字。线性化 $\lhd$ 的两个枚举只差不可比片段之转置，引理 71 又使端点不变，因此两个规范序列一致。与定理 66 的终止一起，生命周期联系因此有唯一规范形。□

失败被排除在声明之外，因为它是真正的分歧来源，而演算不应被读作否认它：步骤是否抛错取决于它运行针对的状态，因此一个调度可能使纤维失败而另一个完成它，两个静止状态随后在该纤维的生命周期状态上不同。它们在其他任何东西上不不同，由推论 62，它把失败纤维对状态的贡献置零。

在第 4.2 节的基础演算中，相同定理成立，而证明除了去掉一个条款外无需替换。L-Unload 在那里不携带守卫，因此引理 72 的最后一段是空洞的；该引理其余部分只诉诸 quiett ，基础演算原样提供。

该定理是允许把 Cordis 应用当作静态装配来推理的东西。添加组件、移除它、替换提供者、再逆转替换的编排器被保证到达它通过在一开始写下最终组合本应得到的状态，而推理哪些余效应在作用域内的组件作者可以只推理静止状态。它也划出保证的界限：它谈状态，不谈系统沿途产生的排放，这正是第 6.1 节在边界内跟踪的获取与跨越它的排放之间划的区分。

5. 实现与案例研究（Implementation and Case Study）

本节介绍 Cordis，它把第 3 节的诸形式模型实现为一种实用的编程抽象。Cordis 是一个时空可组合性的元框架：与针对特定领域（如 Web 路由、ORM、UI 渲染）的应用框架不同，它不规定任何具体场景；其唯一职责是提供通用的动态组合语义。实现分三层：(1) 核心库（第 5.1 节）直接实现效应与余效应系统；(2) 组件加载器（第 5.2 节）用配置协调与热模块替换扩展核心；(3) 像 Koishi（第 5.3 节）这样的应用框架在前两层之上构建领域特定功能。

5.1. 核心库（Core Library）

表 2 总结了理论构造与其运行时对应物之间的对应关系。特别地，我们在本节通篇使用下面引入的运行时名称，把理论符号保留给形式对应。我们也写作@@name 表示框架内部符号键，因此 ctx[@@store] 中的方括号表示对上下文中一个不透明槽的符号键访问，而不是索引进一个字符串键映射。

Theory (Section 3, Section 4)	Implementation
Γ_∞	ctx, the first-class context
$\gamma \in \Gamma$	the context tree together with everything the running system has touched
$\mathfrak{E}_\Gamma, \mathfrak{E}_\Gamma^{\text{iter}}$	Effect callback returning / yielding inverses
$effect_\Gamma(e)$	ctx.effect(callback)
$\Sigma, \Sigma^{\text{iso}}, \Sigma^{\text{inter}}$	ctx[@@store], ctx[@@isolate], ctx[@@intercept]
$get(k), set(k, v)$	ctx.get(key), ctx.set(key, value)
$isolate(k, r)$	ctx.isolate(key, realm)
$intercept(k, \nu)$	ctx.intercept(key, metadata)
$\langle d, p, e, \pi, \sigma, \tau, \theta \rangle$	fiber, the instantiation of a component in $\mathfrak{C}_\Gamma$
$dom(F_\gamma)$	enumerated through ctx.registry
$n : \mathfrak{N}$	fiber.uid
$d : \mathfrak{D}_\Gamma$	fiber.inject
$p : \mathfrak{P}_\Gamma$	the component's provide
$e : \mathfrak{E}_\Gamma^*$	fiber.apply
$\pi : \mathfrak{N}$	fiber.parent.fiber.uid, the fiber owning the context it was instantiated on
derived realization (Definition 27)	fiber. ctx, the child context the fiber runs in
θ (Definition 44)	fiber.state, the lifecycle state, whose LOADING is Reloading and whose FAILED is Inactive(ξ)
recover, accumulator g	fiber.dispose, the accumulator
ω (Definition 44)	fiber.committed, the committed view
$provider_k(\gamma)$	an Impl whose provider fiber is ACTIVE
$target(\gamma, n)$	fiber.target, recomputed by refresh (Algorithm 5), where $\perp$ is INACTIVE
Future, inertia (Section 4.3.3)	fiber.inertia, the handle of the transition in flight
O-Insert, O-Retire (Definition 47)	ctx.use and the inverse of its callback (Algorithm 4)
O-Remove	the fiber dropped from its runtime, with uid cleared
L-Begin, L-Iter, L-Finish	execute's iteration loop (Algorithm 1)
L-Divert	the guard failing at an iteration boundary (Algorithm 1), or reload chaining into unload
L-Leave	refresh marking the fiber UNLOADING (Line 10)
L-Unload	unload and its inertial chaining (Algorithm 5)
guard on L-Unload	unload awaiting the notified dependents (Line 25)
L-Raise	the error recorded on the fiber, with its target set to $\perp$

Table 2 | Theory-to-implementation correspondence

The remainder of this section builds the core library from the bottom up. Section 5.1.1 realizes revertible efects, the sole primitive through which a context is mutated; Section 5.1.2 realizes reactive coefects over it; Section 5.1.3 composes both into the component lifecycle; and Section 5.1.4 exposes the context-level operations built on them.

理论（第 3 节、第 4 节）	实现
Γ_∞	ctx，一等上下文
$\gamma \in \Gamma$	上下文树连同运行系统所触碰的一切
$\mathfrak{E}_\Gamma, \mathfrak{E}_\Gamma^{\text{iter}}$	返回/产出逆的效应回调
$effect_\Gamma(e)$	ctx.effect(callback)
$\Sigma, \Sigma^{\text{iso}}, \Sigma^{\text{inter}}$	ctx[@@store], ctx[@@isolate], ctx[@@intercept]
$get(k), set(k, v)$	ctx.get(key), ctx.set(key, value)
$isolate(k, r)$	ctx.isolate(key, realm)
$intercept(k, \nu)$	ctx.intercept(key, metadata)
$\langle d, p, e, \pi, \sigma, \tau, \theta \rangle$	fiber，$\mathfrak{C}_\Gamma$ 中一个组件的实例化
$dom(F_\gamma)$	通过 ctx.registry 枚举
$n : \mathfrak{N}$	fiber.uid
$d : \mathfrak{D}_\Gamma$	fiber.inject
$p : \mathfrak{P}_\Gamma$	组件的 provide
$e : \mathfrak{E}_\Gamma^*$	fiber.apply
$\pi : \mathfrak{N}$	fiber.parent.fiber.uid，拥有其被实例化所在上下文的纤维
派生实现（定义 27）	fiber.ctx，纤维运行所在的子上下文
θ（定义 44）	fiber.state，生命周期状态，其 LOADING 即 Reloading、其 FAILED 即 Inactive(ξ)
recover、累加器 g	fiber.dispose，累加器
ω（定义 44）	fiber.committed，已提交视图
$provider_k(\gamma)$	其提供者纤维为 ACTIVE 的一个 Impl
$target(\gamma, n)$	fiber.target，由 refresh（算法 5）重算，其中 $\perp$ 即 INACTIVE
Future、惯性（第 4.3.3 节）	fiber.inertia，飞行中转移的句柄
O-Insert、O-Retire（定义 47）	ctx.use 与其回调的逆（算法 4）
O-Remove	从运行时丢弃的纤维，uid 被清除
L-Begin、L-Iter、L-Finish	execute 的迭代循环（算法 1）
L-Divert	迭代边界处守卫失败（算法 1），或 reload 链入 unload
L-Leave	refresh 把纤维标记为 UNLOADING（第 10 行）
L-Unload	unload 及其惯性链（算法 5）
L-Unload 上的守卫	unload 等待被通知的依赖方（第 25 行）
L-Raise	记录在纤维上的错误，其 target 被设为 $\perp$

表 2 | 理论到实现的对应

本节的其余部分自下而上构建核心库。第 5.1.1 节实现可逆效应，即上下文被修改所经由的唯一原语；第 5.1.2 节在其上实现响应式余效应；第 5.1.3 节把两者组合成组件生命周期；第 5.1.4 节暴露建立在其上的上下文级操作。

5.1.1. Effect Tracking

This section realizes revertible efects (Section 3.1). Every context mutation in Cordis flows through a single primitive, ctx.effect: coeffect provision, component instantiation, and every other context-mutating operation reduces to a ctx.effect call, so any operation performed through the context is automatically tracked and recovered upon component unloading. Operationally, ctx.effect is the realization of effectiter (Definition 52): it takes a callback of type $\mathfrak{E}_\Gamma^{\text{iter}}$ and lifts it to $\mathfrak{E}_{\partial\Gamma}^{\text{iter}}$, yielding a dispose closure that, when invoked, recovers the efect. Cordis accepts both $\mathfrak{E}_\Gamma$ and $\mathfrak{E}_\Gamma^{\text{iter}}$ through this one operation (ad-hoc polymorphism); we take the iterator form as representative, since a plain effect function is the degenerate iterator that yields a single inverse. What the operation does not check is the witness that $\mathfrak{E}_\Gamma^*$ carries: the callback supplies an inverse, and that the inverse recovers the effect it accompanies is an obligation on the component author rather than a property the runtime verifies. Theorem 61 is where the calculus appeals to it, and Section 6.1 is where the obligation is delimited.

Algorithm 1 shows the construction of ctx.effect. We write $f \circ g$ for the disposer that runs f after g, and id for the no-op; prepending each new inverse therefore yields LIFO recovery.

```
Algorithm 1 Effect tracking
1 async function execute(callback, guard)
2 iter ← callback()
3 inverse ← id
4 while guard()
5 (value, done) ← await iter.next()
6 if value then inverse ← value ∘ inverse
7 if done then break
8 return inverse
9 function effect(ctx, callback)
10 armed ← true
11 task ← execute(callback, () → armed)
12 async function dispose()
13 if not armed then return
14 armed ← false
15 recover ← await task
16 recover()
17 ctx.dispose ← dispose ∘ ctx.dispose
18 return dispose
```

The engine execute drives the callback as an effect iterator ($\mathfrak{E}_\Gamma^{\text{iter}}$, Definition 51) and folds the inverse yielded at each step into a single composite. Before each step it consults a callersupplied guard; once the guard trips, iteration stops and only the inverses accumulated so far remain. This is the step-boundary interruption of Section 4.3.2: the Maybe ($\mathfrak{E}^{\text{iter}}$) continuation is realized by the iterator's done flag together with guard.

ctx.effect is a thin wrapper over execute that adds two things. First, self-disposal: the guard reports the armed flag, and the returned dispose flips armed to false, which simultaneously halts any in-flight iteration and makes recovery fire at most once. Firing twice would apply an inverse at a state no application of the effect produced, where nothing holds it to reverting anything. Second, parent composition: dispose is prepended to the enclosing context's accumulated inverse ctx.dispose, so a child effect's inverse is itself an effect on the parent,

5.1.1. 效应跟踪（Effect Tracking）

本节实现可逆效应（第 3.1 节）。Cordis 中的每个上下文修改都流经一个唯一原语 ctx.effect：余效应提供、组件实例化以及每个其他修改上下文的操作都归结为一次 ctx.effect 调用，因此任何经上下文执行的操作都会被自动跟踪并在组件卸载时被恢复。操作上，ctx.effect 是 effectiter（定义 52）的实现：它接受一个类型为 $\mathfrak{E}_\Gamma^{\text{iter}}$ 的回调，并将其提升到 $\mathfrak{E}_{\partial\Gamma}^{\text{iter}}$ ，产生一个 dispose 闭包，该闭包在被调用时恢复该效应。Cordis 通过这一个操作同时接受 $\mathfrak{E}_\Gamma$ 和 $\mathfrak{E}_\Gamma^{\text{iter}}$（临时多态）；我们以迭代器形式为代表，因为普通效应函数是产生单个逆的退化迭代器。该操作不检查的是 $\mathfrak{E}_\Gamma^*$ 所携带的见证：回调提供一个逆，而该逆恢复其所伴随的效应是对组件作者的义务，而非运行时验证的性质。定理 61 是演算诉诸它的地方，第 6.1 节是对该义务加以界定的地方。

算法 1 展示 ctx.effect 的构造。我们写 $f \circ g$ 表示先运行 g 再运行 f 的处置器，id 表示无操作；因此前置每个新逆会产生 LIFO 恢复。

```
Algorithm 1 Effect tracking
1 async function execute(callback, guard)
2 iter ← callback()
3 inverse ← id
4 while guard()
5 (value, done) ← await iter.next()
6 if value then inverse ← value ∘ inverse
7 if done then break
8 return inverse
9 function effect(ctx, callback)
10 armed ← true
11 task ← execute(callback, () → armed)
12 async function dispose()
13 if not armed then return
14 armed ← false
15 recover ← await task
16 recover()
17 ctx.dispose ← dispose ∘ ctx.dispose
18 return dispose
```

引擎 execute 把回调作为效应迭代器（$\mathfrak{E}_\Gamma^{\text{iter}}$ ，定义 51）驱动，并把每一步产出的逆折叠进单个复合逆中。每一步之前它查询一个调用方提供的守卫；一旦守卫触发，迭代停止，只保留迄今累积的逆。这是第 4.3.2 节的步边界中断：Maybe ($\mathfrak{E}^{\text{iter}}$) 续延由迭代器的 done 标志连同守卫实现。

ctx.effect 是 execute 之上的一个薄包装，它添加两样东西。第一，自处置：守卫报告 armed 标志，返回的 dispose 把 armed 翻转为 false，这同时中止任何飞行中的迭代并使恢复至多触发一次。触发两次会在没有任何效应应用产生的状态处应用逆，那里没有东西约束它恢复任何东西。第二，父组合：dispose 被前置到包围它的上下文所累积的逆 ctx.dispose 上，因此子效应的逆本身是父上的一个效应，这正是 $\partial^2\Gamma$ 的递归结构。组件级（第 5.1.3 节）复用同一个 execute，其守卫测试 fiber.target 的稳定性而非 armed。

which is the recursive structure of $\partial^2\Gamma$. The component level (Section 5.1.3) reuses the same execute with a guard that tests the stability of fiber.target instead of armed.

5.1.2. Coeffect Operations

This section realizes reactive coefects (Section 3.2). All coeffect operations act on three symbolkeyed slots that each context carries:

- @@store: the value store $\sigma : (r : R)\mathcal{V}_r$ from realm symbols to typed values;
- @@isolate: the realm table $\rho : \mathrm{Map}(K, R)$ from coeffect keys to realm symbols;
- @@intercept: the interception table $\iota : (k : K) \to \mathcal{M}_k$ assigning each key its metadata.

The first two compose into the two-layer resolution $k \to \rho(k) \to \sigma(\rho(k))$: ctx.get(key) (Algorithm 2) reads the realm symbol $\rho(k)$ from @@isolate, then the bound value $\sigma(\rho(k))$ from @@store. The ρ indirection lets isolation redirect a key to an independent binding, whereas @@intercept is consulted only when a binding is accessed, adjusting how it is used rather than what it resolves to. We realize these operations in two parts: (1) provision and notification, which install or retract bindings and propagate the change to dependents; and (2) isolation and interception, which reshape how a key resolves.

Provision and notification. Since set (k, v) has type $\mathfrak{E}_\Sigma$ (Section 3.1), coeffect provision is a ctx.effect call and inherits its automatic tracking and recovery. Algorithm 2 implements ctx.set(key, value), the concrete set (k, v) : the callback binds a value into the store under the realm symbol $\rho(k)$, and the returned dispose function removes it. Both installation and removal invoke notify to propagate the change to dependent components.

Algorithm 2 Coeffect operations

```
1 function get(ctx, key)
2    realm ← ctx[@@isolate][key] ▷ ρ(k)
3    return ctx[@@store][realm] ▷ σ(ρ(k))
4 function set(ctx, key, value)
5    function callback()
6    realm ← ctx[@@isolate][key] ▷ ρ(k)
7    ctx[@@store][realm] ← value ▷ σ[ρ(k) ↦ v]
8    notify(ctx, [key])
9    return function()
10    delete ctx[@@store][realm] ▷ σ \ ρ(k)
11    notify(ctx, [key])
12    return ctx.effect(callback)
```

Algorithm 3 propagates each binding change to dependents by testing, for each live fiber, whether a changed key appears in its fiber.inject and resolves to the same realm; if so, it calls refresh (Section 5.1.3) to re-evaluate that fiber against the new state, and it returns the fibers it re-evaluated so that a caller can wait for them. This is the reactive classification of Definition 26: a change that flips satisfaction activates or deactivates the fiber, and refresh's idempotence renders a neutral change harmless. The interaction of this re-evaluation with diverse control flows is developed in Section 5.1.3.

Algorithm 3 Reactive notification

```
1 function notify(ctx, keys)
2    affected ← ∅
```

5.1.2. 余效应操作（Coeffect Operations）

本节实现响应式余效应（第 3.2 节）。所有余效应操作作用于每个上下文携带的三个符号键槽：

- @@store：值存储 $\sigma : (r : R) \to \mathcal{V}_r$ ，从领域符号到类型化值；
- @@isolate：领域表 $\rho : \mathrm{Map}(K, R)$ ，从余效应键到领域符号；
- @@intercept：拦截表 $\iota : (k : K) \to \mathcal{M}_k$ ，把每个键分配其元数据。

前两者组合成两层解析 $k \to \rho(k) \to \sigma(\rho(k))$ ：ctx.get(key)（算法 2）从 @@isolate 读取领域符号 $\rho(k)$ ，然后从 @@store 读取绑定值 $\sigma(\rho(k))$ 。ρ 间接层让隔离把一个键重定向到独立绑定，而 @@intercept 只在访问绑定时才被查询，调整它如何使用而非它解析到什么。我们分两部分实现这些操作：(1) 提供与通知，安装或撤除绑定并把变化传播给依赖方；以及 (2) 隔离与拦截，重塑一个键如何解析。

提供与通知。由于 set (k, v) 具有类型 $\mathfrak{E}_\Sigma$（第 3.1 节），余效应提供是一次 ctx.effect 调用并继承其自动跟踪与恢复。算法 2 实现 ctx.set(key, value)，即具体的 set (k, v) ：回调把值在领域符号 $\rho(k)$ 下绑定进存储，返回的 dispose 函数移除它。安装与移除都调用 notify 把变化传播给依赖组件。

Algorithm 2 Coeffect operations

```
1 function get(ctx, key)
2    realm ← ctx[@@isolate][key] ▷ ρ(k)
3    return ctx[@@store][realm] ▷ σ(ρ(k))
4 function set(ctx, key, value)
5    function callback()
6    realm ← ctx[@@isolate][key] ▷ ρ(k)
7    ctx[@@store][realm] ← value ▷ σ[ρ(k) ↦ v]
8    notify(ctx, [key])
9    return function()
10    delete ctx[@@store][realm] ▷ σ \ ρ(k)
11    notify(ctx, [key])
12    return ctx.effect(callback)
```

算法 3 通过为每个存活纤维测试已改变的键是否出现在其 fiber.inject 中并解析到同一领域，把每次绑定变化传播给依赖方；若如此，它调用 refresh（第 5.1.3 节）对照新状态重新求值该纤维，并返回它重新求值的纤维，以便调用方能等待它们。这是定义 26 的响应式分类：使满足性翻转的变化激活或停用纤维，而 refresh 的幂等性使中性变化无害。这种重新求值与多种控制流的相互作用在第 5.1.3 节展开。

Algorithm 3 Reactive notification

```
1 function notify(ctx, keys)
2    affected ← ∅
```

```
3    for fiber in all_fibers do
4    for key in keys do
5    if key ∈ fiber.inject and fiber ctx[@@isolate][key] = ctx[@@isolate][key] then
6    refresh(fiber)
7    affected ← affected ∪ {fiber}
8    break
9    return affected
```

A binding counts as available to a dependent only while the fiber that installed it is ACTIVE, so refresh resolves each declared key against an active provider rather than against the store alone. This is the provided by relation of Definition 46, and it is what makes a withdrawal visible to dependents one step before it happens: a provider that has entered UNLOADING has stopped providing, so its dependents recompute an unsatisfied target view and begin their own teardown while its bindings are all still in place.

Isolation and interception. The two operations do structurally the same thing: each derives a child context that adjusts one inherited table for key, leaving the parent untouched, so recovery is implicit: discarding the child context sufices, with no explicit inverse to run. ctx.isolate(key, realm) overrides the realm mapping ρ with realm, or a freshly generated symbol by default (realizing isolate, Definition 29), so two contexts that assign diferent symbols to the same key resolve to independent bindings. ctx.intercept(key, metadata) merges metadata into the interception table ι (realizing intercept, Definition 31): following that definition, the new metadata is combined with whatever the context already carries for key and takes priority over it.

5.1.3. Component Lifecycle

A component is instantiated as a fiber by ctx.use. This section gives the fiber (introduced in Section 5.1) operational meaning as the inertial state machine of Section 4.3.3. Two fields drive the algorithm below: fiber.parent, the parent context of fiber.ctx that forms the component hierarchy (the recursive structure of Γ_∞, Section 3.3.1), and fiber.inertia, a handle to the inflight asynchronous transition (or null if idle).

Algorithm 4 shows component instantiation. A component pairs a coeffect specification component.inject (d) with an effect function component.apply; instantiation binds the component's config into fiber.apply (Line 9), the config-applied effect function (e) that the lifecycle then runs. The callback function (Line 2) is the effect tracked in the parent fiber: when executed, it initiates the child's lifecycle by calling refresh (Algorithm 5); when recovered, it forces the child's target to $\bot$ and triggers unload. This is the registration primitive of Definition 47, with callback as its O-Insert and the closure callback returns as its O-Retire: an instantiation is an ordinary tracked effect of the parent, so unloading a parent cascades to its children.

```
function callback()
    refresh(fiber)
    return function()
    fiber.target ← ⊥
    unload(fiber)

fiber ← Fiber(parent: ctx, inject: component.inject)
fiber.ctx ← ctx[fiber → fiber]
fiber.apply ← () → component.apply(fiber.ctx, config)
```


```
3    for fiber in all_fibers do
4    for key in keys do
5    if key ∈ fiber.inject and fiber ctx[@@isolate][key] = ctx[@@isolate][key] then
6    refresh(fiber)
7    affected ← affected ∪ {fiber}
8    break
9    return affected
```


一个绑定只在安装它的纤维是 ACTIVE 时才对依赖方算作可用，因此 refresh 对照活跃提供者而非仅对照存储解析每个声明键。这是定义 46 的 provided-by 关系，也是使撤出在发生前一步就对依赖方可见的东西：已进入 UNLOADING 的提供者已停止提供，因此其依赖方重算出一个不满足的目标视图并在其绑定仍全部在位时开始自己的拆卸。

隔离与拦截。这两个操作在结构上做同一件事：每个都派生一个子上下文，为一个键调整一个继承的表，让父上下文原封不动，因此恢复是隐式的：丢弃子上下文即可，无需运行显式逆。ctx.isolate(key, realm) 用 realm（默认新生成的符号）覆盖领域映射 ρ（实现 isolate，定义 29），因此把不同符号赋给同一键的两个上下文解析到独立绑定。ctx.intercept(key, metadata) 把 metadata 合并进拦截表 ι（实现 intercept，定义 31）：依该定义，新元数据与上下文为 key 已经携带的任何东西组合并对其优先。

5.1.3. 组件生命周期（Component Lifecycle）

组件经 ctx.use 被实例化为一个纤维。本节把（第 5.1 节引入的）纤维作为第 4.3.3 节的惯性状态机赋予操作意义。两个字段驱动下面的算法：fiber.parent，fiber.ctx 的父上下文，它形成组件层级（Γ_∞, 第 3.3.1 节的递归结构），以及 fiber.inertia，飞行中异步转移的句柄（空闲时为 null）。

算法 4 展示组件实例化。一个组件把余效应规范 component.inject（d）与效应函数 component.apply 配对；实例化把组件的 config 绑定进 fiber.apply（第 9 行），即生命周期随后运行的经配置应用的效应函数（e）。回调函数（第 2 行）是父纤维中跟踪的效应：被执行时，它通过调用 refresh（算法 5）启动子组件的生命周期；被恢复时，它迫使子的 target 为 $\bot$ 并触发 unload。这是定义 47 的注册原语，以 callback 为其 O-Insert、以 callback 返回的闭包为其 O-Retire：一次实例化是父的一个普通跟踪效应，因此卸载父会级联到其子。


```
function callback()
    refresh(fiber)
    return function()
    fiber.target ← ⊥
    unload(fiber)

fiber ← Fiber(parent: ctx, inject: component.inject)
fiber.ctx ← ctx[fiber → fiber]
fiber.apply ← () → component.apply(fiber.ctx, config)
```

```
ctx.effect(callback)
return fiber
```

Algorithm 5 realizes the inertial state machine of Section 4.3.3, in which reload and unload are inertial: once entered, a transition runs to completion before the system responds to a targetstate change. It uses two auxiliary lookups over the coeffect store: resolve(inject) returns the bindings the declared keys currently resolve to, and provided(fiber) returns the keys whose binding this fiber installed. The refresh function recomputes fiber.target from the coeffect store and, if the fiber is not already in a transition, initiates either a reload or unload task2. The reload function records the current target and executes the component's effect function apply. Upon completion, it checks whether the target still matches: if so, the fiber enters ACTIVE; if not (regardless of whether the new target is $\perp$ or a diferent set of providers), it chains into unload. Symmetrically, unload recovers all tracked efects in LIFO order and then either enters INACTIVE or chains into reload. This mutual recursion implements the inertial property: once a transition begins, it completes before any new transition can start.

Algorithm 5 Component lifecycle

```
function refresh(fiber)
target ← target(γ, n)
if target = fiber.target then return
fiber.target ← target
if fiber.inertia then return
if target ≠ ⊥ then
   fiber.state ← LOADING
   fiber.inertia ← create_task(reload(fiber))
else
    fiber.state ← UNLOADING ▷ out of service before any inverse is scheduled
   fiber.inertia ← create_task(unload(fiber))
async function reload(fiber)
target_0 ← fiber.target
fiber.committed ← resolve(fiber.inject) ▷ commit the view
recover ← await execute(fiber.apply, () ↦ fiber.target = target_0)
fiber.dispose ← recover ∘ fiber.dispose
if fiber.target = target_0 then
   fiber.state ← ACTIVE

| notify(fiber.ctx, provided(fiber))
| fiber.inertia ← null
else
| fiber.state ← UNLOADING
| fiber.inertia ← create_task(unload(fiber))
async function unload(fiber)
await all.notify(fiber.ctx, provided(fiber)).map(f → f.await())) ▷ drain dependents
await fiber.dispose()
```


```
ctx.effect(callback)
return fiber
```


算法 5 实现第 4.3.3 节的惯性状态机，其中 reload 和 unload 是惯性的：一旦进入，一次转移运行到完成，之后系统才响应目标状态变化。它使用余效应存储上的两个辅助查询：resolve(inject) 返回声明键当前解析到的绑定，provided(fiber) 返回该纤维安装其绑定的键。refresh 函数从余效应存储重算 fiber.target，并且若纤维尚未处于转移中，则启动一个 reload 或 unload 任务 2。reload 函数记录当前目标并执行组件的效应函数 apply。完成后，它检查目标是否仍匹配：若匹配，纤维进入 ACTIVE；若不匹配（无论新目标是 $\perp$ 还是不同的提供者集），它链入 unload。对称地，unload 以 LIFO 顺序恢复所有被跟踪的效应，然后进入 INACTIVE 或链入 reload。这种互递归实现惯性性质：一旦一次转移开始，它在任何新转移能开始之前完成。

Algorithm 5 Component lifecycle

```
function refresh(fiber)
target ← target(γ, n)
if target = fiber.target then return
fiber.target ← target
if fiber.inertia then return
if target ≠ ⊥ then
   fiber.state ← LOADING
   fiber.inertia ← create_task(reload(fiber))
else
    fiber.state ← UNLOADING ▷ out of service before any inverse is scheduled
   fiber.inertia ← create_task(unload(fiber))
async function reload(fiber)
target_0 ← fiber.target
fiber.committed ← resolve(fiber.inject) ▷ commit the view
recover ← await execute(fiber.apply, () ↦ fiber.target = target_0)
fiber.dispose ← recover ∘ fiber.dispose
if fiber.target = target_0 then
   fiber.state ← ACTIVE

| notify(fiber.ctx, provided(fiber))
| fiber.inertia ← null
else
| fiber.state ← UNLOADING
| fiber.inertia ← create_task(unload(fiber))
async function unload(fiber)
await all.notify(fiber.ctx, provided(fiber)).map(f → f.await())) ▷ drain dependents
await fiber.dispose()
```

```
27 fiber.dispose ← id
28 fiber.committed ← ⊥
29 if fiber.target = ⊥ then
30 | fiber.state ← INACTIVE
31 | fiber.inertia ← null
32 else
33 | fiber.state ← LOADING
34 | fiber.inertia ← create_task(reload(fiber))
```

fiber.target is computed by resolving each declared key against the current coeffect store and tupling the uid of the fiber that provides it, so it is a digest of target(n, γ) (Definition 46). Identifying a binding by its provider rather than by its value is what makes a single comparison against the recorded target suficient: a uid is drawn fresh and never reused, so a provider that is replaced cannot be mistaken for the one it replaced, even when the two provide equal values. Since notify (Section 5.1.2) recomputes the target on every coeffect change, a fiber reloads precisely when one of its declared keys comes to be provided by a diferent fiber. A provider that overwrites its own binding in place is therefore not observed; a component that wants its replacement to propagate withdraws the binding and installs it afresh.

The algorithm operates at two complementary levels. At the transition level, reload and unload check the target at completion, enabling inertial chaining across transitions. At the iteration level within each transition, the effect execution (Algorithm 1) checks the target at each iteration boundary, enabling partial rollback within a single transition. These two mechanisms correspond to the inter-transition chaining of Section 4.3.3 and the intra-transition staleness check that Theorem 64 rests on.

Three lines carry the coeffect ordering of Theorem 63, and where each of them sits is what makes the ordering hold. reload commits the resolved view at Line 14 and unload discards it only after every inverse has run, so a fiber reads the same bindings for as long as it is loaded, its own teardown included. refresh marks the fiber UNLOADING at Line 10 before the transition task is created, which is the L-Leave step: the fiber stops providing, and the dependents recompute against that before any of its inverses is scheduled. unload then waits at Line 25 for each notified dependent to reach INACTIVE, which is the guard on L-Unload; notify admits a dependent only when its declared key resolves to the same realm symbol as the provider's, which is the runtime form of the guard's demand that the dependent see the key from this fiber rather than merely declare it. The wait sits ahead of the whole recovery rather than inside one of the inverses being waited on, since fiber.dispose initiates a fiber's efects concurrently and a wait placed within one of them would leave the rest unordered. Termination follows Theorem 66: a fiber only ever waits on dependents that have already stopped being satisfiable, and a dependent that is itself a provider waits the same way for its own, so the provider graph is traversed on demand rather than analyzed in advance.

5.1.4. Context Access

The coeffect operations of Section 5.1.2 form a reflective API: a coeffect is written with ctx.set(key, value) and read with ctx.get(key), both keyed by name. Cordis layers a second, more native way to extend and consume the context on top of this reflective API: property access. A component can access a coeffect as the property ctx[key], as if it were native structure of the context, rather than through a method call. In TypeScript, Cordis realizes this with a Proxy whose get trap mediates every property access. Algorithm 6 shows how a context resolves such an access to a coeffect, atop the primitive get of Section 5.1.2.


```
27 fiber.dispose ← id
28 fiber.committed ← ⊥
29 if fiber.target = ⊥ then
30 | fiber.state ← INACTIVE
31 | fiber.inertia ← null
32 else
33 | fiber.state ← LOADING
34 | fiber.inertia ← create_task(reload(fiber))
```


fiber.target 通过对照当前余效应存储解析每个声明键并元组化提供它的纤维的 uid 来计算，因此它是 target(n, γ)（定义 46）的一个摘要。用提供者而非其值来识别绑定，是使对已记录目标的一次比较即足够的东西：uid 是全新抽取且永不重用的，因此被替换的提供者不能被误认为替换它的那个，即使两者提供相等的值。由于 notify（第 5.1.2 节）在每次余效应变化时重算目标，一个纤维恰在其声明键之一改由不同纤维提供时重新加载。因此，在其原位覆写自身绑定的提供者不被观察到；想让其替换传播的组件会撤出绑定并全新安装它。

该算法在两个互补的层面运作。在转移层面，reload 和 unload 在完成时检查目标，使惯性链能够跨越转移。在每个转移内部的迭代层面，效应执行（算法 1）在每次迭代边界检查目标，使单次转移内的部分回滚成为可能。这两个机制分别对应第 4.3.3 节的转移间链接与定理 64 所依赖的转移内过期检查。

三行承载定理 63 的余效应排序，而每一行所在的位置正是使排序成立的东西。reload 在第 14 行提交解析视图，unload 只在每个逆都运行之后才丢弃它，因此只要纤维被加载（包括其自身的拆卸），它就读到相同的绑定。refresh 在转移任务创建之前于第 10 行把纤维标记为 UNLOADING，即 L-Leave 步骤：纤维停止提供，依赖方在其任何逆被调度之前据此重算。unload 随后在第 25 行等待每个被通知的依赖方到达 INACTIVE，即 L-Unload 上的守卫；notify 只在依赖方的声明键解析到与提供者相同的领域符号时才接纳它，这是守卫之要求的运行时形式，即依赖方从该纤维看到键而非仅仅声明它。等待位于整个恢复之前而非被等待的某个逆内部，因为 fiber.dispose 并发启动一个纤维的效应，而置于其中之一内的等待会让其余部分失去排序。终止遵循定理 66：一个纤维只等待那些已停止可满足的依赖方，而自身是提供者的依赖方对其自己的依赖方以同样方式等待，因此提供者图按需遍历而非预先分析。

5.1.4. 上下文访问（Context Access）

第 5.1.2 节的余效应操作构成一个反射式 API：余效应用 ctx.set(key, value) 写入、用 ctx.get(key) 读取，两者都按键名。Cordis 在此反射式 API 之上叠加第二种更原生地扩展与消费上下文的方式：属性访问。组件可以把余效应作为属性 ctx[key] 访问，就像它是上下文的原生结构，而非经由方法调用。在 TypeScript 中，Cordis 用 get 陷阱中介每次属性访问的 Proxy 实现这一点。算法 6 展示上下文如何在第 5.1.2 节的原语 get 之上把这样的访问解析为一个余效应。

```
Algorithm 6 Proxy-mediated context access
1 function resolve(ctx, key)
2 fiber ← ctx.fiber
3 repeat
4 if key ∈ fiber.committed then return fiber.committed[key]
5 if key ∈ fiber.inject then throw INACTIVE_ACCESS
6 if fiber = root then throw UNDECLARED_ACCESS
7 fiber ← fiber.parent.fiber
```

Algorithm 6 walks the fiber chain upward from the accessing context: at the first fiber whose committed view binds key, the access is authorized and that binding is returned; if the walk reaches a fiber that declares key without having committed it, the fiber is not loaded and the access fails; and if it reaches the root without any declaration, the access is rejected as undeclared. This is where the proxy difers from the bare ctx.get: ctx.get(key) is a lookup against the store that returns the bound value or nothing and never fails, whereas the proxy resolves against the accessing fiber's own view and enforces the coeffect specification d at the point of use. Reading the view rather than the store is also what Theorem 63 rests on, since it is what keeps a dependency readable to a component whose teardown was triggered by that dependency going away.

This rejection is a runtime check performed at the point of access. Because a component's coeffect specification d is declared statically, the same violation is in principle detectable at compile time, by resolving each ctx[key] against the declared d before execution; Section 6.4 discusses how a host language's type-level dependency declarations and compile-time metaprogramming can carry out exactly this mediation.

5.2. Component Loader

The core library equips component developers with imperative primitives for dynamic composition, such as ctx.effect, ctx.use, and ctx.set. A separate concern arises for application orchestrators, who assemble pre-existing components into a running system and adjust the composition over its lifetime. The component loader addresses this concern by introducing a declarative configuration layer: the orchestrator specifies the desired composition as a persistent data structure, and the loader translates changes to this specification into the corresponding imperative fiber operations.

5.2.1. Declarative Configuration

Section 4 decomposes a running system into fibers, each an instantiation of one component. Everything an instantiation needs can be declared, so an orchestrator can describe a whole system as a declarative configuration: a persistent record that the loader realizes as fibers and keeps in step with them.

Entries. A configuration consists of entries. Each entry specifies a fiber and manages it, and the binding runs in both directions: the loader responds to a change in an entry's fields by adjusting the fiber, and a component that revises its own configuration or disables itself has the change written back to its entry.

Definition 74. An entry declares a single fiber, recording:

- id — a stable identifier, used as the reconciliation key when its group's child list changes;
- url — the URL of the component module to instantiate;
- isolate — an isolation annotation applied to the entry's context;
- intercept — an interception annotation applied to the entry's context;


```
Algorithm 6 Proxy-mediated context access
1 function resolve(ctx, key)
2 fiber ← ctx.fiber
3 repeat
4 if key ∈ fiber.committed then return fiber.committed[key]
5 if key ∈ fiber.inject then throw INACTIVE_ACCESS
6 if fiber = root then throw UNDECLARED_ACCESS
7 fiber ← fiber.parent.fiber
```


算法 6 从访问上下文沿纤维链向上走：在已提交视图绑定 key 的第一个纤维处，访问被授权并返回该绑定；若走到一个声明了 key 而未提交它的纤维，则纤维未加载且访问失败；若到达根而无任何声明，则访问被拒为未声明。这正是 proxy 与裸 ctx.get 不同之处：ctx.get(key) 是对存储的一次查询，返回绑定值或什么也不返回且从不失败，而 proxy 对照访问纤维自身的视图解析并强制使用点的余效应规范 d。读取视图而非存储也是定理 63 所依赖的，因为正是它让依赖对因该依赖消失而触发其拆卸的组件仍可读。

这种拒绝是访问点执行的一次运行时检查。由于组件的余效应规范 d 是静态声明的，同样的违反原则上可在编译期检测，通过在执行前把每个 ctx[key] 对照声明的 d 解析；第 6.4 节讨论宿主语言的类型级依赖声明与编译期元编程如何能恰好执行这种中介。

5.2. 组件加载器（Component Loader）

核心库为组件开发者装备了动态组合的命令式原语，如 ctx.effect、ctx.use 和 ctx.set。应用编排者面临一个独立的问题，他们把预先存在的组件组装成运行系统并在其生命周期内调整组合。组件加载器通过引入一个声明式配置层解决这个问题：编排者把期望的组合指定为持久数据结构，加载器把对该规范的变化翻译成相应的命令式纤维操作。

5.2.1. 声明式配置（Declarative Configuration）

第 4 节把运行系统分解为纤维，每个纤维是一个组件的一次实例化。一次实例化所需的一切都可以声明，因此编排者可以把整个系统描述为一个声明式配置：一个加载器实现为纤维并与之保持同步的持久记录。

条目。一个配置由条目组成。每个条目指定一个纤维并管理它，而绑定双向运行：加载器通过调整纤维响应条目字段的变化，而修订自身配置或禁用自身的组件会把变化写回其条目。

定义 74. 一个条目声明单个纤维，记录：

- id —— 稳定标识符，其组的子列表改变时用作协调键；
- url —— 要实例化的组件模块的 URL；
- isolate —— 应用到条目上下文的隔离注解；
- intercept —— 应用到条目上下文的拦截注解；

• config — the configuration bound into the component to form its effect function apply;

• disabled — whether the entry is administratively turned of.

An entry can serve as a faithful specification because what supports a fiber is exactly what an entry records. The support set of Definition 67 reads $\tau, \pi, d,$, and p and nothing else, and an entry gives all four: disabled gives τ, the entry's parent in the tree gives π, and url selects the component which declares d and p. The fields the support set leaves unread are the fiber's runtime state, which an instantiation does not need either, and Lemma 70 identifies the support set with the Active fibers of a quiescent state (Definition 49) as far as each component installs every key it declares (Definition 69).

These entries form a configuration tree that is the authoritative record of what the system loads. An entry may be a leaf mapping to a single fiber, or its component may in turn load further components, making the entry a branch node. Cordis provides components for such grouped and nested loading: @cordisjs/group takes a list of child entries as its configuration and loads them as a subgroup, and @cordisjs/include loads an external configuration file (YAML or JSON) and grafts its entries in as a nested subtree. Both are ordinary components resting on the registration primitive of Definition 47 (Algorithm 4), so a nested tree stays within the calculus and the results below hold of it.

Reconciliation. When an entry's record changes, the loader reconciles incrementally rather than tearing the fiber down and rebuilding it wholesale. Reconciling this way is sound for reasons the metatheory supplies.

• Theorem 73 makes the quiescent state a function of the final configuration alone: whatever instantiations and retirements the loader performs on the way, and in whatever order, the system quiesces where a load of the final configuration from scratch would have left it. Which components end up loaded is read of the declarations only as far as each of them installs every key it declares (Definition 69); a component that declares a key and installs it under some configurations alone is one the loader can still reconcile, but the set of loaded components then answers to those configurations as well.

• Theorem 66 proves that the system does quiesce, so a reconciliation is complete once its instantiations and retirements have been issued.

• Corollary 62 puts a departing fiber's contribution to the state at nothing, so rebuilding one entry withdraws what its fiber installed and leaves the fibers around it as they were.

• Theorem 63 lets the entries be instantiated together, with no load order for the orchestrator to arrange: a fiber whose declared keys are not yet provided waits at its L-Begin, and one whose provider leaves is deactivated ahead of it. A dependency therefore constrains when a fiber activates rather than when its module is fetched and evaluated, so the loader loads modules concurrently, where bringing up a large configuration spends its time.

On top of the fiber that an entry declares, the loader dispatches on which of the entry's fields changed and applies the least disruptive operation for each.

• id, url — rebuilds the entry, since its identity or its component has changed;

• isolate — reassigns the entry's realms (Algorithm 7);

• intercept — updated in place, as interception metadata is consulted at read time and needs no reload;

• config — handed to the component, which decides how to apply the new payload, typically by difing it against the previous one and reloading only on a material change. In particular, an @cordisjs/group entry's config is its list of child entries, so it applies the update as a keyed dif over child ids, creating, removing, or updating each child; since updating a surviving child re-enters this same per-field dispatch, group reconciliation and entry update recurse together down the tree;

• disabled — unloads the fiber when set and reloads it when cleared.

Managed realms. Isolation in the core derives a child context overriding the realm table ρ at one key

• config —— 绑定进组件以形成其效应函数 apply 的配置；

• disabled —— 条目是否被管理上关闭。

一个条目可以作为忠实的规范，因为支撑一个纤维的东西恰好是条目记录的东西。定义 67 的支撑集只读 $\tau, \pi, d,$ 和 p 而不读别的，而一个条目给出全部四个：disabled 给出 τ，树中条目的父给出 π，url 选择声明 d 和 p 的组件。支撑集留下不读的字段是纤维的运行时状态，一次实例化也不需要它，而引理 70 在每组件安装它声明的每个键（定义 69）之处把支撑集与静止状态（定义 49）的 Active 纤维等同。

这些条目形成一个配置树，即系统加载什么的可信记录。一个条目可以是映射到单个纤维的叶子，或其组件可以转而加载更多组件，使条目成为分支节点。Cordis 为这种分组与嵌套加载提供组件：@cordisjs/group 把一个子条目列表作为其配置并把它们作为子组加载，@cordisjs/include 加载一个外部配置文件（YAML 或 JSON）并把其条目作为嵌套子树嫁接进来。两者都是建立在定义 47（算法 4）的注册原语之上的普通组件，因此一棵嵌套树保持在演算之内且下面的结果对之成立。

协调。当一个条目的记录变化时，加载器增量协调而非拆毁纤维并整体重建。这样协调是健全的，原因由元理论提供。

• 定理 73 使静止状态成为仅最终配置的函数：无论加载器沿途执行什么实例化与退役、以什么顺序，系统都在从零加载最终配置本应达到的地方静止。最终哪些组件被加载只按声明读出，只要每组件安装它声明的每个键（定义 69）；只把某键在某些配置下声明并安装的组件，加载器仍可协调，但加载的组件集随后也应答那些配置。

• 定理 66 证明系统确实静止，因此一旦其实例化与退役已发出，一次协调即完成。

• 推论 62 把离开的纤维对状态的贡献置零，因此重建一个条目撤出它的纤维所安装的、让周围纤维保持原状。

• 定理 63 让条目一起实例化，编排者无需安排加载顺序：声明键尚未被提供的纤维在其 L-Begin 处等待，而提供者离开的那个在它之前被停用。依赖因此约束纤维何时激活而非何时取回并求值其模块，所以加载器并发加载模块，拉起大型配置的时间花在那里。

在条目声明的纤维之上，加载器按条目的哪个字段变化分派，并为每个应用扰动最小的操作。

• id、url —— 重建条目，因为其身份或其组件已变；

• isolate —— 重分配条目的领域（算法 7）；

• intercept —— 原位更新，因为拦截元数据在读取时被查询且无需重新加载；

• config —— 交给组件，组件决定如何应用新载荷，通常把它与先前的 diff 并只在实质变化时重新加载。特别地，一个 @cordisjs/group 条目的 config 是其子条目列表，因此它把更新作为子 id 上的键控 diff 应用，创建、移除或更新每个子；由于更新存活的子会重新进入这个相同的逐字段分派，组协调与条目更新一起沿树递归；

• disabled —— 设置时卸载纤维、清除时重新加载。

受管领域。核心中的隔离派生一个子上下文，在一个键处覆盖领域表 ρ（第 5.1.2 节），这在上下文树静止时

(Section 5.1.2), which sufices while the context tree stands still. An entry may be moved between groups at runtime, so the loader manages realms of its own, and the isolate field selects between two scoping rules per key. A value of true asks for a local realm, private to the entry and tagged by its id, which the entry carries with it wherever it moves; a string asks for a global realm shared by every entry naming that string, so moving such an entry changes which entries it shares a binding with rather than which realm it belongs to. A realm is discarded once no entry names it.

Reassigning an entry's realms turns on which keys changed realm, whether the entry is itself the provider at a changed key, and which dependents to notify. The middle question is the hard one, since a realm symbol may be shared by several fibers of which only one is the provider. The loader answers it with delimiters: one symbol ρ per key, under which each context stores a tag of its own. A delimiter is written on a context and inherited by its descendants, so the entry's tag and the provider's agree exactly when the two were derived within one isolate scope for ρ, which is the case in which the binding at ρ is the entry's own and has to move with it.

Algorithm 7 Isolation realm reassignment 1 function patch_isolation(entry, ρ') 2 $\rho \leftarrow$ entry ctx[@@isolate] 3 store $\leftarrow$ entry ctx[@@store] 4 $\Delta \leftarrow \{k \mid \rho(k) \neq \rho'(k)\}$▷ keys whose realm changes 5 for k in Δ do 6 entry ctx[δ_k] $\leftarrow$ fresh tag 7 diff[k] $\leftarrow$ ($\rho(k)$, $\rho'(k)$, entry ctx[δ_k], store[$\rho(k)$].fiber ctx[δ_k])
8 entry ctx[@@isolate] $\leftarrow \rho'$ 9 reload(entry.fiber) 10 for k in Δ do

$(s_1, s_2, d_1, d_2) \leftarrow$ diff[k] if $d_1 = d_2$ and store [s_1] and not store [s_2] then ▷ the binding is the entry's own store [s_2] $\leftarrow$ store[s_1] delete store [s_1] function affected(fiber, k) $(s_1, s_2, d_1, d_2) \leftarrow$ diff[k] return fiber.ctx[@@isolate][k] $\in \{s_1, s_2\}$ and (fiber.ctx[δ_k] $= d_1$) $\neq (d_2 = d_1)$ notify(entry.ctx, Δ, affected) ▷ in place of the realm test of Algorithm 3

The test turns on one property of delimiters. The tag under δ_k is written on the entry's context and inherited by every context derived from it, and it is drawn afresh at each reassignment, so for a context γ'

$$\gamma'[\delta_k] = d_1 \quad \Longleftrightarrow \quad \gamma' \text{is derived from the entry's context} \tag{65}$$

Write own (γ') for that condition, of which $d_2 = d_1$ is the instance at the provider. The reassignment moves the contexts satisfying own from s_1 to s_2 and leaves the others where they are, and by the loop above it moves the binding to s_2 exactly when the provider satisfies own. A dependent sees the binding while its own realm at k is the realm the binding sits in. Where own agrees on the dependent and the provider, both move or neither does, so the dependent sees the binding afterwards exactly when it saw it before. Where own separates them, one side moves and the other stays, so the dependent gains or loses the binding. The inequality is that separation, and the membership test drops the dependents resolving k in neither realm, which no part of the move reaches.

5.2.2. Hot Module Replacement

Hot module replacement (HMR) applies the revertible-effect pattern at the module level: when source files change, typically during development, the system replaces the afected modules in-place without restarting the process. Because a fiber already bounds all of its component's efects and coefects, a module that is itself a component can be replaced through fiber operations alone: disposing the old fiber recovers everything the component installed, and a new fiber instantiated from the reloaded module reinstalls it. HMR therefore needs no developerannotated acceptance boundaries, as opposed to Webpack [46] or Vite [47] HMR.

The @cordisjs/hmr component provides the HMR engine, which operates in three phases.

Phase 1: Module classification. The engine takes two inputs: the stashed set (file URLs whose contents have changed since the last reload) and the externals set (modules that cannot be hot-replaced and instead trigger a full

足够。条目可在运行时在组之间移动，因此加载器管理自己的领域，isolate 字段按键在两种作用域规则之间选择。值 true 请求一个本地领域，对条目私有并以它的 id 打标签，条目带着它无论移动到何处；字符串请求一个全局领域，由每个命名该字符串的条目共享，因此移动这样的条目改变的是它与哪些条目共享绑定，而非它属于哪个领域。一旦没有条目命名一个领域，它即被丢弃。

重分配条目的领域取决于哪些键改变了领域、条目自身是否在已改变键处是提供者、以及要通知哪些依赖方。中间那个问题是困难的，因为一个领域符号可能被多个纤维共享，其中只有一个是提供者。加载器用定界符回答它：每个键一个符号 ρ，每个上下文在其下存储自己的标签。定界符写在上下文上并被其后代继承，因此条目的标签与提供者的恰在两者在一个 isolate 作用域内为 ρ 派生时一致，这正是 ρ 处的绑定是条目自己的且必须随它移动的情形。

Algorithm 7 Isolation realm reassignment 1 function patch_isolation(entry, ρ') 2 $\rho \leftarrow$ entry ctx[@@isolate] 3 store $\leftarrow$ entry ctx[@@store] 4 $\Delta \leftarrow \{k \mid \rho(k) \neq \rho'(k)\}$▷ keys whose realm changes 5 for k in Δ do 6 entry ctx[δ_k] $\leftarrow$ fresh tag 7 diff[k] $\leftarrow$ ($\rho(k)$, $\rho'(k)$, entry ctx[δ_k], store[$\rho(k)$].fiber ctx[δ_k])
8 entry ctx[@@isolate] $\leftarrow \rho'$ 9 reload(entry.fiber) 10 for k in Δ do

$(s_1, s_2, d_1, d_2) \leftarrow$ diff[k] if $d_1 = d_2$ and store [s_1] and not store [s_2] then ▷ the binding is the entry's own store [s_2] $\leftarrow$ store[s_1] delete store [s_1] function affected(fiber, k) $(s_1, s_2, d_1, d_2) \leftarrow$ diff[k] return fiber.ctx[@@isolate][k] $\in \{s_1, s_2\}$ and (fiber.ctx[δ_k] $= d_1$) $\neq (d_2 = d_1)$ notify(entry.ctx, Δ, affected) ▷ in place of the realm test of Algorithm 3

该测试取决于定界符的一个性质。δ_k 下的标签写在条目的上下文上并被从它派生的每个上下文继承，且在每次重分配时全新抽取，因此对上下文 γ'

$$\gamma'[\delta_k] = d_1 \quad \Longleftrightarrow \quad \gamma' \text{is derived from the entry's context} \tag{65}$$

把 own (γ') 写作该条件，其中 $d_2 = d_1$ 是其在提供者处的实例。重分配把满足 own 的上下文从 s_1 移到 s_2 而把其他留在原处，且由上面的循环，恰在提供者满足 own 时把绑定移到 s_2 。一个依赖方在其自身于 k 处的领域是绑定所在的领域时看到绑定。在 own 对依赖方和提供者一致处，两者都移动或都不移动，因此依赖方在之后看到绑定恰与之前相同。在 own 把它们分开处，一侧移动而另一侧停留，因此依赖方获得或失去绑定。不等式就是那个分离，而成员资格测试丢弃在任一领域中都不解析 k 的依赖方，移动的任何部分都够不到它们。

5.2.2. 热模块替换（Hot Module Replacement）

热模块替换（HMR）在模块级应用可逆效应模式：当源文件改变时（通常在开发期间），系统原地替换受影响的模块而不重启进程。由于纤维已经约束其组件的所有效应与余效应，本身是组件的模块可以仅通过纤维操作被替换：处置旧纤维恢复组件安装的一切，而从重新加载的模块实例化的新纤维重新安装它。因此 HMR 无需开发者标注的接受边界，与 Webpack [46] 或 Vite [47] 的 HMR 相反。

@cordisjs/hmr 组件提供 HMR 引擎，它分三阶段运作。

阶段 1：模块分类。引擎接受两个输入：stashed 集（内容自上次重新加载以来已改变的文件 URL）与 externals 集（不能热替换而触发完整重启的模块）。把 get_imports(url) 写作 url 直接导入的模块，它对变化的依赖子图分

restart). Writing get_imports(url) for the modules that url directly imports, it classifies the changes' dependency subgraph, marking each module accepted or declined:

```
Algorithm 8 Module classification
1 function classify(stashed, externals)
2 accepted ← stashed
3 declined ← externals
4 pending ← ∅
5 for url in stashed do

6 | pending ← pending ∪ (get_imports(url) \ (accepted ∪ declined))
7 repeat
8 | progress ← false
9 for url in pending do
10    if get_imports(url) ∩ accepted ≠ ∅ then
11    accepted ← accepted ∪ {url}
12    pending ← pending \ {url}
13    progress ← true
14 else if get_imports(url) ⊆ declined then
15    declined ← declined ∪ {url}
16    pending ← pending \ {url}
17    progress ← true
18 else
19    pending ← pending ∪ (get_imports(url) \ (accepted ∪ declined))
20 until not progress
21 declined ← declined ∪ pending
22 return (accepted, declined)
```

Seeded with the imports of the stashed files, the fixed point accepts a module once one of its imports is accepted and declines one once all of its imports are declined; any module left undecided, caught in an import cycle, defaults to declined.

Phase 2: Stale-entry detection. Using accepted and declined, the engine then filters the component entries down to the stale ones, whose dependency tree reaches a changed module. It walks each entry's tree with get_dependencies, which collects the transitive imports of a module while respecting declined as a boundary:

```
Algorithm 9 Stale-entry detection

1 function get_dependencies(root, declined)
2    deps ← ∅
3    function traverse(url)
4    if url ∈ deps or url ∈ declined then return
5    deps ← deps ∪ {url}
6    for child in get_imports(url) do traverse(child)
```

类，把每个模块标记为 accepted 或 declined：

```
Algorithm 8 Module classification
1 function classify(stashed, externals)
2 accepted ← stashed
3 declined ← externals
4 pending ← ∅
5 for url in stashed do

6 | pending ← pending ∪ (get_imports(url) \ (accepted ∪ declined))
7 repeat
8 | progress ← false
9 for url in pending do
10    if get_imports(url) ∩ accepted ≠ ∅ then
11    accepted ← accepted ∪ {url}
12    pending ← pending \ {url}
13    progress ← true
14 else if get_imports(url) ⊆ declined then
15    declined ← declined ∪ {url}
16    pending ← pending \ {url}
17    progress ← true
18 else
19    pending ← pending ∪ (get_imports(url) \ (accepted ∪ declined))
20 until not progress
21 declined ← declined ∪ pending
22 return (accepted, declined)
```

以 stashed 文件的导入为种子，固定点在其一个导入被接受时接受一个模块，在其所有导入都被拒绝时拒绝它；任何未决的、被困在导入环中的模块，默认为 declined。

阶段 2：过期条目检测。使用 accepted 和 declined，引擎随后把组件条目过滤为过期的那些，即其依赖树达到一个已改变模块的条目。它用 get_dependencies 走每个条目的树，该函数收集一个模块的传递导入并尊重 declined 作为边界：

```
Algorithm 9 Stale-entry detection

1 function get_dependencies(root, declined)
2    deps ← ∅
3    function traverse(url)
4    if url ∈ deps or url ∈ declined then return
5    deps ← deps ∪ {url}
6    for child in get_imports(url) do traverse(child)
```

```
7    traverse(root)
8    return deps
9 function detect(entries, accepted, declined)
10    stale_entries ← ∅
11    for entry in entries do
12    tree ← get_dependencies(entry.url, declined)
13    if tree ∩ accepted ≠ ∅ then
14    accepted ← accepted ∪ tree
15    stale_entries ← stale_entries ∪ {entry}
16    return stale_entries
```

An entry is stale exactly when its tree intersects accepted; that tree is then folded into accepted, so every stale module along it is invalidated in the next phase.

Phase 3: Transactional reload. Finally, the engine reloads the stale entries. It invalidates the accepted modules' caches3, backing up each removed module to enable rollback, then reimports each stale entry's component module by its url and swaps in a fresh fiber:

```
Algorithm 10 Transactional module reload
1 function reload(ctx, accepted, stale_entries)
2 backup ← invalidate_caches(accepted)
3 try
4    for entry in stale_entries do
5    entry.fiber.dispose()
6    entry.fiber ← ctx.use(import(entry.url), entry.config)
7 catch error
8    restore_caches(backup)
9    for entry in stale_entries do
10    entry.fiber.dispose()
11    entry.fiber ← ctx.use(backup[entry.url], entry.config)
12 throw error
```

The transactional guarantee ensures that the system never enters a half-reloaded state: if any module fails to import (e.g., due to a syntax error), the caches are restored and every stale entry is rebuilt from backup[entry.url], the previous component whose cache was just restored, undoing the swaps already made.

5.3. Case Study: Koishi

Koishi is an open-source chatbot application framework built on Cordis4. Over four years of development, it has accumulated over 4000 community-contributed plugins5, ranging from instant-messaging (IM) adapters and database drivers to administrative consoles and enduser features. Its scale and diversity make it a representative validation of Cordis's dynamic composability in a production setting.

Expressiveness and generality of the meta-framework. Koishi runs as a server-side bot whose every feature is realized as a plugin over the context primitives of Section 5.1; Koishi itself contributes only the chatbot-domain vocabulary. The same model reappears in a wholly diferent runtime: Koishi's web console is a second, independent


```
7    traverse(root)
8    return deps
9 function detect(entries, accepted, declined)
10    stale_entries ← ∅
11    for entry in entries do
12    tree ← get_dependencies(entry.url, declined)
13    if tree ∩ accepted ≠ ∅ then
14    accepted ← accepted ∪ tree
15    stale_entries ← stale_entries ∪ {entry}
16    return stale_entries
```


一个条目恰在其树与 accepted 相交时过期；该树随后被折叠进 accepted，因此沿它的每个过期模块在下一阶段被失效。

阶段 3：事务性重新加载。最后，引擎重新加载过期的条目。它使 accepted 模块的缓存失效 3，备份每个被移除的模块以支持回滚，然后按 url 重新导入每个过期条目的组件模块并换入一个新纤维：


```
Algorithm 10 Transactional module reload
1 function reload(ctx, accepted, stale_entries)
2 backup ← invalidate_caches(accepted)
3 try
4    for entry in stale_entries do
5    entry.fiber.dispose()
6    entry.fiber ← ctx.use(import(entry.url), entry.config)
7 catch error
8    restore_caches(backup)
9    for entry in stale_entries do
10    entry.fiber.dispose()
11    entry.fiber ← ctx.use(backup[entry.url], entry.config)
12 throw error
```


事务性保证确保系统从不到达半重新加载状态：若任何模块导入失败（例如由于语法错误），缓存被恢复且每个过期条目从 backup[entry.url]（缓存刚被恢复的前一个组件）重建，撤销已做的交换。

5.3. 案例研究：Koishi（Case Study: Koishi）

Koishi 是一个构建在 Cordis 之上的开源聊天机器人应用框架 4。历经四年多的开发，它积累了超过 4000 个社区贡献的插件 5，从即时通讯（IM）适配器与数据库驱动到管理控制台与终端用户功能。它的规模与多样性使其成为 Cordis 动态可组合性在生产环境中的代表性验证。

元框架的表达力与通用性。Koishi 作为服务端机器人运行，其每个功能都实现为第 5.1 节上下文原语之上的一个插件；Koishi 自身只贡献聊天机器人领域的词汇。同样的模型出现在一个完全不同的运行时中：Koishi 的 Web 控制台是第二个独立的 Cordis 应用，其插件组合的是浏览器及其用户界面的原语，而非服务器的。上述迥异的设

Cordis application whose plugins compose the primitives of the browser and its user interface rather than those of the server. The disparate settings above establish two properties of the model of Section 3. (1) It is expressive: its primitives sufice to carry a complete production system, the host framework supplying only domain vocabulary. (2) It is general: it fixes how efects and coefects compose while leaving their meaning to each application, and so presupposes neither a particular domain nor a particular runtime.

Temporal composability without cognitive overhead. The plugin systems surveyed in Section 1.2.1 cannot unload an individual extension's efects without restarting the extension host. Koishi routinely performs this operation: an orchestrator disables a plugin from the console and its efects are withdrawn in place; during development, the HMR engine re-applies edited plugins on save while preserving cache state and live connections elsewhere in the system. Cordis makes such removal not merely possible but efortless for the plugin author. Because efects performed through the context are tracked and their inverses composed automatically (Section 3.1), even an inexperienced author obtains ordered cleanup for a plugin's context-mediated efects without writing an uninstall path. This achieves the locality of concern whose absence Section 1.2.1 identifies: correctness that would otherwise rest on each author's diligence is instead discharged once, by the abstraction.

Spatial composability across an open ecosystem. In contrast to the plugin systems of Section 1.2.1, where inter-plugin dependencies are largely absent, Koishi's ecosystem exhibits a genuine dependency topology: IM adapters provide access to each messaging platform, database drivers provide persistent storage, and functional plugins declare these as coefects and access them. Reconfiguring a provider at runtime, such as switching the storage backend or reconnecting an adapter, reactivates only the dependents whose resolved dependency changed (Section 3.2); a plugin whose dependency is unavailable stays inactive until it appears, without erroring. What the case study substantiates is that this composition holds across independently authored code: a plugin and its dependencies are typically written by diferent authors who coordinate on nothing beyond the coeffect that connects them, so reactive coefects keep the assembly consistent across an open ecosystem of independent contributors.

Threats to validity. The evidence here is drawn from a single ecosystem in a single host language, so it cannot separate the merits of the paradigm from those of its TypeScript realization or of Koishi's particular domain, and it is observational rather than a controlled comparison against an alternative architecture. What the case study establishes is thus an existence-andadoption result rather than a quantitative one; measuring the abstraction's overhead and its effect on developer productivity against a baseline remains future work.

6. Discussion

The formal model and implementation presented in the preceding sections introduce a programming paradigm for dynamic composability. This section examines how the paradigm extends to broader engineering concerns, and discusses the design tensions and open problems.

6.1. System Boundary

Every effect in Section 3.1 carries an inverse, and what that inverse amounts to is settled by the system boundary. The boundary divides the environment a system runs against into two parts. (1) A location lies inside when the system is able to modify it exclusively and to restore the state before that modification, so an operation on it is tracked in Γ and can be recovered later. (2) A location lies outside when either ability fails, so an operation on it acts as id_Γ and is therefore neither tracked nor recovered. This section develops the properties of this boundary and their consequences for recovery.

定确立了第 3 节模型的两个性质。(1) 它有表达力：其原语足以承载一个完整的生产系统，宿主框架只提供领域词汇。(2) 它通用：它固定效应与余效应如何组合，而把它们的意思留给每个应用，因此既不预设特定领域也不预设特定运行时。

无需认知开销的时间可组合性。第 1.2.1 节调查的插件系统不能在不重启扩展宿主的情况下卸载单个扩展的效应。Koishi 例行执行这个操作：编排者从控制台禁用一个插件，其效应被原地撤出；在开发期间，HMR 引擎在保存时重新应用已编辑的插件，同时保留缓存状态与系统中其他地方的活跃连接。Cordis 使这种移除不仅可能，而且对插件作者毫不费力。因为经上下文执行的效应被跟踪且其逆被自动组合（第 3.1 节），即使没有经验的作者也能为插件的上下文中介效应获得有序清理，而无需编写卸载路径。这实现了第 1.2.1 节指出的其缺失的关注点局部性：本会依赖每个作者勤勉的正确性改由抽象一次性处理。

跨开放生态的空间可组合性。与第 1.2.1 节插件系统（那里插件间依赖大体缺席）相反，Koishi 的生态呈现真实的依赖拓扑：IM 适配器提供对每个消息平台的访问，数据库驱动提供持久存储，功能插件把这些声明为余效应并访问它们。运行时重配置提供者，如切换存储后端或重连适配器，只重新激活其已解析依赖发生改变的那些依赖方（第 3.2 节）；依赖不可用的插件保持不活跃直到它出现，而不会出错。案例研究证实的正是这种组合在独立编写的代码之间成立：插件与其依赖通常由不同作者编写，他们在连接它们的余效应之外不协调任何东西，因此响应式余效应在独立贡献者的开放生态中保持装配一致。

有效性威胁。这里的证据取自单一宿主语言中的单一生态，因此不能把范式的优点与其 TypeScript 实现或 Koishi 特定领域的优点分开，而且是观察性的而非针对替代架构的受控比较。案例研究所确立的因此是存在性与采纳性结果而非量化结果；对照基线测量抽象的代价及其对开发者生产力的影响仍是未来工作。

6. 讨论（Discussion）

前面各节呈现的形式模型与实现为动态可组合性引入一种编程范式。本节考察该范式如何扩展到更广泛的工程关切，并讨论设计张力与开放问题。

6.1. 系统边界（System Boundary）

第 3.1 节中的每个效应都携带一个逆，而这个逆归结为什么由系统边界决定。边界把系统运行所针对的环境分成两部分。(1) 一个位置在边界内，当系统能够排他地修改它并恢复该修改前的状态，因此对它的一次操作被跟踪进 Γ 且可稍后恢复。(2) 一个位置在边界外，当任一能力失败，因此对它的一次操作充当 id_Γ 从而既不跟踪也不恢复。本节展开这个边界的性质及其对恢复的后果。

Boundaries from coefects. A coeffect moves the boundary by reifying an external location: it confines every access to that location to a set of operations it provides, each of which it can supply an inverse for, so operations that acted as id_Γ come to be tracked in Γ and recovered. The boundary is therefore drawn per location rather than per medium, since both aforementioned abilities are properties of a location, and reification changes how a location is accessed while leaving its medium as it was. For example, a memory region lies inside when the system alone writes it, and outside when other processes write it too; a file lies inside when only the system can reach it, as with a scratch file under a private path, and outside when it is a path other programs read or write. Moving the boundary is itself a trade-of, between whether the environment provides revertible semantics for a location and what supplying those semantics costs on every access. We take up the co-design this suggests in Section 6.7.

来自余效应的边界。一个余效应通过把一个外部位置具体化来移动边界：它把对该位置的每次访问约束到它提供的一组操作上，每个操作它都能提供一个逆，因此充当 id_Γ 的操作开始被跟踪进 Γ 并被恢复。边界因此按位置而非按媒介划定，因为前述两种能力都是位置的性质，而具体化改变的是位置如何被访问、让它的媒介保持原样。例如，当系统单独写一块内存区域时它在边界内，当其他进程也写它时在边界外；当只有系统能到达一个文件时它在边界内，如私有路径下的临时文件，当它是其他程序读写的一个路径时在边界外。移动边界本身是一种权衡，在环境是否为位置提供可逆语义与提供那些语义在每个访问上的代价之间。我们在第 6.7 节接续这暗示的协同设计。

Acquisition and emission. An operation that reaches outside the boundary generally proceeds in two stages. (1) In the acquisition stage, the operation obtains access and installs a record inside the boundary: open installs a descriptor that close removes, malloc reserves a block that free releases, fork starts a child process that kill terminates. The record itself is part of the coeffect that reifies the location, e.g. an entry in a map it keeps, and installing that entry is a revertible efect. That record is at the same time the channel along which data can leave. (2) In the emission stage, the operation pushes data through that channel, as with the bytes a write hands to the file or the datagram a send puts on the wire, and the push acts as id_T , leaving the data where other parties may read and write it. The two stages therefore fall on opposite sides of the boundary: the acquisition stays inside it, whereas the emission crosses to the outside.

获取与排放。触达边界之外的操作通常分两阶段进行。(1) 在获取阶段，操作获得访问并在边界内安装一条记录：open 安装一个 close 移除的描述符，malloc 保留一个 free 释放的块，fork 启动一个 kill 终止的子进程。记录本身是具体化该位置的余效应的一部分，如它所保留的一个映射中的条目，而安装该条目是一个可逆效应。该记录同时是数据可沿其离去的通道。(2) 在排放阶段，操作通过该通道推送数据，如 write 交给文件的字节或 send 放到线上的数据报，而推送充当 id_Γ ，把数据留在其他方可能读写的状态。两阶段因此落在边界相对的两侧：获取留在边界内，而排放跨越到外部。

Withholding and compensation. A system that must nonetheless recover from an emission has two approaches available. One is to withhold an emission until the state that produced it is certain to persist, which is the output commit problem of rollback-recovery [48]. The other is compensation [49]: an action that restores the state up to an equivalence the application supplies, coarser than the $\simeq$ of Definition 33, as in deleting a file that was created or refunding a charge that was made. Such actions compose in the same LIFO order as inverses do, so the composition of Section 3.1 transfers to them. The metatheory does not: the commutation of Definition 60 is proved against $\simeq$ and has to be re-established against the coarser one.

扣留与补偿。一个仍须从排放中恢复的系统有两种可用方法。一种是把排放扣留到产生它的状态确定持久为止，即回滚恢复的输出提交问题 [48]。另一种是补偿 [49]：一个把状态恢复到应用提供的等价（比定义 33 的 $\simeq$ 更粗）的动作，如删除被创建的文件或退还已收的费。这样的动作以与逆相同的 LIFO 顺序组合，因此第 3.1 节的组合迁移到它们。元理论不迁移：定义 60 的交换是对 $\simeq$ 证明的，必须对照更粗的等价重新确立。

6.2. Service Multiplexing

6.2. 服务多路复用（Service Multiplexing）

Dynamic component platforms such as OSGi [50] organize composition around services: units of functionality that a provider publishes under an interface and a consumer binds to. The Cordis coeffect model echoes this notion, with a service corresponding to the interface behind a key. Components that provide a service are its providers, and components that inject a service are its consumers. A single service may be implemented by multiple providers, and this multiplicity can be realized in two forms. (1) Exclusive binding: several implementations share one interface but at most one is bound at a time; the orchestrator selects which implementation is bound, and switching between them requires unloading one provider and loading another, momentarily perturbing every consumer's dependency. (2) Service broker: a central service that acts as the entrypoint for the interface is injected by both the backing providers and the consumers, so that multiple providers coexist and the broker dispatches each request among them. Compared to exclusive binding, the broker absorbs this perturbation: updating a backing provider leaves the broker in place, so consumers see no change to their dependency and no reload is triggered.

动态组件平台如 OSGi [50] 围绕服务组织组合：提供者在接口下发布、消费者绑定到它的功能单元。Cordis 余效应模型呼应这个观念，服务对应键背后的接口。提供服务的是其提供者，注入服务的是其消费者。单个服务可由多个提供者实现，而这种多重性可以两种形式实现。(1) 排他绑定：多个实现共享一个接口但一次至多一个被绑定；编排者选择哪个实现被绑定，在它们之间切换需要卸载一个提供者并加载另一个，瞬间扰动每个消费者的依赖。(2) 服务代理：一个充当接口入口的中心服务被背后的提供者和消费者都注入，使多个提供者共存且代理在它们之间分派每个请求。与排他绑定相比，代理吸收这种扰动：更新背后的提供者让代理留在原位，因此消费者对其依赖看不到变化且不触发重新加载。

The service broker underlies three capabilities: load balancing, rolling updates, and crossprocess invocation.

服务代理支撑三种能力：负载均衡、滚动更新与跨进程调用。

Load balancing. When several providers coexist, the broker distributes requests among them according to a configurable policy (e.g., round-robin, least-loaded, latency-weighted) or an explicit target named by the consumer. Because providers are ordinary components, they can be added or removed to scale capacity up or down; each

负载均衡。当多个提供者共存时，代理按可配置策略（如轮询、最少负载、时延加权）或消费者命名的显式目标在它们之间分派请求。因为提供者是普通组件，它们可被添加或移除以扩大或缩小容量；每个提供者通过一个可逆效应向代理注册，因此卸载它会撤销注册并自动把它从代理的路由集丢弃。

provider registers with the broker through a revertible effect, so unloading it reverts the registration and drops it from the broker's routing set automatically.

Rolling updates. Upgrading a service implementation at runtime reduces to a controlled provider transition [51, 52]. To carry out the transition, the new provider is loaded as an additional fiber and registers with the broker; once it becomes ACTIVE, trafic is gradually shifted from the old providers to the new one (e.g., by adjusting selection weights), and the old providers are unloaded once they no longer carry in-flight requests. This provider transition turns what is traditionally an infrastructure-level operation (e.g., container orchestration, bluegreen deployment) into an application-level composition pattern.

Cross-process invocation. The service broker can also be applied across process boundaries [53]. Each process hosts its own Cordis context with local providers; a coordinating component links them, treating each as a remote provider. Cross-process service access is mediated by an RPC mechanism that preserves the interface, making the distribution transparent to consumers. One caveat is that a cross-process call incurs latency and may fail mid-flight, so exposing it synchronously would block the caller. An interface intended to be exposed across processes must therefore be designed against an asynchronous contract.

6.3. Access Control and Sandboxing

Given an application assembled from independent components, securing the application calls for two complementary mechanisms: (1) constraining what dependencies a component may access, and (2) sandboxing untrusted code from the host environment. Cordis supports the first through dependency declarations and interception; the second requires an external sandbox.

Capability-based access control. The dependency access mechanism (Section 5.1.4) already constitutes a form of access control over proxy-mediated properties: a component can only access dependencies it has declared; an undeclared access raises an error. This is structurally similar to capability-based security [54–56], where authority is conferred by possession of a reference rather than by ambient authority. The inject declaration acts as a capability request, and the context proxy acts as a capability mediator. Since these requests are declared statically, the complete set of proxy-mediated capabilities a component requires is known before it runs, letting the orchestrator review and approve them at load time rather than discovering accesses as they happen.

This mediation generalizes to fine-grained policy through the interception mechanism. Access-control metadata can be carried by contexts or declared by components (Definition 30), and the provider consults it when the dependency is invoked to decide whether a request is permitted. For example, a filesystem dependency may carry metadata declaring which paths a component may read or write, and the provider checks each call against the metadata. Because this interception lives on the context rather than in either party's code, an orchestrator can adjust it to constrain any component's access to a dependency without modifying the provider, e.g., granting read-only database access to a community component whereas a core component retains full access. Moreover, since interception afects only how a dependency is invoked, not whether it is satisfied, it can be installed, reconfigured, or removed at runtime without triggering any reload or perturbing the dependency graph.

Sandboxing untrusted components. When a component's code cannot be trusted, language-level access control is insuficient, since a malicious component with access to the host runtime can reach the underlying objects directly, rendering such checks moot. Sandboxing requires an execution boundary beyond the reach of language-level means, such as software fault isolation [57], a separate language runtime, a sandboxed process, or a virtualized container [58]. Whatever the mechanism, the untrusted component runs in its own sandboxed context and reaches host-provided dependencies through a bridge, generalizing the crossprocess invocation of Section 6.2:

滚动更新。运行时升级一个服务实现归结为受控的提供者转移 [51, 52]。为执行该转移，新提供者作为一个附加纤维被加载并向代理注册；一旦它变为 ACTIVE，流量逐渐从旧提供者移向新提供者（例如通过调整选择权重），而旧提供者一旦不再承载飞行中的请求即被卸载。这种提供者转移把传统上是基础设施级操作（如容器编排、蓝绿部署）的东西变成应用级组合模式。

跨进程调用。服务代理也可应用于跨进程边界 [53]。每个进程托管自己的带本地提供者的 Cordis 上下文；一个协调组件链接它们，把每个作为远程提供者对待。跨进程服务访问由保留接口的 RPC 机制中介，使分布对消费者透明。一个告诫是跨进程调用招致时延且可能飞行中失败，因此同步暴露它会阻塞调用方。打算跨进程暴露的接口因此必须对照异步契约设计。

6.3. 访问控制与沙箱（Access Control and Sandboxing）

给定一个由独立组件组装的应用，保护该应用需要两种互补机制：(1) 约束组件可访问哪些依赖，以及 (2) 把不可信代码与宿主环境沙箱化。Cordis 通过依赖声明与拦截支持前者；后者需要外部沙箱。

基于能力访问控制。依赖访问机制（第 5.1.4 节）已经构成对 proxy 中介属性的一种访问控制形式：组件只能访问它声明的依赖；未声明的访问引发错误。这在结构上类似于基于能力的安全 [54–56]，那里权威由持有引用而非环境权威授予。inject 声明充当能力请求，上下文 proxy 充当能力中介。由于这些请求是静态声明的，组件所需的全部 proxy 中介能力在它运行前已知，让编排者能在加载时审查与批准它们，而非在访问发生时发现。

这种中介通过拦截机制推广到细粒度策略。访问控制元数据可由上下文携带或由组件声明（定义 30），提供者在依赖被调用时查询它以决定请求是否被允许。例如，文件系统依赖可携带声明组件可读写哪些路径的元数据，提供者对照元数据检查每次调用。由于这种拦截活在上下文上而非任一方代码中，编排者可调整它以约束任何组件对依赖的访问而无需修改提供者，e.g., 授予社区组件只读数据库访问而核心组件保留完全访问。再者，由于拦截只影响依赖如何被调用、不影响它是否被满足，它可以在运行时被安装、重配置或移除而不触发任何重新加载或扰动依赖图。

沙箱化不可信组件。当组件的代码不可信时，语言级访问控制不够，因为能访问宿主运行时的恶意组件可以直接触达底层对象，使这样的检查形同虚设。沙箱需要超出语言级手段可达范围的执行边界，如软件故障隔离 [57]、独立语言运行时、沙箱进程或虚拟化容器 [58]。无论何种机制，不可信组件在自己的沙箱化上下文中运行并通过桥接触达宿主提供的依赖，推广第 6.2 节的跨进程调用：同样的透明论证使这种桥接访问与本地注入不可区分。在宿主侧，桥接是一个普通纤维，其能力可被上述访问控制衰减。

the same transparency argument renders this bridged access indistinguishable from local injection. On the host side, the bridge is an ordinary fiber whose capabilities can be attenuated by the access control described above.

6.4. Language Independence and Selection

Although Cordis is implemented in TypeScript, the context paradigm is language-agnostic: spatiotemporal composability is defined only by its two composability dimensions, and thus can be realized in any language that meets certain requirements along both. We analyze these requirements along each dimension in turn.

Temporal composability. At its most basic, temporal composability requires closures: a revertible effect pairs an action with an inverse, and that inverse must be captured as a value, along with the state it restores, so it can be replayed on teardown. Beyond this, a component's code and the side efects of loading it must be introducible and retractable at runtime.

How a language meets this second requirement depends on its execution model. In managed runtimes, this takes the form of a programmatic module registry, where a loaded module can be evicted from the registry and garbage-collected once unreferenced; Node.js, for instance, exposes such a registry.6 Native code exposes no module registry, so introduction and retraction take the form of explicit dynamic linking and unlinking (e.g., dlopen/dlclose on Unix, LoadLibrary/FreeLibrary on Windows) [59], i.e., loading object code into a running process and later detaching it. WebAssembly takes one path or the other depending on its embedder: a module instance is reclaimed by the host's collector under a managed embedder (e.g., a JavaScript host), or released when a native embedder drops it (e.g., Wasmtime). Across these mechanisms, the revertible efects model treats loading as an effect on the context, with inverses that undo the registration of symbols, types, or handlers the module introduced.

Spatial composability. Spatial composability requires a mechanism for components to declare their dependencies and for the runtime to provide and inject these dependencies. This reduces to a dependency injection (DI) problem [38], which manifests at two levels that difer across languages: how dependencies are typed and how their access is mediated.

At the type level, the language should provide a way for developers to express well-typed dependency access. A consumer obtains a coeffect by reading its key from the context, so the context type (Section 3.2.1) must record each key's coefect. Typeclasses (Haskell) [60] and traits (Rust) [61] achieve this by letting a provider extend the context type from its own module through an instance or impl [62]. TypeScript's module augmentation [63] likewise lets a provider module merge declarations into the context type.

At the runtime level, dependency access must be dynamically mediated: the coeffect behind a key may change as providers are loaded and unloaded, and may be resolved diferently across contexts. The language therefore needs a way to interpose on access transparently, leaving the consumer's code unchanged, e.g., via JavaScript's Proxy object [64] or Python's descriptor protocol (`qet`) [65]. Absent such a primitive, runtime reflection [66, 67] can mediate access dynamically, at the cost of type safety and developer experience.

Across both levels, metaprogramming facilities supply the typing and the mediation together. Annotations [68] and decorators attach metadata to a declaration, which a processor expands into the accessor that mediates access; compile-time metaprogramming (e.g., Rust procedural macros, Scala macros [69], Zig comptime) emits, for each dependency, a typed declaration together with such an accessor, dispensing with a general-purpose interception primitive.

6.5. Mutual Dependencies and Component Granularity

6.4. 语言无关性与选择（Language Independence and Selection）

虽然 Cordis 用 TypeScript 实现，上下文范式是语言无关的：时空可组合性只由其两个组合维度定义，因此可在沿两者满足某些要求的任何语言中实现。我们沿每个维度依次分析这些要求。

时间可组合性。在最基本层面，时间可组合性需要闭包：可逆效应把一个动作与一个逆配对，那个逆必须作为值被捕获，连同它恢复的状态，以便能在拆卸时重放。除此之外，组件的代码与加载它的副作用必须在运行时可引入且可撤除。

语言如何满足这第二个要求取决于其执行模型。在受管运行时中，这采取程序化模块注册表的形式，其中已加载的模块可在无引用后从注册表驱逐并被垃圾回收；例如 Node.js 暴露这样的注册表。6 原生代码不暴露模块注册表，因此引入与撤除采取显式动态链接与解链的形式（如 Unix 上的 dlopen/dlclose，Windows 上的 LoadLibrary/FreeLibrary）[59]，即把目标代码加载进运行进程并稍后分离它。WebAssembly 视其嵌入者走这两条路之一：模块实例在受管嵌入者（如 JavaScript 宿主）下由宿主的回收器回收，或在原生嵌入者丢弃它时释放（如 Wasmtime）。跨这些机制，可逆效应模型把加载当作上下文上的一个效应，逆撤销模块引入的符号、类型或处理器的注册。

空间可组合性。空间可组合性需要一种让组件声明依赖、运行时提供并注入这些依赖的机制。这归结为一个依赖注入（DI）问题 [38]，它出现在跨语言不同的两个层面：依赖如何类型化，以及它们的访问如何被中介。

在类型层面，语言应提供一种让开发者表达良类型依赖访问的方式。消费者通过从其上下文读取键获得余效应，因此上下文类型（第 3.2.1 节）必须记录每个键的余效应。类型类（Haskell）[60] 与 trait（Rust）[61] 通过让提供者从其自身模块经由 instance 或 impl 扩展上下文类型实现这一点 [62]。TypeScript 的模块扩充 [63] 同样让提供者模块把声明合并进上下文类型。

在运行时层面，依赖访问必须被动态中介：键背后的余效应可能随提供者被加载与卸载而改变，且可能跨上下文不同地解析。语言因此需要一种透明地介入访问、让消费者代码不变的方式，如经 JavaScript 的 Proxy 对象 [64] 或 Python 的描述符协议 (`qet`) [65]。缺少这样的原语时，运行时反射 [66, 67] 可动态中介访问，代价是类型安全与开发者体验。

跨这两个层面，元编程设施一起提供类型化与中介。注解 [68] 与装饰器把元数据附加到声明上，处理器把它展开成中介访问的访问器；编译期元编程（如 Rust 过程宏、Scala 宏 [69]、Zig comptime）为每个依赖发射一个类型化声明连同这样的访问器，省去通用拦截原语。

6.5. 相互依赖与组件粒度（Mutual Dependencies and Component Granularity）

In the reactive coeffect model, a dependency cycle simply leaves the involved components permanently inactive: given two components A and B, if A requires a key provided by B and B a key provided by A, neither's satisfaction predicate can ever become true. Unlike deadlock in concurrent systems, which depends on the schedule and must be detected as it happens, this condition is predictable from the dependency declarations alone, so a runtime can report it when components are loaded.

In practice, most apparently mutual dependencies can be decomposed into finer-grained components that eliminate the cycle. Consider two components: a server (providing a network interface) and an access controller (enforcing authorization policies). The two components interact bidirectionally: the access controller mediates requests arriving at the server, and the server exposes an endpoint for modifying access-control policies. A monolithic design would make each component depend on the other. However, the two interaction directions are logi cally independent concerns. Decomposing them yields four components: server-core, accesscontrol-core, request-mediation (depending on both cores to apply access control to incoming requests), and policy-management (depending on both cores to expose policy modification via the server). Through this approach, the cycle is eliminated because neither core depends on the other; only the integration components depend on both.

This decomposition is always possible in principle, since every bidirectional interaction can be factored into independent unidirectional bindings, but it increases the number of components: in the general case, given n mutually interacting components, the number of integration components can grow quadratically with n, since each pair of interacting components may require a distinct component for each direction of interaction. This does not afect correctness or runtime performance (components are lightweight), and finer granularity can be beneficial: users gain the ability to load only the specific integration bindings they need, efectively increasing the system's composability. However, it may afect developer experience: more components require more configuration, more naming, and more cognitive overhead in understanding the dependency graph.

Mitigating this granularity cost is an engineering concern rather than a theoretical one. Practical strategies include package bundling (i.e., grouping related fine-grained components into a single installable unit), convention-based wiring (i.e., automatically connecting components whose names or types match a pattern), and scafold tooling (i.e., generating boilerplate integration components from declarative specifications). These strategies preserve the formal guarantees of the acyclic model while reducing the authoring burden to something closer to the monolithic case.

6.6. Dependency Typing and Versioning

In the formal model, a dependency link is established purely by key identity: a component providing key k satisfies any component declaring k in its dependency set. The type family ν_k ensures type-level agreement within a single compilation unit, but this guarantee breaks down when components are developed and built independently, which is a common scenario in component ecosystems. This breakage leads to two distinct problems.

Interface drift. A provider may modify the interface associated with k (adding fields, changing method signatures, altering behavioral contracts) between versions, while a consumer compiled against an earlier interface continues to declare the same key k. The dependency is satisfied at the coeffect level ($k \in \mathrm{dom}(\sigma)$) , yet the runtime value no longer conforms to the consumer's expectations, leading to type errors, method-not-found failures, or silent behavioral divergence [70].

Key collision. Two independently developed providers may use the same key name k to denote entirely unrelated interfaces. Since key identity alone establishes the link, a consumer expecting one provider's interface will accept the other's value without any compatibility check. Unlike interface drift, where the provider and

在响应式余效应模型中，依赖环只是让涉及的组件永久不活跃：给定两个组件 A 和 B，若 A 需要 B 提供的键而 B 需要 A 提供的键，两者的满足谓词都不能变为真。与并发系统中的死锁（取决于调度且必须在发生时检测）不同，这个条件仅从依赖声明就可预测，因此运行时可在组件加载时报告它。

实践中，大多数看似相互的依赖可分解成消除环的更细粒度组件。考虑两个组件：服务器（提供网络接口）与访问控制器（强制授权策略）。两者双向交互：访问控制器中介到达服务器的请求，而服务器暴露修改访问控制策略的端点。整体式设计会让每个组件依赖另一个。然而，两个交互方向是逻辑上独立的关切。分解它们产生四个组件：server-core、access-control-core、request-mediation（依赖两个核心以对入站请求应用访问控制）与 policy-management（依赖两个核心以经由服务器暴露策略修改）。通过这种方法，环被消除，因为没有一个核心依赖另一个；只有集成组件依赖两者。

这种分解原则上总是可能的，因为每个双向交互可分解为独立的单向绑定，但它增加组件数量：一般情形下，给定 n 个相互交互的组件，集成组件的数量可随 n 二次增长，因为每对交互组件可能每个交互方向都需要一个独立组件。这不影响正确性或运行时性能（组件是轻量的），且更细粒度可能有益：用户获得只加载他们需要的特定集成绑定的能力，有效提高系统的可组合性。然而它可能影响开发者体验：更多组件需要更多配置、更多命名，以及理解依赖图时更多认知开销。

缓解这种粒度代价是工程关切而非理论关切。实用策略包括包打包（把相关细粒度组件分组进单个可安装单元）、基于约定的接线（自动连接名字或类型匹配模式的组件）与脚手架工具（从声明式规范生成样板集成组件）。这些策略在保持无环模型的正式保证的同时把编写负担降到更接近整体式情形。

6.6. 依赖类型化与版本化（Dependency Typing and Versioning）

在形式模型中，依赖链接纯粹由键同一性建立：提供键 k 的组件满足在其依赖集中声明 k 的任何组件。类型族 ν_k 在单个编译单元内确保类型级一致，但当组件被独立开发与构建时这个保证瓦解，这是组件生态中的常见场景。这种瓦解导致两个不同的问题。

接口漂移。提供者可能在版本之间修改与 k 关联的接口（添加字段、改变方法签名、改变行为契约），而对照早期接口编译的消费者继续声明同一个键 k。依赖在余效应级被满足 ($k \in \mathrm{dom}(\sigma)$) ，然而运行时值不再符合消费者的期望，导致类型错误、找不到方法失败或静默行为分歧 [70]。

键冲突。两个独立开发的提供者可能用同一个键名 k 表示完全无关的接口。由于仅键同一性建立链接，期望一个提供者接口的消费者会在没有任何兼容性检查的情况下接受另一个的值。与接口漂移（提供者与消费者至少共享共同血统）不同，键冲突在期望与实际类型之间不涉及任何关系，使由此产生的失败不可预测且难以诊断。

consumer at least share a common lineage, key collision involves no relationship whatsoever between the expected and actual types, making the resulting failures unpredictable and dificult to diagnose.

Both problems point to the same gap: the coeffect model provides only nominal linking (by key name) but no versioned or structural linking (by interface compatibility) [71]. We discuss three approaches to the gap, from most infrastructure-coupled to most language-agnostic.

Key namespacing. Extending the key space from K to $K \times P$, where P identifies the interface-defining package, eliminates key collision by construction: independently developed interfaces with the same local name occupy distinct keys. This is the most direct solution but also the most coupled: it embeds the package namespace into the formal model itself, making the system dependent on an external package registry for key identity.

Peer dependencies. A lighter coupling is to declare version constraints through the hostlanguage package manager [72]. This is the approach Cordis currently adopts. Component dependencies are semantically peer dependencies: a component does not bundle its dependencies internally but expects the runtime context to supply them. Package managers with peer dependency support (e.g., npm) can enforce version compatibility: if the version of the package providing a key falls outside a consumer's declared peer range, the incompatibility is caught at install time rather than surfacing as a runtime failure. However, this approach has two limitations: (1) it depends on providers faithfully adhering to semantic versioning, which is an unenforceable convention; (2) package managers typically resolve each dependency to a single version, which prevents loading components from multiple versions of the same package within one application.

Structural compatibility. A fully language-agnostic approach would replace the membership check $k \in \mathrm{dom}(\sigma)$ with a compatibility predicate that verifies the provider's actual interface structurally subsumes the consumer's expectation. This is analogous to structural subtyping [73]: a provider satisfies a consumer if the provided interface is a subtype of the required interface. The challenge lies in defining this predicate language-agnostically: structural compatibility is straightforward for record types (width subtyping) but becomes complex for behavioral contracts (e.g., pre/postconditions [74], effect specifications [22]), and undecidable once parametric polymorphism introduces bounded quantification [75].

These three approaches address diferent aspects of the problem. Designing a unified dependency model that combines these approaches while preserving the dynamic composition guarantees of the coeffect model remains an open problem.

6.7. Co-Design with Languages and Operating Systems

Section 6.4 identifies the minimum a host language must supply for the context paradigm. This section takes up the converse question, what a language or operating system co-designed with the paradigm can ofer beyond that minimum.

Co-design with languages. A language designed around the context paradigm can improve on a library in two respects: the semantics it gives to contexts, and the primitives it gives to efects and coefects.

Such a language can make the context implicit again while preserving the context semantics of Section 3.3. An imperative language already runs every statement against an implicit context, and that single context neither tracks efects nor resolves coefects. The context paradigm instead distinguishes multiple contexts, where an operation either modifies the context it runs against or derives another from it (Definition 27). An in-place realization modifies the ambient context, just as an imperative language does. A derived realization instead introduces a separate context, for which the language must provide a construct. Making the context implicit brings both an ergonomic and a safety benefit. (1) In a library realization, every function involving efects or

两个问题都指向同一个缺口：余效应模型只提供名义链接（按键名）而无版本化或结构化链接（按接口兼容性）[71]。我们讨论该缺口的三种方法，从最基础设施耦合到最语言无关。

键命名空间化。把键空间从 K 扩展到 $K \times P$, ，其中 P 标识定义接口的包，从构造上消除键冲突：具有相同本地名的独立开发接口占据不同键。这是最直接的解法也是最耦合的：它把包命名空间嵌入形式模型本身，使系统依赖外部包注册表来键同一性。

对等依赖。更轻的耦合是通过宿主语言包管理器声明版本约束 [72]。这是 Cordis 目前采纳的方法。组件依赖在语义上是对等依赖：组件不在内部捆绑其依赖，而期望运行时上下文提供它们。带对等依赖支持的包管理器（如 npm）可强制版本兼容：若提供键的包的版本落在消费者的声明对等范围之外，不兼容在安装时被抓到，而非作为运行时失败浮现。然而，这种方法有两个局限：(1) 它依赖提供者忠实遵循语义版本化，这是不可强制执行的约定；(2) 包管理器通常把每个依赖解析到单个版本，这阻止在一个应用内加载同一包的多个版本的组件。

结构兼容性。一种完全语言无关的方法会用兼容性谓词替换成员资格检查 $k \in \mathrm{dom}(\sigma)$ ，该谓词验证提供者的实际接口在结构上涵盖消费者的期望。这类似于结构子类型 [73]：若提供的接口是被要求的接口的子类型，提供者满足消费者。挑战在于语言无关地定义该谓词：结构兼容性对记录类型（宽度子类型）是直接的，但对行为契约（如前置/后置条件 [74]、效应规范 [22]）变得复杂，一旦参数多态引入有界量化就不可判定 [75]。

这三种方法处理问题的不同方面。设计一个统一依赖模型，结合这些方法同时保持余效应模型的动态组合保证，仍是开放问题。

6.7. 与语言和操作系统的协同设计（Co-Design with Languages and Operating Systems）

第 6.4 节识别出宿主语言必须为上下文范式提供的最低要求。本节接续相反的问题，一个与该范式协同设计的语言或操作系统在该最低要求之外能提供什么。

与语言协同设计。一个围绕上下文范式设计的语言可以在两方面优于库：它赋予上下文的语义，以及它赋予效应与余效应的原语。

这样的语言可以让上下文再次隐式，同时保留第 3.3 节的上下文语义。命令式语言已让每个语句对着隐式上下文运行，而那个单一上下文既不跟踪效应也不解析余效应。上下文范式相反区分多个上下文，那里一个操作要么修改它运行所针对的上下文，要么从它派生另一个（定义 27）。原地实现修改环境上下文，正如命令式语言所做。派生实现转而引入一个独立上下文，语言必须为它提供一个构造。让上下文隐式带来人体工程学与安全两方面的好处。(1) 在库实现中，涉及效应或余效应的每个函数把上下文作为普通参数或接收者，如第 5.1 节。语言隐式提供上下文处，函数不再需要取它。(2) 每个上下文携带自己的生命周期状态与已提交视图（第 4.1 节）。库实现把上下文作为普通变量传递，因此组件可能经闭包或全局变量错误地触达另一组件的上下文。它在其中安装的效应

coefects takes the context as an ordinary argument or a receiver, as in Section 5.1. Where the language supplies the context implicitly, functions no longer need to take it. (2) Every context carries its own lifecycle state and committed view (Section 4.1). A library realization passes a context as an ordinary variable, so a component may reach another component's context by mistake, through a closure or a global variable. An effect it installs there then leaks out of its own lifecycle, and a coeffect it reads there escapes its dependency specification. Making the context implicit closes both.

Such a language can also make efects and coefects known to its compiler. (1) For efects, an effect iterator (Definition 51) allocates a closure at every step to hold the inverse together with the state it restores. With syntax for performing an effect, a compiler can emit a single state machine for the whole iteration and hold those inverses in its frame. (2) For coefects, the coeffect specification can be admitted into the type system, with two benefits. First, a dependency cycle is reported at compile time instead of being left to the runtime (Section 6.5). Second, a dependency can be compared by the structure of its type rather than by key identity alone, as row types do [28], which is type-level support for the structural compatibility of Section 6.6.

Co-design with operating systems. Section 1.2.3 observes a coarse-grained substitute for dynamic composability, where the operating system supplies temporal composability at the granularity of a process, and the container orchestrator above it supplies spatial composability at the granularity of a service. An operating system co-designed with the paradigm would support fine-grained composition, by making the coeffect specification a component declares the whole of what it can reach, and by providing its own resources as coefects.

Such an operating system can supply the sandbox that Section 6.3 defers to a mechanism outside the language. It does so by bounding a component to the dependencies it declares, supplying them when the component is loaded and leaving nothing else reachable from within it, as a WebAssembly module receives its imports from its embedder at instantiation [76]. It can also provide the coeffect isolation and interception of Section 3.2.3 as abilities of its own, binding a key diferently for each component and mediating the accesses it supplies.

Such an operating system can also provide its own resources as coefects. A resource lying outside the boundary is made revertible where the runtime records each acquisition against the component that made it (Section 6.1), and every runtime keeps a record of its own. An operating system that provides the resource as a coeffect keeps that record once, since it is the party that hands the resource out and can attribute it to the component that asked. Memory and file descriptors are the immediate candidates, and tracking them for the sake of recovery has been done at the kernel interface [77, 78]. Furthermore, an operating system can make revertible some of the operations Section 6.1 can only withhold or compensate for. A system that performs a write to persistent storage transactionally can roll it back [79], and one built on copy-on-write or immutable storage reaches an earlier state by moving a pointer [80, 81].

7. Related Work

Dynamic composability intersects several established research areas. We survey the most relevant lines of work and distinguish our contribution from each of them.

7.1. Effect and Coeffect Systems

Section 2 reviewed efects and coefects as the theoretical pillars underlying our work. We first situate the monadic effect systems now common in industrial practice, then survey three research lines that extend efects and coefects in directions relevant to Cordis: recasting algebraic efects as capabilities, giving efects a reversible semantics, and

随后泄漏出其生命周期，而它在其中读取的余效应逃出其依赖规范。让上下文隐式封上两者。

这样的语言还可以让效应与余效应为其编译器所知。(1) 对效应，一个效应迭代器（定义 51）在每一步分配一个闭包以把逆连同它恢复的状态放在一起。有了执行效应的语法，编译器可以为整个迭代发射单个状态机并把那些逆保存在其帧中。(2) 对余效应，余效应规范可被接纳进类型系统，有两个好处。第一，依赖环在编译期报告而非留给运行时（第 6.5 节）。第二，依赖可按其类型的结构而非仅键同一性比较，如行类型所做 [28]，这是第 6.6 节结构兼容性的类型级支持。

与操作系统协同设计。第 1.2.3 节观察到动态可组合性的一种粗粒度替代，那里操作系统以进程粒度提供时间可组合性，其上的容器编排器以服务粒度提供空间可组合性。与该范式协同设计的操作系统会支持细粒度组合，把组件声明的余效应规范做成它能触达的一切，并把自己的资源作为余效应提供。

这样的操作系统可以提供第 6.3 节推迟到语言外机制的那个沙箱。它通过把组件约束到它声明的依赖、在组件加载时提供它们并让它内部其余一切都不可达来实现，正如 WebAssembly 模块在实例化时从其嵌入者接收导入 [76]。它还可以把第 3.2.3 节的余效应隔离与拦截作为自己的能力提供，为每个组件不同地绑定一个键并中介它提供的访问。

这样的操作系统还可以把自己的资源作为余效应提供。边界外的资源被做成可逆的，其中运行时对照提出它的组件记录每次获取（第 6.1 节），而每个运行时保留自己的记录。把资源作为余效应提供的操作系统保留该记录一次，因为它是把资源分发出去并能把它归因于提出它的组件的一方。内存与文件描述符是直接候选，而为了恢复跟踪它们已在内核接口处完成 [77, 78]。再者，操作系统可以让第 6.1 节只能扣留或补偿的一些操作可逆。对持久存储事务性地执行写入的系统可以回滚它 [79]，而建立在写时复制或不可变存储上的系统通过移动指针到达较早状态 [80, 81]。

7. 相关工作（Related Work）

动态可组合性与几个成熟研究领域相交。我们调查最相关的几条工作线并把我们的贡献与每条区分开。

7.1. 效应与余效应系统（Effect and Coefect Systems）

第 2 节把效应与余效应作为我们工作的理论支柱做了回顾。我们先定位工业实践中常见的单子效应系统，然后调查在 Cordis 相关方向上扩展效应与余效应的三条研究线：把代数效应重铸为能力，给效应可逆语义，以及把效应与余效应统一在单一分级纪律之下。

unifying efects and coefects under a single graded discipline.

Monadic effect systems. One family of libraries encodes efects in the type systems of existing general-purpose languages, representing them as monadic values that a runtime executes. ZIO in Scala [82] models a computation as ZIO[R,E,A] and Effect-TS in TypeScript [83] as Effect<A,E,R>, a generic type whose parameters describe its result, its typed errors, and the services its context must supply; the fp-ts library [84] encodes the same error and requirement channels through Reader-based monad transformers. Two traits separate these systems from Cordis. First, the tracking is bought with a monadic embedding: a program obtains it only by being written inside the effect type, whereas Cordis tracks efects as an overlay over ordinary host code. Second, a requirement is discharged by interpretation, an installed service that supplies its operations, and when that service is withdrawn what its operations performed remains in place; Cordis instead pairs each effect with an inverse and re-resolves requirements as providers come and go (Section 3.1, Section 3.2).

Algebraic efects as capabilities. Algebraic efects (Section 2.1) make effect operations visible to the type system. The extension closest to our work is Brachthäuser et al.'s Efekt language, which reinterprets effect types as capabilities [85, 86]: an effect type expresses what a computation requires from its context rather than what side efects it may produce. This perspective, like ours, treats the context as a mediator of capabilities. Cordis and Efekt difer in two respects. (1) In purpose, algebraic efects make efects visible to enable modular interpretation, giving one operation many handler semantics, whereas Cordis makes them visible to enable tracking and reversion, pairing every context transformation with an inverse. (2) In setting, Efekt disciplines efects statically at the type level, defaulting to scope-based reasoning in which capabilities are second-class and confined to their lexical scope, and recovering first-class use through boxing, which lifts that restriction by tracking captured capabilities in types; Cordis instead disciplines efects at runtime, aiming at complete resource recovery on component removal; Section 6.7 takes up what a language that made the context second class in this sense would ofer.

Reversible effect semantics. A parallel line gives efects a reversible semantics rather than an interpretive one. Heunen et al. [87] model side efects in a reversible setting by adapting Hughes' arrows to dagger arrows and inverse arrows, capturing efects such as serialization and mutable store whose operations admit inverses. This is the formal account closest to our revertible efects: both pair each effect with the means to undo it rather than discharging it through a handler. The two difer in where reversibility resides, and in how much of it they demand. Heunen et al. work in a denotational, categorical setting where reversibility is a global property, guaranteed by construction since every computation is invertible, and the inverse is two-sided and recovered from the categorical structure. Cordis tracks inverses at runtime and requires less of them: not that the whole computation be reversible, but that each atomic effect admit a one-sided inverse, supplied by the caller at the point of application rather than derived, from which the inverse of any composite follows by composition (Section 3.1).

Graded types as unified efects and coefects. Orchard et al. [88] proposed graded modal types as an umbrella notion encompassing both effect reasoning (via graded monads) and coeffect reasoning (via graded comonads), realized in the Granule language, demonstrating that a single type system can track both what a computation does and what it needs; more recent work extends coefects to imperative Java-like languages [89, 90] and to call-by-push-value [91]. All of these operate at the type level: efects and coefects are static annotations checked at compile time over lexically fixed scopes. Our contribution is orthogonal to this analysis: we lift the same two notions to runtime mechanisms, which lets Cordis handle dynamic composition. Temporal retraction and spatial dependency are re-resolved as the set of loaded components evolves, instead of being settled once over a fixed program text.

单子效应系统。一族库把效应编码进现有通用语言的类型系统，把它们表示为运行时执行的单子值。Scala 中的 ZIO [82] 把计算建模为 ZIO[R,E,A]，TypeScript 中的 Effect-TS [83] 建模为 Effect<A,E,R>，一个其参数描述其结果、其类型化错误及其上下文必须提供的服务的泛型类型；fp-ts 库 [84] 通过基于 Reader 的单子变换器编码同样的错误与要求通道。两个特征把这些系统与 Cordis 分开。第一，跟踪是用单子嵌入买来的：程序只有写在效应类型内部才能获得它，而 Cordis 把效应作为普通宿主代码之上的一个叠加层跟踪。第二，要求由解释处理，一个提供其操作的已安装服务，而当该服务被撤出时其操作所执行的仍留在原位；Cordis 相反把每个效应与一个逆配对，并随提供者来去重新解析要求（第 3.1 节、第 3.2 节）。

作为能力的代数效应。代数效应（第 2.1 节）让效应操作对类型系统可见。离我们工作最近的扩展是 Brachthäuser 等人的 Efekt 语言，它把效应类型重新解释为能力 [85, 86]：一个效应类型表达计算从其上下文要求什么，而非它可能产生什么副作用。这个视角与我们一样把上下文当作能力的中介。Cordis 与 Efekt 有两个方面不同。(1) 在目的上，代数效应让效应可见以启用模块化解释，给一个操作许多处理程序语义，而 Cordis 让它们可见以启用跟踪与逆转，把每个上下文变换与一个逆配对。(2) 在设定上，Efekt 在类型级静态约束效应，默认基于作用域推理，其中能力是二等的并被约束在其词法作用域内，通过装箱恢复一等使用，装箱通过在类型中跟踪被捕获的能力解除该限制；Cordis 相反在运行时约束效应，旨在组件移除时完全恢复资源；第 6.7 节接续一个在此意义上让上下文成为二等的语言会提供什么。

可逆效应语义。一条平行线给效应可逆语义而非解释语义。Heunen 等人 [87] 通过把 Hughes 箭头改编为 dagger 箭头与逆箭头在可逆设定中建模副作用，捕获串行化与可变存储等操作（其操作允许逆）。这是离我们的可逆效应最近的形式化说明：两者都把每个效应与撤销它的手段配对，而非经由处理程序处理它。两者不同在于可逆性驻留何处，以及它们要求多少可逆性。Heunen 等人工作在指称的范畴设定中，那里可逆性是全局性质，因每个计算可逆而由构造保证，逆是双侧的并从范畴结构恢复。Cordis 在运行时跟踪逆并要求更少：不要求整个计算可逆，而要求每个原子效应承认单侧逆，由调用方在应用点提供而非推导，任何复合的逆随之由组合得出（第 3.1 节）。

作为统一效应与余效应的分级类型。Orchard 等人 [88] 提出分级模态类型作为涵盖效应推理（经分级单子）与余效应推理（经分级余单子）的伞形概念，在 Granule 语言中实现，证明单一类型系统可同时跟踪计算做什么与需要什么；更近的工作把余效应扩展到命令式类 Java 语言 [89, 90] 与按值调用 (call-by-push-value) [91]。所有这些都在类型级运作：效应与余效应是在词法固定作用域上于编译期检查的静态注解。我们的贡献与该分析正交：我们把同样的两个概念提升为运行时机制，这让 Cordis 能处理动态组合。时间撤除与空间依赖随已加载组件集的演化被重新解析，而非在固定程序文本上一次性敲定。

7.2. Programming Paradigms

Section 3.3.3 established the context paradigm as a discipline that mediates efects and coefects through an explicit context. Two established paradigms warrant explicit comparison: one shares our terminology, the other our treatment of crosscutting concerns.

Context-oriented programming. COP [92, 93] equips a language with layers—partial method and class definitions that are activated and deactivated at runtime according to the execution context, so that behavior adapts without the base code naming its context dependencies [94]. COP and Cordis coincide in treating context as a first-class, runtime-mutable entity and in activating and deactivating behavior dynamically, but the resemblance is nominal. In COP, "context" denotes the ambient execution situation (e.g., location, user, mode), and activation changes method dispatch within a dynamically scoped extent; a layer neither tracks the side efects it induces nor reverts them, and activation is not governed by dependency satisfaction. In Cordis, the context is the Γ_∞ entity mediating efects and coefects: activation runs a component's revertible efects and is driven by reactive coeffect satisfaction (Section 3.2), and deactivation reverts them in full. COP varies what behavior runs; Cordis composes and reverts what efects and dependencies a component installs. Their diference is one of trade-of. COP folds activation into the host language's method dispatch, gaining dynamically-scoped layer extents at the cost of language specificity, whereas Cordis, as a language-agnostic overlay, resolves activation reactively over a shared context. Cordis can thus express as a coeffect only

COP's global, value-driven fragment: context-dependent selection among implementations, but not dynamically-scoped activation.

Aspect-oriented programming. AOP [95, 96] modularizes a crosscutting concern into an aspect: a pointcut that quantifies over join points selected in the base program, and advice woven in at each. Cordis addresses the same problem of contextual behavior that would otherwise scatter across components, but its analogue of an aspect is a coeffect: a shared point of mediation many components declare a dependence on, so that crosscutting behavior can be reshaped there without editing any of them. The two paradigms then difer on two axes. (1) Declaration versus obliviousness: an AOP pointcut is oblivious and quantified, matching arbitrary join points whose code is unaware it is advised, whereas Cordis confines crosscutting to the coefects each component declares, so its reach is exactly that declared surface. This yields determinacy and traceability: an application orchestrator can inspect and govern what cross-cuts a component at the configuration layer, without reading or analyzing its source, whereas an AOP concern is legible only through the aspects that quantify over it. (2) Lifecycle integration: a crosscutting change in Cordis is carried by a component's efects, reverted when the component unloads and propagated reactively to its dependents, so it is one move within the dynamic composition model; dynamic-AOP systems [97, 98] can also weave and unweave at runtime, but as a standalone operation, neither bound to a component's lifecycle nor triggering re-resolution among the advised code.

7.3. Temporal Composability

Temporal composability concerns replacing or removing a component in a running program while recovering the efects it installed. Prior approaches divide by how they treat a departing component's state and efects: carrying state forward to a successor version, recovering efects through developer-authored cleanup, reversing efects automatically within a scope fixed in advance, or reclaiming resources from a record the runtime accumulates by interposing on an interface.

Stateful forward migration. A broad family of systems replaces components in a running program without downtime by carrying their state forward across versions. All observe the same timing discipline: a component

7.2. 编程范式（Programming Paradigms）

第 3.3.3 节把上下文范式确立为一种通过显式上下文中介效应与余效应的纪律。两个既定范式值得显式比较：一个共享我们的术语，另一个共享我们对横切关切的处理。

面向上下文编程。COP [92, 93] 为语言装备层——部分方法与类定义，它们根据执行上下文在运行时被激活与停用，使行为无需基础代码命名其上下文依赖就能适应 [94]。COP 与 Cordis 在把上下文当作一等的、运行时可变实体并动态激活与停用行为上一致，但相似是名义上的。在 COP 中，"上下文" 表示环境执行情形（如位置、用户、模式），激活在动态作用域范围内改变方法分派；层既不跟踪它诱发的副作用也不逆转它们，且激活不受依赖满足约束。在 Cordis 中，上下文是中介效应与余效应的 Γ_∞ 实体：激活运行组件的可逆效应并由响应式余效应满足驱动（第 3.2 节），停用完全逆转它们。COP 变化运行什么行为；Cordis 组合并逆转组件安装什么效应与依赖。它们的差别是权衡的差别。COP 把激活折叠进宿主语言的方法分派，以语言特定性为代价获得动态作用域的层范围，而 Cordis 作为语言无关叠加层在共享上下文上响应式地解析激活。Cordis 因此只能把 COP 的全局、值驱动片段表达为余效应

——即实现之间的上下文相关选择，而非动态作用域激活。

面向方面编程。AOP [95, 96] 把横切关切模块化为一个方面：一个量化基础程序中选定连接点的切入点，以及织入每处的建议。Cordis 处理同一类会散落各组件间的上下文行为问题，但其方面对应物是余效应：许多组件声明依赖的共享中介点，因此横切行为可在那里重塑而无需编辑任何组件。两个范式随后在两个轴上不同。(1) 声明对无感：AOP 切入点是量化且无感的，匹配其代码不知其被建议的任意连接点，而 Cordis 把横切约束到每个组件声明的余效应，因此其触达恰好是那个声明表面。这带来确定性与可追踪性：应用编排者可在配置层检查并治理什么横切一个组件，而无需读取或分析其源码，而 AOP 关切只有通过量化它的方面才可读。(2) 生命周期集成：Cordis 中的横切变化由组件的效应承载，在组件卸载时被逆转并响应式传播给其依赖方，因此它是动态组合模型中的一步；动态 AOP 系统 [97, 98] 也可以在运行时织入与解织，但作为独立操作，既不绑定组件的生命周期也不在被建议代码间触发重新解析。

7.3. 时间可组合性（Temporal Composability）

时间可组合性关切在运行程序中替换或移除一个组件同时恢复它安装的效应。先前的方法按它们如何处理离开组件的状态与效应划分：把状态前向迁移给后继版本，通过开发者编写的清理恢复效应，在预先固定的作用域内自动逆转效应，或从运行时经介入接口累积的记录回收资源。

有状态前向迁移。一族广泛的系统无停机地替换运行程序中的组件，把它们的状态前向迁移过版本。全部遵守同一时序纪律：组件只在其到达一个安全、无交互的点之后才能被交换。Kramer 与 Magee 把该标准确立为

may be swapped only once it reaches a safe, interaction free point. Kramer and Magee established this criterion as quiescence [51], which Vandewoude et al. later relaxed to the less disruptive tranquility [52]; our rolling-update pattern (Section 6.2) enforces it by draining in-flight requests before unloading a provider. Dynamic software updating (DSU) then migrates state forward through hand-written transformation functions: Hicks et al.'s general-purpose DSU for C [99], Stoyle et al.'s type-safe update points via con-freeness analysis [100], and Hayden et al.'s Kitsune [101] all map old-version data to newversion representations, inheriting heap objects, open files, and connections in place while re-initializing whatever is left unmigrated. The same discipline extends to persistent state: Overeem et al. [102] convert a running event store's data between schema versions through hand-written upgrade operations while keeping the system available. Erlang/OTP [15] takes the same stance at the process level, migrating state through code_change/3 and recovering from faults by restarting supervised processes rather than reverting their efects; JavaScript's Hot Module Replacement (e.g., webpack [46], Vite [47]) does the same at the module level, handing state forward through the module.hot or import.meta.hot API across a reload. Compared with Cordis's module replacement (Section 5.2), these approaches migrate in-memory state more gracefully: Cordis reverts the old component's tracked efects and reapplies the new component's from a clean slate, so a component's own in-memory state does not survive a reload unless placed in a longer-lived dependency, and layering DSU-style forward migration atop revertible efects is future work. Cordis's approach is nonetheless more general in two respects: it needs no hand-written migration functions of the kind DSU and HMR require, and it supports unloading a component entirely and recovering its resources, not merely updating one in place.

Developer-authored recovery. A second family recovers a component's efects through cleanup or compensation logic that the developer writes by hand. Plugin lifecycle conventions (e.g., OSGi [50], Eclipse's extension points, IntelliJ and VSCode) delegate cleanup to developerwritten unload callbacks; the Command pattern [103] encapsulates an operation together with an undo method for undo/redo stacks; the saga model [49] structures a long-lived transaction as steps each paired with a compensating action; algebraic effect handlers can attach finalizers that run on teardown [104]; and event sourcing [105] retracts state by appending compensating events rather than executing an inverse at all. In all of them the inverse is an unenforced duty, decoupled from the operation, so that a forgotten one leaks resources silently (as documented empirically in Section 1.2.1). React's useEffect hook [106] comes closest to pairing an effect with its inverse structurally, returning a cleanup the runtime invokes before each re-execution and on unmount. Its shortfall is composability: a hook may be called only at the top level of a component or another hook, never inside a conditional, loop, or nested function, and its effect body accepts neither an async function nor an iterator. Effects thus cannot be assembled from other efects or interleaved with control flow, leaving nothing from which a composite inverse could be derived. Cordis efects carry no such restriction: they are ordinary operations that compose freely and may run asynchronously, and require a hand-written inverse only for each atomic effect, from which the inverse of any composite is derived by composition, so that assembling existing efects requires writing no inverses at all. This structural pairing of every effect with its inverse makes complete recovery an invariant of the system rather than a matter of developer discipline.

Statically scoped reversal. A third family reverses efects automatically, by construction, but confines reversal to a scope fixed in advance. Software transactional memory [107, 108], descended from hardware transactional memory [109], records a read/write log so that a group of memory operations either commits or aborts, rolling memory back to its pre-transaction state. Reversible computing, from Landauer and Bennett's thermodynamic analyses [110, 111] to reversible languages such as Janus [112], goes further and makes every step of a whole computation globally invertible. Reversible process calculi build backtracking into the semantics itself: RCCS [113] carries a memory alongside each process and admits a step to be taken back when the past it leads to is causally

quiescence（静止）[51]，Vandewoude 等人后来把它放宽为扰动更小的 tranquility（安宁）[52]；我们的滚动更新模式（第 6.2 节）通过在卸载提供者之前排空飞行中的请求强制它。动态软件更新（DSU）随后经手写变换函数前向迁移状态：Hicks 等人的通用 C DSU [99]、Stoyle 等人经共自由性分析的类型安全更新点 [100] 与 Hayden 等人的 Kitsune [101] 都把旧版数据映射到新版表示，原地继承堆对象、打开的文件与连接，同时重新初始化任何未迁移的东西。同样的纪律扩展到持久状态：Overeem 等人 [102] 通过手写升级操作在保持系统可用的同时转换运行中事件存储的 schema 版本之间的数据。Erlang/OTP [15] 在进程级采取同样立场，经 code_change/3 迁移状态并经重启受监督进程而非逆转其效应从故障恢复；JavaScript 的热模块替换（如 webpack [46]、Vite [47]）在模块级做同样的事，跨一次重新加载经 module.hot 或 import.meta.hot API 把状态交给后继。与 Cordis 的模块替换（第 5.2 节）相比，这些方法更优雅地迁移内存态：Cordis 逆转旧组件的被跟踪效应并从一个干净起点重新应用新组件的，因此组件自身的内存态在重新加载后不存活，除非放入更长命的依赖，而在可逆效应之上叠加 DSU 式前向迁移是未来工作。Cordis 的方法仍有两方面更通用：它不需要 DSU 与 HMR 所需的那种手写迁移函数，且它支持完全卸载一个组件并恢复其资源，而不仅仅原地更新一个。

开发者编写的恢复。第二族通过开发者手写的清理或补偿逻辑恢复组件的效应。插件生命周期约定（如 OSGi [50]、Eclipse 的扩展点、IntelliJ 与 VSCode）把清理委托给开发者编写的卸载回调；Command 模式 [103] 把操作连同用于撤销/重做栈的 undo 方法封装；saga 模型 [49] 把长生命周期事务结构化为每步配一个补偿动作的步骤；代数效应处理程序可以附加在拆卸时运行的终结器 [104]；事件溯源 [105] 通过追加补偿事件而非执行逆来撤销状态。在所有这些中逆是不受强制执行的义务，与操作解耦，因此被遗忘的一个静默泄漏资源（如第 1.2.1 节经验地记录）。React 的 useEffect hook [106] 最接近把效应与其逆结构上配对，返回运行时在每次重新执行前与卸载时调用的清理。其短板是可组合性：hook 只能在组件或另一个 hook 的顶层调用，绝不能在条件、循环或嵌套函数内，其效应体既不接受异步函数也不接受迭代器。效应因此不能从其他效应组装或与控制流交错，没有可据以推导复合逆的东西。Cordis 效应没有这样的限制：它们是自由组合的普通操作、可异步运行，只要求每个原子效应一个手写逆，任何复合的逆由组合推导，因此组装既有效应根本不需要写逆。这种每个效应与其逆的结构配对使完全恢复成为系统的不变量而非开发者纪律之事。

静态作用域的逆转。第三族自动逆转效应，由构造如此，但把逆转约束到预先固定的作用域。软件事务内存 [107, 108] 源自硬件事务内存 [109]，记录读写日志使一组内存操作要么提交要么中止，把内存回滚到事务前状态。可逆计算，从 Landauer 与 Bennett 的热力学分析 [110, 111] 到 Janus 等可逆语言 [112]，走得更远使整个计算的每一步全局可逆。可逆进程演算把回溯构建进语义本身：RCCS [113] 为每个进程携带一个记忆，并允许在其通向的过去因果等价时取回一步，Phillips 与 Ulidowski [114] 统一为 CCS、ACP 与 CSP 推导可逆算子同时保留它们的前向操作语义。它们的因果一致性标准是 Cordis 恢复所遵循顺序的并发对应，一个以后进先出顺序应用组件自身逆的累加器，以及第 4.3.1 节把提供者的撤出推迟到其消费者停用的守卫（定理 63）。然而触达由语义固定，所执

equivalent, and Phillips and Ulidowski [114] derive reversible operators for CCS, ACP, and CSP uniformly while preserving their forward operational semantics. Their causal-consistency criterion is the concurrent counterpart of the order Cordis's recovery follows, an accumulator applying a component's own inverses in lastin-first-out order and the guard of Section 4.3.1 deferring a provider's withdrawal until its consumers have deactivated (Theorem 63). The reach, however, is fixed by the semantics, every action performed remaining undoable, whereas a Cordis component supplies an inverse for each atomic effect and its accumulator brings the context back to where its composition began. Linear types [115], RAII [4], and Rust's ownership system [61] tie a resource's release to a lexical region. Each fixes the scope and reach of reversal statically; Cordis, by contrast, fixes no such scope in advance: it reverts arbitrary context operations over a component's lifecycle, and treats lexical resource management as complementary, appropriate for local resources within a single component.

Interposed reclamation. A fourth family reclaims what a component acquired without the component itself supplying the inverses, by recording its acquisitions at an interface the runtime controls. Nooks [77] wraps every call crossing the boundary between the Linux kernel and its loadable extensions, so that the kernel objects an extension touches pass through an object tracker whose record tells the recovery manager what to release when the extension fails; shadow drivers [78] tap the same calls from the other side, recording the requests and configuration that determine a driver's state so that a restarted instance can be restored to it. Akeso [116] obtains the record by compiler instrumentation instead, dividing kernel execution into nestable recovery domains that log their state changes and cross-thread dependencies, and rolling a faulting request back together with every domain that depends on it. Reclamation thus follows from a record the runtime maintains rather than from cleanup the developer remembers to write, which makes this family the closest systems-level precedent for revertible efects. It difers from Cordis in vocabulary and in reach. The platform fixes what can be recorded, whether as release code per kernel object type, one shadow per driver class, or an inverse per instrumented allocator, so a component may hold only resources the platform already knows how to release; a Cordis component instead introduces efects of its own and supplies an inverse for each atomic one (Section 3.1). Reclamation is likewise bounded by a request that commits or a restart of the same extension, whereas Cordis reverts over a component's whole lifetime and propagates removal to its dependents, which release their own efects in turn (Section 3.2).

7.4. Spatial Composability

Spatial composability concerns how a component's dependencies on others are declared and bound. Prior mechanisms divide by how binding responds to change: wiring dependencies once at initialization, reacting to the availability of whole components, or propagating change at the granularity of individual values.

Initialization-time dependency wiring. Two established mechanisms wire components together at initialization time. Dependency injection frameworks [38] (e.g., Spring [117], Guice, Angular, Inversify) inject dependencies into components at initialization, and UI framework context (e.g., Vue.js's provide/inject and React's Context API) passes them along a component tree. Some support dynamic scoping (e.g., Spring's prototype/request scopes, Angular's hierarchical injectors), but neither re-resolves reactively: when a provider is replaced or removed at runtime, existing dependents are neither deactivated nor re-initialized, and none ofers lifecycle management of the kind our component state machine provides. Cordis's reactive coefects (Section 3.2) supply this: the notification mechanism triggers lifecycle transitions whenever the satisfaction predicate changes.

Availability-reactive component models. The closest precedent to our reactive coefects reacts to service availability. OSGi's Declarative Services and iPOJO [118, 119] let components declare provided and required services, with the runtime automatically activating and deactivating them as services appear and disappear;

行的每个动作保持可撤销，而 Cordis 组件为每个原子效应提供一个逆、其累加器把上下文带回其组合开始之处。线性类型 [115]、RAII [4] 与 Rust 的所有权系统 [61] 把资源的释放绑到词法区域。每个都在静态上固定逆转的作用域与触达；Cordis 相反不预先固定这样的作用域：它逆转组件生命周期上的任意上下文操作，并把词法资源管理当作互补，适合单组件内的本地资源。

介入式回收。第四族回收组件不经组件自身提供逆而获得的东西，通过在其可被运行系统控制的接口处记录其获取。Nooks [77] 包装跨 Linux 内核与其可加载扩展之间边界的每次调用，使扩展触碰的内核对象经对象跟踪器通过，其记录告诉恢复管理器在扩展失败时释放什么；shadow drivers [78] 从另一侧接入同样的调用，记录决定驱动状态的请求与配置，使重启的实例可恢复。Akeso [116] 经编译器插桩获得记录，把内核执行分成记录其状态变化与跨线程依赖的可嵌套恢复域，并把故障请求连同依赖它的每个域一起回滚。回收因此遵循运行时维护的记录而非开发者记得编写的清理，这使该族成为可逆效应的最接近的系统级先例。它与 Cordis 的差别在词汇与触达。平台固定什么可被记录，无论按内核对象类型的释放代码、按驱动类的一个 shadow 还是一个被插桩分配器的一个逆，因此组件只能持有平台已知如何释放的资源；Cordis 组件相反引入自己的效应并为每个原子效应提供一个逆（第 3.1 节）。回收同样被一次提交的请求或同一扩展的重启所界定，而 Cordis 在组件整个生命周期上逆转并把它移除传播给其依赖方，后者依次释放自己的效应（第 3.2 节）。

7.4. 空间可组合性（Spatial Composability）

空间可组合性关切组件的依赖如何被声明与绑定。先前机制按绑定如何响应变化划分：在初始化时接线依赖一次，响应整个组件的可用性，或以单个值的粒度传播变化。

初始化时依赖接线。两种既定机制在初始化时把组件接在一起。依赖注入框架 [38]（如 Spring [117]、Guice、Angular、Inversify）在初始化时把依赖注入组件，UI 框架上下文（如 Vue.js 的 provide/inject 与 React 的 Context API）沿组件树传递它们。一些支持动态作用域（如 Spring 的 prototype/request 作用域、Angular 的分层注入器），但两者都不响应式地重新解析：当提供者在运行时被替换或移除时，既有依赖方既不停用也不重新初始化，且没有一个提供我们组件状态机提供的那类生命周期管理。Cordis 的响应式余效应（第 3.2 节）供给这一点：通知机制在满足谓词改变时触发生命周期转移。

可用性响应式组件模型。离我们的响应式余效应最近的先例对服务可用性做出响应。OSGi 的 Declarative Services 与 iPOJO [118, 119] 让组件声明提供与要求的服务，运行时随服务出现与消失自动激活与停用它们；iPOJO 的 Gravity 项目 [119] 显式针对随服务可用性变化的自主运行时适应，其 provide/require 模型直接预示 Cordis 的

iPOJO's Gravity project [119] explicitly targets autonomous runtime adaptation to changing service availability, and its provide/require model directly prefigures Cordis's ctx.provide/ctx.get pattern. R-OSGi [53] extends the same abstraction transparently to distributed settings via RPC, mapping network failures to servicewithdrawal events, a pattern Section 6.2 discusses as an extension of the Cordis model. All these systems recover through a deactivation callback, which is limited in two ways. First, the callback is hand-written, so resource safety rests on developer discipline and a forgotten one leaks silently. Second, the callback is synchronous: should teardown require an asynchronous exchange with the departing dependency, the frameworks ofer no protocol to await it, forcing a blocking wait against a reference that may already be stale. Cordis's reactive coefects close both gaps: deactivation reverts the dependents' accumulated efects, and its inertial Unloading state (Section 4.3.3) runs asynchronous teardown to completion before acting on further change.

Value-level reactivity. Functional reactive programming (FRP) [120] and its modern incarnations (e.g., signals [121, 122] in SolidJS, Vue's reactivity system, Angular Signals) propagate change at a value-level granularity: when a signal changes, derived computations are re-evaluated synchronously or under a scheduler [123]. Cordis's reactive coefects act at a componentlevel granularity, adding asynchronous lifecycle semantics that value-level propagation does not model. The same granularity diference runs the other way for consistency: propagating in a turn, in an order the dependency graph fixes, lets FRP require that no derived computation read a mixture of updated and stale inputs, which is glitch freedom [124], whereas Cordis has no counterpart of a turn, orchestration actions arriving one at a time, and guarantees only that no single transition straddles two resolutions of its coefects (Theorem 64). The two are complementary rather than competing: a Cordis coeffect can itself carry reactive values, and a component updates on only the parts it actually consumes, refining component-level reactivity into finer-grained reactive coefects that span both levels.

ctx.provide/ctx.get 模式。R-OSGi [53] 经 RPC 把同一抽象透明扩展到分布式设定，把网络故障映射为服务撤出事件，第 6.2 节把这一模式讨论为 Cordis 模型的扩展。所有这些系统经停用回调恢复，该回调有两个局限。第一，回调是手写的，因此资源安全取决于开发者纪律，被遗忘的一个静默泄漏。第二，回调是同步的：若拆卸需要与离开的依赖异步交换，框架不提供等待它的协议，迫使对可能已过期的引用的阻塞等待。Cordis 的响应式余效应封上两个缺口：停用逆转依赖方累积的效应，其惯性 Unloading 状态（第 4.3.3 节）在响应进一步变化前把异步拆卸运行到完成。

值级响应性。函数式响应式编程（FRP）[120] 及其现代化身（如 SolidJS 中的 signals [121, 122]、Vue 的响应式系统、Angular Signals）以值级粒度传播变化：当 signal 改变时，派生计算被同步或在调度器下重新求值 [123]。Cordis 的响应式余效应以组件级粒度运作，添加值级传播不建模的异步生命周期语义。同样的粒度差异在一致性上反向运行：在回合中、以依赖图固定的顺序传播，让 FRP 能要求没有派生计算读取更新与陈旧输入的混合，即无毛刺 (glitch freedom) [124]，而 Cordis 没有回合的对应物，编排动作一次一个到达，且只保证没有单个转移跨越其余效应的两次解析（定理 64）。两者互补而非竞争：Cordis 余效应本身可携带响应式值，组件只在其实际消费的部分上更新，把组件级响应性精化为跨两个层面的更细粒度响应式余效应。

8. Conclusion

We have presented a formal foundation for dynamic composability by lifting the classical concepts of efects and coefects to runtime mechanisms. Revertible efects address local temporal composability: every context transformation carries an inverse that the runtime tracks, and both tracking and recovery preserve composition, so the context is recovered upon component removal. Reactive coefects address local spatial composability: a component is notified against its coeffect specification whenever the context changes, each change classified as activating, deactivating, or neutral, with coeffect isolation varying what a declared key resolves to and coeffect interception varying how the binding is used. We unify the effect context and the coeffect context into a single context type, in which an observational equivalence on the coefects supplies the efects with independence, constituting a programming paradigm for spatiotemporal composability. Combining these mechanisms into the notion of a component then gives a calculus of dynamic composition, whose metatheory carries spatiotemporal composability from a single component to a whole system of interleaved components. We realize this paradigm as the Cordis meta-framework, with a core library providing effect tracking and coeffect resolution, as well as a declarative component loader with configuration reconciliation and hot module replacement. The Koishi case study validates the design of Cordis in a production system with over 4000 community plugins.

Beyond human-curated plugin ecosystems, a compelling direction for future validation is self-evolving agent harnesses (Section 1.2.2), where an AI agent generates and replaces its own harness components continuously and with little human oversight. Applying Cordis in such a setting would validate the temporal guarantees of complete recovery under rapid component replacement, as well as the spatial guarantees of dependency coordination under frequent topological change. Such validation would demonstrate the paradigm's applicability as a foundation for

8. 结论（Conclusion）

我们通过把经典效应与余效应概念提升为运行时机制，为动态可组合性呈现了一个形式基础。可逆效应处理局部时间可组合性：每个上下文变换携带一个运行时跟踪的逆，而跟踪与恢复都保持组合，因此上下文在组件移除时被恢复。响应式余效应处理局部空间可组合性：每当上下文改变时，组件对照其余效应规范被通知，每次变化被分类为激活、停用或中性，余效应隔离变化声明的键解析到什么，余效应拦截变化绑定如何使用。我们把效应上下文与余效应上下文统一进单一上下文类型，其中余效应上的一个观测等价为效应供给独立性，构成时空可组合性的编程范式。把这些机制组合进组件的概念随后给出一个动态组合演算，其元理论把时空可组合性从单个组件带到整个交错组件系统。我们把这个范式实现为 Cordis 元框架，核心库提供效应跟踪与余效应解析，以及带配置协调与热模块替换的声明式组件加载器。Koishi 案例研究在拥有超过 4000 个社区插件的生产系统中验证了 Cordis 的设计。

超越人工策划的插件生态，一个令人信服的未来验证方向是自我演化的 agent harness（第 1.2.2 节），那里 AI agent 持续且几乎无需人工监督地生成并替换自己的 harness 组件。在这种设定中应用 Cordis 会验证快速组件替换下完全恢复的时间保证，以及频繁拓扑变化下依赖协调的空间保证。这样的验证会证明该范式作为 agent harness 与其他自主系统中可恢复、协调、连续自我演化的基础的适用性。

recoverable, coordinated, and continuous self-evolution in agent harnesses and other autonomous systems.

References

[1] D. L. Parnas, “On the criteria to be used in decomposing systems into modules,” Communications of the ACM, vol. 15, no. 12, pp. 1053–1058, 1972, doi: 10.1145/361598.361623.

[2] D. Birsan, “On Plug-ins and Extensible Architectures,” ACM Queue, vol. 3, no. 2, pp. 40– 46, 2005, doi: 10.1145/1053331.1053345.

[3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016, doi: 10.1145/2890784.

[4] B. Stroustrup, The Design and Evolution of C++. Addison-Wesley, 1994.

[5] S. Marlow, S. Peyton Jones, A. Moran, and J. Reppy, “Asynchronous Exceptions in Haskell,” in Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation, in PLDI ’01. New York, NY, USA: Association for Computing Machinery, 2001, pp. 274–285. doi: 10.1145/378795.378858.

[6] L. Cardelli, “Program Fragments, Linking, and Modularization,” in Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 1997), ACM Press, 1997, pp. 266–277. doi: 10.1145/263699.263735.

[7] C. Szyperski, Component Software: Beyond Object-Oriented Programming, 2nd ed. Addison-Wesley, 2002.

[8] R. Lopopolo, “Harness Engineering: Leveraging Codex in an Agent-First World.” [Online]. Available: https://openai.com/index/harness-engineering/

[9] Anthropic, “Harness Design for Long-Running Application Development.” [Online]. Available: https://www.anthropic.com/engineering/harness-design-longrunning-apps

[10] L. Wang et al., “A Survey on Large Language Model Based Autonomous Agents,” Frontiers of Computer Science, vol. 18, no. 6, p. 186345, 2024, doi: 10.1007/s11704-024-40231-1.

[11] Y. Qin et al., “Tool Learning with Foundation Models,” ACM Computing Surveys, 2025, doi: 10.1145/3704435.

[12] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” CoRR, vol. abs/2310.08560, 2023.

[13] T. Guo et al., “Large Language Model Based Multi-Agents: A Survey of Progress and Challenges,” in Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, in IJCAI 2024. 2024, pp. 8048–8057. doi: 10.24963/ijcai.2024/890.

[14] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, “Large Language Models as Tool Makers,” in Proceedings of the Twelfth International Conference on Learning Representations, in ICLR 2024. 2024. [Online]. Available: https://openreview.net/forum?id=qV83K9d5WB

[15] J. Armstrong, “Making Reliable Distributed Systems in the Presence of Software Errors,” Doctoral dissertation, 2003. [Online]. Available: https://erlang.org/download/ armstrong_thesis_2003.pdf

[16] E. Moggi, “Notions of computation and monads,” Information and Computation, vol. 93, no. 1, pp. 55–92, 1991, doi: 10.1016/0890-5401(91)90052-4.

[17] G. Plotkin and J. Power, “Adequacy for Algebraic Efects,” in Foundations of Software Science and Computation Structures, F. Honsell and M. Miculan, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 1–24.

[18] T. Petricek, D. Orchard, and A. Mycroft, “Coefects: unified static analysis of contextdependence,” in Proceedings of the 40th International Conference on Automata, Languages, and Programming - Volume Part II, in ICALP’13. Riga, Latvia: Springer-Verlag, 2013, pp. 385–397. doi: 10.1007/978-3-642-39212-2_35.

参考文献（References）

[1] D. L. Parnas, “On the criteria to be used in decomposing systems into modules,” Communications of the ACM, vol. 15, no. 12, pp. 1053–1058, 1972, doi: 10.1145/361598.361623.

[2] D. Birsan, “On Plug-ins and Extensible Architectures,” ACM Queue, vol. 3, no. 2, pp. 40– 46, 2005, doi: 10.1145/1053331.1053345.

[3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016, doi: 10.1145/2890784.

[4] B. Stroustrup, The Design and Evolution of C++. Addison-Wesley, 1994.

[5] S. Marlow, S. Peyton Jones, A. Moran, and J. Reppy, “Asynchronous Exceptions in Haskell,” in Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation, in PLDI ’01. New York, NY, USA: Association for Computing Machinery, 2001, pp. 274–285. doi: 10.1145/378795.378858.

[6] L. Cardelli, “Program Fragments, Linking, and Modularization,” in Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 1997), ACM Press, 1997, pp. 266–277. doi: 10.1145/263699.263735.

[7] C. Szyperski, Component Software: Beyond Object-Oriented Programming, 2nd ed. Addison-Wesley, 2002.

[8] R. Lopopolo, “Harness Engineering: Leveraging Codex in an Agent-First World.” [Online]. Available: https://openai.com/index/harness-engineering/

[9] Anthropic, “Harness Design for Long-Running Application Development.” [Online]. Available: https://www.anthropic.com/engineering/harness-design-longrunning-apps

[10] L. Wang et al., “A Survey on Large Language Model Based Autonomous Agents,” Frontiers of Computer Science, vol. 18, no. 6, p. 186345, 2024, doi: 10.1007/s11704-024-40231-1.

[11] Y. Qin et al., “Tool Learning with Foundation Models,” ACM Computing Surveys, 2025, doi: 10.1145/3704435.

[12] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, “MemGPT: Towards LLMs as Operating Systems,” CoRR, vol. abs/2310.08560, 2023.

[13] T. Guo et al., “Large Language Model Based Multi-Agents: A Survey of Progress and Challenges,” in Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, in IJCAI 2024. 2024, pp. 8048–8057. doi: 10.24963/ijcai.2024/890.

[14] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, “Large Language Models as Tool Makers,” in Proceedings of the Twelfth International Conference on Learning Representations, in ICLR 2024. 2024. [Online]. Available: https://openreview.net/forum?id=qV83K9d5WB

[15] J. Armstrong, “Making Reliable Distributed Systems in the Presence of Software Errors,” Doctoral dissertation, 2003. [Online]. Available: https://erlang.org/download/ armstrong_thesis_2003.pdf

[16] E. Moggi, “Notions of computation and monads,” Information and Computation, vol. 93, no. 1, pp. 55–92, 1991, doi: 10.1016/0890-5401(91)90052-4.

[17] G. Plotkin and J. Power, “Adequacy for Algebraic Efects,” in Foundations of Software Science and Computation Structures, F. Honsell and M. Miculan, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 1–24.

[18] T. Petricek, D. Orchard, and A. Mycroft, “Coefects: unified static analysis of contextdependence,” in Proceedings of the 40th International Conference on Automata, Languages, and Programming - Volume Part II, in ICALP’13. Riga, Latvia: Springer-Verlag, 2013, pp. 385–397. doi: 10.1007/978-3-642-39212-2_35.

[19] M. Gaboardi, S.-ya Katsumata, D. Orchard, F. Breuvart, and T. Uustalu, “Combining efects and coefects via grading,” in Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, in ICFP 2016. Nara, Japan: Association for Computing Machinery, 2016, pp. 476–489. doi: 10.1145/2951913.2951939.

[20] A. Church, “A Formulation of the Simple Theory of Types,” The Journal of Symbolic Logic, vol. 5, no. 2, pp. 56–68, 1940, doi: 10.2307/2266170.

[21] B. C. Pierce, Types and Programming Languages. MIT Press, 2002.

[22] J. M. Lucassen and D. K. Giford, “Polymorphic Efect Systems,” in Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’88. San Diego, California, USA: Association for Computing Machinery, 1988, pp. 47–57. doi: 10.1145/73560.73564.

[23] P. Wadler, “Monads for functional programming,” in Program Design Calculi, M. Broy, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 233–264.

[24] G. Plotkin and J. Power, “Notions of Computation Determine Monads,” in Foundations of Software Science and Computation Structures, Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 342–356. doi: 10.1007/3-540-45931-6_24.

[25] G. Plotkin and M. Pretnar, “Handlers of Algebraic Efects,” in Programming Languages and Systems (ESOP), Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 80–94. doi: 10.1007/978-3-642-00590-9_7.

[26] M. Pretnar, “An Introduction to Algebraic Efects and Handlers. Invited tutorial paper,” Electron. Notes Theor. Comput. Sci., vol. 319, no. C, pp. 19–35, Dec. 2015, doi: 10.1016/ j.entcs.2015.12.003.

[27] D. Leijen, “Koka: Programming with Row Polymorphic Efect Types,” Electronic Proceedings in Theoretical Computer Science, vol. 153, pp. 100–126, Jun. 2014, doi: 10.4204/ eptcs.153.8.

[28] D. Leijen, “Type directed compilation of row-typed algebraic efects,” in Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, in POPL ’17. Paris, France: Association for Computing Machinery, 2017, pp. 486–499. doi: 10.1145/3009837.3009872.

[29] A. Bauer and M. Pretnar, “Programming with algebraic efects and handlers,” Journal of Logical and Algebraic Methods in Programming, vol. 84, no. 1, pp. 108–123, Jan. 2015, doi: 10.1016/j.jlamp.2014.02.001.

[30] K. Sivaramakrishnan et al., “Retrofitting parallelism onto OCaml,” Proc. ACM Program. Lang., vol. 4, no. ICFP, Aug. 2020, doi: 10.1145/3408995.

[31] T. Petricek, D. Orchard, and A. Mycroft, “Coefects: a calculus of context-dependent computation,” in Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming, in ICFP ’14. Gothenburg, Sweden: Association for Computing Machinery, 2014, pp. 123–135. doi: 10.1145/2628136.2628160.

[32] T. Uustalu and V. Vene, “Comonadic Notions of Computation,” Electronic Notes in Theoretical Computer Science, vol. 203, no. 5, pp. 263–284, 2008, doi: 10.1016/j.entcs.2008.05.029.

[33] A. Brunel, M. Gaboardi, D. Mazza, and S. Zdancewic, “A Core Quantitative Coefect Calculus,” in Proceedings of the 23rd European Symposium on Programming Languages and Systems - Volume 8410, Berlin, Heidelberg: Springer-Verlag, 2014, pp. 351–370. doi: 10.1007/978-3-642-54833-8_19.

[34] J. Reed and B. C. Pierce, “Distance makes the types grow stronger: a calculus for diferential privacy,” SIGPLAN Not., vol. 45, no. 9, pp. 157–168, Sep. 2010, doi: 10.1145/1932681.1863568.

[35] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A core calculus of dependency,” in Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’99. San Antonio, Texas, USA: Association for Computing Machinery, 1999, pp. 147–160. doi: 10.1145/292540.292555.

[36] D. E. Denning, “A lattice model of secure information flow,” Commun. ACM, vol. 19, no. 5, pp. 236–243, May 1976, doi: 10.1145/360051.360056.

[37] U. Dal Lago and F. Gavazzo, “A relational theory of efects and coefects,” Proc. ACM Program. Lang.,

[19] M. Gaboardi, S.-ya Katsumata, D. Orchard, F. Breuvart, and T. Uustalu, “Combining efects and coefects via grading,” in Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming, in ICFP 2016. Nara, Japan: Association for Computing Machinery, 2016, pp. 476–489. doi: 10.1145/2951913.2951939.

[20] A. Church, “A Formulation of the Simple Theory of Types,” The Journal of Symbolic Logic, vol. 5, no. 2, pp. 56–68, 1940, doi: 10.2307/2266170.

[21] B. C. Pierce, Types and Programming Languages. MIT Press, 2002.

[22] J. M. Lucassen and D. K. Giford, “Polymorphic Efect Systems,” in Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’88. San Diego, California, USA: Association for Computing Machinery, 1988, pp. 47–57. doi: 10.1145/73560.73564.

[23] P. Wadler, “Monads for functional programming,” in Program Design Calculi, M. Broy, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1993, pp. 233–264.

[24] G. Plotkin and J. Power, “Notions of Computation Determine Monads,” in Foundations of Software Science and Computation Structures, Berlin, Heidelberg: Springer Berlin Heidelberg, 2002, pp. 342–356. doi: 10.1007/3-540-45931-6_24.

[25] G. Plotkin and M. Pretnar, “Handlers of Algebraic Efects,” in Programming Languages and Systems (ESOP), Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 80–94. doi: 10.1007/978-3-642-00590-9_7.

[26] M. Pretnar, “An Introduction to Algebraic Efects and Handlers. Invited tutorial paper,” Electron. Notes Theor. Comput. Sci., vol. 319, no. C, pp. 19–35, Dec. 2015, doi: 10.1016/ j.entcs.2015.12.003.

[27] D. Leijen, “Koka: Programming with Row Polymorphic Efect Types,” Electronic Proceedings in Theoretical Computer Science, vol. 153, pp. 100–126, Jun. 2014, doi: 10.4204/ eptcs.153.8.

[28] D. Leijen, “Type directed compilation of row-typed algebraic efects,” in Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages, in POPL ’17. Paris, France: Association for Computing Machinery, 2017, pp. 486–499. doi: 10.1145/3009837.3009872.

[29] A. Bauer and M. Pretnar, “Programming with algebraic efects and handlers,” Journal of Logical and Algebraic Methods in Programming, vol. 84, no. 1, pp. 108–123, Jan. 2015, doi: 10.1016/j.jlamp.2014.02.001.

[30] K. Sivaramakrishnan et al., “Retrofitting parallelism onto OCaml,” Proc. ACM Program. Lang., vol. 4, no. ICFP, Aug. 2020, doi: 10.1145/3408995.

[31] T. Petricek, D. Orchard, and A. Mycroft, “Coefects: a calculus of context-dependent computation,” in Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming, in ICFP ’14. Gothenburg, Sweden: Association for Computing Machinery, 2014, pp. 123–135. doi: 10.1145/2628136.2628160.

[32] T. Uustalu and V. Vene, “Comonadic Notions of Computation,” Electronic Notes in Theoretical Computer Science, vol. 203, no. 5, pp. 263–284, 2008, doi: 10.1016/j.entcs.2008.05.029.

[33] A. Brunel, M. Gaboardi, D. Mazza, and S. Zdancewic, “A Core Quantitative Coefect Calculus,” in Proceedings of the 23rd European Symposium on Programming Languages and Systems - Volume 8410, Berlin, Heidelberg: Springer-Verlag, 2014, pp. 351–370. doi: 10.1007/978-3-642-54833-8_19.

[34] J. Reed and B. C. Pierce, “Distance makes the types grow stronger: a calculus for diferential privacy,” SIGPLAN Not., vol. 45, no. 9, pp. 157–168, Sep. 2010, doi: 10.1145/1932681.1863568.

[35] M. Abadi, A. Banerjee, N. Heintze, and J. G. Riecke, “A core calculus of dependency,” in Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’99. San Antonio, Texas, USA: Association for Computing Machinery, 1999, pp. 147–160. doi: 10.1145/292540.292555.

[36] D. E. Denning, “A lattice model of secure information flow,” Commun. ACM, vol. 19, no. 5, pp. 236–243, May 1976, doi: 10.1145/360051.360056.

[37] U. Dal Lago and F. Gavazzo, “A relational theory of efects and coefects,” Proc. ACM Program. Lang.,

vol. 6, no. POPL, Jan. 2022, doi: 10.1145/3498692.

[38] M. Fowler, “Inversion of Control Containers and the Dependency Injection pattern.” [Online]. Available: https://martinfowler.com/articles/injection.html

[39] A. M. Pitts and I. D. B. Stark, “Observable Properties of Higher Order Functions that Dynamically Create Local Names, or What’s New?,” in Mathematical Foundations of Computer Science 1993 (MFCS 1993), in Lecture Notes in Computer Science, vol. 711. Springer, 1993, pp. 122–141. doi: 10.1007/3-540-57182-5_8.

[40] G. D. Plotkin, “LCF Considered as a Programming Language,” Theoretical Computer Science, vol. 5, no. 3, pp. 223–255, 1977, doi: 10.1016/0304-3975(77)90044-5.

[41] D. R. Ghica, K. Muroya, and T. Waugh Ambridge, “A Robust Graph-Based Approach to Observational Equivalence,” Logical Methods in Computer Science, vol. 21, no. 2, p. 8:1– 8:95, 2025, doi: 10.46298/LMCS-21(2:8)2025.

[42] X. Leroy and S. Blazy, “Formal Verification of a C-like Memory Model and Its Uses for Verifying Program Transformations,” Journal of Automated Reasoning, vol. 41, no. 1, pp. 1–31, 2008, doi: 10.1007/s10817-008-9099-0.

[43] R. P. James and A. Sabry, “Yield: Mainstream Delimited Continuations,” in First International Workshop on the Theory and Practice of Delimited Continuations (TPDC 2011), 2011, pp. 20–32. [Online]. Available: https://homes.luddy.indiana.edu/sabry/files/yield.pdf

[44] A. W. Mazurkiewicz, “Trace Theory,” in Petri Nets: Central Models and Their Properties, Advances in Petri Nets 1986, Part II, in Lecture Notes in Computer Science, vol. 255. Springer, 1986, pp. 279–324. doi: 10.1007/3-540-17906-2_30.

[45] U. A. Acar, G. E. Blelloch, and R. Harper, “Adaptive functional programming,” ACM Transactions on Programming Languages and Systems, vol. 28, no. 6, pp. 990–1034, 2006, doi: 10.1145/1186632.1186634.

[46] webpack, “Hot Module Replacement.” [Online]. Available: https://webpack.js.org/api/ hot-module-replacement

[47] Vite, “HMR API.” [Online]. Available: https://vite.dev/guide/api-hmr

[48] E. N. (M. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, “A Survey of Rollback-Recovery Protocols in Message-Passing Systems,” ACM Computing Surveys, vol. 34, no. 3, pp. 375–408, 2002, doi: 10.1145/568522.568525.

[49] H. Garcia-Molina and K. Salem, “Sagas,” in Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data, in SIGMOD ’87. 1987, pp. 249–259. doi: 10.1145/38713.38742.

[50] OSGi Alliance, OSGi Core Release 8. OSGi Alliance, 2020. [Online]. Available: https:// docs.osgi.org/specification/osgi.core/8.0.0/

[51] J. Kramer and J. Magee, “The Evolving Philosophers Problem: Dynamic Change Management,” IEEE Transactions on Software Engineering, vol. 16, no. 11, pp. 1293–1306, 1990, doi: 10.1109/32.60317.

[52] Y. Vandewoude, P. Ebraert, Y. Berbers, and T. D’Hondt, “Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates,” IEEE Transactions on Software Engineering, vol. 33, no. 12, pp. 856–868, 2007, doi: 10.1109/tse.2007.70733.

[53] J. S. Rellermeyer, G. Alonso, and T. Roscoe, “R-OSGi: Distributed Applications Through Software Modularization,” in Proceedings of the ACM/IFIP/USENIX 8th International Middleware Conference, in Middleware ’07. 2007, pp. 1–20. doi: 10.1007/978-3-540-76778-7_1.

[54] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multiprogrammed Computations,” Communications of the ACM, vol. 9, no. 3, pp. 143–155, 1966, doi: 10.1145/365230.365252.

[55] M. S. Miller, K.-P. Yee, and J. Shapiro, “Capability Myths Demolished,” technical report SRL2003–2,

vol. 6, no. POPL, Jan. 2022, doi: 10.1145/3498692.

[38] M. Fowler, “Inversion of Control Containers and the Dependency Injection pattern.” [Online]. Available: https://martinfowler.com/articles/injection.html

[39] A. M. Pitts and I. D. B. Stark, “Observable Properties of Higher Order Functions that Dynamically Create Local Names, or What’s New?,” in Mathematical Foundations of Computer Science 1993 (MFCS 1993), in Lecture Notes in Computer Science, vol. 711. Springer, 1993, pp. 122–141. doi: 10.1007/3-540-57182-5_8.

[40] G. D. Plotkin, “LCF Considered as a Programming Language,” Theoretical Computer Science, vol. 5, no. 3, pp. 223–255, 1977, doi: 10.1016/0304-3975(77)90044-5.

[41] D. R. Ghica, K. Muroya, and T. Waugh Ambridge, “A Robust Graph-Based Approach to Observational Equivalence,” Logical Methods in Computer Science, vol. 21, no. 2, p. 8:1– 8:95, 2025, doi: 10.46298/LMCS-21(2:8)2025.

[42] X. Leroy and S. Blazy, “Formal Verification of a C-like Memory Model and Its Uses for Verifying Program Transformations,” Journal of Automated Reasoning, vol. 41, no. 1, pp. 1–31, 2008, doi: 10.1007/s10817-008-9099-0.

[43] R. P. James and A. Sabry, “Yield: Mainstream Delimited Continuations,” in First International Workshop on the Theory and Practice of Delimited Continuations (TPDC 2011), 2011, pp. 20–32. [Online]. Available: https://homes.luddy.indiana.edu/sabry/files/yield.pdf

[44] A. W. Mazurkiewicz, “Trace Theory,” in Petri Nets: Central Models and Their Properties, Advances in Petri Nets 1986, Part II, in Lecture Notes in Computer Science, vol. 255. Springer, 1986, pp. 279–324. doi: 10.1007/3-540-17906-2_30.

[45] U. A. Acar, G. E. Blelloch, and R. Harper, “Adaptive functional programming,” ACM Transactions on Programming Languages and Systems, vol. 28, no. 6, pp. 990–1034, 2006, doi: 10.1145/1186632.1186634.

[46] webpack, “Hot Module Replacement.” [Online]. Available: https://webpack.js.org/api/ hot-module-replacement

[47] Vite, “HMR API.” [Online]. Available: https://vite.dev/guide/api-hmr

[48] E. N. (M. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, “A Survey of Rollback-Recovery Protocols in Message-Passing Systems,” ACM Computing Surveys, vol. 34, no. 3, pp. 375–408, 2002, doi: 10.1145/568522.568525.

[49] H. Garcia-Molina and K. Salem, “Sagas,” in Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data, in SIGMOD ’87. 1987, pp. 249–259. doi: 10.1145/38713.38742.

[50] OSGi Alliance, OSGi Core Release 8. OSGi Alliance, 2020. [Online]. Available: https:// docs.osgi.org/specification/osgi.core/8.0.0/

[51] J. Kramer and J. Magee, “The Evolving Philosophers Problem: Dynamic Change Management,” IEEE Transactions on Software Engineering, vol. 16, no. 11, pp. 1293–1306, 1990, doi: 10.1109/32.60317.

[52] Y. Vandewoude, P. Ebraert, Y. Berbers, and T. D’Hondt, “Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates,” IEEE Transactions on Software Engineering, vol. 33, no. 12, pp. 856–868, 2007, doi: 10.1109/tse.2007.70733.

[53] J. S. Rellermeyer, G. Alonso, and T. Roscoe, “R-OSGi: Distributed Applications Through Software Modularization,” in Proceedings of the ACM/IFIP/USENIX 8th International Middleware Conference, in Middleware ’07. 2007, pp. 1–20. doi: 10.1007/978-3-540-76778-7_1.

[54] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multiprogrammed Computations,” Communications of the ACM, vol. 9, no. 3, pp. 143–155, 1966, doi: 10.1145/365230.365252.

[55] M. S. Miller, K.-P. Yee, and J. Shapiro, “Capability Myths Demolished,” technical report SRL2003–2,

2003. [Online]. Available: http://zesty.ca/capmyths/usenix.pdf

[56] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, "Capsicum: Practical Capabilities for UNIX," in Proceedings of the 19th USENIX Security Symposium, 2010, pp. 29–46. [Online]. Available: https://www.usenix.org/legacy/events/sec10/tech/full_papers/ Watson.pdf

[57] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, "Eficient Software-Based Fault Isolation," in Proceedings of the 14th ACM Symposium on Operating Systems Principles, in SOSP '93. 1993, pp. 203–216. doi: 10.1145/168619.168635.

[58] A. Barth, A. P. Felt, P. Saxena, and A. Boodman, "Protecting Browsers from Extension Vulnerabilities," in Proceedings of the 17th Annual Network and Distributed System Security Symposium, in NDSS '10. 2010. [Online]. Available: https://www.ndss-symposium.org/ ndss2010/protecting-browsers-extension-vulnerabilities/

[59] W. W. Ho and R. A. Olsson, "An Approach to Genuine Dynamic Linking," Software: Practice and Experience, vol. 21, no. 4, pp. 375–390, 1991, doi: 10.1002/SPE.4380210404.

[60] P. Wadler and S. Blott, "How to Make Ad-hoc Polymorphism Less Ad Hoc," in Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL '89. 1989, pp. 60–76. doi: 10.1145/75277.75283.

[61] N. D. Matsakis and F. S. K. II, "The Rust Language and Type System," in ACM SIGPLAN ML Family Workshop, Gothenburg, Sweden, Sep. 2014.

[62] D. Dreyer, R. Harper, M. M. T. Chakravarty, and G. Keller, "Modular Type Classes," in Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL '07. 2007, pp. 63–70. doi: 10.1145/1190216.1190229.

[63] Microsoft, "Declaration Merging." [Online]. Available: https://www.typescriptlang. org/docs/handbook/declaration-merging.html

[64] T. Van Cutsem and M. S. Miller, "Proxies: Design Principles for Robust Object-oriented Intercession APIs," in Proceedings of the 6th Symposium on Dynamic Languages, in DLS '10. 2010, pp. 59–72. doi: 10.1145/1869631.1869638.

[65] R. Hettinger, "Descriptor HowTo Guide." [Online]. Available: https://docs.python.org/ 3/howto/descriptor.html

[66] P. Maes, "Concepts and Experiments in Computational Reflection," in Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), 1987, pp. 147–155. doi: 10.1145/38765.38821.

[67] G. Bracha and D. M. Ungar, "Mirrors: design principles for meta-level facilities of objectoriented programming languages," in Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOP-SLA), 2004, pp. 331–344. doi: 10.1145/1028976.1029004.

[68] R. Rouvoy and P. Merle, "Leveraging component-based software engineering with Fraclet," Annals of Telecommunications, vol. 64, no. 1–2, pp. 65–79, 2009, doi: 10.1007/ s12243-008-0072-z.

[69] E. Burmako, "Scala Macros: Let Our Powers Combine!," in Proceedings of the 4th Workshop on Scala, in SCALA@ECOOP '13. 2013, p. 3:1–3:10. doi: 10.1145/2489837.2489840.

[70] S. Raemaekers, A. van Deursen, and J. Visser, "Semantic Versioning and Impact of Breaking Changes in the Maven Repository," Journal of Systems and Software, vol. 129, pp. 140–158, 2017, doi: 10.1016/j.jss.2016.04.008.

[71] P. Lam, J. Dietrich, and D. J. Pearce, "Putting the Semantics into Semantic Versioning," in Proceedings of the 2020 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, in Onward! '20. 2020, pp. 157– 179. doi: 10.1145/3426428.3426922.

[72] P. Abate, R. Di Cosmo, R. Treinen, and S. Zacchiroli, "Dependency Solving: A Separate Concern in Component Evolution Management," Journal of Systems and Software, vol. 85, no. 10, pp. 2228–2240, 2012, doi:

2003. [Online]. Available: http://zesty.ca/capmyths/usenix.pdf

[56] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, "Capsicum: Practical Capabilities for UNIX," in Proceedings of the 19th USENIX Security Symposium, 2010, pp. 29–46. [Online]. Available: https://www.usenix.org/legacy/events/sec10/tech/full_papers/ Watson.pdf

[57] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, "Eficient Software-Based Fault Isolation," in Proceedings of the 14th ACM Symposium on Operating Systems Principles, in SOSP '93. 1993, pp. 203–216. doi: 10.1145/168619.168635.

[58] A. Barth, A. P. Felt, P. Saxena, and A. Boodman, "Protecting Browsers from Extension Vulnerabilities," in Proceedings of the 17th Annual Network and Distributed System Security Symposium, in NDSS '10. 2010. [Online]. Available: https://www.ndss-symposium.org/ ndss2010/protecting-browsers-extension-vulnerabilities/

[59] W. W. Ho and R. A. Olsson, "An Approach to Genuine Dynamic Linking," Software: Practice and Experience, vol. 21, no. 4, pp. 375–390, 1991, doi: 10.1002/SPE.4380210404.

[60] P. Wadler and S. Blott, "How to Make Ad-hoc Polymorphism Less Ad Hoc," in Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL '89. 1989, pp. 60–76. doi: 10.1145/75277.75283.

[61] N. D. Matsakis and F. S. K. II, "The Rust Language and Type System," in ACM SIGPLAN ML Family Workshop, Gothenburg, Sweden, Sep. 2014.

[62] D. Dreyer, R. Harper, M. M. T. Chakravarty, and G. Keller, "Modular Type Classes," in Proceedings of the 34th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL '07. 2007, pp. 63–70. doi: 10.1145/1190216.1190229.

[63] Microsoft, "Declaration Merging." [Online]. Available: https://www.typescriptlang. org/docs/handbook/declaration-merging.html

[64] T. Van Cutsem and M. S. Miller, "Proxies: Design Principles for Robust Object-oriented Intercession APIs," in Proceedings of the 6th Symposium on Dynamic Languages, in DLS '10. 2010, pp. 59–72. doi: 10.1145/1869631.1869638.

[65] R. Hettinger, "Descriptor HowTo Guide." [Online]. Available: https://docs.python.org/ 3/howto/descriptor.html

[66] P. Maes, "Concepts and Experiments in Computational Reflection," in Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), 1987, pp. 147–155. doi: 10.1145/38765.38821.

[67] G. Bracha and D. M. Ungar, "Mirrors: design principles for meta-level facilities of objectoriented programming languages," in Proceedings of the 19th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOP-SLA), 2004, pp. 331–344. doi: 10.1145/1028976.1029004.

[68] R. Rouvoy and P. Merle, "Leveraging component-based software engineering with Fraclet," Annals of Telecommunications, vol. 64, no. 1–2, pp. 65–79, 2009, doi: 10.1007/ s12243-008-0072-z.

[69] E. Burmako, "Scala Macros: Let Our Powers Combine!," in Proceedings of the 4th Workshop on Scala, in SCALA@ECOOP '13. 2013, p. 3:1–3:10. doi: 10.1145/2489837.2489840.

[70] S. Raemaekers, A. van Deursen, and J. Visser, "Semantic Versioning and Impact of Breaking Changes in the Maven Repository," Journal of Systems and Software, vol. 129, pp. 140–158, 2017, doi: 10.1016/j.jss.2016.04.008.

[71] P. Lam, J. Dietrich, and D. J. Pearce, "Putting the Semantics into Semantic Versioning," in Proceedings of the 2020 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, in Onward! '20. 2020, pp. 157– 179. doi: 10.1145/3426428.3426922.

[72] P. Abate, R. Di Cosmo, R. Treinen, and S. Zacchiroli, "Dependency Solving: A Separate Concern in Component Evolution Management," Journal of Systems and Software, vol. 85, no. 10, pp. 2228–2240, 2012, doi:

10.1016/j.jss.2012.02.018.

[73] L. Cardelli, “Structural Subtyping and the Notion of Power Type,” in Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’88. 1988, pp. 70–79. doi: 10.1145/73560.73566.

[74] B. Meyer, “Applying “Design by Contract”,” Computer, vol. 25, no. 10, pp. 40–51, 1992, doi: 10.1109/2.161279.

[75] B. C. Pierce, “Bounded Quantification is Undecidable,” Information and Computation, vol. 112, no. 1, pp. 131–165, 1994, doi: 10.1006/inco.1994.1055.

[76] A. Haas et al., “Bringing the web up to speed with WebAssembly,” in Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), ACM, 2017, pp. 185–200. doi: 10.1145/3062341.3062363.

[77] M. M. Swift, B. N. Bershad, and H. M. Levy, “Improving the reliability of commodity operating systems,” in Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP), ACM, 2003, pp. 207–222. doi: 10.1145/945445.945466.

[78] M. M. Swift, M. Annamalai, B. N. Bershad, and H. M. Levy, “Recovering device drivers,” ACM Transactions on Computer Systems, vol. 24, no. 4, pp. 333–360, 2006, doi: 10.1145/1189256.1189257.

[79] D. E. Porter, O. S. Hofmann, C. J. Rossbach, A. Benn, and E. Witchel, “Operating System Transactions,” in Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP), ACM, 2009, pp. 161–176. doi: 10.1145/1629575.1629591.

[80] O. Kiselyov and C.-chieh Shan, “Delimited Continuations in Operating Systems,” in Modeling and Using Context (CONTEXT 2007), in Lecture Notes in Computer Science, vol. 4635. Springer, 2007, pp. 291–302. doi: 10.1007/978-3-540-74255-5_22.

[81] E. Dolstra and A. Löh, “NixOS: a purely functional Linux distribution,” in Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming (ICFP), ACM, 2008, pp. 367–378. doi: 10.1145/1411204.1411255.

[82] ZIO, “ZIO: Type-safe, composable asynchronous and concurrent programming for Scala.” [Online]. Available: https://zio.dev/

[83] Efect, “Efect: A TypeScript library for building robust applications.” [Online]. Available: https://efect.website/

[84] G. Canti, “fp-ts: Functional programming in TypeScript.” [Online]. Available: https:// github.com/gcanti/fp-ts

[85] J. I. Brachthäuser, P. Schuster, and K. Ostermann, “Efects as capabilities: efect handlers and lightweight efect polymorphism,” Proc. ACM Program. Lang., vol. 4, no. OOPSLA, 2020, doi: 10.1145/3428194.

[86] J. I. Brachthäuser, P. Schuster, E. Lee, and A. Boruch-Gruszecki, “Efects, capabilities, and boxes: from scope-based reasoning to type-based reasoning and back,” Proc. ACM Program. Lang., vol. 6, no. OOPSLA1, 2022, doi: 10.1145/3527320.

[87] C. Heunen, R. Kaarsgaard, and M. Karvonen, “Reversible Efects as Inverse Arrows,” in Proceedings of the Thirty-Fourth Conference on the Mathematical Foundations of Programming Semantics (MFPS XXXIV), in Electronic Notes in Theoretical Computer Science, vol. 341. 2018, pp. 179–199. doi: 10.1016/j.entcs.2018.11.009.

[88] D. Orchard, V.-B. Liepelt, and H. Eades III, “Quantitative program reasoning with graded modal types,” Proc. ACM Program. Lang., vol. 3, no. ICFP, 2019, doi: 10.1145/3341714.

[89] R. Bianchini, F. Dagnino, P. Giannini, E. Zucca, and M. Servetto, “Coefects for sharing and mutation,” Proc. ACM Program. Lang., vol. 6, no. OOPSLA2, Oct. 2022, doi: 10.1145/3563319.

[90] R. Bianchini, F. Dagnino, P. Giannini, and E. Zucca, “A Java-like calculus with heterogeneous coefects,”

10.1016/j.jss.2012.02.018.

[73] L. Cardelli, “Structural Subtyping and the Notion of Power Type,” in Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’88. 1988, pp. 70–79. doi: 10.1145/73560.73566.

[74] B. Meyer, “Applying “Design by Contract”,” Computer, vol. 25, no. 10, pp. 40–51, 1992, doi: 10.1109/2.161279.

[75] B. C. Pierce, “Bounded Quantification is Undecidable,” Information and Computation, vol. 112, no. 1, pp. 131–165, 1994, doi: 10.1006/inco.1994.1055.

[76] A. Haas et al., “Bringing the web up to speed with WebAssembly,” in Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), ACM, 2017, pp. 185–200. doi: 10.1145/3062341.3062363.

[77] M. M. Swift, B. N. Bershad, and H. M. Levy, “Improving the reliability of commodity operating systems,” in Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP), ACM, 2003, pp. 207–222. doi: 10.1145/945445.945466.

[78] M. M. Swift, M. Annamalai, B. N. Bershad, and H. M. Levy, “Recovering device drivers,” ACM Transactions on Computer Systems, vol. 24, no. 4, pp. 333–360, 2006, doi: 10.1145/1189256.1189257.

[79] D. E. Porter, O. S. Hofmann, C. J. Rossbach, A. Benn, and E. Witchel, “Operating System Transactions,” in Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP), ACM, 2009, pp. 161–176. doi: 10.1145/1629575.1629591.

[80] O. Kiselyov and C.-chieh Shan, “Delimited Continuations in Operating Systems,” in Modeling and Using Context (CONTEXT 2007), in Lecture Notes in Computer Science, vol. 4635. Springer, 2007, pp. 291–302. doi: 10.1007/978-3-540-74255-5_22.

[81] E. Dolstra and A. Löh, “NixOS: a purely functional Linux distribution,” in Proceedings of the 13th ACM SIGPLAN International Conference on Functional Programming (ICFP), ACM, 2008, pp. 367–378. doi: 10.1145/1411204.1411255.

[82] ZIO, “ZIO: Type-safe, composable asynchronous and concurrent programming for Scala.” [Online]. Available: https://zio.dev/

[83] Efect, “Efect: A TypeScript library for building robust applications.” [Online]. Available: https://efect.website/

[84] G. Canti, “fp-ts: Functional programming in TypeScript.” [Online]. Available: https:// github.com/gcanti/fp-ts

[85] J. I. Brachthäuser, P. Schuster, and K. Ostermann, “Efects as capabilities: efect handlers and lightweight efect polymorphism,” Proc. ACM Program. Lang., vol. 4, no. OOPSLA, 2020, doi: 10.1145/3428194.

[86] J. I. Brachthäuser, P. Schuster, E. Lee, and A. Boruch-Gruszecki, “Efects, capabilities, and boxes: from scope-based reasoning to type-based reasoning and back,” Proc. ACM Program. Lang., vol. 6, no. OOPSLA1, 2022, doi: 10.1145/3527320.

[87] C. Heunen, R. Kaarsgaard, and M. Karvonen, “Reversible Efects as Inverse Arrows,” in Proceedings of the Thirty-Fourth Conference on the Mathematical Foundations of Programming Semantics (MFPS XXXIV), in Electronic Notes in Theoretical Computer Science, vol. 341. 2018, pp. 179–199. doi: 10.1016/j.entcs.2018.11.009.

[88] D. Orchard, V.-B. Liepelt, and H. Eades III, “Quantitative program reasoning with graded modal types,” Proc. ACM Program. Lang., vol. 3, no. ICFP, 2019, doi: 10.1145/3341714.

[89] R. Bianchini, F. Dagnino, P. Giannini, E. Zucca, and M. Servetto, “Coefects for sharing and mutation,” Proc. ACM Program. Lang., vol. 6, no. OOPSLA2, Oct. 2022, doi: 10.1145/3563319.

[90] R. Bianchini, F. Dagnino, P. Giannini, and E. Zucca, “A Java-like calculus with heterogeneous coefects,”

Theoretical Computer Science, vol. 971, p. 114063, 2023, doi: https://doi. org/10.1016/j.tcs.2023.114063.

[91] C. Torczon, E. Suárez Acevedo, S. Agrawal, J. Velez-Ginorio, and S. Weirich, “Efects and Coefects in Call-by-Push-Value,” Proc. ACM Program. Lang., vol. 8, no. OOPSLA2, Oct. 2024, doi: 10.1145/3689750.

[92] R. Hirschfeld, P. Costanza, and O. Nierstrasz, “Context-oriented Programming,” Journal of Object Technology, vol. 7, no. 3, pp. 125–151, 2008, doi: 10.5381/jot.2008.7.3.a4.

[93] P. Costanza and R. Hirschfeld, “Language constructs for context-oriented programming: an overview of ContextL,” in Proceedings of the 2005 Symposium on Dynamic Languages (DLS ’05), ACM, 2005, pp. 1–10. doi: 10.1145/1146841.1146842.

[94] G. Salvaneschi, C. Ghezzi, and M. Pradella, “Context-oriented programming: A software engineering perspective,” Journal of Systems and Software, vol. 85, no. 8, pp. 1801–1817, 2012, doi: 10.1016/j.jss.2012.03.024.

[95] G. Kiczales et al., “Aspect-Oriented Programming,” in ECOOP’97 — Object-Oriented Programming, 11th European Conference, in Lecture Notes in Computer Science, vol. 1241. Springer, 1997, pp. 220–242. doi: 10.1007/BFb0053381.

[96] G. Kiczales, E. Hilsdale, J. Hugunin, M. Kersten, J. Palm, and W. G. Griswold, “An Overview of AspectJ,” in ECOOP 2001 — Object-Oriented Programming, 15th European Conference, in Lecture Notes in Computer Science, vol. 2072. Springer, 2001, pp. 327–353. doi: 10.1007/3-540-45337-7_18.

[97] A. Popovici, T. Gross, and G. Alonso, “Dynamic Weaving for Aspect-Oriented Programming,” in Proceedings of the 1st International Conference on Aspect-Oriented Software Development (AOSD 2002), ACM, 2002, pp. 141–147. doi: 10.1145/508386.508404.

[98] J. Bonér, “What Are the Key Issues for Commercial AOP Use: How Does AspectWerkz Address Them?,” in Proceedings of the 3rd International Conference on Aspect-Oriented Software Development (AOSD 2004), ACM, 2004, pp. 5–6. doi: 10.1145/976270.976273.

[99] M. Hicks, J. T. Moore, and S. Nettles, “Dynamic Software Updating,” in Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation, in PLDI ’01. 2001, pp. 13–23. doi: 10.1145/378795.378798.

[100] G. Stoyle, M. Hicks, G. Bierman, P. Sewell, and I. Neamtiu, “Mutatis Mutandis: Safe and Predictable Dynamic Software Updating,” in Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’05. 2005, pp. 183– 194. doi: 10.1145/1040305.1040321.

[101] C. M. Hayden, K. Saur, E. K. Smith, and M. Hicks, “Kitsune: Eficient, General-Purpose Dynamic Software Updating for C,” ACM Trans. Program. Lang. Syst., vol. 36, no. 4, 2014, doi: 10.1145/2629460.

[102] M. Overeem, M. Spoor, and S. Jansen, “The Dark Side of Event Sourcing: Managing Data Conversion,” in IEEE 24th International Conference on Software Analysis, Evolution and Reengineering, in SANER ’17. 2017, pp. 193–204. doi: 10.1109/SANER.2017.7884621.

[103] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA: Addison-Wesley, 1994.

[104] D. Leijen, “Algebraic Efect Handlers with Resources and Deep Finalization,” technical report MSR-TR-2018-10, Apr. 2018. [Online]. Available: https://www.microsoft.com/ en-us/research/publication/algebraic-efect-handlers-resources-deep-finalization/

[105] M. Fowler, “Event Sourcing.” 2005.

[106] J. Lee, J. Ahn, and K. Yi, “React-tRace: A Semantics for Understanding React Hooks,” Proc. ACM Program. Lang., vol. 9, no. OOPSLA2, pp. 471–498, 2025, doi: 10.1145/3763067.

[107] N. Shavit and D. Touitou, “Software Transactional Memory,” in Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing, in PODC ’95. 1995, pp. 204–213. doi: 10.1145/224964.224987.

Theoretical Computer Science, vol. 971, p. 114063, 2023, doi: https://doi. org/10.1016/j.tcs.2023.114063.

[91] C. Torczon, E. Suárez Acevedo, S. Agrawal, J. Velez-Ginorio, and S. Weirich, “Efects and Coefects in Call-by-Push-Value,” Proc. ACM Program. Lang., vol. 8, no. OOPSLA2, Oct. 2024, doi: 10.1145/3689750.

[92] R. Hirschfeld, P. Costanza, and O. Nierstrasz, “Context-oriented Programming,” Journal of Object Technology, vol. 7, no. 3, pp. 125–151, 2008, doi: 10.5381/jot.2008.7.3.a4.

[93] P. Costanza and R. Hirschfeld, “Language constructs for context-oriented programming: an overview of ContextL,” in Proceedings of the 2005 Symposium on Dynamic Languages (DLS ’05), ACM, 2005, pp. 1–10. doi: 10.1145/1146841.1146842.

[94] G. Salvaneschi, C. Ghezzi, and M. Pradella, “Context-oriented programming: A software engineering perspective,” Journal of Systems and Software, vol. 85, no. 8, pp. 1801–1817, 2012, doi: 10.1016/j.jss.2012.03.024.

[95] G. Kiczales et al., “Aspect-Oriented Programming,” in ECOOP’97 — Object-Oriented Programming, 11th European Conference, in Lecture Notes in Computer Science, vol. 1241. Springer, 1997, pp. 220–242. doi: 10.1007/BFb0053381.

[96] G. Kiczales, E. Hilsdale, J. Hugunin, M. Kersten, J. Palm, and W. G. Griswold, “An Overview of AspectJ,” in ECOOP 2001 — Object-Oriented Programming, 15th European Conference, in Lecture Notes in Computer Science, vol. 2072. Springer, 2001, pp. 327–353. doi: 10.1007/3-540-45337-7_18.

[97] A. Popovici, T. Gross, and G. Alonso, “Dynamic Weaving for Aspect-Oriented Programming,” in Proceedings of the 1st International Conference on Aspect-Oriented Software Development (AOSD 2002), ACM, 2002, pp. 141–147. doi: 10.1145/508386.508404.

[98] J. Bonér, “What Are the Key Issues for Commercial AOP Use: How Does AspectWerkz Address Them?,” in Proceedings of the 3rd International Conference on Aspect-Oriented Software Development (AOSD 2004), ACM, 2004, pp. 5–6. doi: 10.1145/976270.976273.

[99] M. Hicks, J. T. Moore, and S. Nettles, “Dynamic Software Updating,” in Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation, in PLDI ’01. 2001, pp. 13–23. doi: 10.1145/378795.378798.

[100] G. Stoyle, M. Hicks, G. Bierman, P. Sewell, and I. Neamtiu, “Mutatis Mutandis: Safe and Predictable Dynamic Software Updating,” in Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, in POPL ’05. 2005, pp. 183– 194. doi: 10.1145/1040305.1040321.

[101] C. M. Hayden, K. Saur, E. K. Smith, and M. Hicks, “Kitsune: Eficient, General-Purpose Dynamic Software Updating for C,” ACM Trans. Program. Lang. Syst., vol. 36, no. 4, 2014, doi: 10.1145/2629460.

[102] M. Overeem, M. Spoor, and S. Jansen, “The Dark Side of Event Sourcing: Managing Data Conversion,” in IEEE 24th International Conference on Software Analysis, Evolution and Reengineering, in SANER ’17. 2017, pp. 193–204. doi: 10.1109/SANER.2017.7884621.

[103] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA: Addison-Wesley, 1994.

[104] D. Leijen, “Algebraic Efect Handlers with Resources and Deep Finalization,” technical report MSR-TR-2018-10, Apr. 2018. [Online]. Available: https://www.microsoft.com/ en-us/research/publication/algebraic-efect-handlers-resources-deep-finalization/

[105] M. Fowler, “Event Sourcing.” 2005.

[106] J. Lee, J. Ahn, and K. Yi, “React-tRace: A Semantics for Understanding React Hooks,” Proc. ACM Program. Lang., vol. 9, no. OOPSLA2, pp. 471–498, 2025, doi: 10.1145/3763067.

[107] N. Shavit and D. Touitou, “Software Transactional Memory,” in Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing, in PODC ’95. 1995, pp. 204–213. doi: 10.1145/224964.224987.

[108] T. Harris, S. Marlow, S. Peyton Jones, and M. Herlihy, “Composable Memory Transactions,” in Proceedings of the Tenth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, in PPoPP ’05. 2005, pp. 48–60. doi: 10.1145/1065944.1065952.

[109] M. Herlihy and J. E. B. Moss, “Transactional Memory: Architectural Support for Lock-Free Data Structures,” in Proceedings of the 20th Annual International Symposium on Computer Architecture, in ISCA ’93. 1993, pp. 289–300. doi: 10.1145/165123.165164.

[110] R. Landauer, “Irreversibility and Heat Generation in the Computing Process,” IBM Journal of Research and Development, vol. 5, no. 3, pp. 183–191, 1961, doi: 10.1147/rd.53.0183.

[111] C. H. Bennett, “Logical Reversibility of Computation,” IBM Journal of Research and Development, vol. 17, no. 6, pp. 525–532, 1973, doi: 10.1147/rd.176.0525.

[112] T. Yokoyama and R. Glück, “A Reversible Programming Language and its Invertible Self-Interpreter,” in Proceedings of the 2007 ACM SIGPLAN Workshop on Partial Evaluation and Semantics-Based Program Manipulation, in PEPM ’07. 2007, pp. 144–153. doi: 10.1145/1244381.1244404.

[113] V. Danos and J. Krivine, “Reversible Communicating Systems,” in CONCUR 2004 — Concurrency Theory, 15th International Conference, in Lecture Notes in Computer Science, vol. 3170. Springer, 2004, pp. 292–307. doi: 10.1007/978-3-540-28644-8_19.

[114] I. Phillips and I. Ulidowski, “Reversing Algebraic Process Calculi,” in Foundations of Software Science and Computation Structures, 9th International Conference (FOSSACS 2006), in Lecture Notes in Computer Science, vol. 3921. Springer, 2006, pp. 246–260. doi: 10.1007/11690634_17.

[115] P. Wadler, “Linear Types Can Change the World!,” in Programming Concepts and Methods: Proceedings of the IFIP Working Group 2.2/2.3 Working Conference, North-Holland, 1990, pp. 561–581. [Online]. Available: https://homepages.inf.ed.ac.uk/wadler/papers/ linear/linear.ps

[116] A. Lenharth, V. S. Adve, and S. T. King, “Recovery domains: an organizing principle for recoverable operating systems,” in Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM, 2009, pp. 49–60. doi: 10.1145/1508244.1508251.

[117] C. Walls, Spring in Action, 6th ed. Manning Publications, 2022. [Online]. Available: https://www.manning.com/books/spring-in-action-sixth-edition

[118] C. Escofier, R. S. Hall, and P. Lalanda, “iPOJO: an Extensible Service-Oriented Component Framework,” in IEEE International Conference on Services Computing, 2007, pp. 474– 481. doi: 10.1109/SCC.2007.74.

[119] H. Cervantes and R. S. Hall, “Autonomous Adaptation to Dynamic Availability Using a Service-Oriented Component Model,” in Proceedings of the 26th International Conference on Software Engineering, in ICSE ’04. 2004, pp. 614–623. doi: 10.1109/ICSE.2004.1317483.

[120] C. Elliott and P. Hudak, “Functional Reactive Animation,” in Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming, in ICFP ’97. 1997, pp. 263–273. doi: 10.1145/258948.258973.

[121] G. H. Cooper and S. Krishnamurthi, “Embedding Dynamic Dataflow in a Call-by-Value Language,” in Programming Languages and Systems (ESOP 2006), in Lecture Notes in Computer Science, vol. 3924. Springer, 2006, pp. 294–308. doi: 10.1007/11693024_20.

[122] I. Maier and M. Odersky, “Deprecating the Observer Pattern with Scala.React,” technical report EPFL-REPORT-176887, 2012. [Online]. Available: https://infoscience.epfl.ch/ record/176887

[123] E. Bainomugisha, A. L. Carreton, T. Van Cutsem, W. De Meuter, and others, “A Survey on Reactive Programming,” ACM Comput. Surv., vol. 45, no. 4, 2013, doi: 10.1145/2501654.2501666.

[124] A. Margara and G. Salvaneschi, “On the Semantics of Distributed Reactive Programming: The Cost of Consistency,” IEEE Trans. Software Eng., vol. 44, no. 7, pp. 689–711, 2018, doi: 10.1109/TSE.2018.2833109.

[108] T. Harris, S. Marlow, S. Peyton Jones, and M. Herlihy, “Composable Memory Transactions,” in Proceedings of the Tenth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, in PPoPP ’05. 2005, pp. 48–60. doi: 10.1145/1065944.1065952.

[109] M. Herlihy and J. E. B. Moss, “Transactional Memory: Architectural Support for Lock-Free Data Structures,” in Proceedings of the 20th Annual International Symposium on Computer Architecture, in ISCA ’93. 1993, pp. 289–300. doi: 10.1145/165123.165164.

[110] R. Landauer, “Irreversibility and Heat Generation in the Computing Process,” IBM Journal of Research and Development, vol. 5, no. 3, pp. 183–191, 1961, doi: 10.1147/rd.53.0183.

[111] C. H. Bennett, “Logical Reversibility of Computation,” IBM Journal of Research and Development, vol. 17, no. 6, pp. 525–532, 1973, doi: 10.1147/rd.176.0525.

[112] T. Yokoyama and R. Glück, “A Reversible Programming Language and its Invertible Self-Interpreter,” in Proceedings of the 2007 ACM SIGPLAN Workshop on Partial Evaluation and Semantics-Based Program Manipulation, in PEPM ’07. 2007, pp. 144–153. doi: 10.1145/1244381.1244404.

[113] V. Danos and J. Krivine, “Reversible Communicating Systems,” in CONCUR 2004 — Concurrency Theory, 15th International Conference, in Lecture Notes in Computer Science, vol. 3170. Springer, 2004, pp. 292–307. doi: 10.1007/978-3-540-28644-8_19.

[114] I. Phillips and I. Ulidowski, “Reversing Algebraic Process Calculi,” in Foundations of Software Science and Computation Structures, 9th International Conference (FOSSACS 2006), in Lecture Notes in Computer Science, vol. 3921. Springer, 2006, pp. 246–260. doi: 10.1007/11690634_17.

[115] P. Wadler, “Linear Types Can Change the World!,” in Programming Concepts and Methods: Proceedings of the IFIP Working Group 2.2/2.3 Working Conference, North-Holland, 1990, pp. 561–581. [Online]. Available: https://homepages.inf.ed.ac.uk/wadler/papers/ linear/linear.ps

[116] A. Lenharth, V. S. Adve, and S. T. King, “Recovery domains: an organizing principle for recoverable operating systems,” in Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM, 2009, pp. 49–60. doi: 10.1145/1508244.1508251.

[117] C. Walls, Spring in Action, 6th ed. Manning Publications, 2022. [Online]. Available: https://www.manning.com/books/spring-in-action-sixth-edition

[118] C. Escofier, R. S. Hall, and P. Lalanda, “iPOJO: an Extensible Service-Oriented Component Framework,” in IEEE International Conference on Services Computing, 2007, pp. 474– 481. doi: 10.1109/SCC.2007.74.

[119] H. Cervantes and R. S. Hall, “Autonomous Adaptation to Dynamic Availability Using a Service-Oriented Component Model,” in Proceedings of the 26th International Conference on Software Engineering, in ICSE ’04. 2004, pp. 614–623. doi: 10.1109/ICSE.2004.1317483.

[120] C. Elliott and P. Hudak, “Functional Reactive Animation,” in Proceedings of the Second ACM SIGPLAN International Conference on Functional Programming, in ICFP ’97. 1997, pp. 263–273. doi: 10.1145/258948.258973.

[121] G. H. Cooper and S. Krishnamurthi, “Embedding Dynamic Dataflow in a Call-by-Value Language,” in Programming Languages and Systems (ESOP 2006), in Lecture Notes in Computer Science, vol. 3924. Springer, 2006, pp. 294–308. doi: 10.1007/11693024_20.

[122] I. Maier and M. Odersky, “Deprecating the Observer Pattern with Scala.React,” technical report EPFL-REPORT-176887, 2012. [Online]. Available: https://infoscience.epfl.ch/ record/176887

[123] E. Bainomugisha, A. L. Carreton, T. Van Cutsem, W. De Meuter, and others, “A Survey on Reactive Programming,” ACM Comput. Surv., vol. 45, no. 4, 2013, doi: 10.1145/2501654.2501666.

[124] A. Margara and G. Salvaneschi, “On the Semantics of Distributed Reactive Programming: The Cost of Consistency,” IEEE Trans. Software Eng., vol. 44, no. 7, pp. 689–711, 2018, doi: 10.1109/TSE.2018.2833109.